

## UNIT STUDY NOTES

# SQL and Relational Databases

SQL is the interface between your analysis and durable, concurrent, correctly stored data. This unit covers relational modelling, DDL and constraints, the full DML/DQL surface, joins, aggregation, subqueries and window functions, plus transaction and index fundamentals.

 1 source notebook 962 topic sections extracted 8 study sections

## Unit Overview

SQL is one of the foundational languages of data engineering, analytics, backend systems, and practical artificial intelligence.

A machine-learning model may eventually consume a NumPy array or tensor, but real organizational data rarely begins in that form. Customer records, transactions, subscriptions, payments, products, events, experiment metadata, predictions, and monitoring information frequently live in databases.

An AI engineer therefore needs substantially more than the ability to write a few `SELECT` statements.

You should understand:

- why databases exist;
- how relational information is represented;
- how tables relate through keys;
- how constraints protect integrity;
- how SQL differs from ordinary imperative programming;
- how data types affect semantics;
- how joins change dataset grain;
- how aggregation changes grain;
- how transactions protect state;
- how indexes affect performance;
- how SQL interacts with Python;
- how SQL preprocessing affects machine-learning correctness;
- how temporal leakage can occur inside otherwise valid SQL.

This notebook develops SQL and relational database engineering from first principles.

Most executable examples use SQLite through Python's built-in `sqlite3` module. This allows the notebook to execute without a separate database server.

SQLite is the execution environment for learning; it is not the entire SQL subject. PostgreSQL, MySQL, SQL Server, Oracle, BigQuery, Snowflake, DuckDB, and other systems differ in syntax, type systems, functions, concurrency, query optimization, and operational architecture.

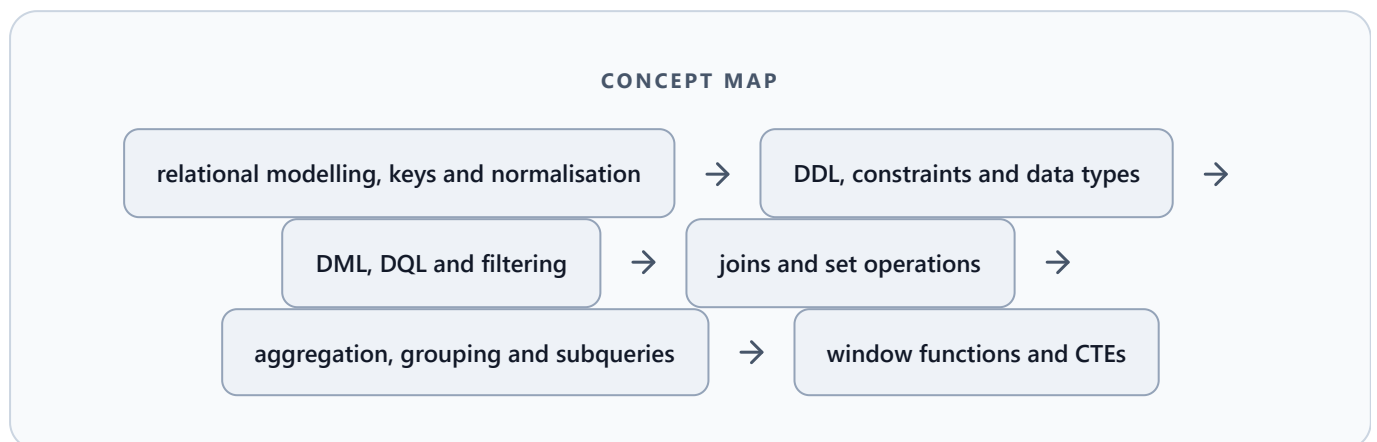
## Prerequisites

→ Python Core Fundamentals

## What you will be able to do

- ✓ Design a normalised schema with appropriate keys and constraints
- ✓ Write correct DDL and DML, including safe ALTER and DELETE patterns
- ✓ Join tables across keys without dropping or duplicating rows
- ✓ Aggregate, group and pivot results for reporting
- ✓ Use subqueries, CTEs and window functions for analytical queries
- ✓ Reason about transactions, isolation and index usage

## What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

## Core Concepts

### SQL and Relational Database Engineering

#### Learning Philosophy

SQL is simple to begin but deep to master.

A beginner can quickly write:

but that does not mean the learner understands relational databases.

A query can execute successfully and still be logically wrong.

For example, joining a customer table to order items can silently change the grain from:

to:

Likewise, an ML feature query can be perfectly valid SQL while being methodologically wrong because it includes future information unavailable at prediction time.

We will therefore learn SQL at several layers:

The objective is not simply to make SQL execute. It is to understand exactly what the result represents and why it is correct.

#### SQL

```
SELECT *  
FROM customers;
```

## Data and Persistent Storage

A running Python process can hold information in variables:

Those values normally disappear when the process terminates unless they are persisted.

Persistent storage allows information to survive beyond one execution.

Examples include:

- text files;
- CSV files;
- JSON;
- binary files;
- object storage;
- relational databases;
- key-value databases;
- document databases;
- analytical warehouses.

A database is a structured system for storing and retrieving persistent information.

In engineering, it is useful to distinguish the stored data from the software responsible for managing that data.

#### PYTHON

```
customer_name = "Aisha"  
balance = 2500
```

## Database Management Systems

A Database Management System, or DBMS, is software that manages databases.

A DBMS may provide mechanisms for:

- defining data structures;
- storing records;
- querying records;
- modifying records;
- enforcing constraints;
- managing concurrent users;
- maintaining indexes;
- managing transactions;
- controlling permissions;
- recovering from failures;
- optimizing queries.

Examples include:

- PostgreSQL;
- MySQL;
- MariaDB;
- SQLite;
- Microsoft SQL Server;
- Oracle Database.

Instead of every application inventing its own storage engine, locking system, indexing strategy, query processor, and recovery mechanism, applications can delegate many of these responsibilities to a database-management system.

## Why Not Store Everything in CSV Files?

CSV files are useful, but they do not naturally provide the same guarantees and capabilities as relational database systems.

Imagine:

An application creates an order for customer `5001`.

Several questions appear:

- Does customer 5001 exist?
- What if two applications write simultaneously?
- What if the process crashes halfway through an update?

- How do we prevent duplicate customer identifiers?
- How do we efficiently find every order for one customer?
- How do we update several related pieces of information atomically?
- Who is allowed to modify customer data?
- How do we query millions of records efficiently?

These problems can be solved with systems built around files, but databases are specifically designed to address many of them.

Files and databases therefore solve overlapping but different engineering problems.

PYTHON

```
customers.csv  
orders.csv
```

## Relational Databases

A relational database organizes information using relations.

In practical SQL systems, relations are commonly represented as tables.

For example:

Each row represents a record.

Each column represents an attribute.

The relational model is deeper than "a spreadsheet in a database." It provides concepts for identity, relationships, dependencies, constraints, and relational operations.

These concepts eventually explain:

- primary keys;
- foreign keys;
- normalization;
- joins;
- grouping;
- relational algebra;
- query correctness.

## PYTHON

customers

| customer_id | name  | city   |
|-------------|-------|--------|
| 1           | Aisha | Delhi  |
| 2           | Rohan | Pune   |
| 3           | Meera | Jaipur |

**Tables, Rows, Columns, and Values**

Consider:

The table represents a collection of employee records.

`employee_id`, `name`, `department`, and `salary` are columns.

The record:

is one row.

`90000` is one value.

Later SQL operations act at different levels: some transform individual values, some filter rows, some group many rows, and some combine entire relations.

## PYTHON

| employee_id | name  | department | salary |
|-------------|-------|------------|--------|
| 101         | Aisha | AI         | 90000  |
| 102         | Rohan | Data       | 85000  |

## PYTHON

101 | Aisha | AI | 90000

**Relation Versus SQL Table**

In mathematical relational theory, a relation is conceptually a set of tuples.

A mathematical set:

- contains no duplicates;
- has no inherent order.

Practical SQL differs in important ways.

SQL query results can contain duplicates unless they are prevented or removed.

SQL results also have no guaranteed presentation order unless an `ORDER BY` is explicitly requested.

SQL additionally introduces `NULL`, which makes its logic more complicated than classical two-valued relational reasoning.

It is useful to distinguish:

#### PYTHON

```
relational theory
  ↓
conceptual foundation

SQL database
  ↓
practical implementation
```

## Schema

A schema describes the structure of data.

A table schema can define:

- table name;
- column names;
- data types;
- primary keys;
- foreign keys;
- nullability;
- uniqueness rules;
- checks;
- default values.

For example:

A schema is a data contract.

For AI systems, schema changes matter because a feature pipeline can break even when a database connection itself continues working.

#### PYTHON

```
customers

customer_id  INTEGER  PRIMARY KEY
name         TEXT     NOT NULL
email        TEXT     UNIQUE
city         TEXT
```

## Schema Versus Data

Schema describes structure.

Data contains actual records.

Example schema:

Example data:

The distinction is similar to the difference between a structural definition and instances conforming to it.

Database schema design influences correctness, maintainability, storage, querying, and downstream application behavior.

PYTHON

```
student_id : integer
name       : text
score      : numeric
```

PYTHON

```
1 | Aisha | 88.5
2 | Rohan | 91.0
```

## Metadata

Metadata is data describing other data.

Database metadata can describe:

- tables;
- columns;
- data types;
- constraints;
- indexes;
- views;
- relationships.

Database systems maintain metadata internally so applications and developer tools can inspect database structure programmatically.

Later we will inspect SQLite schema information directly.

## Domains

An attribute conceptually draws values from a domain.

Examples:



A database type is only part of the domain.

An integer column can technically hold:

but that might be invalid for an age.

Therefore domain correctness can involve:

- data types;
- `CHECK` constraints;
- foreign keys;
- reference tables;
- application validation.

An important principle is:

PYTHON

|             |   |                         |
|-------------|---|-------------------------|
| age         | → | plausible ages          |
| salary      | → | valid monetary values   |
| signup_date | → | valid dates             |
| status      | → | allowed business states |

PYTHON

-500

## Tuples and Attributes

Relational terminology often uses:

For:

the complete record is a tuple.

`customer_id`, `name`, and `city` are attributes.

SQL documentation often uses row and column terminology, but relational terminology becomes useful when studying theory and normalization.

PYTHON

|           |   |        |
|-----------|---|--------|
| tuple     | → | row    |
| attribute | → | column |

## Degree and Cardinality

The degree of a relation refers to the number of attributes.

A table containing:

has degree 4.

Cardinality in relational theory refers to the number of tuples.

In practical database engineering, the word cardinality is also used when discussing:

- number of distinct values;
- relationship multiplicity;
- query-planner estimates.

Context matters.

Later, join cardinality will be especially important.

PYTHON

```
customer_id
name
city
signup_date
```

## Keys

Keys provide identity and relationships.

Important key concepts include:

- superkeys;
- candidate keys;
- primary keys;
- alternate keys;
- composite keys;
- natural keys;
- surrogate keys;
- foreign keys.

These terms describe different roles in relational modeling.

## Superkeys

A superkey is a set of attributes capable of uniquely identifying a row.

Suppose:

and `user_id` is guaranteed unique.

Then:

is a superkey.

The larger combination:

also uniquely identifies a row, but it contains unnecessary information.

This leads to candidate keys.

A **superkey** is any set of attributes that uniquely identifies a row.

Suppose:

uniquely identifies customers.

Then these can all technically be superkeys:

Once a unique identifier is included, adding additional attributes still leaves the set unique.

But the larger sets contain unnecessary attributes.

That leads to the idea of a candidate key.

PYTHON

```
user_id
email
username
full_name
```

PYTHON

```
{user_id}
```

## Candidate Keys

A candidate key is a minimal superkey.

Minimal means no unnecessary attribute can be removed while preserving uniqueness.

Suppose both:

are independently guaranteed unique.

Both can be candidate keys.

One candidate key may be chosen as the primary key, while another can remain protected through a uniqueness constraint.

A candidate key is a minimal superkey.

"Minimal" means that if any attribute is removed, the remaining attributes no longer guarantee uniqueness.

Suppose both of these are guaranteed unique:

Then each can be a candidate key.

One may be chosen as the primary key while the other remains an alternate candidate key and may receive a **UNIQUE** constraint.

Candidate keys are determined by business rules, not just observed duplicate counts.

```
PYTHON
```

```
user_id  
email
```

## Primary Keys

A primary key is the candidate key chosen as the table's main identifier.

Example:

A primary key supports:

- record identity;
- relationships;
- updates;
- joins;
- references from other tables.

Primary-key design is therefore a structural decision rather than merely a syntactic requirement.

```
PYTHON
```

```
customer_id INTEGER PRIMARY KEY
```

## Composite Keys

A key can contain multiple columns.

Consider:

Neither column is individually unique.

But the pair:

can uniquely identify one enrollment.

That pair can therefore serve as a composite key.

A key may contain multiple columns.

Consider course enrollment:

The combination may uniquely identify one enrollment.

Then:

directly represents the business rule.

Composite keys are particularly common in:

- junction tables
- association tables
- mappings
- line items
- temporal records

A surrogate key can be added when useful, but it should not accidentally weaken the natural uniqueness rule.

PYTHON

| student_id | course_id |
|------------|-----------|
| 1          | 101       |
| 1          | 102       |
| 2          | 101       |

PYTHON

```
(student_id, course_id)
```

## Natural Keys

A natural key comes from meaningful domain data.

Examples might include:

- an externally assigned account code;
- a standardized identifier;
- an immutable serial number.

Natural keys may be useful, but real-world identifiers can:

- change;
- contain errors;
- be reused;
- be sensitive;
- differ across systems.

Their suitability must be evaluated rather than assumed.

A natural key is derived from meaningful domain data.

Examples might include:

Natural keys can be attractive because they already have business meaning.

However, they can also be problematic if they:

- change

- are long
- are sensitive
- are externally controlled
- have awkward formats
- are only "usually unique"
- have jurisdiction-dependent rules

A natural key should not be selected merely because a column happens to look unique.

PYTHON

```
ISO country code
SKU
vehicle identification number
government-issued identifier
```

## Surrogate Keys

A surrogate key is an artificial identifier introduced mainly for database identity.

Examples:

Surrogate keys can provide stable internal identity.

However, adding a surrogate key does not eliminate business uniqueness requirements.

If email must be unique, introducing `user_id` does not remove the need to enforce unique email semantics.

A surrogate key is an artificial identifier introduced primarily for row identity.

Examples:

The identifier need not have domain meaning.

Surrogate keys can simplify:

- foreign-key references
- joins
- schema evolution
- integration when natural identifiers change

However, adding a surrogate key does not remove business uniqueness requirements.

If email must be unique, this schema is insufficient:

We may still need:

Surrogate identity and business integrity solve different problems.

## PYTHON

```
customer_id = 100001
order_id = 900501
```

## PYTHON

```
customer_id INTEGER
order_id INTEGER
model_version_id UUID
```

## Alternate Keys

If a table has multiple candidate keys and one becomes the primary key, the remaining candidate keys are often called alternate keys.

Example:

If email truly must remain unique, that rule should still be represented in the schema.

## PYTHON

```
user_id → primary key
email → alternate key
```

## Foreign Keys

A foreign key establishes a relationship between tables.

Consider:

`orders.customer_id` can reference:

Conceptually:

This says that an order belongs to an existing customer.

## PYTHON

```
customers
-----
customer_id

orders
-----
order_id
customer_id
```

## Referential Integrity

Referential integrity means relationships remain valid.

Suppose an order contains:

but no customer `999999` exists.

That order is orphaned relative to the intended customer relationship.

A foreign-key constraint can prevent such invalid states.

Broken relationships can cause serious analytical problems such as:

- missing features;
- incorrect aggregates;
- silently dropped rows;
- invalid business reports;
- inconsistent model inputs.

```
PYTHON
```

```
customer_id = 999999
```

## One-to-One Relationships

A one-to-one relationship means one row on one side corresponds to at most one row on the other side.

Example:

True one-to-one table separation should have a meaningful design reason, such as security, lifecycle, optional data, or architectural boundaries.

A one-to-one relationship means one row in entity A corresponds to at most one relevant row in entity B, and vice versa according to the domain rule.

Examples might include:

A one-to-one relationship can often be enforced using a foreign key plus `UNIQUE`, or by sharing the same primary key.

Do not split a table one-to-one without a reason.

Possible reasons include:

- different security requirements
- optional large attributes
- lifecycle differences
- modular domain design



SQL

```

user
|
| 1
v
user_profile

```

SQL

```

design_conn.executescript("""
CREATE TABLE app_users (
    user_id INTEGER PRIMARY KEY,
    username TEXT NOT NULL UNIQUE
);

CREATE TABLE user_profiles (
    user_id INTEGER PRIMARY KEY,
    bio TEXT,
    timezone TEXT,
    FOREIGN KEY (user_id)
        REFERENCES app_users(user_id)
        ON DELETE CASCADE
);
""")

design_conn.commit()

print("One-to-one user/profile schema created.")

```

## One-to-Many Relationships

One-to-many is extremely common.

Example:

The foreign key usually lives on the many side:

This relationship later explains why joining customers to orders can duplicate customer-level information across multiple result rows.

A one-to-many relationship is extremely common.

Examples:

The foreign key normally resides on the "many" side.

For example:

This allows many order rows to reference the same customer.

## PYTHON

```

one customer
  ↓
many orders

```

## Many-to-Many Relationships

Many-to-many relationships are normally modeled with an intermediate table.

Example:

One student can join many courses.

One course can contain many students.

The bridge table may hold additional facts such as:

Many-to-many relationships become especially important when analyzing join multiplication.

Orders and products have a many-to-many business relationship:

The relationship is represented through:

which contains foreign keys to both relations.

Conceptually:

This pattern is fundamental in relational modeling.

A many-to-many relationship means many rows from A may relate to many rows from B.

Examples:

A direct foreign key in only one table cannot represent the general many-to-many relationship correctly.

Relational databases normally model it through a junction table.

## PYTHON

```

students
  ↓
enrollments
  ↑
courses

```

## PYTHON

```

enrollment_date
grade
status

```

## SQL Is Declarative

Python is often written imperatively:

SQL is primarily declarative.

You describe the result you want:

The database query planner decides how to execute that request.

It may choose:

- table scans;
- index lookups;
- different join algorithms;
- sorting strategies;
- temporary structures.

This separation between logical request and physical execution is fundamental to SQL performance.

SQL

```
iterate
check a condition
append a result
update state
```

SQL

```
SELECT name, salary
FROM employees
WHERE salary > 80000;
```

## SQL Language Categories

SQL statements are often grouped into educational categories:

Different sources classify some commands differently, but the categories help organize responsibilities.

PYTHON

```
DDL
DML
DQL
TCL
DCL
```

## Data Definition Language

DDL means Data Definition Language.

Common DDL statements include:

They define or modify database structures.

Examples:

SQL

```
CREATE
ALTER
DROP
```

SQL

```
CREATE TABLE customers (...);
```

## Data Manipulation Language

DML commonly includes statements such as:

These modify stored records.

Some definitions include `SELECT` within DML, while educational material sometimes separates querying into DQL.

SQL

```
INSERT
UPDATE
DELETE
```

## Data Query Language

DQL is commonly used as a teaching category for:

`SELECT` becomes the foundation of analytical SQL and will receive detailed coverage in Part 2.

SQL

```
SELECT
```

## Transaction Control Language

Transaction-control concepts include:

Transactions allow multiple operations to form a reliable logical unit.

They will receive detailed coverage later.

PYTHON

```
COMMIT
ROLLBACK
SAVEPOINT
```

## Data Control Language

DCL concerns authorization and permissions.

Typical server-database commands include:

SQLite uses a different security model because it is an embedded file-based database rather than a conventional multi-user server.

PYTHON

GRANT

REVOKE

## SQL Statements and Semicolons

SQL statements commonly end with a semicolon:

The semicolon separates statements.

Interactive tools sometimes execute one statement without requiring the final semicolon, but using it is standard and improves multi-statement clarity.

SQL

```
SELECT *  
FROM customers;
```

## SQL Keyword Case

SQL keywords are commonly formatted in uppercase:

This is mainly a readability convention.

Many SQL systems treat keywords case-insensitively.

Identifier case semantics are more complicated, particularly when quoted identifiers are involved, so do not generalize keyword behavior to every database object name.

SQL

```
SELECT name  
FROM customers  
WHERE city = 'Delhi';
```

## SQL Formatting

Whitespace is generally flexible.

This can be valid:

and so can:

As queries become larger, consistent formatting substantially improves readability, reviewability, debugging, and optimization work.

SQL

```
SELECT name FROM customers WHERE city = 'Delhi';
```

SQL

```
SELECT
  name
FROM customers
WHERE city = 'Delhi';
```

## SQL Comments

Common SQL comment styles include:

and often:

Comments should explain intent or non-obvious reasoning rather than merely repeating obvious syntax.

PYTHON

```
-- single-line comment
```

## SQL Data Types

SQL databases provide data types describing intended value representations.

Common type families include:

- integer;
- exact numeric;
- approximate numeric;
- character/text;
- Boolean;
- date/time;
- binary;
- structured/vendor-specific types.

Type names and exact semantics vary substantially across database products.

## Integer Types

Many database systems provide integer types such as:

Integers are appropriate for whole-number quantities and many identifiers.

However, a numeric storage type does not automatically mean a column is a quantitative ML feature.

For example:

does not imply customer 1000 is mathematically twice customer 500.

PYTHON

SMALLINT  
INTEGER  
BIGINT

PYTHON

```
customer_id = 1000
```

## Exact Numeric Types

Many databases provide exact decimal types:

A declaration such as:

typically represents decimal precision and scale according to vendor rules.

Exact decimal semantics are important for financial and accounting data where binary floating-point approximation may be inappropriate.

PYTHON

NUMERIC  
DECIMAL

PYTHON

```
DECIMAL(10, 2)
```

## Floating-Point Types

Approximate numerical types often include:

Binary floating-point values efficiently represent a large numerical range, but many decimal fractions cannot be represented exactly.

The same principle encountered in NumPy applies:

PYTHON

REAL  
FLOAT  
DOUBLE PRECISION

## Text Types

Common textual types include:

Exact storage and length semantics vary by database.

Text processing also involves:

- encoding;
- collation;
- case sensitivity;
- locale;
- Unicode normalization.

Those concerns affect sorting, equality, searching, and indexing.

#### PYTHON

CHAR  
VARCHAR  
TEXT

## Boolean Types

Some database systems provide a native Boolean type.

Other systems historically represent Boolean-like states with integers or constrained values.

SQLite commonly uses integer `0` and `1` representations for Boolean-like expressions.

Always understand the target database rather than assuming Python Boolean behavior.

## Date and Time Types

Database systems may provide:

Temporal correctness involves more than storing a date-looking string.

Important issues include:

- time zones;
- daylight-saving transitions;
- precision;
- UTC;
- local business time;
- date-only values versus instants.

Temporal errors can create major failures in analytics and ML feature pipelines.

#### PYTHON

DATE  
TIME  
TIMESTAMP  
TIMESTAMP WITH TIME ZONE



## Binary Types

Binary types store bytes rather than text.

Names vary and can include:

Large ML model artifacts are generally better stored in artifact/object storage, while relational databases hold metadata and durable references.

The correct architecture depends on system requirements.

PYTHON

BLOB  
BYTEA  
VARBINARY

## SQLite Type Affinity

SQLite is more permissive than stricter database systems.

Its core storage classes are conceptually:

SQLite columns have type affinity, but values remain dynamically typed to a greater degree than in systems such as PostgreSQL.

This flexibility is useful for teaching and embedded applications, but it means:

does not prove:

We will identify important dialect-specific behavior where appropriate.

PYTHON

NULL  
INTEGER  
REAL  
TEXT  
BLOB

PYTHON

SQLite accepted this value

## Setting Up SQLite with Python

Python includes `sqlite3` in the standard library.

This allows us to create a complete relational learning environment without installing a database server.

The special database name:

creates an in-memory database.

It exists only for the lifetime of the connection.

PYTHON

```
import sqlite3

print("SQLite version:", sqlite3.sqlite_version)
```

PYTHON

```
connection = sqlite3.connect(":memory:")

print("Connection created:", connection)
```

## Database Connections

A connection represents an active database session.

Through it, an application can:

- execute SQL;
- obtain cursors;
- begin transactions;
- commit;
- roll back;
- configure connection behavior.

Server databases often use connection pooling because repeatedly opening network/database sessions can be relatively expensive.

## Cursors

A cursor is an interface for executing SQL and consuming results.

A simplified Python database workflow is:

PYTHON

```
cursor = connection.cursor()

print("Cursor created:", cursor)
```

## SQLite Row Objects

SQLite normally returns row-like tuples.

Using:

allows rows to support column-name-based access in addition to positional access.

We will recreate the in-memory database connection with this configuration before building the learning schema.

PYTHON

```
connection.close()

connection = sqlite3.connect(":memory:")
connection.row_factory = sqlite3.Row

cursor = connection.cursor()

print("Fresh configured connection created.")
```

PYTHON

```
connection.row_factory = sqlite3.Row
```

## Foreign-Key Enforcement in SQLite

SQLite supports foreign-key constraints, but enforcement should be enabled explicitly for the connection in common SQLite usage.

This is SQLite-specific operational behavior.

Server databases such as PostgreSQL have different foreign-key enforcement defaults and architecture.

PYTHON

```
cursor.execute("PRAGMA foreign_keys = ON;")

enabled = cursor.execute(
    "PRAGMA foreign_keys;"
).fetchone()[0]

print("Foreign keys enabled:", bool(enabled))
```

## CREATE TABLE

**CREATE TABLE** defines a table schema.

General form:

For example:

A table definition should be treated as a data contract, not merely a place to assign column names.

SQL

```
CREATE TABLE table_name (
    column_name data_type constraints,
    column_name data_type constraints
);
```

## Designing the Learning Schema

Our educational database models a small commerce system:

Relationships:

A customer can place many orders.

An order can contain multiple products.

A product can appear in many orders.

Therefore orders and products form a many-to-many business relationship resolved through `order_items`.

PYTHON

```
customers
products
orders
order_items
```

PYTHON

```
customers
  |
  | one
  |
  | many
  v
orders
  |
  | one
  |
  | many
  v
order_items
  ^
  |
  | many
  |
  | one
products
```

## Creating the Customers Table

The customer table will contain:

- a primary key;
- required name;
- required unique email;
- optional city;
- required signup date.

The city is intentionally nullable so later SQL sections can study missing-data semantics.

SQL

```
cursor.execute(
    """
    CREATE TABLE customers (
        customer_id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        city TEXT,
        signup_date TEXT NOT NULL
    );
    """
)

print("customers created.")
```

## NOT NULL

**NOT NULL** means a column cannot contain SQL NULL.

Example:

This prevents an absent name.

However, it does not automatically prevent every semantically invalid string.

Depending on constraints:

may still be valid text.

Therefore:

PYTHON

```
name TEXT NOT NULL
```

## UNIQUE

A **UNIQUE** constraint prevents duplicate non-NULL values according to the database's uniqueness semantics.

Our email column is:

so every customer must provide an email and duplicate email addresses are rejected.

NULL behavior under unique constraints can differ from ordinary equality intuition and should be checked for the target database.

PYTHON

```
email TEXT NOT NULL UNIQUE
```

## Creating the Products Table

The products table includes a nonnegative price rule.

This gives us an opportunity to use a `CHECK` constraint.

SQL

```
cursor.execute(
    """
    CREATE TABLE products (
        product_id INTEGER PRIMARY KEY,
        product_name TEXT NOT NULL,
        category TEXT NOT NULL,
        price REAL NOT NULL,
        CHECK (price >= 0)
    );
    """
)

print("products created.")
```

## CHECK Constraints

A `CHECK` constraint defines a rule that inserted or updated values must satisfy according to database semantics.

Example:

prevents negative prices in our educational schema.

Application-level validation is still useful for better user feedback and business logic, while database constraints provide an integrity boundary close to the data itself.

PYTHON

```
CHECK (price >= 0)
```

## Creating the Orders Table

Each order belongs to one customer.

The `orders.customer_id` foreign key references:

Status is restricted to a known set:

SQL

```
cursor.execute(
    """
    CREATE TABLE orders (
        order_id INTEGER PRIMARY KEY,

        customer_id INTEGER NOT NULL,

        order_date TEXT NOT NULL,

        status TEXT NOT NULL
        CHECK (
            status IN (
                'pending',
                'completed',
                'cancelled'
            )
        ),

        FOREIGN KEY (customer_id)
            REFERENCES customers(customer_id)
    );
    """
)

print("orders created.")
```

SQL

```
customers.customer_id
```

## Domain Rules with CHECK

Without a constraint, an order-status column might accumulate inconsistent values such as:

Our simplified schema defines a closed set:

For categories that evolve frequently or carry additional attributes, a separate reference table may be more maintainable.

PYTHON

```
completed
Complete
done
xyz
cancel
```

## PYTHON

```
CHECK (
    status IN (
        'pending',
        'completed',
        'cancelled'
    )
)
```

## Creating the Order Items Table

`order_items` resolves the many-to-many relationship between orders and products.

Each row means approximately:

The pair:

forms a composite primary key in this educational design.

## SQL

```
cursor.execute(
    """
    CREATE TABLE order_items (
        order_id INTEGER NOT NULL,

        product_id INTEGER NOT NULL,

        quantity INTEGER NOT NULL
        CHECK (quantity > 0),

        unit_price REAL NOT NULL
        CHECK (unit_price >= 0),

        PRIMARY KEY (
            order_id,
            product_id
        ),

        FOREIGN KEY (order_id)
            REFERENCES orders(order_id),

        FOREIGN KEY (product_id)
            REFERENCES products(product_id)
    );
    """
)

print("order_items created.")
```

## Why Store unitprice in orderitems?

The products table already stores price.



Why store another price in `order_items`?

Because these can represent different facts.

`products.price` can represent the current catalog price.

`order_items.unit_price` represents the historical price charged for a specific order.

Suppose:

A March order should not suddenly become worth 120 merely because the current product price changed.

This teaches an important normalization lesson:

#### PYTHON

```
March catalog price = 100
April catalog price = 120
```

#### PYTHON

```
values that happen to match today
are not redundant
if they represent different facts
```

## DEFAULT Values

A column can define a default.

For example:

If an insert omits the status, the database supplies the default.

A default is not the same as missingness.

It says:

Use defaults only when that rule is legitimate.

## SQL

```
cursor.execute(
    """
    CREATE TABLE demo_defaults (
        id INTEGER PRIMARY KEY,

        state TEXT NOT NULL
        DEFAULT 'new'
    );
    """
)

cursor.execute(
    """
    INSERT INTO demo_defaults (id)
    VALUES (1);
    """
)

row = cursor.execute(
    """
    SELECT *
    FROM demo_defaults;
    """
).fetchone()

print(dict(row))
```

## SQL

```
cursor.execute("DROP TABLE demo_defaults;")

print("Temporary default-value table removed.")
```

## ALTER TABLE

**ALTER TABLE** modifies an existing table structure.

Possible operations across SQL systems include:

- adding columns;
- dropping columns;
- renaming columns;
- renaming tables;
- changing types;
- adding constraints;
- removing constraints.

Support differs substantially across database systems.

Production applications commonly use database migration tools so schema changes are versioned and reproducible.

SQL

```
cursor.execute(
    """
    CREATE TABLE alter_demo (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL
    );
    """
)

cursor.execute(
    """
    ALTER TABLE alter_demo
    ADD COLUMN description TEXT;
    """
)

columns = cursor.execute(
    "PRAGMA table_info(alter_demo);"
).fetchall()

for column in columns:
    print(dict(column))
```

## DROP TABLE

**DROP TABLE** removes a table object and its stored data.

Example:

This is destructive.

It is fundamentally different from deleting selected rows while retaining the table schema.

SQL

```

cursor.execute(
    """
    CREATE TABLE temporary_table (
        id INTEGER
    );
    """
)

cursor.execute(
    """
    DROP TABLE temporary_table;
    """
)

print("Temporary table created and dropped.")

```

SQL

```
DROP TABLE temporary_data;
```

## DELETE, DROP, and TRUNCATE

These operations are conceptually different.

**DELETE** removes rows while retaining the table:

**DROP TABLE** removes the entire table object:

Database systems supporting **TRUNCATE TABLE** generally use it to remove all rows efficiently while keeping the table, but transaction, identity, locking, and foreign-key behavior is vendor-specific.

SQLite does not provide a separate **TRUNCATE TABLE** statement.

Never treat these operations as interchangeable.

SQL

```

DELETE FROM customers
WHERE customer_id = 10;

```

SQL

```
DROP TABLE customers;
```

## INSERT INTO

**INSERT** adds rows.

A clear pattern is:

Explicit target columns are generally preferable to relying on physical column order.

## SQL

```
INSERT INTO table_name (  
    column_a,  
    column_b  
)  
VALUES (  
    value_a,  
    value_b  
);
```

## Parameterized Inserts

Python values should be sent using the database driver's parameter mechanism.

With Python's SQLite driver:

The placeholder represents a value.

This is safer and more correct than constructing SQL values through string concatenation.

Parameterized SQL will receive much deeper coverage in Part 2.

## PYTHON

```
customer_rows = [  
    (  
        1,  
        "Aisha",  
        "aisha@example.com",  
        "Delhi",  
        "2026-01-05",  
    ),  
    (  
        2,  
        "Rohan",  
        "rohan@example.com",  
        "Pune",  
        "2026-01-08",  
    ),  
    (  
        3,  
        "Meera",  
        "meera@example.com",  
        "Delhi",  
        "2026-02-10",  
    ),  
    (  
        4,  
        "Kabir",  
        "kabir@example.com",  
        "Mumbai",  
        "2026-02-15",  
        # ...  
    ),  
    customer_rows,  
)  
  
print("Customers inserted:", len(customer_rows))
```

## PYTHON

```
cursor.execute(  
    "INSERT INTO users (name) VALUES (?);",  
    ("Aisha",)  
)
```

**executemany**

`executemany()` applies one parameterized statement to multiple parameter sequences.

This is cleaner than manually constructing separate SQL text for every record.

For very large ingestion workloads, database-native bulk-loading utilities may be much faster than ordinary row-oriented insertion.

## SQL

```

product_rows = [
    (101, "Laptop", "Electronics", 75000.0),
    (102, "Mouse", "Electronics", 1200.0),
    (103, "Keyboard", "Electronics", 3000.0),
    (104, "Desk", "Furniture", 15000.0),
    (105, "Chair", "Furniture", 9000.0),
    (106, "Notebook", "Stationery", 150.0),
]

cursor.executemany(
    """
    INSERT INTO products (
        product_id,
        product_name,
        category,
        price
    )
    VALUES (?, ?, ?, ?);
    """,
    product_rows,
)

print("Products inserted:", len(product_rows))

```

## SQL

```

order_rows = [
    (1001, 1, "2026-03-01", "completed"),
    (1002, 2, "2026-03-02", "completed"),
    (1003, 1, "2026-03-05", "pending"),
    (1004, 3, "2026-03-06", "completed"),
    (1005, 4, "2026-03-08", "cancelled"),
    (1006, 2, "2026-03-09", "completed"),
]

cursor.executemany(
    """
    INSERT INTO orders (
        order_id,
        customer_id,
        order_date,
        status
    )
    VALUES (?, ?, ?, ?);
    """,
    order_rows,
)

print("Orders inserted:", len(order_rows))

```

## COMMIT

Database changes generally participate in transaction semantics.

`commit()` makes the current transaction's changes permanent according to the database's guarantees.

Transactions will receive dedicated coverage later.

For now, commit the learning data so later examples begin from a stable state.

`COMMIT` makes a successful transaction's changes durable according to the guarantees provided by the database.

Before commit, changes belong to the active transaction. Other transactions may or may not be able to observe them depending on the database and isolation model.

After commit, the database treats the transaction as successfully completed.

Transaction boundaries should correspond to meaningful units of work. Committing after every tiny statement may destroy the atomicity of a multi-step operation, while keeping transactions open unnecessarily long can increase contention and locking.

PYTHON

```
connection.commit()

print("Learning data committed.")
```

## Testing Constraint Enforcement

A constraint is useful only if invalid states are rejected.

We can deliberately attempt invalid inserts and catch the expected database errors.

The failed statements do not become part of our committed learning dataset.

## Testing UNIQUE

The email:

already belongs to customer 1.

A second customer using the same email should violate the uniqueness rule.



SQL

```
try:
    cursor.execute(
        """
        INSERT INTO customers (
            customer_id,
            name,
            email,
            city,
            signup_date
        )
        VALUES (?, ?, ?, ?, ?);
        """,
        (
            99,
            "Duplicate Email",
            "aisha@example.com",
            "Delhi",
            "2026-03-10",
        ),
    )

except sqlite3.IntegrityError as exc:
    print("IntegrityError:", exc)
```

SQL

```
aisha@example.com
```

## Testing CHECK

A negative product price violates:

## SQL

```
try:
    cursor.execute(
        """
        INSERT INTO products (
            product_id,
            product_name,
            category,
            price
        )
        VALUES (?, ?, ?, ?);
        """,
        (
            999,
            "Invalid Product",
            "Demo",
            -100.0,
        ),
    )

except sqlite3.IntegrityError as exc:
    print("IntegrityError:", exc)
```

## SQL

```
CHECK (price >= 0)
```

## Testing a Foreign Key

Customer 999999 does not exist.

An order referencing that customer violates referential integrity.

## SQL

```

try:
    cursor.execute(
        """
        INSERT INTO orders (
            order_id,
            customer_id,
            order_date,
            status
        )
        VALUES (?, ?, ?, ?);
        """,
        (
            9999,
            999999,
            "2026-03-10",
            "pending",
        ),
    )

except sqlite3.IntegrityError as exc:
    print("IntegrityError:", exc)

```

## Testing NOT NULL

A required customer name cannot be SQL NULL.

## SQL

```

try:
    cursor.execute(
        """
        INSERT INTO customers (
            customer_id,
            name,
            email,
            city,
            signup_date
        )
        VALUES (?, ?, ?, ?, ?);
        """,
        (
            100,
            None,
            "missing-name@example.com",
            "Delhi",
            "2026-03-10",
        ),
    )

except sqlite3.IntegrityError as exc:
    print("IntegrityError:", exc)

```

## Database Constraints and Application Validation

Should validation happen in Python or in the database?

Both layers can have legitimate responsibilities.

Application validation provides:

- fast feedback;
- user-friendly messages;
- complex business logic.

Database constraints provide:

- centralized integrity rules;
- protection across multiple applications;
- protection against accidental direct writes;
- rules close to persistent data.

Fundamental invariants should not depend solely on every application remembering to enforce them correctly.

## Inspecting Database Metadata

SQLite stores schema metadata internally.

The `sqlite_master` table can expose information about objects such as:

- tables;
- indexes;
- views.

Other database systems provide system catalogs with different structures.

## SQL

```
schema_objects = cursor.execute(
    """
    SELECT
        type,
        name,
        tbl_name
    FROM sqlite_master
    WHERE type IN (
        'table',
        'index',
        'view'
    )
    ORDER BY type, name;
    """
).fetchall()

for row in schema_objects:
    print(dict(row))
```

## Inspecting Table Columns

SQLite provides:

to inspect table columns.

**PRAGMA** commands are SQLite-specific and should not be confused with portable SQL syntax.

## PYTHON

```
columns = cursor.execute(
    "PRAGMA table_info(customers);")
).fetchall()

for column in columns:
    print(dict(column))
```

## Inspecting Foreign-Key Metadata

SQLite provides:

for inspecting declared foreign-key relationships.

## PYTHON

```
foreign_keys = cursor.execute(
    "PRAGMA foreign_key_list(order_items);")
).fetchall()

for foreign_key in foreign_keys:
    print(dict(foreign_key))
```

## PYTHON

```
PRAGMA foreign_key_list(table_name);
```

## First SELECT

Now that the database contains data, we can retrieve rows.

The query:

asks for all selected columns from the customer relation.

Part 2 will develop `SELECT` comprehensively.

## SQL

```
rows = cursor.execute(
    """
    SELECT *
    FROM customers;
    """)
.fetchall()

for row in rows:
    print(dict(row))
```

## SQL

```
SELECT *
FROM customers;
```

## Selecting Specific Columns

Rather than retrieving every column, a query can explicitly request only those needed:

Explicit projection is often preferable for stable production interfaces.

## SQL

```
rows = cursor.execute(
    """
    SELECT
        customer_id,
        name,
        city
    FROM customers;
    """)
.fetchall()

for row in rows:
    print(dict(row))
```

## Displaying SQL Results with Pandas

SQL does not depend on Pandas.

However, Pandas is useful for displaying database query results in a Jupyter notebook.

The database still executes the SQL. Pandas simply converts the result into a DataFrame.

SQL

```
import pandas as pd

def query(sql, params=None):
    if params is None:
        params = ()

    return pd.read_sql_query(
        sql,
        connection,
        params=params,
    )

print("Pandas version:", pd.__version__)
```

SQL

```
query(
    """
    SELECT
        customer_id,
        name,
        city
    FROM customers;
    """
)
```

## Learning Database Relationship Map

Our current schema is:

This schema provides enough structure for increasingly advanced querying later in the notebook.

## PYTHON

```

customers
-----
customer_id PK
name
email
city
signup_date
|
| 1
|
| many
v
orders
-----
order_id PK
customer_id FK
order_date
status
|
| 1
|
| many
v
order_items
-----
order_id PK/FK
product_id PK/FK
quantity
# ...
product_id PK
product_name
category
price

```

**Table Grain**

Grain describes what one row represents.

Our tables have different grains:

This is one of the most important concepts in analytical SQL.

Joining tables can change grain.

If customer Aisha has two orders, joining customers to orders can turn:

into:

Many SQL errors are actually grain errors.



## PYTHON

```
customers
→ one row per customer

products
→ one row per product

orders
→ one row per order

order_items
→ one row per order-product pair
```

## Grain and Machine Learning

Suppose a customer-level model requires:

You join customers to orders and then order items.

The intermediate data may become:

If that result is fed directly into a customer-level model:

- frequent customers appear more times;
- target distribution may change;
- train/test splitting can become invalid;
- metrics can be biased;
- customer-level features can be duplicated.

Before constructing a feature query, explicitly define:

## PYTHON

```
one row per customer
```

## PYTHON

```
one row per customer-order-product combination
```

## SQL and Pandas

SQL and Pandas overlap conceptually:

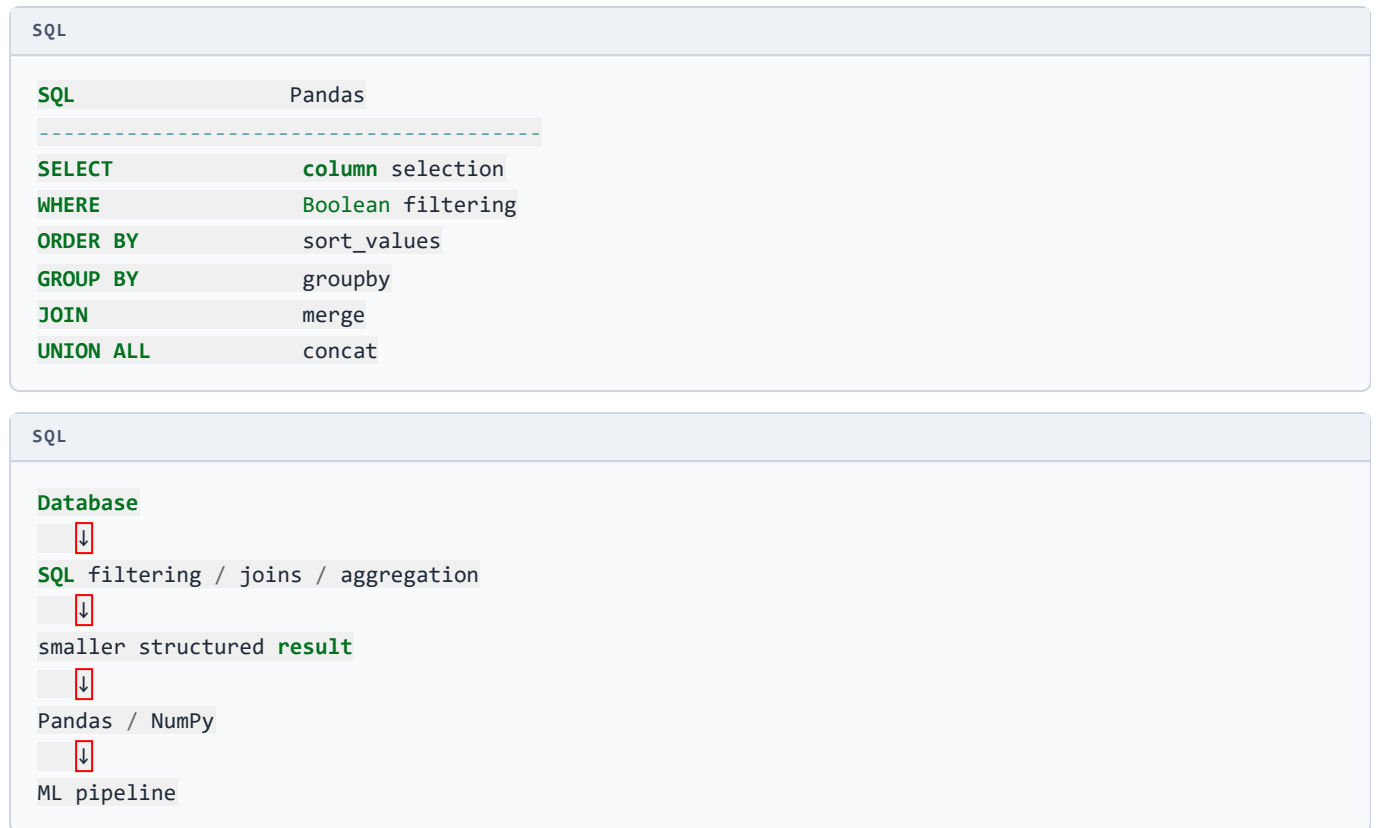
But they execute in different environments.

SQL often runs close to persistent data inside a database engine.

Pandas normally works with in-process DataFrames.

A common AI workflow is:

Good engineering uses each tool according to the workload.



## OLTP and OLAP

Database workloads are often discussed using two broad categories.

OLTP — Online Transaction Processing:

- creating orders;
- recording payments;
- updating accounts;
- retrieving one user.

These workloads often involve many small concurrent operations.

OLAP — Online Analytical Processing:

- aggregating millions of events;
- computing monthly revenue;
- analyzing customer segments;
- creating training datasets;
- building reports.

Modern architectures can blur the boundaries, but the distinction helps explain why database systems and schemas may be optimized differently.

## Row-Oriented and Column-Oriented Storage

Traditional transactional systems commonly use row-oriented storage because applications frequently retrieve several attributes from a small number of records.

Analytical systems often use column-oriented storage because analytical queries frequently scan selected columns across many rows.

Conceptually:

versus:

Columnar layouts can improve analytical scanning and compression.

This physical-storage topic should not be confused with the simple fact that a Pandas DataFrame also has columns.

Transactional relational databases commonly organize data in row-oriented forms.

Conceptually:

Analytics often reads only a few columns across huge numbers of rows.

Column-oriented storage instead groups values by column conceptually:

This can improve:

- compression
- analytical scanning
- vectorized processing

Storage layout is one reason OLTP and OLAP systems make different performance trade-offs.

PYTHON

Row storage

```
row 1 → id, city, age, income
row 2 → id, city, age, income
```

## Relational and Non-Relational Databases

Not every database uses the relational model.

Other categories include:

- document databases;
- key-value stores;
- graph databases;
- wide-column stores;
- specialized time-series systems;

- other domain-specific stores.

This course focuses here on relational SQL because SQL remains fundamental across application engineering, analytics, data engineering, and AI systems.

Different database models solve different problems.

### Common Early Database Mistakes

Several misconceptions should be corrected before advanced querying begins.

A database table is not merely a CSV stored somewhere else.

A primary key is not simply "the first column."

A foreign key is not merely a column containing similar-looking numbers.

A numeric identifier is not automatically a quantitative ML feature.

`NOT NULL` does not guarantee semantic validity.

A successful insert does not prove business correctness.

A query that executes successfully can still produce an analytically incorrect result.

Relational modeling, constraints, grain, and semantics matter just as much as SQL syntax.

### Hinglish Intuition

Database ko sirf ek file mat samjho.

CSV mein tum rows store kar sakte ho, but CSV automatically enforce nahi karta:

ya:

Database constraints ye rules enforce kar sakte hain.

Suppose:

and:

`orders.customer_id = 1` sirf number `1` nahi hai.

Uska semantic meaning hai:

Aur grain abhi se strongly yaad rakho.

`customers` mein:

`orders` mein:

Join karne ke baad row ka meaning change ho sakta hai.

SQL mein syntax correct hona aur analysis logically correct hona alag cheezein hain.

**WHERE** ko simple filter samajh sakte ho, lekin SQL mein NULL ki wajah se ek third logical state aati hai:

Suppose:

Iska meaning city **"NULL"** string nahi hai.

Meaning hai:

Ab:

true nahi bola ja sakta.

False bhi confidently nahi bola ja sakta, because unknown city Delhi ho sakti hai.

Isliye result **UNKNOWN** hota hai.

Aur **WHERE** sirf TRUE rows retain karta hai.

Isi wajah se:

NULL cities ko automatically include nahi karta.

Doosri important cheez:

exactly three rows return karta hai. Ye automatically top three ranks with all ties nahi hota.

SQL mein query chalna enough nahi hai. Exact semantics samajhna zaroori hai.

Aggregation ka sabse important concept function ka naam nahi hai. Sabse important concept hai grain.

Suppose:

Yahan one row ka meaning hai:

Agar tum directly:

karte ho, tum average line value calculate kar rahe ho.

Agar question hai:

to pehle:

karke:

banana hoga.

Uske baad order totals ka average calculate hoga.

So:

**WHERE** aur **HAVING** ka difference bhi important hai:

Har analytical SQL stage mein ye question poochho:

Join ko sirf:

mat samjho.

Join ka main question hai:

Suppose:

ke:

hain.

Customer ko orders se join karoge to customer 1 ki information 3 result rows mein appear karegi.

Ye accidental duplicate nahi hai.

Result ka grain change hua:

Ab agar each order mein multiple products hain aur `order_items` join kar diya:

Isi liye join ke baad blindly `SUM`, `COUNT`, ya ML training nahi karni chahiye.

Aur LEFT JOIN ka ek critical rule:

customers ko preserve kar sakta hai.

Lekin:

un customers ko remove kar sakta hai jinke matching orders nahi hain.

Join SQL ka syntax easy hai. Grain aur cardinality ka reasoning real skill hai.

Subquery ko:

samajhna starting point hai, but better mental model hai:

Example:

Inner query ek number nikalti hai:

Outer query har product ko us benchmark se compare karti hai.

CTE same idea ko readable stages mein organize karta hai:

Aur har stage par grain clear hona chahiye.

`EXISTS` ka intuition:

`UNION ALL` ka intuition:

`UNION` :

Aur ML ke liye important point: CTE query ko clean bana sakta hai, but clean query automatically leakage-free nahi hoti. Time boundary phir bhi manually correct define karni padegi.

`GROUP BY` aur window function ka difference strongly samjho.

Suppose products:

sab Electronics category mein hain.

`GROUP BY category` karoge to:

Individual products disappear ho jaate hain.

Lekin window:

har product row ko preserve karta hai aur uske saath category average attach karta hai.

Matlab:

`PARTITION BY` ko bolo:

Window `ORDER BY` ko bolo:

Frame ko bolo:

ML mein sabse dangerous part future-looking window hai.

Agar prediction time par future available nahi tha aur feature frame future rows use kar raha hai, to SQL correct hone ke bawajood model feature leaked hai.

Transaction ko ek sealed package ki tarah socho.

Agar tum bank transfer mein pehle account A se ₹100 minus kar do aur uske baad program crash ho jaye, lekin B mein ₹100 add na hua ho, to database logically corrupt ho gaya.

Transaction bolta hai:

`COMMIT` ka matlab hai package successfully finalize karna.

`ROLLBACK` ka matlab hai uncommitted package ko cancel karke previous valid state par lautna.

Isolation ka intuition alag hai. Agar do log ek hi time par database use kar rahe hain, to database ko decide karna padta hai ki unke operations kis tarah ek doosre ko visible honge aur conflicting writes ko kaise coordinate kiya jayega.

Isliye transaction sirf `BEGIN` aur `COMMIT` syntax nahi hai. Ye application ke logical operation aur database ke permanent state ke beech correctness boundary hai.

Index ko dictionary ke alphabetical guide ki tarah socho.

Agar tumhe ek 1000-page book mein "gradient descent" dhoondhna hai aur koi index nahi hai, tum theoretically page 1 se page 1000 tak check kar sakte ho.

Database table scan bhi similar idea hai:

Index hone par database ke paas pehle se organized shortcut hota hai.

Lekin shortcut maintain karna bhi padta hai.

Agar table mein new row insert hui, database ko:

dono karne pad sakte hain.

Isliye index read ko fast kar sakta hai, but writes aur storage free nahi hote.

Aur har column par index banana smart optimization nahi hai. Index tab useful hai jab actual query pattern us data structure ka benefit le sakta hai.

Normalization ka main idea sirf "tables ko todna" nahi hai.

Imagine customer ka email 500 order rows mein copied hai.

Ab customer email change karta hai.

Agar tumhe 500 rows update karni pad rahi hain, database ek hi fact ko 500 jagah store kar raha hai.

Problem ye hai:

Normalized design mein current customer email `customers` table mein ek authoritative location par rahega, aur orders sirf `customer_id` reference karenge.

Lekin har repetition galat nahi hoti.

Historical invoice par purchase-time product price intentionally preserve karna useful ho sakta hai.

Question ye hona chahiye:

Good schema design dependency aur meaning ko model karta hai, blindly maximum number of tables nahi banata.

Connection pool ko parking lot ki tarah socho.

Suppose database ke liye application ne 20 parking spots rakhe hain.

Har request ek connection borrow karti hai.

Agar request connection lekar 30 seconds tak LLM API ka response wait karti rahe, connection unnecessarily occupied hai.

20 requests aise kar dein to saare spots full.

21st request ke paas query simple hone ke bawajood connection hi available nahi hoga.

Problem SQL query ki speed nahi bhi ho sakti.

Problem ho sakta hai:

Isliye production database engineering mein query ke saath-saath ye bhi important hai ki connection kab liya, kitni der rakha, transaction kab commit hua, aur connection pool mein kab return hua.



Retention query mein sabse dangerous mistake SQL syntax nahi, denominator aur grain ka confusion hota hai.

Suppose January mein 100 users signup hue.

February mein unmein se 40 active the.

A simple month-1 retention ho sakta hai:

Ab agar those 40 users ne total 500 events kiye, numerator 500 nahi banega.

Why?

Kyuki metric hai:

not:

Isliye intermediate grain ko consciously define karo:

Advanced analytics mein query ka result tabhi trustworthy hai jab tum har intermediate table ka meaning ek sentence mein explain kar sako.

PYTHON

```
customer_id unique hona chahiye
```

PYTHON

```
order ka customer actually exist karna chahiye
```

## Database Foundations Summary

Part 1 established the database model beneath SQL syntax.

The major concepts covered were:

- persistent storage;
- DBMS;
- relational databases;
- relations and SQL tables;
- rows and columns;
- tuples and attributes;
- schemas;
- metadata;
- domains;
- degree and cardinality;
- superkeys;
- candidate keys;

- primary keys;
- composite keys;
- natural keys;
- surrogate keys;
- alternate keys;
- foreign keys;
- referential integrity;
- one-to-one relationships;
- one-to-many relationships;
- many-to-many relationships;
- declarative SQL;
- DDL;
- DML;
- DQL;
- TCL;
- DCL;
- SQL formatting and comments;
- integer types;
- exact numeric types;
- floating-point types;
- text types;
- Boolean types;
- temporal types;
- binary types;
- SQLite type affinity;
- SQLite connections;
- cursors;
- foreign-key enforcement;
- `CREATE TABLE` ;
- `PRIMARY KEY` ;
- `FOREIGN KEY` ;
- `NOT NULL` ;
- `UNIQUE` ;
- `CHECK` ;
- `DEFAULT` ;

- `ALTER TABLE` ;
- `DROP TABLE` ;
- `INSERT` ;
- parameterized insertion;
- `executemany` ;
- commits;
- constraint enforcement;
- metadata inspection;
- introductory `SELECT` ;
- table grain;
- SQL versus Pandas;
- OLTP versus OLAP;
- row versus column storage;
- AI implications of grain.

Part 2 develops the SQL querying language in depth, including `SELECT` , expressions, filtering, NULL, three-valued logic, membership, ranges, pattern matching, sorting, pagination, scalar functions, conditional expressions, parameterized queries, and AI-oriented filtering implications.

## Querying Data with `SELECT`

Part 1 established the relational database model, schemas, keys, constraints, data types, SQLite setup, table creation, and insertion.

We can now focus on the language used to retrieve and transform data.

`SELECT` is the foundation of SQL querying, but serious SQL querying is much more than:

A query can:

- choose columns;
- calculate expressions;
- rename outputs;
- filter records;
- combine logical conditions;
- handle missing information;
- perform membership tests;
- match textual patterns;
- eliminate duplicate result combinations;
- sort rows;
- restrict result size;

- paginate;
- convert values;
- construct derived categories;
- calculate temporal expressions;
- safely incorporate Python values.

Throughout this section, repeatedly ask:

These semantic questions are more important than memorizing isolated keywords.

SQL

```
SELECT *  
FROM table;
```

SQL

What does one **result row** represent?

Which **input rows** qualify?

Which expressions appear **in** the **output**?

How does **NULL** affect this condition?

**Is** the **output order** guaranteed?

**Are** repeated **rows** expected?

Could this filtering decision change the ML population?

## Reproducible Notebook State

Part 1 used an in-memory SQLite database.

An in-memory database disappears when its connection or Python process ends. If someone opens this notebook later and starts execution from this section after restarting the kernel, hidden state from Part 1 may no longer exist.

For Part 2 we therefore construct a fresh in-memory learning database.

This is deliberate notebook engineering:

The schema remains conceptually the same, but the example data is slightly expanded so that filtering, NULL behavior, pattern matching, sorting, and later aggregation examples are richer.

## PYTHON

```
import sqlite3
import pandas as pd
from IPython.display import display

print("SQLite version:", sqlite3.sqlite_version)
print("Pandas version:", pd.__version__)

connection = sqlite3.connect(":memory:")
connection.row_factory = sqlite3.Row

cursor = connection.cursor()

cursor.execute("PRAGMA foreign_keys = ON;")

enabled = cursor.execute(
    "PRAGMA foreign_keys;"
).fetchone()[0]

print("Foreign-key enforcement:", bool(enabled))
```

## Query Display Helper

SQL itself does not require Pandas.

The database engine executes the SQL.

Pandas simply makes tabular results convenient to display inside Jupyter.

The helper below also supports bound SQL parameters.

## SQL

```
def query(sql, params=None):
    if params is None:
        params = ()

    return pd.read_sql_query(
        sql,
        connection,
        params=params,
    )

display(
    query(
        """
        SELECT
            (SELECT COUNT(*) FROM customers)
            AS customers,

            (SELECT COUNT(*) FROM products)
            AS products,

            (SELECT COUNT(*) FROM orders)
            AS orders,

            (SELECT COUNT(*) FROM order_items)
            AS order_items;
        """
    )
)
```

## The SELECT Statement

A basic SQL retrieval statement has the form:

**FROM** establishes source rows.

**SELECT** defines the expressions appearing in the result.

For example:

returns the **name** expression for each source row.

It is useful to stop thinking of SELECT as merely "printing columns." Later, selected expressions can include: and window functions.

The SELECT list defines the result attributes.

SQL

```
query(  
  ""  
  SELECT name  
  FROM customers;  
  ""  
)
```

SQL

```
SELECT expression  
FROM source;
```

## Projection

Selecting particular attributes is related to relational projection.

Suppose `customers` contains:

but a task requires only:

SQL can return exactly those fields.

Explicit projection:

- documents the result schema;
- reduces unnecessary transfer;
- avoids exposing irrelevant fields;
- can reduce downstream memory;
- makes dependencies clearer;
- can enable efficient plans in some workloads.

A useful engineering principle is:

SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    name  
  FROM customers;  
  ""  
)
```

## SELECT \*

The wildcard:

requests every column from the source.

This is useful for exploration and debugging.

However, production queries often benefit from explicit fields.

If a table later gains a large or sensitive column, a wildcard query may start retrieving it even though the original consumer never needed it.

`SELECT *` is therefore not inherently wrong. It simply establishes a broad data dependency.

SQL

```
query(  
  ""  
  SELECT *  
  FROM customers;  
  ""  
)
```

SQL

```
SELECT *  
FROM customers;
```

## SELECT Expressions

A selected item does not have to be a stored column.

SQL can calculate an expression for each qualifying row.

For example:

produces a derived value.

Selecting the expression does not modify the underlying stored price.

SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
    price * 1.18 AS price_with_tax  
  FROM products;  
  ""  
)
```

SQL

```
price * 1.18
```

## Arithmetic Expressions

Common arithmetic operators include:



Exact type behavior depends on the database and operands.

SQL arithmetic is frequently used for:

- revenue;
- discounts;
- ratios;
- quantities;
- derived features;
- financial metrics.

A syntactically valid calculation is not automatically a meaningful metric. The underlying units and grain still matter.

SQL

```
query(  
  """  
  SELECT  
    product_name,  
    price,  
    price * 0.10 AS discount_amount,  
    price - (price * 0.10)  
      AS discounted_price  
  FROM products;  
  """  
)
```

## Column Aliases

Calculated fields need meaningful output names.

**AS** assigns an alias:

Aliases improve:

- readability;
- DataFrame output;
- feature naming;
- reports;
- complex expressions.

They rename only the query result field, not the stored database column.

SQL

```
query(  
    ""  
    SELECT  
        product_name,  
        price AS base_price,  
        ROUND(price * 1.18, 2)  
        AS price_with_tax  
    FROM products;  
    ""  
)
```

SQL

```
price * 1.18 AS price_with_tax
```

## Optional AS

Many databases allow:

instead of:

Using `AS` is often clearer.

Table alias syntax can have additional dialect differences and will matter more when we reach joins.

PYTHON

```
price total_price
```

PYTHON

```
price AS total_price
```

## Literal Expressions

SQL can return constants as selected expressions.

For example:

Every qualifying row receives the literal value `"customer"`.

Literals are useful for:

- source labels;
- constant flags;
- synthetic metadata;
- set operations;
- standardized output schemas.

SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    name,  
    'customer' AS record_type,  
    1 AS example_flag  
  FROM customers;  
  ""  
)
```

## SELECT Without FROM

SQLite and PostgreSQL can evaluate expressions without a table source:

This is useful for experimenting with SQL expressions.

Other database systems may have different conventions.

SQL

```
query(  
  ""  
  SELECT  
    2 + 3 AS addition,  
    10 * 5 AS multiplication,  
    ROUND(10.0 / 3.0, 3)  
    AS division;  
  ""  
)
```

SQL

```
SELECT 2 + 3;
```

## String Concatenation

SQLite and PostgreSQL support:

for concatenation.

For example:

MySQL commonly uses `CONCAT()`.

This is an early example of SQL dialect differences.

SQL

```
query(  
    ""  
    SELECT  
        customer_id,  
        'Customer: ' || name  
        AS display_name  
    FROM customers;  
    ""  
)
```

SQL

```
||
```

## WHERE

**WHERE** filters source rows.

Example:

Conceptually:

Later, this will matter because **WHERE** operates before grouped aggregation.

SQL

```
query(  
    ""  
    SELECT  
        product_name,  
        category,  
        price  
    FROM products  
    WHERE price > 10000;  
    ""  
)
```

## Comparison Operators

Common comparisons include:

SQL equality uses:

not Python's:

## SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    customer_id,  
    status  
  FROM orders  
  WHERE status = 'completed'  
  ORDER BY order_id;  
  ""  
)
```

## SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price  
  FROM products  
  WHERE price <= 3000  
  ORDER BY price;  
  ""  
)
```

**AND**

**AND** requires both conditions to qualify as true.

Example:

## SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    category,  
    price  
  FROM products  
  WHERE  
    category = 'Electronics'  
    AND price < 10000  
  ORDER BY price;  
  ""  
)
```

## SQL

```
category = 'Electronics'  
AND price < 10000
```

## OR

**OR** requires at least one component condition to qualify as true.

Example:

SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    status  
  FROM orders  
  WHERE  
    status = 'pending'  
    OR status = 'cancelled'  
  ORDER BY order_id;  
  ""  
)
```

## NOT

**NOT** negates a logical condition.

For simple non-NULL data:

behaves as expected.

However, SQL NULL introduces an **UNKNOWN** logical state, and:

remains:

This becomes important shortly.

SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    status  
  FROM orders  
  WHERE NOT status = 'completed'  
  ORDER BY order_id;  
  ""  
)
```

SQL

```
NOT status = 'completed'
```

## Logical Operator Precedence

`AND` normally binds more strongly than `OR`.

Consider:

It behaves conceptually like:

not:

Use parentheses when they make intent clearer.

SQL

```
print("Without grouped category logic:")

display(
    query(
        """
        SELECT
            product_name,
            category,
            price
        FROM products
        WHERE
            category = 'Electronics'
            OR category = 'Furniture'
            AND price > 10000
        ORDER BY product_id;
        """
    )
)

print("\nWith explicit grouping:")

display(
    query(
        """
        SELECT
            product_name,
            category,
            price
        # ...
        ORDER BY product_id;
        """
    )
)
```

SQL

```
WHERE
    category = 'Electronics'
    OR category = 'Furniture'
    AND price > 10000
```

## NULL

NULL represents the absence of a known value.

Depending on the data model, it may represent:

- unknown;
- unavailable;
- uncollected;
- not applicable;
- not yet determined.

Those business meanings are not necessarily equivalent.

NULL is not automatically equivalent to:

Those are actual values.

SQL gives NULL special semantics.

SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    name,  
    city  
  FROM customers  
  WHERE city IS NULL;  
  ""  
)
```

### Why = NULL Does Not Work

A common mistake is:

This does not test missingness correctly.

Use:

or:

NULL is not an ordinary value participating in normal equality semantics.



SQL

```
print("Using = NULL:")

display(
  query(
    """
    SELECT
      customer_id,
      name,
      city
    FROM customers
    WHERE city = NULL;
    """
  )
)

print("\nUsing IS NULL:")

display(
  query(
    """
    SELECT
      customer_id,
      name,
      city
    FROM customers
    WHERE city IS NULL;
    """
  )
)
```

SQL

```
WHERE city = NULL
```

## Three-Valued Logic

Ordinary Boolean logic contains:

SQL adds:

because some comparisons cannot be determined when information is missing.

Suppose a city is NULL.

Evaluate:

SQL cannot conclude true.

But it also cannot conclude false, because the unknown city could actually be Delhi.

The result is `UNKNOWN`.

A `WHERE` clause keeps true rows. False and unknown rows are filtered out.

This produces one of SQL's most important rules:

does not automatically include rows where city is NULL.

An unknown city is not known to be different from Delhi.

SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    name,  
    city,  
    city = 'Delhi'  
      AS equals_delhi,  
    city <> 'Delhi'  
      AS not_equal_delhi  
  FROM customers  
  ORDER BY customer_id;  
  ""  
)
```

SQL

```
TRUE  
FALSE
```

## Explicit Missing-Value Policies

Suppose the requirement is:

"Return customers whose known city is not Delhi, and also include customers whose city is missing."

The query must express both cases:

SQL makes missing-data policy explicit.

## SQL

```
query(  
    ""  
    SELECT  
        customer_id,  
        name,  
        city  
    FROM customers  
    WHERE  
        city <> 'Delhi'  
        OR city IS NULL  
    ORDER BY customer_id;  
    ""  
)
```

**UNKNOWN with AND, OR, and NOT**

Useful three-valued logic relationships include:

This behavior follows from what can and cannot be concluded despite missing information.

It becomes especially important when nullable fields participate in complex filters.

## PYTHON

|                   |   |         |
|-------------------|---|---------|
| TRUE AND UNKNOWN  | → | UNKNOWN |
| FALSE AND UNKNOWN | → | FALSE   |
|                   |   |         |
| TRUE OR UNKNOWN   | → | TRUE    |
| FALSE OR UNKNOWN  | → | UNKNOWN |
|                   |   |         |
| NOT UNKNOWN       | → | UNKNOWN |

**IS NOT NULL**

**IS NOT NULL** keeps rows with known non-NULL values.

This is common in data completeness analysis.

However, removing missing records changes the population. For ML, that can create selection bias if missingness is systematic.

## SQL

```
query(  
    """  
    SELECT  
        customer_id,  
        name,  
        city  
    FROM customers  
    WHERE city IS NOT NULL  
    ORDER BY customer_id;  
    """  
)
```

## NULL-Safe Comparison Awareness

SQL dialects provide different NULL-safe comparison mechanisms.

PostgreSQL and standards-oriented SQL support constructs such as:

SQLite has `IS` and `IS NOT` semantics of its own.

For portable foundational missingness checks, use:

and consult the target database when NULL-safe value comparison is needed.

## PYTHON

```
IS DISTINCT FROM  
IS NOT DISTINCT FROM
```

## BETWEEN

`BETWEEN` defines an inclusive range.

For example:

is conceptually equivalent to:

The endpoints are included.

## SQL

```
query(  
    """  
    SELECT  
        product_name,  
        price  
    FROM products  
    WHERE price BETWEEN 1000 AND 10000  
    ORDER BY price;  
    """  
)
```

SQL

```
price BETWEEN 1000 AND 10000
```

## NOT BETWEEN

**NOT BETWEEN** selects values outside the inclusive range:

NULL still participates according to three-valued logic.

SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price  
  FROM products  
  WHERE price NOT BETWEEN 1000 AND 10000  
  ORDER BY price;  
  ""  
)
```

SQL

```
price NOT BETWEEN 1000 AND 10000
```

## IN

**IN** tests membership.

Instead of:

we can write:

This often expresses membership more clearly.

SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    status  
  FROM orders  
  WHERE status IN (  
    'pending',  
    'cancelled'  
  )  
  ORDER BY order_id;  
  ""  
)
```

## NOT IN

For a set containing no NULL:

asks for values matching neither member.

However, NULL inside `NOT IN` creates a major SQL trap.

SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    status  
  FROM orders  
  WHERE status NOT IN (  
    'pending',  
    'cancelled'  
  )  
  ORDER BY order_id;  
  ""  
)
```

SQL

```
status NOT IN (  
  'pending',  
  'cancelled'  
)
```

### The NOT IN and NULL Trap

Consider:

A useful conceptual approximation is:

For value 3:

becomes:

and does not survive `WHERE`.

This is especially dangerous when `NOT IN` receives values from a subquery and that subquery unexpectedly returns NULL.

Later we will learn why `NOT EXISTS` is often a better anti-match pattern.

Now compare the same query without NULL.

## SQL

```

query(
  """
  WITH numbers(value) AS (
    VALUES
      (1),
      (2),
      (3),
      (4)
  )

  SELECT value
  FROM numbers
  WHERE value NOT IN (
    1,
    2,
    NULL
  );
  """
)

```

## SQL

```

query(
  """
  WITH numbers(value) AS (
    VALUES
      (1),
      (2),
      (3),
      (4)
  )

  SELECT value
  FROM numbers
  WHERE value NOT IN (
    1,
    2
  );
  """
)

```

**LIKE**

**LIKE** performs SQL pattern matching.

Its fundamental wildcards are:

For example:

means the value begins with **A**.

**LIKE** is not a full regular-expression language.

SQL

```

query(
    """
    SELECT
        customer_id,
        name
    FROM customers
    WHERE name LIKE 'A%'
    ORDER BY customer_id;
    """
)

```

## Percent Wildcard

Patterns include:

Case behavior depends on:

- database;
- collation;
- configuration;
- character set.

SQLite's common ASCII `LIKE` behavior should not be generalized to every SQL platform or every Unicode string.

SQL

```

query(
    """
    SELECT
        product_id,
        product_name
    FROM products
    WHERE product_name LIKE '%o%'
    ORDER BY product_id;
    """
)

```

SQL

```

'A%' starts with A
'%a' ends with a
'%book%' contains book

```

## Underscore Wildcard

`_` matches exactly one character.

For example:



matches `Rohan`.

SQL

```
query(
  """
  SELECT
    customer_id,
    name
  FROM customers
  WHERE name LIKE '_ohan';
  """
)
```

SQL

```
name LIKE '_ohan'
```

## Escaping LIKE Wildcards

Sometimes `%` or `_` should be treated as a literal character.

A query can specify an escape character.

For example:

Here `!%` means a literal percent sign.

SQL

```
query(
  """
  WITH examples(text_value) AS (
    VALUES
      ('discount 10%'),
      ('discount 20'),
      ('100_percent')
  )

  SELECT text_value
  FROM examples
  WHERE text_value
        LIKE '%!%%' ESCAPE '!';
  """
)
```

## SQLite GLOB

SQLite also provides `GLOB`, which uses shell-like wildcards:

`GLOB` is SQLite-specific knowledge.

For general SQL foundations, `LIKE` is more portable.

SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    name  
  FROM customers  
  WHERE name GLOB 'A*'  
  ORDER BY customer_id;  
  ""  
)
```

SQL

```
* zero or more characters  
? one character
```

## DISTINCT

Query results can contain repeated combinations.

**DISTINCT** eliminates duplicate result rows from the selected expressions.

For example:

returns each distinct city representation once.

SQL

```
query(  
  ""  
  SELECT DISTINCT city  
  FROM customers;  
  ""  
)
```

SQL

```
SELECT DISTINCT city  
FROM customers;
```

## DISTINCT Applies to the Complete Selected Combination

Consider:

The pair:

is deduplicated.

**DISTINCT** does not independently deduplicate only the first column.

SQL

```
query(  
  ""  
  SELECT DISTINCT  
    city,  
    signup_date  
  FROM customers  
  ORDER BY  
    city,  
    signup_date;  
  ""  
)
```

## DISTINCT and NULL

Duplicate elimination handles NULL according to distinct-result semantics.

Multiple corresponding NULL results can collapse into one distinct row.

This should not be interpreted as proving:

under ordinary equality.

SQL treats NULL differently according to operation context.

SQL

```
query(  
  ""  
  SELECT DISTINCT city  
  FROM customers  
  ORDER BY city;  
  ""  
)
```

SQL

```
NULL = NULL
```

## DISTINCT Is Not a Join Repair Tool

Later, joins can produce more rows than expected.

A dangerous reaction is:

without understanding why duplication happened.

First ask:

Then decide whether duplicate elimination is logically correct.

SQL

`SELECT DISTINCT ...`

SQL

`What is each table's grain?``Are join keys unique?``Is the relationship one-to-many?``Did the join condition omit a key?``What should one final row represent?`

## ORDER BY

SQL query results have no guaranteed presentation order unless you request one.

Do not rely on:

- insertion order;
- physical storage;
- primary-key appearance;
- observed development output.

If order matters:

is part of the query's semantics.

SQL

```

query(
  """
  SELECT
    product_name,
    price
  FROM products
  ORDER BY price;
  """
)

```

## ASC and DESC

`ASC` means ascending.

`DESC` means descending.

Ascending is commonly the default, but explicitly stating direction can make intent clearer.

SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price  
  FROM products  
  ORDER BY price DESC;  
  ""  
)
```

## Sorting by Multiple Expressions

Later ordering expressions act as tie-breakers.

For example:

means:

1. category ascending;
2. price descending within category;
3. product ID ascending when category and price tie.

SQL

```
query(  
  ""  
  SELECT  
    category,  
    product_id,  
    product_name,  
    price  
  FROM products  
  ORDER BY  
    category ASC,  
    price DESC,  
    product_id ASC;  
  ""  
)
```

SQL

```
ORDER BY  
  category ASC,  
  price DESC,  
  product_id ASC
```

## ORDER BY and Aliases

Many databases allow selected aliases in `ORDER BY`.

For example:

Alias visibility varies by clause and SQL dialect.

Do not assume an alias that works in `ORDER BY` will also be available in `WHERE`.

SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
    ROUND(  
      price * 1.18,  
      2  
    ) AS adjusted_price  
  
  FROM products  
  
  ORDER BY adjusted_price DESC;  
  ""  
)
```

## NULL Ordering

NULL sorting rules differ among database systems.

SQLite commonly places NULL before ordinary values for ascending order and after them for descending order.

Some databases provide:

When missing-value placement matters, explicitly understand the target database's rules.

## SQL

```
print("Ascending city order:")

display(
  query(
    """
    SELECT
      customer_id,
      name,
      city
    FROM customers
    ORDER BY
      city ASC,
      customer_id ASC;
    """
  )
)

print("\nDescending city order:")

display(
  query(
    """
    SELECT
      customer_id,
      name,
      city
    FROM customers
    ORDER BY
      city DESC,
      customer_id ASC;
    """
  )
)
```

## SQL

```
NULLS FIRST
NULLS LAST
```

## Deterministic Ordering

Suppose several rows have equal values for the primary ordering field.

This:

does not fully specify the relative order among tied scores.

If deterministic ordering matters, add a stable tie-breaker:

Deterministic order matters for:

- pagination;

- reproducible reports;
- top-N results;
- APIs;
- stable batch processing.

**PYTHON**

```
ORDER BY score DESC
```

**PYTHON**

```
ORDER BY
    score DESC,
    record_id ASC
```

## LIMIT

SQLite and several other databases support:

to return at most `n` rows.

Other systems may use syntax such as:

When `LIMIT` represents a top-N question, `ORDER BY` must define what "top" means.

**SQL**

```
query(
    """
    SELECT
        product_id,
        product_name,
        price
    FROM products
    ORDER BY
        price DESC,
        product_id ASC
    LIMIT 3;
    """
)
```

## LIMIT Without ORDER BY

This:

means:

It does not logically mean:

If "first" has business meaning, ordering must define that meaning.



SQL

```
SELECT *  
FROM products  
LIMIT 3;
```

SQL

```
return no more than three rows
```

## OFFSET

**OFFSET** skips a number of rows from the ordered result.

For example:

can represent a simple second page.

SQL

```
query(  
  ""  
  SELECT  
    product_id,  
    product_name,  
    price  
  FROM products  
  ORDER BY product_id  
  LIMIT 3  
  OFFSET 3;  
  ""  
)
```

SQL

```
ORDER BY product_id  
LIMIT 3  
OFFSET 3
```

## Offset Pagination

A common UI pagination query is:

It is simple and often appropriate.

However:

- very large offsets can become expensive;
- concurrent insertions/deletions can move rows between pages;
- unstable ordering can cause duplicates or omissions across requests.

These limitations motivate keyset pagination for some systems.

## PYTHON

```
ORDER BY id
LIMIT 20
OFFSET 100;
```

## Keyset Pagination

Keyset pagination continues after the last observed ordering key.

Example:

Instead of:

the query says:

This can be efficient with a suitable index and stable ordering.

Suppose records are ordered by:

Instead of asking:

the client remembers the final key from the previous page and requests:

With a compatible composite index, the database can navigate from the last known key.

This is commonly called keyset, seek, or cursor-style pagination.

## SQL

```
last_seen_product_id = 103

query(
    """
    SELECT
        product_id,
        product_name,
        price
    FROM products
    WHERE product_id > ?
    ORDER BY product_id
    LIMIT 3;
    """,
    params=(last_seen_product_id,),
)
```

## SQL

```
WHERE product_id > ?
ORDER BY product_id
LIMIT 20;
```

## Scalar Functions

Scalar functions transform individual values and normally produce one result per qualifying row.

Common families include:

- text functions;
- numerical functions;
- temporal functions;
- conversion functions;
- NULL-handling functions.

Aggregate functions, which combine multiple rows, are covered in Part 3.

## LOWER and UPPER

`LOWER()` and `UPPER()` convert text case.

They are useful for display and some normalization tasks.

However, robust case-insensitive comparison across international text can involve Unicode and collation semantics beyond simple case conversion.

SQL

```
query(  
  ""  
  SELECT  
    name,  
    LOWER(name) AS lower_name,  
    UPPER(name) AS upper_name  
  FROM customers;  
  ""  
)
```

## LENGTH

SQLite's `LENGTH()` measures text length.

Other SQL systems may expose:

Character length and byte length are not necessarily equivalent for Unicode text.

## SQL

```

query(
    """
    SELECT
        product_name,
        LENGTH(product_name)
        AS name_length
    FROM products
    ORDER BY
        name_length DESC,
        product_name;
    """
)

```

**TRIM**

Dirty textual data often contains unintended leading or trailing whitespace.

Common functions include:

Cleaning can repair representation, but robust pipelines should also detect and investigate recurring upstream data-quality problems.

## SQL

```

query(
    """
    WITH dirty_text(value) AS (
        VALUES
            (' AI '),
            ('Data '),
            (' SQL')
    )

    SELECT
        value,
        TRIM(value) AS cleaned,
        LENGTH(value)
        AS original_length,
        LENGTH(TRIM(value))
        AS cleaned_length
    FROM dirty_text;
    """
)

```

## SQL

```

TRIM()
LTRIM()
RTRIM()

```

## SUBSTR

SQLite provides `SUBSTR()` for extracting text.

SQLite's normal positive substring indexing is one-based.

Do not assume Python's zero-based slicing rules apply to SQL functions.

SQL

```
query(  
    """  
    SELECT  
        name,  
        SUBSTR(name, 1, 2)  
        AS first_two_characters  
    FROM customers;  
    """  
)
```

## REPLACE

SQLite's scalar `REPLACE()` returns text with occurrences substituted.

This should not be confused with vendor-specific SQL statements that may also use the keyword `REPLACE`.

SQL

```
query(  
    """  
    SELECT  
        email,  
  
        REPLACE(  
            email,  
            '@example.com',  
            '@course.local'  
        ) AS transformed_email  
  
    FROM customers;  
    """  
)
```

## Numeric Functions

SQL systems expose mathematical functions such as:

- `ABS` ;
- `ROUND` ;
- powers;
- roots;
- logarithms.

Availability varies by database.

This section uses broadly available SQLite examples.

SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
  
    ROUND(  
      price / 3.0,  
      2  
    ) AS one_third,  
  
    ABS(  
      price - 10000  
    ) AS distance_from_10000  
  
  FROM products;  
  ""  
)
```

## Division and Operand Types

Numerical division can depend on operand types.

In SQLite:

with integer operands yields integer-style behavior.

Introducing a real operand:

produces real-number division.

This matters when calculating ratios and ML features.

SQL

```
query(  
  ""  
  SELECT  
    5 / 2  
    AS integer_operand_result,  
  
    5.0 / 2  
    AS real_operand_result,  
  
    CAST(5 AS REAL) / 2  
    AS cast_result;  
  ""  
)
```

SQL

5 / 2

## CAST

**CAST** requests conversion:

Conversions may be needed for:

- numerical arithmetic;
- imported text data;
- schema compatibility;
- output formatting;
- comparisons.

Conversion behavior is dialect-specific.

SQL

```
query(  
  ""  
  SELECT  
    '123' AS original_text,  
  
    CAST(  
      '123'  
      AS INTEGER  
    ) AS integer_value,  
  
    CAST(  
      123  
      AS TEXT  
    ) AS text_value;  
  ""  
)
```

## Conversion Is Not Validation

SQLite is permissive in many conversions.

Malformed text can sometimes partially convert rather than raising the error a stricter database would produce.

Therefore:

does not necessarily mean:

Validation should be explicit.

Do not generalize this SQLite conversion behavior to PostgreSQL or other database systems.

SQL

```
query(  
  ""  
  SELECT  
    '123abc' AS original_value,  
  
    CAST(  
      '123abc'  
      AS INTEGER  
    ) AS sqlite_cast_result;  
  ""  
)
```

SQL

```
CAST returned a value
```

## CASE Expressions

**CASE** provides conditional expressions.

A searched CASE has the form:

It is widely used for:

- categories;
- flags;
- rule-based transformations;
- feature engineering;
- data validation;
- conditional aggregation.



## SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
  
    CASE  
      WHEN price >= 50000  
        THEN 'high'  
  
      WHEN price >= 10000  
        THEN 'medium'  
  
      ELSE 'low'  
    END AS price_band  
  
  FROM products  
  ORDER BY price DESC;  
  ""  
)
```

## SQL

```
CASE  
  WHEN condition_1 THEN result_1  
  WHEN condition_2 THEN result_2  
  ELSE fallback  
END
```

## CASE Condition Order

CASE uses the first matching condition.

Consider:

A price of 75,000 satisfies the first condition, so the second branch is never reached.

Overlapping conditions must be ordered intentionally.

## SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
  
    CASE  
      WHEN price >= 10000  
        THEN 'at_least_10000'  
  
      WHEN price >= 50000  
        THEN 'at_least_50000'  
  
      ELSE 'small'  
    END AS poorly_ordered_logic,  
  
    CASE  
      WHEN price >= 50000  
        THEN 'at_least_50000'  
  
      WHEN price >= 10000  
        THEN 'at_least_10000'  
  
      ELSE 'small'  
    END AS correctly_ordered_logic  
  
  FROM products  
  ORDER BY price DESC;  
  ""  
)
```

## Simple CASE

SQL also provides a value-comparison form:

This is useful for exact mappings.

Searched CASE is more flexible for inequalities and complex conditions.

SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    status,  
  
    CASE status  
      WHEN 'completed'  
        THEN 'done'  
  
      WHEN 'pending'  
        THEN 'waiting'  
  
      WHEN 'cancelled'  
        THEN 'stopped'  
  
      ELSE 'unexpected'  
    END AS display_status  
  
  FROM orders  
  
  ORDER BY order_id;  
  ""  
)
```

SQL

```
CASE status  
  WHEN 'completed' THEN 'done'  
  WHEN 'pending' THEN 'waiting'  
  ELSE 'other'  
END
```

## CASE Without ELSE

If no `WHEN` matches and no `ELSE` is provided, the CASE expression normally produces NULL.

This can be deliberate.

It can also accidentally create missing values when a category was forgotten.

## SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    status,  
  
    CASE  
      WHEN status = 'completed'  
      THEN 1  
    END AS completed_only_flag  
  
  FROM orders  
  
  ORDER BY order_id;  
  ""  
)
```

## CASE and NULL

Do not use ordinary equality logic to detect NULL.

Prefer:

## SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    name,  
    city,  
  
    CASE  
      WHEN city IS NULL  
      THEN 'missing'  
      ELSE 'known'  
    END AS city_state  
  
  FROM customers  
  
  ORDER BY customer_id;  
  ""  
)
```

## COALESCE

**COALESCE** returns the first non-NULL argument.

Example:

But remember:

and:

are not automatically the same semantic state.

The replacement should reflect the domain.

SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    name,  
    city,  
  
    COALESCE(  
      city,  
      'Unknown'  
    ) AS city_for_display  
  
  FROM customers  
  
  ORDER BY customer_id;  
  ""  
)
```

SQL

```
COALESCE(city, 'Unknown')
```

## Multiple COALESCE Alternatives

`COALESCE` can express an ordered fallback:

The first non-NULL value wins.

## SQL

```

query(
  """
  WITH people (
    preferred_name,
    legal_name,
    username
  ) AS (
    VALUES
    (
      'Aisha',
      'Aisha Singh',
      'aisha01'
    ),
    (
      NULL,
      'Rohan Mehta',
      'rohan02'
    ),
    (
      NULL,
      NULL,
      'user03'
    ),
    (
      NULL,
      NULL,
      NULL
    )
  )
  # ...

  FROM people;
  """
)

```

## SQL

```

COALESCE(
  preferred_name,
  legal_name,
  username,
  'Unknown'
)

```

**NULLIF**

`NULLIF(a, b)` returns NULL when `a` equals `b`; otherwise it returns `a`.

A common pattern is:

This represents an undefined zero-denominator result as NULL.

SQL

```

query(
    """
    SELECT
        10.0 / NULLIF(2, 0)
        AS valid_ratio,

        10.0 / NULLIF(0, 0)
        AS guarded_zero_ratio;
    """
)

```

## NULL Propagation

Many expressions involving NULL produce NULL.

For example:

cannot produce a known sum.

This becomes important in feature engineering because one missing input can propagate into several derived values.

SQL

```

query(
    """
    SELECT
        10 + NULL
        AS addition_result,

        10 * NULL
        AS multiplication_result,

        'AI' || NULL
        AS concatenation_result;
    """
)

```

SQL

```
10 + NULL
```

## Predicates as Expressions

SQLite can select comparison expressions directly.

For example:

commonly produces integer-like `1` or `0`.

Other databases may return native Boolean types.

The conceptual point is that predicates can become derived fields.

SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
    price >= 10000  
    AS is_expensive  
  FROM products  
  ORDER BY product_id;  
  ""  
)
```

SQL

```
price >= 10000 AS is_expensive
```

## Date and Time Functions in SQLite

SQLite does not provide the same native temporal type system as PostgreSQL.

It commonly represents dates and timestamps using:

- ISO-style text;
- Unix timestamps;
- Julian day numbers.

SQLite provides functions including:

The syntax here is SQLite-specific, while the underlying temporal-engineering principles are broadly applicable.



## SQL

```
query(  
    ""  
    SELECT  
        order_id,  
        order_date,  
  
        DATE(  
            order_date,  
            '+7 day'  
        ) AS seven_days_later  
  
    FROM orders  
  
    ORDER BY  
        order_date,  
        order_id;  
    ""  
)
```

## Extracting Date Components

SQLite's `STRFTIME()` can format or extract temporal components.

Examples:

Other databases may use:

or other dialect-specific functions.

## SQL

```
query(  
    ""  
    SELECT  
        order_id,  
        order_date,  
  
        STRFTIME(  
            '%Y',  
            order_date  
        ) AS year,  
  
        STRFTIME(  
            '%m',  
            order_date  
        ) AS month,  
  
        STRFTIME(  
            '%d',  
            order_date  
        ) AS day  
  
    FROM orders  
  
    ORDER BY  
        order_date,  
        order_id;  
    ""  
)
```

## SQL

```
EXTRACT  
DATE_PART  
YEAR  
MONTH
```

## ISO-Style Date Text

The learning database stores dates as:

For valid consistently formatted dates, lexicographic ordering corresponds to chronological ordering.

Arbitrary date strings such as:

do not provide the same reliable properties and can also be ambiguous.

A string looking date-like does not guarantee that it is a valid date.

## PYTHON

```
YYYY-MM-DD
```

## PYTHON

```
3/1/2026  
11/2/2026
```

## Half-Open Time Ranges

A useful temporal filtering pattern is:

For March:

This avoids guessing an artificial final instant such as:

Half-open intervals are common in robust data-engineering and ML feature pipelines.

## SQL

```
query(  
    """  
    SELECT  
        order_id,  
        order_date,  
        status  
  
    FROM orders  
  
    WHERE  
        order_date >= '2026-03-01'  
        AND order_date < '2026-03-07'  
  
    ORDER BY  
        order_date,  
        order_id;  
    """  
)
```

## Parameterized Filtering

Values coming from Python should normally be bound through the database driver's parameter interface.

With `sqlite3`, `?` is a positional placeholder.

This keeps values separate from SQL source.

## SQL

```
minimum_price = 5000.0

query(
    """
    SELECT
        product_id,
        product_name,
        price

    FROM products

    WHERE price >= ?

    ORDER BY price;
    """,
    params=(minimum_price,),
)
```

## Multiple Parameters

The order of supplied values corresponds to the order of positional placeholders.

## SQL

```
category = "Electronics"
minimum_price = 1000.0
maximum_price = 25000.0

query(
    """
    SELECT
        product_name,
        category,
        price

    FROM products

    WHERE
        category = ?
        AND price BETWEEN ? AND ?

    ORDER BY price;
    """,
    params=(
        category,
        minimum_price,
        maximum_price,
    ),
)
```

## Named Parameters

Python's SQLite driver also supports named parameters.

Named parameters can improve readability when a statement uses several values.

### SQL

```
query(
    """
    SELECT
        product_name,
        category,
        price

    FROM products

    WHERE
        category = :category
        AND price >= :minimum_price

    ORDER BY price;
    """,
    params={
        "category": "Furniture",
        "minimum_price": 5000.0,
    },
)
```

## SQL Injection

Dangerous code can arise when untrusted values are concatenated into SQL:

The user-controlled text has become SQL source.

An attacker may be able to change the structure or meaning of the statement.

The fundamental defense is:

Use bound parameters.

### PYTHON

```
sql = (
    "SELECT * FROM customers "
    "WHERE email = '" + user_input + "'"
)
```

### PYTHON

```
SQL structure
and
untrusted values
must remain separate
```

## Parameters Are for Values

Bind parameters represent values.

For example:

The placeholder is a value.

You generally cannot bind a table or column name this way:

and expect the bound string `"price"` to become SQL syntax.

Dynamic identifiers require controlled identifier handling rather than ordinary value parameters.

PYTHON

```
WHERE price >= ?
```

PYTHON

```
ORDER BY ?
```

## Safe Dynamic Identifiers

Suppose an application allows exactly two sort choices:

A strict application-controlled mapping can translate those choices into known SQL identifiers.

Arbitrary user SQL is never accepted.

The f-string in the previous example is safe only because the inserted SQL identifier comes from a strict application-owned allow-list.

Do not directly interpolate arbitrary user text into SQL.

## SQL

```

allowed_sort_columns = {
    "price": "price",
    "name": "product_name",
}

requested_sort = "price"

if requested_sort not in allowed_sort_columns:
    raise ValueError(
        "Unsupported sort option."
    )

safe_identifier = (
    allowed_sort_columns[
        requested_sort
    ]
)

sql = f"""
SELECT
    product_id,
    product_name,
    price
FROM products
ORDER BY
    {safe_identifier} ASC,
    product_id ASC;
"""

query(sql)

```

## Logical Query Processing Order

A query is written like:

A useful conceptual order is:

Later, grouped queries expand this model:

This is a logical model.

A query optimizer may physically execute an equivalent plan differently while preserving semantics.

## SQL

```

SELECT ...
FROM ...
WHERE ...
ORDER BY ...
LIMIT ...

```

## SQL

**FROM****WHERE****SELECT****DISTINCT****ORDER BY****LIMIT / OFFSET**

## SELECT Alias Visibility

Consider:

Some systems are more permissive than others, but portable foundational SQL should not assume a **SELECT** alias is available to **WHERE**.

Conceptually, **WHERE** operates before final **SELECT** projection.

**ORDER BY**, however, commonly allows output aliases.

## SQL

```

query(
  """
  SELECT
    product_name,
    price,
    price * 1.18
      AS adjusted_price

  FROM products

  WHERE
    price * 1.18 > 10000

  ORDER BY
    adjusted_price DESC;
  """
)

```

## SQL

```

SELECT
  price * 1.18 AS adjusted_price
FROM products
WHERE adjusted_price > 10000;

```



## Filtering for Data Validation

SQL filtering is useful for finding invalid records.

Suppose a staging dataset should contain only:

If NULL is also invalid, both cases must be expressed.

This is another example where the `NOT IN` NULL interaction matters.

SQL

```
query(  
  ""  
  WITH staging_status(value) AS (  
    VALUES  
      ('pending'),  
      ('completed'),  
      ('xyz'),  
      (NULL)  
  )  
  
  SELECT value  
  
  FROM staging_status  
  
  WHERE  
    value IS NULL  
    OR value NOT IN (  
      'pending',  
      'completed',  
      'cancelled'  
    );  
  ""  
)
```

## Missingness as a Feature

Missingness can itself carry information.

SQL can construct:

as a missingness indicator.

Whether such a feature should be used depends on the problem.

Missingness can reflect:

- user behavior;
- source-system differences;
- data-collection processes;
- operational failures;

- potentially sensitive proxies.

Missingness should therefore be analyzed rather than mechanically encoded.

SQL

```
query(  
    ""  
    SELECT  
        customer_id,  
        city,  
  
        CASE  
            WHEN city IS NULL  
            THEN 1  
            ELSE 0  
        END AS city_missing  
  
    FROM customers  
  
    ORDER BY customer_id;  
    ""  
)
```

SQL

```
CASE  
    WHEN city IS NULL THEN 1  
    ELSE 0  
END
```

## Index-Friendly Filtering Awareness

Indexes will be covered deeply later.

For now, introduce one performance idea: the form of a predicate can affect whether an index is useful.

If a database has an index on:

a direct range such as:

may be easier to optimize than transforming the indexed column for every row before comparison.

This is related to the concept of sargability.

Actual performance must be checked using execution plans and measurements rather than rules of thumb alone.

PYTHON

```
order_date
```

## PYTHON

```
order_date >= '2026-03-01'  
AND order_date < '2026-04-01'
```

## Top-N Semantics and Ties

Suppose the third and fourth products have identical prices.

This:

returns exactly three rows.

But the business requirement might instead be:

That is a different problem.

Ranking window functions such as `RANK()` and `DENSE_RANK()` are better suited to rank-based tie semantics.

Natural-language requirements must be translated into exact relational behavior.

## PYTHON

```
ORDER BY price DESC  
LIMIT 3
```

## Do Not Depend on Physical Predicate Evaluation Order

SQL is declarative.

A database optimizer can transform the physical execution plan.

Do not write expressions whose safety depends on assuming the engine physically evaluates predicates strictly from left to right.

When guarding an unsafe calculation, use SQL constructs that express the desired semantics explicitly, such as `NULLIF` for zero denominators where appropriate.

## SQL Logic Versus Python Logic

Important differences include:

Python equality:

SQL equality:

Python missing object:

SQL missing value:

Python membership:

SQL membership:

Python Boolean operators:

SQL:

Most importantly, SQL includes the logical state `UNKNOWN`, so ordinary two-valued Python reasoning does not fully describe SQL predicates.

PYTHON

```
x == y
```

PYTHON

```
x = y
```

### Filtering Changes an ML Population

Suppose an ML training source contains one million customers.

Then the query applies:

The resulting population is no longer:

It is:

If age missingness correlates with account type, geography, acquisition source, device, or user behavior, the filter can introduce selection bias.

SQL preprocessing therefore belongs to the ML methodology, not merely to data retrieval.

PYTHON

```
WHERE age IS NOT NULL
```

PYTHON

```
all customers
```

### Temporal Leakage Preview

Suppose a model predicts customer churn on March 1.

A feature query accidentally includes an order from March 20.

The query may be syntactically correct and relationally valid, but the feature contains future information.

For time-dependent ML problems, distinguish:

SQL correctness does not automatically imply ML correctness.

## PYTHON

```
prediction time
```

```
feature observation period
```

```
label observation period
```

## Common Querying Mistakes

Important mistakes include:

- using `==` instead of `=`;
- using `= NULL` instead of `IS NULL`;
- assuming `NULL <> value` is true;
- forgetting `UNKNOWN`;
- using `NOT IN` with NULL-containing inputs;
- mixing `AND` and `OR` without clear grouping;
- relying on row order without `ORDER BY`;
- using `LIMIT` as "top N" without ordering;
- failing to define tie-breaking;
- using `DISTINCT` to hide structural duplication;
- assuming `BETWEEN` excludes endpoints;
- confusing `LIKE` with regular expressions;
- assuming case behavior is identical across databases;
- assuming SQLite `CAST` behavior is portable;
- replacing NULL without considering meaning;
- interpolating untrusted SQL values;
- attempting to bind identifiers as ordinary parameters;
- assuming aliases are visible in every clause;
- treating data filters as statistically neutral in ML.

Most of these are semantic mistakes rather than syntax mistakes.

## Querying and Filtering Summary

Part 2 developed SQL row-level querying in depth.

The major concepts covered were:

- `SELECT`;
- projection;
- `SELECT *`;

- expressions;
- arithmetic;
- aliases;
- literals;
- `SELECT` without `FROM`;
- string concatenation;
- `WHERE`;
- comparison operators;
- `AND`;
- `OR`;
- `NOT`;
- operator precedence;
- `NULL`;
- `IS NULL`;
- `IS NOT NULL`;
- three-valued logic;
- explicit missing-value policies;
- `BETWEEN`;
- `NOT BETWEEN`;
- `IN`;
- `NOT IN`;
- the `NOT IN` `NULL` trap;
- `LIKE`;
- `%`;
- `_`;
- escaping wildcards;
- SQLite `GLOB`;
- `DISTINCT`;
- distinct result combinations;
- `DISTINCT` and `NULL`;
- `ORDER BY`;
- `ASC`;
- `DESC`;
- multiple sort expressions;
- deterministic tie-breaking;

- NULL sorting;
- `LIMIT` ;
- `OFFSET` ;
- offset pagination;
- keyset pagination;
- scalar functions;
- text functions;
- numeric functions;
- integer versus real division;
- `CAST` ;
- conversion versus validation;
- searched CASE;
- simple CASE;
- CASE ordering;
- CASE and NULL;
- `COALESCE` ;
- `NULLIF` ;
- NULL propagation;
- Boolean-like expressions;
- SQLite temporal functions;
- ISO date representation;
- half-open temporal ranges;
- positional parameters;
- named parameters;
- SQL injection;
- safe dynamic identifiers;
- logical query-processing order;
- alias visibility;
- data-validation filters;
- missingness indicators;
- index-friendly filtering awareness;
- top-N tie semantics;
- filtering bias;
- temporal ML leakage.

Part 3 moves from row-level querying to aggregation and grouped analysis:

## PYTHON

```
COUNT
SUM
AVG
MIN
MAX
COUNT DISTINCT
GROUP BY
HAVING
conditional aggregation
grouping grain
weighted averages
data-quality profiling
AI feature aggregation
```

## Aggregation and Grouped Analysis

Parts 1 and 2 established relational structure and row-level querying. We now move from questions about individual rows to questions about collections of rows.

Compare:

with:

Compare:

with:

And compare:

with:

The second question in each pair requires aggregation.

Aggregation is central to:

- analytics;
- reporting;
- business metrics;
- monitoring;
- data validation;
- feature engineering;
- experiment analysis;
- machine-learning dataset construction.

The most important idea in this section is not the syntax of `SUM`, `COUNT`, or `AVG`. It is grain.

Before aggregation:



After aggregation:

or:

or:

Aggregation deliberately changes the meaning of a result row.

PYTHON

What **is** each product's price?

PYTHON

What **is** the average product price?

## Execution State

Part 2 creates the current in-memory SQLite learning database and defines:

Part 3 continues using that database.

When the complete notebook is executed from the beginning, the required database state therefore exists before this section.

The next cell validates those assumptions so accidental out-of-order execution fails clearly.

Part 4 continues using the learning database initialized in Part 2.

The expected tables are:

The following cell validates the notebook state before join examples begin.

Part 5 continues using the in-memory learning database created in Part 2.

The next cell verifies the expected Python objects and database tables.

Part 6 uses the learning database created in Part 2.

The following cell verifies that the expected objects and tables remain available.

## PYTHON

```

required_objects = [
    "connection",
    "cursor",
    "query",
]

missing_objects = [
    name
    for name in required_objects
    if name not in globals()
]

if missing_objects:
    raise RuntimeError(
        "Run the previous SQL sections first. "
        f"Missing objects: {missing_objects}"
    )

required_tables = {
    "customers",
    "products",
    "orders",
    "order_items",
}

existing_tables = {
    row[0]
    for row in connection.execute(
        # ...
        f"{sorted(missing_tables)}"
    )
}

print("Part 3 prerequisites verified.")

```

## Aggregate Functions

An aggregate function summarizes a collection of rows.

The foundational aggregate functions are:

Suppose a numerical input contains:

Then:

Without `GROUP BY`, an aggregate query normally treats the complete qualifying input as one aggregate group.

With `GROUP BY`, SQL partitions the input into several groups and calculates aggregates separately for each group.

## PYTHON

```
COUNT
SUM
AVG
MIN
MAX
```

## PYTHON

```
100
200
300
400
500
```

## COUNT(\*)

`COUNT(*)` counts rows.

For example:

asks:

The star inside `COUNT(*)` represents row counting and should not be confused with the output-column wildcard in:

## SQL

```
query(
    """
    SELECT
        COUNT(*) AS customer_count
    FROM customers;
    """
)
```

## SQL

```
SELECT COUNT(*)
FROM customers;
```

## COUNT(expression)

`COUNT(expression)` counts rows where the expression is non-NULL.

Therefore:

and:

can produce different results.

`COUNT(*)` counts every row.

`COUNT(city)` counts only rows whose city is known.

This distinction is very useful for missing-data profiling.

#### SQL

```
query(
    """
    SELECT
        COUNT(*)
        AS total_customers,

        COUNT(city)
        AS customers_with_city,

        COUNT(*) - COUNT(city)
        AS customers_missing_city

    FROM customers;
    """
)
```

### Three Important COUNT Questions

These expressions answer different questions:

Their meanings are:

The choice of aggregate must follow the metric definition.

#### PYTHON

```
COUNT(*)
COUNT(city)
COUNT(DISTINCT city)
```

#### PYTHON

```
How many rows exist?
How many rows contain a known city?
How many different known cities exist?
```

### COUNT(DISTINCT expression)

`COUNT(DISTINCT expression)` counts distinct non-NULL values.

For example:

calculates the number of different known city values represented in the customer table.

## SQL

```

query(
  """
  SELECT
    COUNT(*)
      AS customer_rows,

    COUNT(city)
      AS known_city_rows,

    COUNT(DISTINCT city)
      AS distinct_known_cities

  FROM customers;
  """
)

```

## SQL

```

COUNT(DISTINCT city)

```

**COUNT(1) Versus COUNT(\*)**

You may encounter:

Since `1` is a non-NULL constant for each input row, this normally counts rows too.

Modern database optimizers generally understand row-count operations efficiently.

Do not rely on folklore saying `COUNT(1)` is universally faster than `COUNT(*)`.

`COUNT(*)` directly communicates the intended operation:

## SQL

```

query(
  """
  SELECT
    COUNT(*) AS count_star,
    COUNT(1) AS count_one
  FROM orders;
  """
)

```

**SUM**

`SUM()` adds non-NULL numerical inputs.

For example:

calculates total quantity.

However, numerical storage does not automatically imply additive meaning.

These commonly make sense to sum:

These usually do not:

Semantic meaning matters more than storage type.

SQL

```
query(  
  ""  
  SELECT  
    SUM(quantity)  
    AS total_units  
  FROM order_items;  
  ""  
)
```

SQL

```
SUM(quantity)
```

### SUM over a Calculated Expression

Aggregates can operate on expressions.

For one order-item row:

represents the line value.

Therefore:

calculates the total value of all qualifying item lines.

SQL

```
query(  
  ""  
  SELECT  
    SUM(  
      quantity * unit_price  
    ) AS gross_item_value  
  FROM order_items;  
  ""  
)
```

SQL

```
quantity x unit_price
```

### AVG

`AVG()` calculates the arithmetic mean of non-NULL input values.

Conceptually:

NULL values are normally ignored rather than treated as zero.

Those are different statistical assumptions.

SQL

```
query(  
  ""  
  SELECT  
    ROUND(  
      AVG(price),  
      2  
    ) AS average_product_price  
  
  FROM products;  
  ""  
)
```

## MIN and MAX

`MIN()` returns the smallest non-NULL input.

`MAX()` returns the largest non-NULL input.

These functions are useful for:

- numerical ranges;
- earliest/latest timestamps;
- validation;
- detecting suspicious extremes;
- monitoring.

Textual minimum/maximum behavior depends on collation and comparison rules.

SQL

```
query(  
  ""  
  SELECT  
    MIN(price)  
    AS minimum_price,  
  
    MAX(price)  
    AS maximum_price  
  
  FROM products;  
  ""  
)
```

## Multiple Aggregates in One Query

A single aggregate query can compute several summaries over the same input population.

This is common in exploratory analysis, reporting, and validation.

SQL

```
query(  
  ""  
  SELECT  
    COUNT(*)  
      AS product_count,  
  
    MIN(price)  
      AS minimum_price,  
  
    MAX(price)  
      AS maximum_price,  
  
    ROUND(  
      AVG(price),  
      2  
    ) AS average_price,  
  
    SUM(price)  
      AS sum_of_catalog_prices  
  
  FROM products;  
  ""  
)
```

## NULL Behavior in Aggregate Functions

Most common aggregates ignore NULL values:

`COUNT(*)` is different because it counts rows.

Suppose values are:

Ordinary SQL averaging generally computes:

It does not automatically compute:

Treating missing as zero requires an explicit transformation.



## SQL

```
query(  
  ""  
  WITH measurements(value) AS (  
    VALUES  
      (10.0),  
      (20.0),  
      (NULL)  
    )  
  
  SELECT  
    COUNT(*)  
      AS total_rows,  
  
    COUNT(value)  
      AS observed_values,  
  
    SUM(value)  
      AS observed_sum,  
  
    AVG(value)  
      AS observed_average  
  
  FROM measurements;  
  ""  
)
```

## Replacing NULL Before Averaging Changes the Statistic

Compare:

with:

The first excludes missing observations.

The second changes each missing observation into an actual zero before averaging.

These are not equivalent operations.

For machine learning, imputation policy must be selected deliberately and must respect training/validation/test boundaries when parameters are learned from data.

## SQL

```
query(  
  ""  
  WITH measurements(value) AS (  
    VALUES  
      (10.0),  
      (20.0),  
      (NULL)  
    )  
  
  SELECT  
    AVG(value)  
    AS missing_excluded,  
  
    AVG(  
      COALESCE(  
        value,  
        0  
      )  
    ) AS missing_as_zero  
  
  FROM measurements;  
  ""  
)
```

## SQL

```
AVG(value)
```

## Aggregating an All-NULL Input

If rows exist but all relevant values are NULL, aggregates such as:

typically return NULL.

There is no observed value from which to calculate their ordinary result.

`COUNT(expression)` returns zero because it sees no non-NULL expression values.

## SQL

```

query(
    ""
    WITH measurements(value) AS (
        VALUES
            (NULL),
            (NULL)
        )

    SELECT
        COUNT(*)
        AS rows_present,

        COUNT(value)
        AS observed_values,

        SUM(value)
        AS total,

        AVG(value)
        AS average,

        MIN(value)
        AS minimum,

        MAX(value)
        AS maximum

    FROM measurements;
    ""
)

```

## SQL

```

SUM
AVG
MIN
MAX

```

## Aggregation over No Qualifying Rows

An aggregate query without `GROUP BY` can still return a summary row even when no row satisfies the filter.

Typical results are:

This distinction matters in feature engineering.

"No events occurred" and "events existed but values were unknown" can represent different states.

## SQL

```
query(  
  ""  
  SELECT  
    COUNT(*)  
    AS matching_rows,  
  
    SUM(price)  
    AS price_sum,  
  
    AVG(price)  
    AS price_average,  
  
    MIN(price)  
    AS minimum_price,  
  
    MAX(price)  
    AS maximum_price  
  
  FROM products  
  
  WHERE price > 1000000;  
  ""  
)
```

**COALESCE after an Aggregate**

Sometimes a domain rule says:

Then:

can represent that rule.

This is different from:

because the latter modifies individual missing values before summation.

The position of **COALESCE** changes the semantics.

SQL

```

query(
    """
    SELECT
        COALESCE(
            SUM(price),
            0
        ) AS total_price

    FROM products

    WHERE price > 1000000;
    """
)

```

SQL

```

no qualifying events
total = 0

```

## WHERE Filters Rows Before Aggregation

Consider:

The conceptual sequence is:

The aggregate therefore describes the filtered population rather than all products.

SQL

```

query(
    """
    SELECT
        COUNT(*)
        AS electronics_products,

        ROUND(
            AVG(price),
            2
        ) AS electronics_average_price

    FROM products

    WHERE category = 'Electronics';
    """
)

```

SQL

```

SELECT AVG(price)
FROM products
WHERE category = 'Electronics';

```

## GROUP BY

Without `GROUP BY`, an aggregate query commonly treats the complete qualifying input as one aggregate group.

`GROUP BY` divides rows according to one or more grouping expressions.

For example:

conceptually performs:

The result grain has changed.

SQL

```
query(  
  ""  
  SELECT  
    category,  
  
    COUNT(*)  
      AS product_count  
  
  FROM products  
  
  GROUP BY category  
  
  ORDER BY category;  
  ""  
)
```

## Multiple Metrics per Group

Each group can produce several summary values.

A common grouped profile contains:

## SQL

```
query(  
  ""  
  SELECT  
    category,  
  
    COUNT(*)  
      AS product_count,  
  
    MIN(price)  
      AS minimum_price,  
  
    MAX(price)  
      AS maximum_price,  
  
    ROUND(  
      AVG(price),  
      2  
    ) AS average_price,  
  
    SUM(price)  
      AS sum_of_catalog_prices  
  
  FROM products  
  
  GROUP BY category  
  
  ORDER BY category;  
  ""  
)
```

## SQL

```
group key  
count  
minimum  
maximum  
average  
sum
```

## GROUP BY Changes Grain

The products table begins with:

After:

the result becomes:

Now ask whether an ordinary `product_name` makes sense at category grain.

Electronics contains several product names. There is no single ordinary product name that represents the entire group.

This is why strict grouped queries normally select:

- grouping expressions;
- aggregate expressions.

SQLite can be permissive with ambiguous grouped queries, but permissiveness should not be treated as portable correctness.

PYTHON

```
one row per product
```

PYTHON

```
GROUP BY category
```

### A Portable GROUP BY Mental Rule

For foundational SQL, use this rule:

If the output is one row per category:

have clear category-level meanings.

An arbitrary individual product name does not.

PYTHON

```
Every selected value must have a clear meaning  
at the output grain.
```

### GROUP BY Multiple Columns

Grouping several expressions creates one group for each distinct combination.

For example:

produces:

rather than one row per customer.



SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    status,  
  
    COUNT(*)  
    AS order_count  
  
  FROM orders  
  
  GROUP BY  
    customer_id,  
    status  
  
  ORDER BY  
    customer_id,  
    status;  
  ""  
)
```

SQL

```
GROUP BY  
  customer_id,  
  status
```

## GROUP BY and NULL

Rows whose grouping expression is NULL participate in a NULL group for grouping purposes.

For example, several customers with missing cities appear together in the NULL city group.

SQL

```
query(  
  ""  
  SELECT  
    city,  
  
    COUNT(*)  
    AS customer_count  
  
  FROM customers  
  
  GROUP BY city  
  
  ORDER BY city;  
  ""  
)
```

## A NULL Group Can Hide Different Business Causes

The NULL city group can contain several kinds of missingness:

The SQL representation is the same, but the underlying business causes need not be.

If missingness reasons matter, model them explicitly.

### PYTHON

```
not collected
not applicable
legacy data
privacy removal
ingestion failure
```

## Grouping by an Expression

Grouping keys can be calculated expressions.

For example, order dates can be converted into month buckets before counting.

Temporal grouping syntax differs across SQL systems.

### SQL

```
query(
    """
    SELECT
        STRFTIME(
            '%Y-%m',
            order_date
        ) AS order_month,

        COUNT(*)
        AS order_count

    FROM orders

    GROUP BY
        STRFTIME(
            '%Y-%m',
            order_date
        )

    ORDER BY order_month;
    """
)
```

## GROUP BY Aliases and Portability

Some SQL systems allow an output alias to be reused in `GROUP BY`, while others have different rules.

For complicated expressions, a clean pattern is:

This can improve both portability and readability.

#### PYTHON

```
inner query / CTE
→ calculate derived field

outer query
→ group by derived field name
```

## WHERE with GROUP BY

**WHERE** removes individual rows before groups are formed.

For example:

first removes cancelled orders.

Only the remaining orders contribute to customer groups.

#### SQL

```
query(
    """
    SELECT
        customer_id,

        COUNT(*)
        AS non_cancelled_orders

    FROM orders

    WHERE status <> 'cancelled'

    GROUP BY customer_id

    ORDER BY customer_id;
    """
)
```

## HAVING

**HAVING** filters groups after aggregation.

For example:

means:

This is different from row-level filtering with **WHERE**.

## SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
  
    COUNT(*)  
    AS order_count  
  
  FROM orders  
  
  GROUP BY customer_id  
  
  HAVING COUNT(*) >= 2  
  
  ORDER BY customer_id;  
  ""  
)
```

## SQL

```
HAVING COUNT(*) >= 2
```

## WHERE Versus HAVING

Question:

"How many completed orders does each customer have?"

The status condition determines which rows enter the groups:

Question:

"Which customers have at least two orders?"

The count exists only after grouping:

A query can contain both because they operate at different logical stages.

SQL

```

query(
  """
  SELECT
    customer_id,

    COUNT(*)
    AS completed_order_count

  FROM orders

  WHERE status = 'completed'

  GROUP BY customer_id

  HAVING COUNT(*) >= 1

  ORDER BY customer_id;
  """
)

```

SQL

```
WHERE status = 'completed'
```

## Logical Processing of Grouped Queries

A useful conceptual sequence is:

This is a logical reasoning model.

A query optimizer may physically execute an equivalent strategy differently.

SQL

```

FROM / JOIN
  ↓
WHERE
  ↓
GROUP BY
  ↓
aggregate calculations
  ↓
HAVING
  ↓
SELECT
  ↓
DISTINCT
  ↓
ORDER BY
  ↓
LIMIT

```

## HAVING with Aggregate Statistics

**HAVING** can filter groups using aggregate values such as:

For example:

SQL

```
query(  
  ""  
  SELECT  
    category,  
  
    COUNT(*)  
      AS product_count,  
  
    ROUND(  
      AVG(price),  
      2  
    ) AS average_price  
  
  FROM products  
  
  GROUP BY category  
  
  HAVING AVG(price) > 5000  
  
  ORDER BY average_price DESC;  
  ""  
)
```

SQL

```
COUNT  
SUM  
AVG  
MIN  
MAX
```

## Multiple HAVING Conditions

Group predicates can be combined using Boolean logic.

For example:

## SQL

```
query(  
    ""  
    SELECT  
        category,  
  
        COUNT(*)  
        AS product_count,  
  
        ROUND(  
            AVG(price),  
            2  
        ) AS average_price  
  
    FROM products  
  
    GROUP BY category  
  
    HAVING  
        COUNT(*) >= 2  
        AND AVG(price) > 1000  
  
    ORDER BY category;  
    ""  
)
```

## SQL

```
at least two products  
AND  
average price above 1000
```

## Ordering Grouped Results

After aggregation, group rows can be sorted according to their aggregate values.

For example, categories can be ordered from highest to lowest average price.

## SQL

```
query(  
  ""  
  SELECT  
    category,  
  
    ROUND(  
      AVG(price),  
      2  
    ) AS average_price  
  
  FROM products  
  
  GROUP BY category  
  
  ORDER BY  
    average_price DESC,  
    category ASC;  
  ""  
)
```

## Top Group Semantics

Combining:

can return the group with the largest aggregate.

However:

returns exactly one row.

It does not automatically return every group tied for first rank.

Tie semantics must be explicitly defined.



## SQL

```
query(  
  ""  
  SELECT  
    category,  
  
    AVG(price)  
    AS average_price  
  
  FROM products  
  
  GROUP BY category  
  
  ORDER BY  
    average_price DESC,  
    category ASC  
  
  LIMIT 1;  
  ""  
)
```

## SQL

```
GROUP BY  
ORDER BY  
LIMIT
```

## Conditional Aggregation

One of SQL's most useful analytical patterns combines `CASE` with an aggregate.

Example:

A matching row contributes `1`.

A nonmatching row contributes `0`.

The resulting sum counts matching rows.

## SQL

```

query(
    """
    SELECT
        COUNT(*)
        AS total_orders,

        SUM(
            CASE
                WHEN status = 'completed'
                THEN 1
                ELSE 0
            END
        ) AS completed_orders,

        SUM(
            CASE
                WHEN status = 'pending'
                THEN 1
                ELSE 0
            END
        ) AS pending_orders,

        SUM(
            CASE
                WHEN status = 'cancelled'
                THEN 1
                ELSE 0
            END
        ) AS cancelled_orders

    FROM orders;
    """
)

```

## SQL

```

SUM(
    CASE
        WHEN condition THEN 1
        ELSE 0
    END
)

```

## Conditional Aggregation per Group

Conditional aggregation becomes especially useful with `GROUP BY`.

For each customer we can calculate several order-status metrics within one query.

## SQL

```

query(
  """
  SELECT
    customer_id,

    COUNT(*)
    AS total_orders,

    SUM(
      CASE
        WHEN status = 'completed'
        THEN 1
        ELSE 0
      END
    ) AS completed_orders,

    SUM(
      CASE
        WHEN status = 'pending'
        THEN 1
        ELSE 0
      END
    ) AS pending_orders,

    SUM(
      CASE
        WHEN status = 'cancelled'
        THEN 1
        # ...

    ORDER BY customer_id;
  """
)

```

**Conditional COUNT with CASE**

Because `COUNT(expression)` ignores NULL, another conditional counting pattern is:

Matching rows produce a non-NULL result.

Nonmatching rows produce NULL and are not counted.

## SQL

```
query(  
  ""  
  SELECT  
    COUNT(  
      CASE  
        WHEN status = 'completed'  
        THEN 1  
      END  
    ) AS completed_with_count,  
  
    SUM(  
      CASE  
        WHEN status = 'completed'  
        THEN 1  
        ELSE 0  
      END  
    ) AS completed_with_sum  
  
  FROM orders;  
  ""  
)
```

## SQL

```
COUNT(  
  CASE  
    WHEN condition THEN 1  
  END  
)
```

## Aggregate FILTER Clause Awareness

Some databases provide:

Modern SQLite and PostgreSQL support this syntax.

Not every SQL database uses the same feature set, so `CASE`-based conditional aggregation remains important portable knowledge.

## SQL

```
query(  
  ""  
  SELECT  
    COUNT(*)  
      AS total_orders,  
  
    COUNT(*) FILTER (  
      WHERE status = 'completed'  
    ) AS completed_orders,  
  
    COUNT(*) FILTER (  
      WHERE status = 'pending'  
    ) AS pending_orders  
  
  FROM orders;  
  ""  
)
```

## SQL

```
COUNT(*) FILTER (  
  WHERE condition  
)
```

## Rates and Ratios

Counts often become ratios.

For example:

Correct ratio construction requires decisions about:

- numerator;
- denominator;
- zero denominators;
- numerical types;
- missing values.

Integer division can produce incorrect rates when real-number division was intended.

## SQL

```

query(
  """
  SELECT
    customer_id,

    COUNT(*)
    AS total_orders,

    SUM(
      CASE
        WHEN status = 'completed'
        THEN 1
        ELSE 0
      END
    ) AS completed_orders,

    ROUND(
      1.0
      * SUM(
        CASE
          WHEN status = 'completed'
          THEN 1
          ELSE 0
        END
      )
      /
      NULLIF(
        COUNT(*),
        # ...

    ORDER BY customer_id;
  """
)

```

## Arithmetic Mean Versus Weighted Mean

Suppose order-item rows contain:

`AVG(unit_price)` gives each row equal influence.

But to calculate average price per actual unit sold, quantity should determine weight.

The weighted mean is:

The question being asked determines the correct weighting.

## SQL

```
query(  
  ""  
  SELECT  
    ROUND(  
      AVG(unit_price),  
      2  
    ) AS average_item_row_unit_price,  
  
    ROUND(  
      SUM(  
        quantity * unit_price  
      )  
      /  
      NULLIF(  
        1.0 * SUM(quantity),  
        0  
      ),  
      2  
    ) AS quantity_weighted_unit_price  
  
  FROM order_items;  
  ""  
)
```

## SQL

```
quantity  
unit_price
```

## Aggregating Order Items into Orders

`order_items` has grain:

An order can occupy multiple rows.

To create one summary row per order:

The result grain becomes:

SQL

```
query(  
  ""  
  SELECT  
    order_id,  
  
    SUM(quantity)  
      AS total_units,  
  
    SUM(  
      quantity * unit_price  
    ) AS order_value  
  
  FROM order_items  
  
  GROUP BY order_id  
  
  ORDER BY order_id;  
  ""  
)
```

SQL

```
one row per order-product pair
```

## Multi-Stage Aggregation

Suppose we need:

The order-item table is too fine-grained.

We first need:

Then we calculate the average across order rows.

Conceptually:

A CTE makes this multi-stage transformation readable.



## SQL

```
query(  
  ""  
  WITH order_totals AS (  
    SELECT  
      order_id,  
  
      SUM(  
        quantity * unit_price  
      ) AS order_value  
  
    FROM order_items  
  
    GROUP BY order_id  
  )  
  
  SELECT  
    COUNT(*)  
      AS number_of_orders,  
  
    ROUND(  
      AVG(order_value),  
      2  
    ) AS average_order_value,  
  
    MIN(order_value)  
      AS minimum_order_value,  
  
    MAX(order_value)  
      AS maximum_order_value  
  
  FROM order_totals;  
  ""  
)
```

**Correct Aggregate Function, Wrong Grain**

This is technically valid:

But it calculates:

not:

An order containing multiple lines contributes multiple observations.

Therefore:

SQL

```

print("Average line value:")

display(
  query(
    """
    SELECT
      ROUND(
        AVG(
          quantity * unit_price
        ),
        2
      ) AS average_line_value

    FROM order_items;
    """
  )
)

print("\nAverage order value:")

display(
  query(
    """
    WITH order_totals AS (
      SELECT
        order_id,

        SUM(
          # ...
        )
      FROM order_totals;
    """
  )
)

```

SQL

```

AVG(quantity * unit_price)
FROM order_items

```

## Counting Entities at Finer Grain

At order-item grain, `order_id` can appear several times.

Therefore:

counts item rows containing an order ID.

But:

counts distinct orders.

This is appropriate when the metric explicitly asks how many orders are represented.

## SQL

```

query(
    """
    SELECT
        COUNT(*)
        AS item_rows,

        COUNT(order_id)
        AS order_id_values,

        COUNT(DISTINCT order_id)
        AS distinct_orders

    FROM order_items;
    """
)

```

## SQL

```
COUNT(order_id)
```

## DISTINCT Is Not a General Repair Strategy

A dangerous workflow is:

Before deduplicating, understand:

- source grain;
- target grain;
- key uniqueness;
- relationship cardinality;
- why rows repeated.

**DISTINCT** should implement the intended metric, not hide relational mistakes.

## PYTHON

```

too many rows
↓
add DISTINCT
↓
number looks correct

```

## SUM(DISTINCT ...) Has Different Semantics

Suppose amounts are:

Ordinary sum:

Distinct-value sum:

because the repeated numerical value 100 contributes once.

Two legitimate business events can have identical amounts, so distinct-value summation should not be used as arbitrary deduplication.

SQL

```
query(  
  ""  
  WITH amounts(amount) AS (  
    VALUES  
      (100),  
      (100),  
      (200)  
  )  
  
  SELECT  
    SUM(amount)  
    AS ordinary_sum,  
  
    SUM(DISTINCT amount)  
    AS distinct_value_sum  
  
  FROM amounts;  
  ""  
)
```

SQL

```
100  
100  
200
```

## DISTINCT Versus GROUP BY

These may produce similar visible values:

and:

But their intent differs.

**DISTINCT** says:

**GROUP BY** says:

Use the operation that matches the intended reasoning.

SQL

```
SELECT DISTINCT city  
FROM customers;
```

## SQL

```
SELECT city
FROM customers
GROUP BY city;
```

## Grouping Derived Categories

`CASE` can construct analytical groups.

For example, products can be classified into:

price bands before grouped aggregation.

## SQL

```
query(
  """
  SELECT
    CASE
      WHEN price >= 50000
      THEN 'high'

      WHEN price >= 10000
      THEN 'medium'

      ELSE 'low'
    END AS price_band,

    COUNT(*)
    AS product_count,

    ROUND(
      AVG(price),
      2
    ) AS average_price

  FROM products

  GROUP BY
    CASE
      WHEN price >= 50000
      THEN 'high'

  # ...

  ORDER BY average_price DESC;
  """
)
```

## Using a CTE to Simplify Derived Grouping

Repeating a large classification expression can make SQL harder to maintain.

A CTE can first calculate a named derived value.

The outer query then groups by that name.

This is an early example of composing SQL in logical stages.

#### SQL

```
query(  
  ""  
  WITH classified_products AS (  
    SELECT  
      product_id,  
      product_name,  
      price,  
  
      CASE  
        WHEN price >= 50000  
        THEN 'high'  
  
        WHEN price >= 10000  
        THEN 'medium'  
  
        ELSE 'low'  
      END AS price_band  
  
    FROM products  
  )  
  
  SELECT  
    price_band,  
  
    COUNT(*)  
    AS product_count,  
  
    ROUND(  
      # ...  
  
    ORDER BY average_price DESC;  
  ""  
)
```

## Missing-Data Profiling with Aggregates

A completeness report can calculate:

A particularly useful pattern is:

for missing-value counts.

## SQL

```
query(  
  ""  
  SELECT  
    COUNT(*)  
    AS total_customers,  
  
    COUNT(city)  
    AS known_city,  
  
    COUNT(*) - COUNT(city)  
    AS missing_city,  
  
    ROUND(  
      100.0  
      * (  
        COUNT(*)  
        - COUNT(city)  
      )  
      /  
      NULLIF(  
        COUNT(*),  
        0  
      ),  
      2  
    ) AS missing_city_percent  
  
  FROM customers;  
  ""  
)
```

## SQL

```
total rows  
known values  
missing values  
missing percentage
```

## Frequency Tables

Grouping a categorical value and counting rows creates a frequency table.

Frequency tables can reveal:

- common categories;
- rare categories;
- missing categories;
- invalid labels;
- spelling inconsistencies;
- class imbalance.

They are one of the first analyses to perform on categorical data.

SQL

```
query(  
  ""  
  SELECT  
    status,  
  
    COUNT(*)  
    AS frequency  
  
  FROM orders  
  
  GROUP BY status  
  
  ORDER BY  
    frequency DESC,  
    status ASC;  
  ""  
)
```

## Category Proportions

Raw counts can be converted into proportions:

A scalar subquery can provide the total denominator.

Later, window functions will provide another approach.



SQL

```

query(
    """
    SELECT
        status,

        COUNT(*)
        AS frequency,

        ROUND(
            1.0
            * COUNT(*)
            /
            (
                SELECT COUNT(*)
                FROM orders
            ),
            3
        ) AS proportion

    FROM orders

    GROUP BY status

    ORDER BY
        frequency DESC,
        status ASC;
    """
)

```

SQL

```

category count
-----
total rows

```

## Temporal Aggregation

Event data is commonly aggregated into periods such as:

- day;
- week;
- month;
- quarter;
- year.

The following SQLite query groups order dates by year-month.

Other databases use different temporal functions.

## SQL

```
query(  
  ""  
  SELECT  
    STRFTIME(  
      '%Y-%m',  
      order_date  
    ) AS order_month,  
  
    COUNT(*)  
    AS order_count  
  
  FROM orders  
  
  GROUP BY  
    STRFTIME(  
      '%Y-%m',  
      order_date  
    )  
  
  ORDER BY order_month;  
  ""  
)
```

## Detecting Duplicate Business Keys

A standard SQL validation pattern is:

It identifies key values that occur multiple times.

A primary-key-protected production table prevents this state, but raw staging data may not have equivalent constraints.

## SQL

```
query(  
    ""  
    WITH staging_customers(  
        customer_id,  
        name  
    ) AS (  
        VALUES  
        (1, 'Aisha'),  
        (2, 'Rohan'),  
        (2, 'Rohan Duplicate'),  
        (3, 'Meera'),  
        (3, 'Meera Again')  
    )  
  
    SELECT  
        customer_id,  
  
        COUNT(*)  
        AS occurrences  
  
    FROM staging_customers  
  
    GROUP BY customer_id  
  
    HAVING COUNT(*) > 1  
  
    ORDER BY customer_id;  
    ""  
)
```

## Duplicate Keys and Duplicate Rows Are Different

These records:

do not form identical full rows.

But they still violate a rule saying customer ID 2 should uniquely identify one customer.

Therefore full-row deduplication and business-key validation solve different problems.

## PYTHON

```
2 | Rohan  
2 | Different Name
```

## Aggregate Data-Quality Checks

Aggregation can efficiently summarize quality conditions.

Useful statistics include:

- row counts;

- missing counts;
- duplicate-key counts;
- minimum values;
- maximum values;
- unexpected category counts;
- invalid-range counts.

These summaries can become automated pipeline checks.

SQL

```
query(  
  ""  
  SELECT  
    COUNT(*)  
      AS item_rows,  
  
    SUM(quantity)  
      AS total_units,  
  
    MIN(quantity)  
      AS minimum_quantity,  
  
    MAX(quantity)  
      AS maximum_quantity  
  
  FROM order_items;  
  ""  
)
```

## Counting Invalid Records

Conditional aggregation can count records violating specific rules.

Pattern:

This can support data-quality dashboards and monitoring.

## SQL

```
query(  
  ""  
  SELECT  
    COUNT(*)  
    AS product_rows,  
  
    SUM(  
      CASE  
        WHEN price < 0  
        THEN 1  
        ELSE 0  
      END  
    ) AS negative_price_rows,  
  
    SUM(  
      CASE  
        WHEN product_name IS NULL  
        THEN 1  
        ELSE 0  
      END  
    ) AS missing_product_name_rows  
  
  FROM products;  
  ""  
)
```

## Target Distribution for Machine Learning

Classification targets should be profiled before modeling.

A common SQL pattern is:

This can reveal:

- severe class imbalance;
- invalid target labels;
- missing labels;
- labeling-pipeline failures.

A model trained on a 99-to-1 class distribution requires different evaluation reasoning from a balanced problem.

## SQL

```
SELECT  
  target,  
  COUNT(*)  
FROM training_data  
GROUP BY target;
```

## Customer-Level Feature Aggregation

Suppose a model requires:

The orders table has:

A customer can have multiple orders.

Aggregation can transform order history into customer-level features such as:

- total orders;
- completed orders;
- pending orders;
- cancelled orders;
- completion rate.

Customers with no orders do not appear in this result yet. Part 4 will use joins to preserve the complete customer population.

## SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
  
    COUNT(*)  
      AS order_count,  
  
    SUM(  
      CASE  
        WHEN status = 'completed'  
        THEN 1  
        ELSE 0  
      END  
    ) AS completed_order_count,  
  
    SUM(  
      CASE  
        WHEN status = 'pending'  
        THEN 1  
        ELSE 0  
      END  
    ) AS pending_order_count,  
  
    SUM(  
      CASE  
        WHEN status = 'cancelled'  
        THEN 1  
      # ...  
    )  
  
    ORDER BY customer_id;  
  ""  
)
```

## SQL

one **row** per customer

## Aggregation and Temporal Leakage

The previous feature query uses every currently available order.

For current reporting, that can be fine.

For historical model training, it can be invalid.

Suppose a training example represents a prediction made on March 6.

An order occurring on March 10 must not contribute to historical features for that example.

Therefore:

## PYTHON

```
correct aggregate
+
wrong observation window
=
data leakage
```

## Time-Bounded Aggregate Features

A global historical cutoff can be applied before grouping:

The resulting aggregate uses only events occurring before the cutoff.

## SQL

```
prediction_cutoff = "2026-03-06"

query(
    """
    SELECT
        customer_id,

        COUNT(*)
        AS historical_order_count,

        SUM(
            CASE
                WHEN status = 'completed'
                THEN 1
                ELSE 0
            END
        ) AS historical_completed_orders

    FROM orders

    WHERE order_date < ?

    GROUP BY customer_id

    ORDER BY customer_id;
    """,
    params=(prediction_cutoff,),
)
```

## SQL

```
WHERE order_date < ?
```

## Per-Observation Prediction Times

A single cutoff is relatively simple.

Real historical training sets may contain prediction rows at many times:



Each feature row may require its own historical event window.

This requires point-in-time SQL reasoning rather than one global filter.

We will return to this after joins and advanced querying are established.

#### PYTHON

```
customer A → March 1
customer B → March 5
customer C → March 10
```

## Aggregates Can Hide Distribution Shape

Two groups can have the same average while having very different underlying observations.

Example:

Both means are 100.

But their distributions are clearly different.

Depending on the analysis, also inspect:

- sample count;
- minimum and maximum;
- variance;
- standard deviation;
- quantiles;
- histograms;
- plots.

An average is a summary, not the full distribution.

#### PYTHON

```
Group A:
95, 100, 105

Group B:
0, 0, 300
```

## Aggregation Does Not Automatically Guarantee Privacy

Aggregate output usually reveals less row-level detail, but aggregation alone is not a privacy guarantee.

A group containing one person can reveal individual information.

Small groups can also become identifiable when combined with external data.

Production systems may require:

- minimum group sizes;
- suppression;
- permissions;
- anonymization;
- differential privacy or related methods.

SQL aggregation is a data transformation, not a complete privacy mechanism.

## GROUP BY Versus Window Functions

Ordinary `GROUP BY` usually collapses rows.

Example:

Window functions can calculate category-level context while preserving:

Conceptually:

Window functions receive a dedicated section later.

The distinction deserves a concrete example.

A grouped query:

produces category-level rows.

The individual products disappear from that result because they were collapsed into their groups.

A window aggregate can instead produce:

for every product.

The original row grain remains one row per product.

PYTHON

one row per product



GROUP BY category



one row per category

PYTHON

one row per product

## SQLite's Permissive Grouping Behavior

SQLite can accept grouped queries that select fields which are neither grouped nor aggregated.

This should not be treated as a general correctness rule.

For example:

A category can contain several product names.

There is no ordinary single `product_name` representing the group.

Write grouped queries so every output field has clear meaning at the result grain.

SQL

```
SELECT
  category,
  product_name,
  AVG(price)
FROM products
GROUP BY category;
```

### Prefer WHERE for Row-Level Conditions

Use `WHERE` when a condition determines which individual rows enter aggregation.

Use `HAVING` when a condition applies to groups or aggregate values.

This distinction makes the query easier to understand and often enables the database to reduce input earlier.

### Aggregate Aliases in HAVING

Alias visibility differs across database systems.

For portable foundational SQL:

is preferable to relying on:

unless the target database's alias rules are known.

Understanding the underlying aggregate expression is the primary skill.

PYTHON

```
HAVING AVG(price) > 5000
```

### GROUP BY Does Not Imply ORDER BY

Grouped output may appear sorted during testing.

Do not rely on this.

Grouping defines analytical partitions.

Ordering defines presentation sequence.

Use explicit `ORDER BY` whenever order matters.

## Aggregation Collapses Detail

Once many detail rows are reduced into one aggregate row, individual detail is not present separately in that result.

For example:

contains customer totals, not the original transaction rows.

If both detailed records and group-level context are needed simultaneously, window functions may be more suitable.

Do not aggregate too early if downstream logic still requires detailed grain.

SQL

```
SELECT
  customer_id,
  SUM(amount)
FROM transactions
GROUP BY customer_id;
```

## Common Aggregation Mistakes

Important mistakes include:

- confusing `COUNT(*)` with `COUNT(column)`;
- forgetting that many aggregates ignore NULL;
- replacing missing values with zero without justification;
- summing identifiers because they are numeric;
- selecting ambiguous detail columns in grouped queries;
- grouping at the wrong grain;
- calculating average line value when average order value was required;
- using `COUNT(DISTINCT ...)` to hide structural mistakes;
- using `SUM(DISTINCT ...)` as arbitrary deduplication;
- placing aggregate predicates in `WHERE`;
- using `HAVING` as a generic substitute for `WHERE`;
- assuming `GROUP BY` provides ordering;
- accidentally using integer division in rates;
- failing to handle zero denominators;
- including future events in historical ML features;
- interpreting aggregate NULL as automatically equal to zero;
- relying on averages without understanding distributions.

The strongest defense is to define the metric and result grain before writing SQL.

## Aggregation and Grouped Analysis Summary

This part developed aggregate SQL from first principles.

The major concepts included:

- aggregate functions;
- `COUNT(*)`;
- `COUNT(expression)`;
- `COUNT(DISTINCT expression)`;
- `COUNT(1)` awareness;
- `SUM`;
- `AVG`;
- `MIN`;
- `MAX`;
- aggregating expressions;
- aggregate NULL semantics;
- all-NULL inputs;
- empty aggregate populations;
- `COALESCE` placement;
- filtering before aggregation;
- `GROUP BY`;
- grouping grain;
- multiple grouping keys;
- NULL grouping;
- expression grouping;
- grouped-query selection rules;
- `WHERE` versus `HAVING`;
- multiple HAVING conditions;
- grouped sorting;
- top-group tie semantics;
- conditional aggregation;
- `SUM(CASE ...)`;
- `COUNT(CASE ...)`;
- aggregate `FILTER` awareness;
- rates;
- denominator protection;
- weighted averages;

- order-item to order aggregation;
- multi-stage aggregation;
- wrong-grain metrics;
- distinct entity counting;
- `SUM(DISTINCT ...)` ;
- `DISTINCT` versus `GROUP BY` ;
- derived groups;
- CTE-assisted grouping;
- missing-data profiling;
- categorical frequency tables;
- category proportions;
- temporal aggregation;
- duplicate-key detection;
- data-quality aggregation;
- ML target inspection;
- customer-level feature engineering;
- temporal leakage;
- cutoff-based features;
- point-in-time awareness;
- limitations of averages;
- privacy considerations;
- aggregation versus window functions;
- SQLite grouping portability.

The next part develops joins comprehensively:

## PYTHON

```
INNER JOIN
LEFT JOIN
CROSS JOIN
self joins
join predicates
join cardinality
one-to-one
one-to-many
many-to-many
row multiplication
ON versus WHERE
semi joins
anti joins
multi-table joins
aggregation after joins
referential validation
AI feature-table construction
```

## Joining Relational Data

Relational databases deliberately separate different kinds of facts into different tables.

Our learning database contains:

A customer row stores customer facts.

An order row stores order facts.

An order-item row records which products belong to an order.

A product row stores product facts.

Real analytical questions usually require information from several of these relations.

For example:

Joins combine related rows from multiple relational sources.

Join syntax is not particularly difficult. Join reasoning is.

A query can be syntactically valid while returning too many rows, silently discarding entities, duplicating monetary values, or changing an ML dataset's grain.

The most important join concepts are therefore:

Before writing a serious join, you should be able to state what one row represents on each side and what one result row is intended to represent.

## PYTHON

```
customers
products
orders
order_items
```

## The Join Mental Model

A join compares rows from two relational inputs according to a condition.

Suppose:

and:

The relationship is:

A join can combine matching facts:

Notice that Aisha appears twice because she has two orders.

The join did not accidentally duplicate Aisha. The result grain changed.

This is why understanding cardinality is essential.

## PYTHON

```
customers

customer_id | name
-----|-----
1           | Aisha
2           | Rohan
```

## PYTHON

```
orders

order_id | customer_id
-----|-----
1001     | 1
1002     | 2
1003     | 1
```

## Define Input Grain Before Joining

Our important table grains are:

If customers are joined to orders:

the output naturally becomes approximately:

If that result is joined to order items, it becomes finer again:



The data did not become "duplicated" in a generic sense. The entity represented by one row changed.

## PYTHON

```
customers
→ one row per customer

orders
→ one row per order

order_items
→ one row per order-product pair

products
→ one row per product
```

## PYTHON

```
one customer
can match
many orders
```

## INNER JOIN

An **INNER JOIN** returns combinations of rows satisfying the join condition.

General form:

The word **INNER** is commonly optional:

normally means inner join.

Rows without a matching partner are excluded.

## SQL

```
query(
    """
    SELECT
        o.order_id,
        o.customer_id,
        c.name,
        o.order_date,
        o.status

    FROM orders AS o

    INNER JOIN customers AS c
        ON o.customer_id = c.customer_id

    ORDER BY o.order_id;
    """
)
```

## Table Aliases

Join queries frequently use aliases:

Then:

clearly identifies which table supplies each column.

Aliases are especially useful when:

- several tables have similarly named columns;
- queries become long;
- a table is joined to itself;
- subqueries and CTEs are involved.

Choose short aliases that remain understandable.

PYTHON

```
orders AS o
customers AS c
```

PYTHON

```
o.customer_id
c.customer_id
```

## Qualified Column Names

Both `customers` and `orders` contain:

A query selecting simply:

can therefore be ambiguous.

A qualified reference is explicit:

Even when the database can infer an unqualified column, explicit qualification often improves complex-query readability.

PYTHON

```
customer_id
```

PYTHON

```
customer_id
```

## INNER JOIN Removes Unmatched Rows

Customers Sara and Zoya have no orders in the learning data.

If customers are inner-joined to orders, they disappear because there is no matching order row.

This is not a bug. It is the definition of an inner join.

The key question is whether that behavior matches the analytical requirement.

#### SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name,  
    o.order_id  
  
  FROM customers AS c  
  
  INNER JOIN orders AS o  
    ON c.customer_id = o.customer_id  
  
  ORDER BY  
    c.customer_id,  
    o.order_id;  
  ""  
)
```

## LEFT JOIN

A **LEFT JOIN** preserves every row from the left input.

Matching right-side data is added when available.

When no right-side row matches, right-side result columns become NULL.

Conceptually:

This is extremely important when the left table represents a complete population that must not be silently reduced.

Sara and Zoya remain in the result even though they have no orders.

For their unmatched result rows:

Those NULL values were introduced by the outer join. They do not mean an order exists with missing fields.

## SQL

```

query(
    """
    SELECT
        c.customer_id,
        c.name,
        o.order_id,
        o.status

    FROM customers AS c

    LEFT JOIN orders AS o
        ON c.customer_id = o.customer_id

    ORDER BY
        c.customer_id,
        o.order_id;
    """
)

```

## SQL

```

LEFT TABLE
every row survives
    ↓
matching right data added
    ↓
no match → right fields NULL

```

**LEFT JOIN Does Not Guarantee One Row per Left Row**

A common misconception is:

That is false.

A left row with three matching right rows produces three result rows.

A left row with no match still produces one NULL-extended result row.

For customers and orders:

A left join preserves left-side membership, not left-side row count.

## PYTHON

```

LEFT JOIN preserves left rows
therefore
one left row stays one result row

```

## PYTHON

```
customer with 2 orders → 2 rows
customer with 1 order  → 1 row
customer with 0 orders → 1 NULL-extended row
```

**RIGHT JOIN**

A **RIGHT JOIN** is conceptually the mirror image of a left join.

It preserves every row from the right input.

Many databases support it.

A right join can often be rewritten as a left join by swapping table positions:

can conceptually become:

SQLite support for right/full joins depends on SQLite version. Because left joins are broadly understood and sufficient to express the conceptual behavior by swapping inputs, this notebook uses left joins for portable foundational reasoning.

## PYTHON

```
A RIGHT JOIN B
```

**FULL OUTER JOIN**

A full outer join preserves unmatched rows from both inputs.

Conceptually:

PostgreSQL and several other database systems support **FULL OUTER JOIN**.

SQLite support depends on version.

The key learning objective is the row-preservation model rather than memorizing one dialect's availability.

## PYTHON

```
matched left/right rows
+
unmatched left rows
+
unmatched right rows
```

**The ON Clause**

**ON** defines how rows relate.

Example:

The join condition is part of the relational meaning.

An incomplete condition can cause severe row multiplication.

A missing condition can produce a Cartesian product.

A condition using the wrong keys can associate unrelated entities.

Join predicates therefore deserve the same care as primary-key and foreign-key design.

PYTHON

```
ON o.customer_id = c.customer_id
```

### Similar Column Names Do Not Automatically Form a Join Key

Two columns should not be joined merely because:

- they share a name;
- both are integers;
- their values appear similar in a small sample.

A valid join requires domain meaning.

For example:

and:

represent the same customer-identity domain.

By contrast, joining:

to:

would be semantically meaningless even though both are integers.

PYTHON

```
orders.customer_id
```

### Join Cardinality

Join cardinality describes how rows on one side can correspond to rows on the other.

Important patterns are:

Before joining, ask:

These questions often matter more than the join keyword itself.

## PYTHON

one-to-one  
one-to-many  
many-to-one  
many-to-many

## PYTHON

Is the left key unique?  
Is the right key unique?  
How many matches can one left row have?  
How many matches can one right row have?  
What result row count should I expect?

## One-to-One Join

In a true one-to-one relationship:

A successful match contributes at most one combined row for each entity.

One-to-one joins generally do not multiply matched entities.

However, the relationship is one-to-one only if uniqueness is actually enforced or validated.

## PYTHON

one left key  
↕  
at most one right key

## One-to-Many Join

Customers to orders is one-to-many:

A customer with multiple orders naturally appears multiple times after the join.

This has major implications when customer-level attributes such as age, income, or account balance are carried into the joined result.

Those values repeat because the result is now at order grain.

## SQL

```
query(  
    """  
    SELECT  
        c.customer_id,  
        c.name,  
        o.order_id,  
        o.status  
  
    FROM customers AS c  
  
    JOIN orders AS o  
        ON c.customer_id = o.customer_id  
  
    WHERE c.customer_id IN (1, 2)  
  
    ORDER BY  
        c.customer_id,  
        o.order_id;  
    """  
)
```

## Many-to-One Join

From the opposite perspective:

Joining orders to customers is many-to-one.

When the customer primary key is unique, each order receives at most one customer match.

The same relationship can therefore be described as one-to-many or many-to-one depending on which table is treated as the starting side.

## PYTHON

```
many orders  
→ one customer
```

## Row Multiplication

Suppose a customer has:

and each order contains:

Joining:

can create:

for that customer.

If a customer-level monetary value is then summed after this join, it may be counted repeatedly.

This is one of the most important join hazards in analytics:



PYTHON

`2 orders`

PYTHON

`3 item rows`

## Measure Row Counts Before and After Joins

A practical debugging habit is to measure row counts and key uniqueness before and after a join.

For our learning tables:

The customer-order inner join does not necessarily contain 7 customer-level entities. It contains one row per matched order because that is the resulting grain.

SQL

```
query(  
    """  
    SELECT  
        (SELECT COUNT(*) FROM customers)  
        AS customer_rows,  
  
        (SELECT COUNT(*) FROM orders)  
        AS order_rows,  
  
        (  
            SELECT COUNT(*)  
            FROM customers AS c  
            JOIN orders AS o  
            ON c.customer_id = o.customer_id  
        ) AS joined_rows;  
    """  
)
```

## Row Count Versus Distinct Entity Count

After a one-to-many join, these are different:

and:

The first counts result rows.

The second counts represented customers.

Neither is universally "more correct." They answer different questions.

SQL

```
query(  
  ""  
  SELECT  
    COUNT(*)  
    AS join_rows,  
  
    COUNT(  
      DISTINCT c.customer_id  
    ) AS represented_customers  
  
  FROM customers AS c  
  
  JOIN orders AS o  
    ON c.customer_id = o.customer_id;  
  ""  
)
```

SQL

```
COUNT(*)
```

## CROSS JOIN

A cross join produces every combination of rows from two inputs.

If table A contains:

and table B contains:

their cross join contains:

This is called a Cartesian product.

Cross joins are useful when every combination is genuinely required, such as:

- scenario grids;
- date × category templates;
- experiment parameter combinations.

They are dangerous when produced accidentally.

SQL

```
query(  
  ""  
  WITH colors(color) AS (  
    VALUES  
      ('Red'),  
      ('Blue')  
  ),  
  
  sizes(size) AS (  
    VALUES  
      ('S'),  
      ('M'),  
      ('L')  
  )  
  
  SELECT  
    color,  
    size  
  
  FROM colors  
  
  CROSS JOIN sizes  
  
  ORDER BY  
    color,  
    size;  
  ""  
)
```

SQL

m rows

## Cartesian Product Size

Two colors × three sizes produces:

result rows.

At large scale, accidental Cartesian products can explode extremely quickly.

## SQL

```

query(
    """
    WITH left_side(value) AS (
        VALUES
            (1),
            (2),
            (3)
    ),

    right_side(value) AS (
        VALUES
            ('A'),
            ('B'),
            ('C'),
            ('D')
    )

    SELECT
        COUNT(*) AS cartesian_rows

    FROM left_side

    CROSS JOIN right_side;
    """
)

```

## Implicit Comma Joins

Older SQL often uses:

This conceptually begins from a Cartesian combination and filters according to the relationship.

Modern explicit join syntax:

is generally clearer because the relationship condition appears directly with the join.

Explicit joins reduce the chance of accidentally forgetting a relationship predicate.

## PYTHON

```

FROM customers c, orders o
WHERE c.customer_id = o.customer_id

```

## PYTHON

```

FROM customers AS c
JOIN orders AS o
    ON c.customer_id = o.customer_id

```

## Self Join

A self join joins a relation to itself.

Aliases are essential because SQL needs to distinguish the two logical roles.

Common uses include:

- employee-manager relationships;
- comparing records within one table;
- hierarchical data;
- finding pairs.

We will build a small employee hierarchy for demonstration.

In the self join:

means the employee role.

means the manager role.

Both aliases refer to the same underlying relation but represent different logical participants in the join.

SQL

```
query(  
  ""  
  WITH employees(  
    employee_id,  
    employee_name,  
    manager_id  
  ) AS (  
    VALUES  
    (1, 'Aisha', NULL),  
    (2, 'Rohan', 1),  
    (3, 'Meera', 1),  
    (4, 'Kabin', 2)  
  )  
  
  SELECT  
    e.employee_id,  
    e.employee_name,  
  
    m.employee_name  
      AS manager_name  
  
  FROM employees AS e  
  
  LEFT JOIN employees AS m  
    ON e.manager_id = m.employee_id  
  
  ORDER BY e.employee_id;  
  ""  
)
```

SQL

e

## ON Versus WHERE in Outer Joins

This is one of the most important join subtleties.

Suppose we want:

A condition on the optional right-side table can be placed in **ON** :

The customer population is preserved.

Customers with no completed order still remain, with NULL order fields.

SQL

```
query(
  """
  SELECT
    c.customer_id,
    c.name,
    o.order_id,
    o.status

  FROM customers AS c

  LEFT JOIN orders AS o
    ON c.customer_id = o.customer_id
    AND o.status = 'completed'

  ORDER BY
    c.customer_id,
    o.order_id;
  """
)
```

## Right-Side Filtering in WHERE Can Remove Unmatched Rows

Now move the same condition into **WHERE** :

Customers with no order receive:

Then:

is unknown, so those rows do not survive **WHERE** .

The outer-join preservation has therefore been undermined by the later filter.

These two queries answer different questions.

Condition inside **ON** :

Condition inside `WHERE` :

This distinction is extremely important in feature engineering and reporting.

SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name,  
    o.order_id,  
    o.status  
  
  FROM customers AS c  
  
  LEFT JOIN orders AS o  
    ON c.customer_id = o.customer_id  
  
  WHERE o.status = 'completed'  
  
  ORDER BY  
    c.customer_id,  
    o.order_id;  
  ""  
)
```

SQL

```
LEFT JOIN orders AS o  
  ON c.customer_id = o.customer_id  
  
WHERE o.status = 'completed'
```

## COUNT(\*) after a LEFT JOIN

Suppose we left join customers to orders and group by customer.

It may seem natural to write:

for order count.

But every left-side customer contributes at least one result row, even when no order matched.

For an unmatched customer, that row is NULL-extended.

Therefore `COUNT(*)` would incorrectly report at least 1.

Instead, count a non-NULL right-side key:

Then customers with no orders correctly receive zero.

SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name,  
  
    COUNT(*)  
      AS joined_row_count,  
  
    COUNT(o.order_id)  
      AS actual_order_count  
  
  FROM customers AS c  
  
  LEFT JOIN orders AS o  
    ON c.customer_id = o.customer_id  
  
  GROUP BY  
    c.customer_id,  
    c.name  
  
  ORDER BY c.customer_id;  
  ""  
)
```

SQL

```
COUNT(*)
```

## Preserving a Complete Feature Population

This pattern is fundamental in ML feature construction.

Suppose the prediction population is:

Start from `customers`.

Then left join historical events such as orders.

Aggregate right-side keys rather than counting the outer-join placeholder row.

The resulting dataset can maintain:

including customers with zero activity.



## SQL

```
query(  
    """  
    SELECT  
        c.customer_id,  
        c.name,  
  
        COUNT(o.order_id)  
        AS order_count,  
  
        SUM(  
            CASE  
                WHEN o.status = 'completed'  
                THEN 1  
                ELSE 0  
            END  
        ) AS completed_order_count  
  
    FROM customers AS c  
  
    LEFT JOIN orders AS o  
        ON c.customer_id = o.customer_id  
  
    GROUP BY  
        c.customer_id,  
        c.name  
  
    ORDER BY c.customer_id;  
    """  
)
```

**Conditional Aggregation after a LEFT JOIN**

The previous `CASE` uses:

so customers with no matching order receive zero completed orders.

This is semantically useful because:

is naturally represented as count 0.

Not every missing metric should be transformed to zero; the choice depends on the domain.

## PYTHON

```
ELSE 0
```

## PYTHON

```
no completed order events
```

## Anti Join Pattern

An anti join answers:

SQL does not universally use an `ANTI JOIN` keyword.

A classic pattern is:

For example:

SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name  
  
  FROM customers AS c  
  
  LEFT JOIN orders AS o  
    ON c.customer_id = o.customer_id  
  
  WHERE o.order_id IS NULL  
  
  ORDER BY c.customer_id;  
  ""  
)
```

SQL

Which `left`-side `rows` have `no` matching `right`-side `row`?

## NOT EXISTS as an Anti-Match Pattern

Another important pattern is:

This asks whether no related row exists.

It avoids the NULL trap associated with some `NOT IN` queries and often communicates anti-match intent very clearly.

Subqueries and `EXISTS` receive dedicated coverage later, but the relational meaning is worth introducing here.

Both previous anti-match queries return customers without orders.

The appropriate form depends on:

- readability;
- database optimizer;
- surrounding query structure;

- NULL semantics;
- team conventions.

Do not select an anti-join strategy based solely on folklore about which syntax is always faster. Inspect actual query plans on the target database.

SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name  
  
  FROM customers AS c  
  
  WHERE NOT EXISTS (  
    SELECT 1  
    FROM orders AS o  
    WHERE o.customer_id = c.customer_id  
  )  
  
  ORDER BY c.customer_id;  
  ""  
)
```

## Semi-Join Pattern

A semi join asks:

But it returns left entities rather than every matching pair.

For example:

Using an inner join can multiply customers who have multiple matching orders.

**EXISTS** naturally expresses existence without multiplication.

SQL

```

query(
  """
  SELECT
    c.customer_id,
    c.name

  FROM customers AS c

  WHERE EXISTS (
    SELECT 1

    FROM orders AS o

    WHERE
      o.customer_id = c.customer_id
      AND o.status = 'completed'
  )

  ORDER BY c.customer_id;
  """
)

```

SQL

Which **left-side rows** have **at least one match**?

## INNER JOIN + DISTINCT Versus EXISTS

You may see:

used to obtain customers with matches.

This can be logically valid, but **EXISTS** often expresses the question more directly:

The choice should follow intent and measured performance.

SQL

```

SELECT DISTINCT
  c.customer_id,
  c.name
FROM customers AS c
JOIN orders AS o
  ON ...

```

SQL

Does **at least one matching order** exist?

## Products with No Order Items

The same anti-match pattern can find products never referenced by order items.

This is useful for:

- catalog analysis;
- quality validation;
- inventory analysis;
- cold-start ML analysis.

SQL

```
query(  
  ""  
  SELECT  
    p.product_id,  
    p.product_name  
  
  FROM products AS p  
  
  LEFT JOIN order_items AS oi  
    ON p.product_id = oi.product_id  
  
  WHERE oi.product_id IS NULL  
  
  ORDER BY p.product_id;  
  ""  
)
```

## Joining Multiple Tables

SQL can combine more than two relations.

Suppose we need:

The relationship path is:

At each step, grain must be tracked.

## SQL

```
query(  
  ""  
  SELECT  
    o.order_id,  
  
    c.customer_id,  
    c.name AS customer_name,  
  
    p.product_id,  
    p.product_name,  
  
    oi.quantity,  
    oi.unit_price,  
  
    oi.quantity * oi.unit_price  
    AS line_value  
  
  FROM orders AS o  
  
  JOIN customers AS c  
    ON o.customer_id = c.customer_id  
  
  JOIN order_items AS oi  
    ON o.order_id = oi.order_id  
  
  JOIN products AS p  
    ON oi.product_id = p.product_id  
  
  ORDER BY  
    o.order_id,  
    p.product_id;  
  ""  
)
```

## SQL

```
order  
customer  
product  
quantity  
historical unit price
```

## Track Grain through Multi-Table Joins

The previous query starts at:

After joining customers:

because each order has one customer.

After joining order items:

because each order can contain several items.

Joining products does not further multiply rows when `product_id` uniquely identifies one product.

At the end, one row represents:

This type of grain tracing should become automatic.

PYTHON

```
orders
→ one row per order
```

PYTHON

```
still approximately one row per order
```

## Aggregation after Joins

Once the joined grain is understood, aggregation can deliberately return to a coarser grain.

For example:

SQL

```
query(
    """
    SELECT
        o.order_id,
        c.name AS customer_name,

        SUM(
            oi.quantity * oi.unit_price
        ) AS order_value

    FROM orders AS o

    JOIN customers AS c
        ON o.customer_id = c.customer_id

    JOIN order_items AS oi
        ON o.order_id = oi.order_id

    GROUP BY
        o.order_id,
        c.name

    ORDER BY
        o.order_id;
    """
)
```

## Customer-Level Spend

We can aggregate further to customer grain.

But a business decision appears:

SQL cannot answer this automatically.

The query must encode the metric definition.

SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name,  
  
    SUM(  
      CASE  
        WHEN o.status = 'completed'  
        THEN oi.quantity * oi.unit_price  
        ELSE 0  
      END  
    ) AS completed_spend  
  
  FROM customers AS c  
  
  LEFT JOIN orders AS o  
    ON c.customer_id = o.customer_id  
  
  LEFT JOIN order_items AS oi  
    ON o.order_id = oi.order_id  
  
  GROUP BY  
    c.customer_id,  
    c.name  
  
  ORDER BY c.customer_id;  
  ""  
)
```

SQL

Should cancelled orders contribute **to** spend?

## Counting Orders after Joining Order Items

After joining orders to order items, one order can occupy multiple rows.

Therefore:

can count order-item rows rather than unique orders.



If the metric is order count:

may be necessary at this joined grain.

Again, DISTINCT is justified by metric semantics, not by a desire to make a number smaller.

SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name,  
  
    COUNT(o.order_id)  
      AS order_id_occurrences_after_item_join,  
  
    COUNT(DISTINCT o.order_id)  
      AS distinct_order_count  
  
  FROM customers AS c  
  
  LEFT JOIN orders AS o  
    ON c.customer_id = o.customer_id  
  
  LEFT JOIN order_items AS oi  
    ON o.order_id = oi.order_id  
  
  GROUP BY  
    c.customer_id,  
    c.name  
  
  ORDER BY c.customer_id;  
  ""  
)
```

SQL

```
COUNT(o.order_id)
```

## Double Counting after Joins

Suppose an `orders` table stored a precomputed `order_total`.

If each order is then joined to several order-item rows and the query calculates:

the same order total can be repeated once per item.

This is a classic double-counting bug.

Possible solutions depend on the metric and schema:

- aggregate items before joining;

- aggregate each table to the desired grain first;
- select one row per entity before summing;
- avoid carrying coarse-grain measures into finer-grain sums.

There is no universal `DISTINCT` patch because two legitimate orders can have identical totals.

```
PYTHON
```

```
SUM(order_total)
```

## Pre-Aggregation before Joining

A powerful pattern is:

For example, order items can first become one row per order.

That summary can then be joined to orders and customers without item-level row multiplication.

Suppose we need:

but transactions contain many rows per customer.

A safe pattern is:

rather than joining several detail tables first and aggregating afterward.

Pre-aggregation can prevent row multiplication and often improves performance.

## SQL

```

query(
  """
  WITH order_totals AS (
    SELECT
      order_id,

      SUM(
        quantity * unit_price
      ) AS order_value

    FROM order_items

    GROUP BY order_id
  )

  SELECT
    o.order_id,
    c.customer_id,
    c.name,
    o.status,
    ot.order_value

  FROM orders AS o

  JOIN customers AS c
    ON o.customer_id = c.customer_id

  JOIN order_totals AS ot
    ON o.order_id = ot.order_id

  ORDER BY o.order_id;
  """
)

```

## SQL

```

fine-grained events
  ↓
aggregate to intended entity grain
  ↓
join the summaries

```

## Pre-Aggregating to Customer Grain

The previous result has one row per order.

We can aggregate those order-level values into one row per customer before joining to the complete customer population.

This pattern makes grain transitions explicit:

## SQL

```
query(  
  ""  
  WITH order_totals AS (  
    SELECT  
      order_id,  
  
      SUM(  
        quantity * unit_price  
      ) AS order_value  
  
    FROM order_items  
  
    GROUP BY order_id  
  ),  
  
  customer_order_features AS (  
    SELECT  
      o.customer_id,  
  
      COUNT(*) AS order_count,  
  
      SUM(  
        CASE  
          WHEN o.status = 'completed'  
          THEN 1  
          ELSE 0  
        END  
      ) AS completed_order_count,  
      # ...  
  
    ORDER BY c.customer_id;  
  ""  
)
```

## SQL

```
items  
→ orders  
→ customers
```

## Grain-Safe Feature Table Construction

The previous query is a strong ML/data-engineering pattern.

The target population is:

Order items are first aggregated into:

Orders are then aggregated into:

Only then is the customer-level summary joined to the complete customer population.

This reduces the chance of accidental feature duplication.

A useful general pattern is:

PYTHON

```
customers  
→ one row per customer
```

## Joining on Composite Keys

Some relationships require more than one join column.

Suppose a table's logical identity is:

Joining only on:

can combine records from different dates.

The correct relationship may require:

Incomplete composite-key joins are a common source of silent row multiplication.

## SQL

```
query(  
  ""  
  WITH left_data(  
    customer_id,  
    snapshot_date,  
    feature_value  
  ) AS (  
    VALUES  
    (1, '2026-03-01', 10),  
    (1, '2026-03-02', 20),  
    (2, '2026-03-01', 30)  
  ),  
  
  right_data(  
    customer_id,  
    snapshot_date,  
    label  
  ) AS (  
    VALUES  
    (1, '2026-03-01', 0),  
    (1, '2026-03-02', 1),  
    (2, '2026-03-01', 1)  
  )  
  
  SELECT  
    l.customer_id,  
    l.snapshot_date,  
    l.feature_value,  
    # ...  
    l.customer_id,  
    l.snapshot_date;  
  ""  
)
```

## SQL

```
(customer_id, snapshot_date)
```

## What Happens When Part of a Composite Key Is Omitted?

If the previous datasets are joined using only:

customer 1 has two rows on each side.

The join can produce:

customer-1 combinations.

This is not a database malfunction. It is the logical result of the supplied join condition.

## SQL

```
query(  
  ""  
  WITH left_data(  
    customer_id,  
    snapshot_date,  
    feature_value  
  ) AS (  
    VALUES  
    (1, '2026-03-01', 10),  
    (1, '2026-03-02', 20)  
  ),  
  
  right_data(  
    customer_id,  
    snapshot_date,  
    label  
  ) AS (  
    VALUES  
    (1, '2026-03-01', 0),  
    (1, '2026-03-02', 1)  
  )  
  
  SELECT  
    l.customer_id,  
    l.snapshot_date  
    AS feature_date,  
    r.snapshot_date  
    AS label_date,  
    # ...  
    feature_date,  
    label_date;  
  ""  
)
```

## SQL

```
customer_id
```

## NULL Join Keys

An equality join condition such as:

does not normally match NULL to NULL.

Why?

Because:

is not true under ordinary SQL equality semantics.

This can matter when joining dirty or incomplete data.

Do not automatically replace NULL keys with arbitrary sentinels merely to force matching. Missing identifiers often mean the relationship itself is unknown.

SQL

```
query(
  """
  WITH left_data(
    id,
    join_key
  ) AS (
    VALUES
      (1, 10),
      (2, NULL)
  ),

  right_data(
    id,
    join_key
  ) AS (
    VALUES
      (101, 10),
      (102, NULL)
  )

  SELECT
    l.id AS left_id,
    r.id AS right_id,
    l.join_key

  FROM left_data AS l

  LEFT JOIN right_data AS r
    ON l.join_key = r.join_key

  ORDER BY l.id;
  """
)
```

## NULL-Safe Join Conditions

Some systems provide NULL-safe comparison semantics such as:

or dialect-specific operators.

Whether NULL should match NULL is a domain decision.

For an identifier, two unknown IDs usually should not automatically be treated as the same entity merely because both are missing.

NULL-safe joins are powerful but must be used intentionally.



## PYTHON

`IS NOT DISTINCT FROM`

## NATURAL JOIN Awareness

SQL includes constructs such as `NATURAL JOIN` in some databases.

A natural join automatically identifies same-named columns as join columns.

This can look convenient, but it creates fragile coupling to column names.

If a new same-named column appears later, the join semantics can change.

For production-quality analytical SQL, explicit `ON` conditions are generally easier to review and safer to maintain.

## USING Clause Awareness

When both tables use the same column name for a relationship, some databases support:

This can be concise.

However, `ON` is more flexible and makes complex multi-column or differently named relationships explicit:

You should be comfortable reading both forms.

## SQL

```
query(  
    """  
    SELECT  
        order_id,  
        customer_id,  
        name  
  
    FROM orders  
  
    JOIN customers  
    USING (customer_id)  
  
    ORDER BY order_id;  
    """  
)
```

## SQL

```
JOIN ...  
USING (customer_id)
```

## Referential Integrity Validation with Joins

Even when a database does not enforce foreign-key constraints, joins can identify orphan records.

An orphan order is an order whose customer key does not exist in the customer table.

A common validation pattern is:

The result is empty because the learning schema enforces its foreign key correctly.

The same anti-join pattern is extremely useful when validating raw staging data where database constraints may not exist.

SQL

```
query(  
  ""  
  SELECT  
    o.order_id,  
    o.customer_id  
  
  FROM orders AS o  
  
  LEFT JOIN customers AS c  
    ON o.customer_id = c.customer_id  
  
  WHERE c.customer_id IS NULL;  
  ""  
)
```

### Validate Join-Key Uniqueness

Before assuming a relationship is many-to-one, validate that the "one" side is actually unique.

A simple check is:

If duplicate keys exist, a supposedly many-to-one join can become many-to-many.

SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    COUNT(*) AS occurrences  
  
  FROM customers  
  
  GROUP BY customer_id  
  
  HAVING COUNT(*) > 1;  
  ""  
)
```

## SQL

```
GROUP BY key
HAVING COUNT(*) > 1
```

## Accidental Many-to-Many Joins

Suppose both inputs contain duplicate join keys.

Left:

Right:

The join produces:

matches for key A.

This multiplicative behavior can create severe analytical errors when duplicate keys were not expected.

## SQL

```
query(
  """
  WITH left_data(
    join_key,
    left_value
  ) AS (
    VALUES
      ('A', 1),
      ('A', 2),
      ('A', 3)
    ),

  right_data(
    join_key,
    right_value
  ) AS (
    VALUES
      ('A', 10),
      ('A', 20),
      ('A', 30),
      ('A', 40)
    )

  SELECT
    COUNT(*) AS joined_rows

  FROM left_data AS l

  JOIN right_data AS r
    ON l.join_key = r.join_key;
  """
)
```

SQL

key A → 3 rows

A Join Validation Workflow

Before performing an important join, especially on large ML or analytics data:

This workflow catches many problems that are otherwise easy to miss.

PYTHON

Define left grain  
↓  
Define right grain  
↓  
Identify join key  
↓  
Check key uniqueness on each side  
↓  
Predict relationship cardinality  
↓  
Estimate expected row behavior  
↓  
Perform join  
↓  
Measure resulting row count  
↓  
Measure distinct entity count  
↓  
Inspect unmatched keys

Join Coverage Metrics

When joining two data sources, useful diagnostics include:

For customers and orders, we can calculate customer coverage.

## SQL

```

query(
    """
    SELECT
        COUNT(*)
        AS total_customers,

        SUM(
            CASE
                WHEN order_count > 0
                THEN 1
                ELSE 0
            END
        ) AS customers_with_orders,

        SUM(
            CASE
                WHEN order_count = 0
                THEN 1
                ELSE 0
            END
        ) AS customers_without_orders,

        ROUND(
            100.0
            * SUM(
                CASE
                    WHEN order_count > 0
                    THEN 1
                # ...
            ) GROUP BY c.customer_id
        ) AS customer_order_counts;
    """
)

```

## SQL

```

number of left entities
number with matches
number without matches
match percentage

```

## Temporal Join Awareness

Ordinary equality joins answer questions such as:

Time-dependent ML can require more complex relationships:

That relationship may look conceptually like:

A temporal condition can change both cardinality and the information available to the model.

Point-in-time correctness will receive deeper treatment after subqueries and window functions.

PYTHON

```
same customer ID
same order ID
same product ID
```

PYTHON

```
same customer
AND
event occurred before prediction time
```

## Joins Can Create Data Leakage

Suppose one table contains:

and another contains all customer transactions, including future transactions.

A simple equality join on customer ID can attach events that happened after the prediction date.

The join is relationally valid but methodologically invalid for historical model training.

The correct relationship may need:

AI feature joins therefore require temporal semantics, not only matching IDs.

PYTHON

```
customer_id
prediction_date
```

## Joining Labels at the Wrong Grain

Suppose a label table has:

and feature events have:

Joining the label directly onto raw events duplicates that customer's label across every event.

That may be appropriate for an event-level model.

It is not automatically appropriate for a customer-level model.

Always ask:

Then construct joins and aggregations around that grain.

PYTHON

```
one row per customer
```

PYTHON

```
many rows per customer
```

### Join Multiplication and Train/Test Leakage

Suppose one customer appears in many joined event rows.

A random row-level train/test split can place some rows from the same customer into training and others into test.

If the modeling problem expects independent customer-level evaluation, this creates leakage across splits.

The SQL grain therefore influences downstream splitting methodology.

Data engineering and ML evaluation cannot be designed independently.

### Written Join Order and Physical Execution

SQL might be written:

but a query optimizer may choose a different physical join order when relational semantics permit it.

Do not infer physical execution strategy solely from textual order.

When performance matters, inspect execution plans.

Logical correctness comes first; physical optimization is a separate concern.

PYTHON

```
A
JOIN B
JOIN C
```

### Join Keys and Index Awareness

Indexes on join keys can improve performance for many workloads.

For example:

are natural candidates to consider for indexes because they participate in relationships.

However, index usefulness depends on:

- table size;
- data distribution;
- query patterns;
- database optimizer;
- write workload;
- existing indexes.

We will study indexing and query plans in a dedicated section.

**PYTHON**

```
orders.customer_id  
order_items.order_id  
order_items.product_id
```

## Joins and Data Exposure

A join can expose sensitive information even when each source table is individually controlled.

For example, joining:

can create a much more sensitive dataset than any source alone.

Access controls, least privilege, data minimization, and privacy review should consider the joined dataset, not merely individual tables.

**PYTHON**

```
customer identity  
+  
transaction history  
+  
location  
+  
health or behavioral data
```

## Common Join Mistakes

Important join mistakes include:

- joining columns merely because their names or types match;
- not knowing the grain of either input;
- assuming a left join preserves the number of left rows;
- assuming a join key is unique without validating it;
- omitting part of a composite join key;
- accidentally creating a many-to-many join;
- forgetting a join condition and producing a Cartesian product;
- using `DISTINCT` to hide row multiplication;
- summing coarse-grain measures after joining to finer-grain rows;
- counting `COUNT(*)` after a left join when unmatched entities should count as zero;
- placing a right-side outer-join filter in `WHERE` and removing unmatched rows;
- treating NULL join keys as ordinary equal values;
- joining future events into historical ML features;
- joining entity labels onto event-level data without checking training grain;



- performing row-level train/test splits after entity-level duplication;
- assuming written join order equals physical execution order;
- overlooking the privacy sensitivity of joined datasets.

Most join bugs are grain or cardinality bugs rather than syntax bugs.

## Join Engineering Checklist

Before an important join, ask:

If these questions have clear answers, the SQL syntax usually becomes straightforward.

### PYTHON

What does one left row represent?

What does one right row represent?

Which key represents the relationship?

Is the key unique on either side?

Is the relationship:

one-to-one,

one-to-many,

many-to-one,

or many-to-many?

What should one output row represent?

Which unmatched rows should survive?

Can NULL appear in the join key?

Will the join duplicate a measure?

Will a later COUNT or SUM operate at the correct grain?

For ML:

does the join include only information available at prediction time?

## Joining Relational Data Summary

This part developed joins as relational transformations rather than merely SQL syntax.

The major concepts included:

- join mental models;
- input grain;
- output grain;
- table aliases;

- qualified column names;
- `INNER JOIN` ;
- unmatched rows;
- `LEFT JOIN` ;
- NULL-extension;
- left-side preservation;
- `RIGHT JOIN` awareness;
- `FULL OUTER JOIN` awareness;
- `ON` ;
- semantic join keys;
- one-to-one relationships;
- one-to-many relationships;
- many-to-one relationships;
- many-to-many relationships;
- row multiplication;
- row counts versus entity counts;
- Cartesian products;
- `CROSS JOIN` ;
- implicit comma-join awareness;
- self joins;
- `ON` versus `WHERE` for outer joins;
- `COUNT(*)` pitfalls after left joins;
- complete-population feature construction;
- anti-join patterns;
- `NOT EXISTS` ;
- semi-join patterns;
- `EXISTS` ;
- multi-table joins;
- tracking grain through join chains;
- aggregation after joins;
- double-counting risks;
- distinct order counting after finer-grain joins;
- pre-aggregation before joins;
- customer-level feature summaries;
- composite join keys;

- incomplete composite-key multiplication;
- NULL join-key behavior;
- NULL-safe join awareness;
- `NATURAL JOIN` risks;
- `USING` awareness;
- referential-integrity validation;
- orphan detection;
- uniqueness validation;
- accidental many-to-many joins;
- join coverage metrics;
- temporal joins;
- AI leakage through future-event joins;
- label-grain mismatches;
- train/test leakage after entity duplication;
- optimizer join-order awareness;
- index awareness;
- privacy implications of relational combination.

The next part will develop subqueries, `EXISTS`, Common Table Expressions, recursive CTE fundamentals, and SQL set operations. Those constructs allow complex transformations to be decomposed into understandable relational stages.

## Subqueries, Common Table Expressions, and Set Operations

SQL queries are themselves relational expressions.

That means the result of one query can often become an input to another query.

This simple idea gives rise to several powerful constructs:

These tools allow complex analytical problems to be decomposed into meaningful stages.

For example:

Trying to force every stage into one flat expression can make SQL difficult to reason about.

Subqueries and CTEs provide structure.

Set operations solve a different relational problem: combining compatible result sets vertically according to set-like semantics.

This part develops all of these constructs from first principles while continuing to emphasize:

- row grain;
- NULL behavior;

- cardinality;
- portability;
- performance awareness;
- AI/data-engineering implications.

## PYTHON

```
subqueries
correlated subqueries
EXISTS
Common Table Expressions
recursive CTEs
set operations
```

## What Is a Subquery?

A subquery is a SQL query nested inside another SQL statement.

For example:

The inner query:

calculates one benchmark value.

The outer query compares individual products against that benchmark.

A useful mental model is:

Subqueries can appear in several locations, including:

- `WHERE` ;
- `HAVING` ;
- `SELECT` ;
- `FROM` ;
- `EXISTS` ;
- membership predicates.

The expected shape of the subquery result depends on where it is used.

## SQL

```
SELECT ...
FROM ...
WHERE price > (
    SELECT AVG(price)
    FROM products
);
```

SQL

```
SELECT AVG(price)
FROM products
```

## Scalar Subqueries

A scalar subquery is used where the surrounding SQL expects one value.

For example:

The subquery should conceptually produce one value: the overall average product price.

The outer query can then use that result like another scalar expression.

SQL

```
query(
  """
  SELECT
    product_id,
    product_name,
    price

  FROM products

  WHERE price > (
    SELECT AVG(price)
    FROM products
  )

  ORDER BY price DESC;
  """
)
```

SQL

```
price > (
  SELECT AVG(price)
  FROM products
)
```

## Scalar Subqueries in the SELECT List

A scalar subquery can also appear as a selected expression.

For example, every product row can be shown alongside the overall average price.

This preserves product-level grain while attaching a global benchmark.

## SQL

```

query(
    """
    SELECT
        product_id,
        product_name,
        price,

        (
            SELECT ROUND(
                AVG(price),
                2
            )
            FROM products
        ) AS overall_average_price

    FROM products

    ORDER BY product_id;
    """
)

```

## Comparing Each Row with a Scalar Benchmark

Once a scalar subquery provides a benchmark, SQL can derive additional values.

For example:

measures how far each product is above or below the global mean.

## SQL

```

query(
    """
    SELECT
        product_name,
        price,

        ROUND(
            price
            -
            (
                SELECT AVG(price)
                FROM products
            ),
            2
        ) AS difference_from_average

    FROM products

    ORDER BY difference_from_average DESC;
    """
)

```

## SQL

```
product price
-
overall average price
```

## Scalar Subqueries Must Match the Expected Shape

A scalar context expects one value.

If a subquery logically returns many rows, using it as a scalar is usually incorrect.

Database behavior differs:

- strict systems may raise an error;
- SQLite can accept some multi-row scalar subqueries and use one row according to its semantics.

Do not rely on permissive behavior.

If the logical question expects one value, design the subquery so its cardinality is clearly one.

## Subqueries with IN

A subquery can produce a collection of values for membership testing.

For example:

One approach is:

The inner query returns a set-like collection of customer IDs.

The outer query retrieves the corresponding customer records.

## SQL

```
query(
    """
    SELECT
        customer_id,
        name
    FROM customers
    WHERE customer_id IN (
        SELECT customer_id
        FROM orders
        WHERE status = 'completed'
    )
    ORDER BY customer_id;
    """
)
```

## SQL

Which customers have **at** least one completed **order**?

### Duplicate Values inside IN Subqueries

The inner query used by **IN** may return the same customer ID several times.

Membership testing does not require those duplicates to become duplicate outer rows.

The outer customer relation remains at customer grain.

This is one reason **IN** can be useful for existence-like questions, although **EXISTS** often communicates correlation more directly.

### NOT IN with Subqueries

A natural-looking anti-match query is:

If the subquery column is guaranteed non-NULL, this can work as intended.

But if the subquery can produce NULL, **NOT IN** becomes dangerous because of SQL's three-valued logic.

This is why **NOT EXISTS** is often preferred for anti-match logic.

## SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    name  
  
  FROM customers  
  
  WHERE customer_id NOT IN (  
    SELECT customer_id  
    FROM orders  
  )  
  
  ORDER BY customer_id;  
  ""  
)
```

### NOT IN with a NULL-Producing Subquery

The NULL trap is easier to see with a synthetic example.

Suppose the inner result contains:

Then an outer value such as **3** cannot establish that it differs from every inner value because comparison with NULL is unknown.

The resulting **NOT IN** predicate becomes unknown rather than true.



## SQL

```

query(
  """
  WITH outer_values(value) AS (
    VALUES
      (1),
      (2),
      (3),
      (4)
  ),
  inner_values(value) AS (
    VALUES
      (1),
      (2),
      (NULL)
  )
  SELECT value
  FROM outer_values
  WHERE value NOT IN (
    SELECT value
    FROM inner_values
  );
  """
)

```

## SQL

```

1
2
NULL

```

**EXISTS**

**EXISTS** asks whether a subquery produces at least one row.

General form:

The actual values selected inside the EXISTS subquery are usually irrelevant. What matters is whether a qualifying row exists.

That is why this is common:

inside **EXISTS** .

It communicates:

## SQL

```
query(  
    ""  
    SELECT  
        c.customer_id,  
        c.name  
  
    FROM customers AS c  
  
    WHERE EXISTS (  
        SELECT 1  
  
        FROM orders AS o  
  
        WHERE o.customer_id = c.customer_id  
    )  
  
    ORDER BY c.customer_id;  
    ""  
)
```

## SQL

```
WHERE EXISTS (  
    SELECT ...  
)
```

## Correlated Subqueries

The previous `EXISTS` subquery is correlated.

Inside the subquery:

references `c.customer_id` from the outer query.

Conceptually, for each outer customer, SQL asks:

This row-by-row mental model helps explain semantics.

However, it should not be interpreted as guaranteed physical execution. A query optimizer may transform correlated logic into a more efficient execution strategy.

## PYTHON

```
o.customer_id = c.customer_id
```

## EXISTS with Additional Conditions

Correlation can include business predicates.

For example:

requires both:

## SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name  
  
  FROM customers AS c  
  
  WHERE EXISTS (  
    SELECT 1  
  
    FROM orders AS o  
  
    WHERE  
      o.customer_id = c.customer_id  
      AND o.status = 'completed'  
  )  
  
  ORDER BY c.customer_id;  
  ""  
)
```

## SQL

```
customers having at least one completed order
```

## NOT EXISTS

**NOT EXISTS** asks whether no qualifying inner row exists.

This naturally expresses anti-match questions:

Because it tests row existence rather than membership against a NULL-containing value list, it avoids the classic **NOT IN** NULL trap.

## SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name  
  
  FROM customers AS c  
  
  WHERE NOT EXISTS (  
    SELECT 1  
  
    FROM orders AS o  
  
    WHERE o.customer_id = c.customer_id  
  )  
  
  ORDER BY c.customer_id;  
  ""  
)
```

## SQL

```
customers with no orders  
  
products never ordered  
  
users with no successful login  
  
accounts with no payment
```

## Products Never Ordered with NOT EXISTS

The same anti-match pattern works across other relationships.

SQL

```

query(
  """
  SELECT
    p.product_id,
    p.product_name

  FROM products AS p

  WHERE NOT EXISTS (
    SELECT 1

    FROM order_items AS oi

    WHERE oi.product_id = p.product_id
  )

  ORDER BY p.product_id;
  """
)

```

## Why SELECT 1 Is Common inside EXISTS

Inside:

the existence predicate asks only whether a qualifying row exists.

The selected literal `1` is conventional.

You may also encounter:

inside EXISTS.

Modern database optimizers understand that the actual selected columns do not need to be materialized merely to answer existence.

`SELECT 1` often makes programmer intent especially clear.

SQL

```

EXISTS (
  SELECT 1
  FROM ...
  WHERE ...
)

```

SQL

```

SELECT *

```

## EXISTS Versus INNER JOIN

Suppose the question is:

An inner join produces one result row per customer-order match.

A customer with several orders therefore appears several times.

You could then use `DISTINCT`, but `EXISTS` expresses the existence requirement directly without introducing match multiplicity into the outer result.

This is a useful distinction:

SQL

```
print("JOIN result:")

display(
  query(
    """
    SELECT
      c.customer_id,
      c.name
    FROM customers AS c
    JOIN orders AS o
      ON c.customer_id = o.customer_id

    ORDER BY
      c.customer_id,
      o.order_id;
    """
  )
)

print("\nEXISTS result:")

display(
  query(
    """
    SELECT
      c.customer_id,
      # ...
    ORDER BY c.customer_id;
    """
  )
)
```

SQL

Which customers have **at** least one **order**?

## Correlated Scalar Subqueries

A correlated subquery can also calculate a value for each outer row.

For example:

can be expressed with a correlated scalar aggregate.

The correlated-count query is readable for small examples.

For large analytical transformations, an aggregate followed by a join may be preferable depending on the database and optimizer.

Do not choose between correlated subqueries and joins based only on slogans such as:

or:

Inspect actual execution plans and workload characteristics.

SQL

```
query(  
  ""  
  SELECT  
    c.customer_id,  
    c.name,  
  
    (  
      SELECT COUNT(*)  
  
      FROM orders AS o  
  
      WHERE o.customer_id = c.customer_id  
    ) AS order_count  
  
  FROM customers AS c  
  
  ORDER BY c.customer_id;  
  ""  
)
```

## Subqueries in FROM

A subquery in `FROM` creates a derived table.

Conceptually:

The derived table normally needs an alias.

For example, calculate one total per order first, then summarize those order totals.

## SQL

```
query(  
  ""  
  SELECT  
    COUNT(*)  
    AS order_count,  
  
    ROUND(  
      AVG(order_value),  
      2  
    ) AS average_order_value  
  
  FROM (  
    SELECT  
      order_id,  
  
      SUM(  
        quantity * unit_price  
      ) AS order_value  
  
    FROM order_items  
  
    GROUP BY order_id  
  ) AS order_totals;  
  ""  
)
```

## SQL

```
inner query  
→ temporary relational result  
  
outer query  
→ operates on that result
```

## Derived Tables

The inner query:

has grain:

The outer query then operates on order-level records.

This is another example of multi-stage grain transformation.

A derived table is especially useful when an intermediate result is needed only once and remains reasonably readable.



SQL

```
SELECT
  order_id,
  SUM(...) AS order_value
FROM order_items
GROUP BY order_id
```

SQL

```
one row per order
```

## Subqueries in HAVING

Subqueries can also provide comparison values inside grouped filters.

For example, retain categories whose average product price is above the overall average product price.

SQL

```
query(
  """
  SELECT
    category,

    ROUND(
      AVG(price),
      2
    ) AS category_average

  FROM products

  GROUP BY category

  HAVING AVG(price) > (
    SELECT AVG(price)
    FROM products
  )

  ORDER BY category_average DESC;
  """
)
```

## Nested Subqueries and Readability

SQL can nest queries several levels deep.

That does not mean every transformation should be deeply nested.

For example:

can become difficult to:

- debug;

- test;
- reason about;
- review;
- modify.

Common Table Expressions often provide a clearer alternative by naming intermediate relational stages.

PYTHON

```
subquery
inside subquery
inside subquery
inside outer query
```

## Common Table Expressions

A Common Table Expression, or CTE, is introduced with `WITH`.

General pattern:

A CTE gives an intermediate query result a meaningful name.

This can transform a difficult nested query into a sequence of understandable relational stages.

For example:

A CTE is part of the SQL statement. It should not automatically be thought of as a permanently stored table.

SQL

```

query(
    """
    WITH order_totals AS (
        SELECT
            order_id,

            SUM(
                quantity * unit_price
            ) AS order_value

        FROM order_items

        GROUP BY order_id
    )

    SELECT
        order_id,
        order_value

    FROM order_totals

    ORDER BY order_id;
    """
)

```

SQL

```

WITH name AS (
    SELECT ...
)
SELECT ...
FROM name;

```

## CTE Mental Model

A helpful conceptual model is:

means:

Then the main query can consume it.

CTEs improve readability when names communicate business meaning.

Good CTE names:

Weak names such as:

provide much less semantic value.

SQL

```

WITH step_a AS (...)

```

## Multiple CTEs

A `WITH` clause can define several named stages.

Conceptually:

This style is common in analytical SQL and data-engineering transformation pipelines.

SQL

```
query(  
  ""  
  WITH order_totals AS (  
    SELECT  
      order_id,  
  
      SUM(  
        quantity * unit_price  
      ) AS order_value  
  
    FROM order_items  
  
    GROUP BY order_id  
  ),  
  
  customer_totals AS (  
    SELECT  
      o.customer_id,  
  
      COUNT(*)  
      AS order_count,  
  
      SUM(  
        CASE  
          WHEN o.status = 'completed'  
          THEN ot.order_value  
          ELSE 0  
        END  
      )  
      # ...  
  
    ORDER BY c.customer_id;  
  ""  
)
```

SQL

```
raw order items  
  ↓  
order totals  
  ↓  
customer totals  
  ↓  
final customer output
```

## Document CTE Grain

A strong SQL engineering habit is to know the grain of every CTE.

For the previous query:

If a CTE's grain is unclear, later joins and aggregates become much harder to reason about.

In production SQL, comments can explicitly document grain:

PYTHON

```
order_totals
→ one row per order

customer_totals
→ one row per customer

final result
→ one row per customer
```

PYTHON

```
-- Grain: one row per customer
```

## Referencing Earlier CTEs

A later CTE can consume a CTE defined earlier in the same `WITH` clause.

This allows transformations to form a readable dependency chain.

However, avoid creating dozens of unnecessary CTEs merely to make the query look modular. Each stage should represent a meaningful conceptual transformation.

## CTEs Are Not Permanent Tables

A normal CTE exists conceptually for the statement in which it is defined.

It does not automatically create a persistent database object.

Compare:

Materialization behavior of CTEs is a separate physical-execution concern and differs across database systems and versions.

## PYTHON

## CTE

 query-local relational expression

## VIEW

 stored query definition

## TABLE

 persistent stored relation

## CTE Materialization Is Database-Dependent

A common oversimplification is:

That is not generally correct.

Depending on database, version, query, and optimization rules, a CTE may be:

- inlined;
- materialized;
- optimized together with the outer query;
- controlled by explicit hints or keywords.

Use CTEs primarily for correct and readable relational structure. When performance matters, inspect the actual execution plan.

## PYTHON

CTE always creates a temporary table

## CTE Versus Derived Table

These can often express the same relational logic.

Derived table:

CTE:

A CTE often improves readability when:

- the intermediate result is conceptually meaningful;
- several stages exist;
- the intermediate relation is referenced multiple times;
- deep nesting would reduce clarity.

A small one-use derived table can still be perfectly clear.

SQL

```
SELECT ...  
FROM (  
    SELECT ...  
) AS x;
```

SQL

```
WITH x AS (  
    SELECT ...  
)  
SELECT ...  
FROM x;
```

## CTEs as Validation Stages

CTEs can make data-quality pipelines readable.

Conceptually:

This approach makes each transformation independently understandable.

## SQL

```

query(
    """
    WITH raw_values(value) AS (
        VALUES
            (' completed '),
            ('pending'),
            ('XYZ'),
            (NULL)
    ),

    normalized AS (
        SELECT
            value,

            LOWER(
                TRIM(value)
            ) AS normalized_value

        FROM raw_values
    ),

    flagged AS (
        SELECT
            value,
            normalized_value,

            CASE
                WHEN normalized_value IS NULL
            # ...
            SELECT *
            FROM flagged;
    """
)

```

## Recursive Common Table Expressions

A recursive CTE allows a query to repeatedly refer to its own accumulating result.

This is useful for structures such as:

- organizational hierarchies;
- category trees;
- filesystem-like trees;
- graph traversal;
- integer/date sequence generation.

A recursive CTE normally contains two conceptual pieces:

The process continues until the recursive step produces no new qualifying rows or another stopping mechanism applies.



## PYTHON

```

anchor query
  ↓
initial rows

recursive query
  ↓
derive new rows from previous rows

```

## Recursive CTE Structure

A simplified structure is:

The anchor establishes the starting state.

The recursive term references the CTE itself and calculates the next state.

A correct termination condition is essential. Poorly designed recursion can generate enormous results or fail to terminate appropriately.

## SQL

```

WITH RECURSIVE cte_name AS (

    anchor_query

    UNION ALL

    recursive_query
)
SELECT ...
FROM cte_name;

```

## Recursive Sequence Example

A simple numerical sequence makes recursive mechanics visible.

We start with:

Then repeatedly calculate:

while `n < 5`.

The recursive flow is conceptually:

This small example demonstrates the same structural mechanism used for hierarchical data.

## SQL

```
query(  
  ""  
  WITH RECURSIVE numbers(n) AS (  
  
    SELECT 1  
  
    UNION ALL  
  
    SELECT n + 1  
    FROM numbers  
    WHERE n < 5  
  )  
  
  SELECT n  
  FROM numbers;  
  ""  
)
```

## Recursive Hierarchy Traversal

Consider employees with manager relationships:

Each employee stores the ID of their manager.

A recursive CTE can begin at the root and repeatedly find employees whose `manager_id` matches someone already discovered.

## SQL

```

query(
  """
  WITH RECURSIVE employee_tree(
    employee_id,
    employee_name,
    manager_id,
    depth,
    path
  ) AS (

    SELECT
      employee_id,
      employee_name,
      manager_id,
      0 AS depth,
      employee_name AS path

    FROM (
      SELECT 1 AS employee_id,
            'Aisha' AS employee_name,
            NULL AS manager_id

      UNION ALL

      SELECT 2, 'Rohan', 1

      UNION ALL

      SELECT 3, 'Meera', 1

      # ...

      depth,
      employee_id;
    """
)

```

## SQL

```

Aisha
├─ Rohan
├─ Kabir
├─ Meera
└─ Zoya

```

## Recursive Queries and Cycles

Hierarchical source data can contain cycles.

For example:

Naïve recursive traversal can repeatedly revisit the same nodes.

Production recursive SQL may therefore need:

- cycle detection;
- path tracking;
- depth limits;
- database-specific cycle features;
- validated acyclic source data.

Recursive SQL should never assume hierarchy integrity without evidence.

#### PYTHON

```
A reports to B
B reports to C
C reports to A
```

### Recursive CTE Performance

Recursive CTEs are elegant, but they are not automatically the best solution to every graph-like problem.

Performance depends on:

- hierarchy depth;
- branching factor;
- indexes;
- recursion termination;
- repeated node visits;
- database optimizer;
- graph size.

Specialized graph databases or application-level algorithms may be more suitable for some graph workloads.

Recursive SQL is one tool, not a universal graph engine.

### SQL Set Operations

Joins combine columns by matching rows horizontally.

Set operations combine compatible query results vertically.

The major set-style operators are:

Conceptually:

The component queries must return compatible result shapes.

## PYTHON

UNION  
UNION ALL  
INTERSECT  
EXCEPT

## UNION

**UNION** combines rows from two compatible query results and removes duplicates.

Conceptually:

## SQL

```
query(  
    """  
    SELECT city AS value  
    FROM customers  
    WHERE city IS NOT NULL  
  
    UNION  
  
    SELECT category AS value  
    FROM products  
  
    ORDER BY value;  
    """)
```

## SQL

```
query A rows  
+  
query B rows  
↓  
duplicate elimination  
↓  
combined distinct result
```

## UNION ALL

**UNION ALL** combines result rows without duplicate elimination.

This means duplicate rows are preserved.

Because duplicate removal requires additional work and changes semantics, **UNION ALL** is commonly preferred when duplicate preservation is correct.

## SQL

```
query(  
    ""  
    SELECT city AS value  
    FROM customers  
    WHERE city IS NOT NULL  
  
    UNION ALL  
  
    SELECT category AS value  
    FROM products  
  
    ORDER BY value;  
    ""  
)
```

## UNION Versus UNION ALL

The difference is semantic:

Do not choose `UNION` merely because it looks cleaner.

If two identical events are both legitimate, removing one changes the data.

Likewise, do not use `UNION ALL` when the requirement explicitly asks for a set of unique values.

The data model determines the correct operation.

## PYTHON

```
UNION  
→ duplicate elimination  
  
UNION ALL  
→ preserve every row
```

## Set Operations Require Compatible Column Counts

The queries on both sides of a set operation must return the same number of output columns.

Conceptually invalid:

The first query returns two columns.

The second returns one.

There is no compatible result row structure.

SQL

```
SELECT customer_id, name
FROM customers

UNION

SELECT product_id
FROM products;
```

## Set-Operation Type Compatibility

Corresponding output positions should have compatible value domains.

Different databases have different type-coercion rules.

For example, combining an integer identifier with arbitrary textual status values may be accepted through coercion in one engine but rejected or represented differently in another.

Do not rely blindly on SQLite's flexible type system when writing portable SQL.

## Column Names in Set Results

The output column names of a compound query are commonly determined primarily by the first SELECT.

Therefore explicit aliases in the first query help define a readable result schema.

SQL

```
query(
    """
    SELECT
        customer_id AS entity_id,
        name AS entity_name,
        'customer' AS entity_type

    FROM customers

    UNION ALL

    SELECT
        product_id,
        product_name,
        'product'

    FROM products

    ORDER BY
        entity_type,
        entity_id;
    """
)
```

## INTERSECT

**INTERSECT** returns rows present in both query results, with duplicate behavior defined according to the operator and database.

Conceptually:

For example:

SQL

```
query(  
  ""  
  SELECT customer_id  
  FROM customers  
  
  INTERSECT  
  
  SELECT customer_id  
  FROM orders  
  
  ORDER BY customer_id;  
  ""  
)
```

## EXCEPT

**EXCEPT** returns rows from the first query that are absent from the second query.

Conceptually:

For example:

produces customers without orders.

SQL

```
query(  
  ""  
  SELECT customer_id  
  FROM customers  
  
  EXCEPT  
  
  SELECT customer_id  
  FROM orders  
  
  ORDER BY customer_id;  
  ""  
)
```

SQL

A - B



## EXCEPT Versus Anti Join

Several SQL constructs can express related "A but not B" questions:

They are not always interchangeable in every detail because:

- duplicates;
- NULL;
- selected columns;
- database dialect;
- optimizer behavior

can differ.

Choose the construct that best expresses the intended semantics, then inspect performance if needed.

PYTHON

EXCEPT

NOT EXISTS

LEFT JOIN ... IS NULL

sometimes NOT IN

## Set Operations and Duplicate Semantics

`UNION`, `INTERSECT`, and `EXCEPT` generally apply set-like duplicate elimination under their standard forms.

Some database systems support `ALL` variants for additional operators, such as:

Support varies by dialect.

Always distinguish:

from:

SQL normally allows duplicate rows, so this distinction matters.

PYTHON

INTERSECT ALL

EXCEPT ALL

## SQL Often Uses Bag Semantics

Pure relational theory is set-oriented.

SQL query results commonly behave more like multisets, or bags, because duplicate rows are permitted.

For example:

can return Delhi twice.

`DISTINCT` or set operators such as `UNION` can explicitly remove duplicates.

Understanding SQL's bag-like behavior explains many differences between theoretical relational algebra and everyday SQL.

SQL

```
SELECT city
FROM customers;
```

## ORDER BY with Set Operations

Ordering belongs to the final combined result.

A common pattern is:

Ordering each branch individually does not necessarily define the final compound-query order unless the dialect and query structure explicitly support that behavior.

As always, if final presentation order matters, request it at the appropriate outer level.

SQL

```
SELECT ...
UNION ALL
SELECT ...
ORDER BY ...
```

## Limiting Individual Set-Operation Inputs

Sometimes the requirement is:

That is different from:

Subqueries or CTEs are often required to establish per-branch ordering/limiting before the set operation.

This is another example where query stages clarify semantics.

PYTHON

```
top 3 rows from source A
plus
top 3 rows from source B
```

## Preserve Source Identity When Stacking Data

When rows from several sources are appended with `UNION ALL`, adding a source label can preserve provenance.

For example:

Without such metadata, later analysis may be unable to determine where a row originated.

SQL

```
query(  
  ""  
  WITH source_a(value) AS (  
    VALUES  
      (10),  
      (20)  
    ),  
  
    source_b(value) AS (  
      VALUES  
        (30),  
        (40)  
    )  
  
  SELECT  
    value,  
    'source_a' AS source  
  
  FROM source_a  
  
  UNION ALL  
  
  SELECT  
    value,  
    'source_b' AS source  
  
  FROM source_b  
  
  ORDER BY source, value;  
  ""  
)
```

SQL

```
source = 'web'  
source = 'store'
```

## Combining CTEs and Set Operations

CTEs and set operations can be composed.

For example:

This is useful for cohort analysis and data validation.

## SQL

```
query(  
    ""  
    WITH completed_customers AS (  
        SELECT DISTINCT customer_id  
        FROM orders  
        WHERE status = 'completed'  
    ),  
  
    pending_customers AS (  
        SELECT DISTINCT customer_id  
        FROM orders  
        WHERE status = 'pending'  
    )  
  
    SELECT customer_id  
    FROM completed_customers  
  
    INTERSECT  
  
    SELECT customer_id  
    FROM pending_customers  
  
    ORDER BY customer_id;  
    ""  
)
```

## SQL

```
define one population  
define another population  
compare them with INTERSECT or EXCEPT
```

## Cohort Difference with EXCEPT

Suppose we need customers who have completed orders but no pending order.

This is a set-difference question.

## SQL

```
query(  
    ""  
    WITH completed_customers AS (  
        SELECT DISTINCT customer_id  
        FROM orders  
        WHERE status = 'completed'  
    ),  
  
    pending_customers AS (  
        SELECT DISTINCT customer_id  
        FROM orders  
        WHERE status = 'pending'  
    )  
  
    SELECT customer_id  
    FROM completed_customers  
  
    EXCEPT  
  
    SELECT customer_id  
    FROM pending_customers  
  
    ORDER BY customer_id;  
    ""  
)
```

## Query Stages for AI Feature Engineering

AI feature SQL frequently needs several transformations:

CTEs can make these stages explicit.

A long query is not automatically bad. A long query whose grain and logic are unclear is dangerous.

Each CTE should ideally answer:

## PYTHON

```
raw events  
↓  
filter to historical window  
↓  
aggregate events  
↓  
calculate entity features  
↓  
join to prediction population  
↓  
validate one row per entity
```

## PYTHON

What does one row represent here?

## CTEs Do Not Automatically Prevent Leakage

A beautifully structured CTE pipeline can still contain temporal leakage.

For example:

is invalid for historical training if `all_transactions` contains events after the prediction point.

Code organization and methodological correctness are separate concerns.

Feature CTEs must still enforce the correct observation window.

## PYTHON

```
all_transactions
↓
customer_totals
↓
training features
```

## Correlated Temporal Logic Preview

Suppose a prediction table contains:

and orders contain:

A correlated subquery could conceptually calculate:

The condition would depend on both outer values:

This demonstrates why correlated SQL can be useful for point-in-time feature definitions.

The same customer can have different feature values at different prediction times.

That is exactly what point-in-time correctness requires.

A customer feature is not always one permanent number.

For time-dependent systems:

This principle is foundational to production ML feature engineering.

## SQL

```

query(
    """
    WITH predictions(
        customer_id,
        prediction_date
    ) AS (
        VALUES
            (1, '2026-03-04'),
            (1, '2026-03-08'),
            (2, '2026-03-10'),
            (3, '2026-03-05')
    )

    SELECT
        p.customer_id,
        p.prediction_date,

        (
            SELECT COUNT(*)

            FROM orders AS o

            WHERE
                o.customer_id = p.customer_id
                AND o.order_date < p.prediction_date
        ) AS historical_order_count

    FROM predictions AS p

    ORDER BY
        p.customer_id,
        p.prediction_date;
    """
)

```

## Scalar Aggregate Subqueries and NULL

Suppose a scalar subquery computes:

over no qualifying non-NULL rows.

The scalar result can be NULL.

The outer query must therefore have an explicit missing-data policy if the value is required.

Nested queries do not remove NULL semantics; they merely move those semantics between query stages.

## PYTHON

```
AVG(...)
```

## Validate CTE Uniqueness before Joining

Suppose a CTE named:

is intended to have:

Before joining it to the customer population, you can validate:

If any row is returned, the CTE violates its intended grain.

This kind of structural assertion is valuable in complex AI/data pipelines.

SQL

```
query(  
  ""  
  WITH customer_features AS (  
    SELECT  
      customer_id,  
      COUNT(*) AS order_count  
  
    FROM orders  
  
    GROUP BY customer_id  
  )  
  
  SELECT  
    customer_id,  
    COUNT(*) AS occurrences  
  
  FROM customer_features  
  
  GROUP BY customer_id  
  
  HAVING COUNT(*) > 1;  
  ""  
)
```

SQL

```
customer_features
```

## Subquery Performance Myths

Avoid oversimplified claims such as:

or:

Modern database optimizers can transform:

- subqueries;
- EXISTS predicates;
- joins;



- CTEs

into efficient physical plans.

Performance depends on:

- indexes;
- statistics;
- data volume;
- selectivity;
- database engine;
- version;
- query shape.

Write correct understandable SQL first, then inspect actual plans when performance matters.

PYTHON

```
subqueries are always slow
```

### CTEs and Performance Boundaries

In some database versions, a CTE can influence optimizer behavior or act as a materialization boundary.

In others, it may be inlined into the surrounding query.

Therefore:

should not automatically be interpreted as either:

or:

Use execution-plan evidence.

PYTHON

```
CTE
```

PYTHON

```
performance optimization
```

### UNION Versus UNION ALL Performance

**UNION** must implement duplicate elimination.

That may require operations such as:

- sorting;
- hashing;

- temporary structures.

`UNION ALL` does not need duplicate elimination.

Therefore `UNION ALL` can often be less expensive.

But performance is secondary to semantics.

If duplicates must be removed, using `UNION ALL` merely because it is faster would be incorrect.

## Subqueries and Security

Nested SQL does not change the fundamental security rule:

A dangerous application remains dangerous even if user input appears several levels inside a CTE or subquery.

Likewise, CTEs should not be used to disguise unauthorized access to sensitive tables.

Database permissions apply to underlying objects according to the database's security model.

PYTHON

```
untrusted values must be parameterized
```

## Common Subquery, CTE, and Set-Operation Mistakes

Important mistakes include:

- using a multi-row subquery where one scalar value is logically expected;
- using `NOT IN` when the inner result can contain NULL;
- using a join when the actual question is only about existence and then adding unnecessary DISTINCT;
- assuming correlated subqueries must physically execute one full query per outer row;
- believing subqueries are always slower than joins;
- creating deeply nested SQL that obscures grain;
- creating many meaningless CTE stages;
- assuming CTEs are permanent tables;
- assuming CTEs always materialize;
- forgetting recursive termination;
- ignoring cycles in hierarchical data;
- using `UNION` when duplicate rows are legitimate;
- using `UNION ALL` when unique set semantics are required;
- combining incompatible set-operation schemas;
- forgetting source provenance after vertically combining datasets;
- assuming branch-level ordering controls final set-operation ordering;
- constructing point-in-time features from future events;

- joining a CTE without validating its intended uniqueness.

The recurring principle is to treat every intermediate query as a relation with its own grain, schema, and semantics.

## Multi-Stage SQL Checklist

For a complex query, ask:

These questions make complex SQL substantially easier to debug.

### PYTHON

What does the inner query **return**?

Is it scalar, one-column, **or** tabular?

What **is** its grain?

Can it **return** NULL?

Can it **return** duplicates?

Does the outer context expect one row **or** many?

Would EXISTS express intent better than a join?

Would a CTE make the stages clearer?

Does a recursive CTE terminate?

Are cycles possible?

For **set** operations:

do both queries have compatible schemas?

Should duplicates be preserved?

For ML:

does every stage respect prediction-time boundaries?

## Subqueries, CTEs, and Set Operations Summary

This part developed multi-stage relational SQL.

The major concepts included:

- subqueries;
- scalar subqueries;
- scalar subqueries in `SELECT`;
- benchmark comparisons;
- subquery cardinality expectations;

- `IN` with subqueries;
- duplicate membership values;
- `NOT IN` ;
- the NULL trap in `NOT IN` ;
- `EXISTS` ;
- correlated subqueries;
- `NOT EXISTS` ;
- existence versus matching-row joins;
- correlated scalar aggregates;
- derived tables;
- subqueries in `FROM` ;
- subqueries in `HAVING` ;
- nested-query readability;
- Common Table Expressions;
- multiple CTEs;
- CTE dependency chains;
- documenting CTE grain;
- query-local versus persistent objects;
- CTE materialization awareness;
- CTE versus derived-table design;
- validation pipelines with CTEs;
- recursive CTEs;
- anchors;
- recursive terms;
- termination conditions;
- sequence generation;
- hierarchy traversal;
- recursion cycles;
- recursive-query performance;
- `UNION` ;
- `UNION ALL` ;
- duplicate semantics;
- compatible result shapes;
- type compatibility;
- result-column naming;

- `INTERSECT` ;
- `EXCEPT` ;
- anti-match alternatives;
- SQL bag semantics;
- final ordering of set operations;
- provenance labels;
- combining CTEs with set operations;
- cohort intersections and differences;
- point-in-time correlated features;
- feature values as functions of entity and time;
- CTE grain validation;
- subquery and CTE performance myths;
- set-operation performance;
- security considerations.

The next part will focus on SQL window functions, one of the most important advanced analytical capabilities:

PYTHON

```
OVER
PARTITION BY
window ORDER BY
ROW_NUMBER
RANK
DENSE_RANK
LAG
LEAD
FIRST_VALUE
LAST_VALUE
aggregate windows
running totals
moving calculations
window frames
ROWS
RANGE
top-N per group
deduplication patterns
time-series analytics
AI feature engineering
```

## SQL Window Functions

Window functions are among the most powerful analytical features in modern SQL.

Ordinary aggregation with `GROUP BY` deliberately collapses rows.

Suppose products contain:

A grouped query:

changes the result to:

That is exactly correct when the desired output is a category summary.

But what if we need:

while also attaching:

We need both:

This is where window functions become essential.

A window function performs a calculation across a collection of rows related to the current row without ordinary `GROUP BY` collapse.

Conceptually:

versus:

Window functions power ranking, running totals, moving averages, previous/next-row comparisons, top-N-per-group queries, sequence analysis, deduplication, cohort analytics, temporal features, and many other advanced SQL patterns.

SQL

one **row** per product

SQL

```
SELECT
  category,
  AVG(price)
FROM products
GROUP BY category;
```

## The OVER Clause

A function becomes a window function when it operates with an `OVER` clause.

Examples:

The `OVER` clause defines the analytical window associated with each row.

A window specification can contain:

These answer different questions.

`PARTITION BY` asks:

Window `ORDER BY` asks:

A window frame asks:

PYTHON

```
AVG(price) OVER (...)
```

## OVER() Across the Entire Result Population

An empty window specification:

means the function operates across the complete qualifying result population.

For example, every product can receive the overall average product price.

Unlike:

the window version does not collapse the product rows.

The overall average is repeated as contextual information alongside each row.

SQL

```
query(  
    """  
    SELECT  
        product_id,  
        product_name,  
        price,  
  
        ROUND(  
            AVG(price) OVER (),  
            2  
        ) AS overall_average_price  
  
    FROM products  
  
    ORDER BY product_id;  
    """  
)
```

SQL

```
OVER ()
```

## PARTITION BY

**PARTITION BY** divides the window calculation into independent groups.

For example:

calculates a separate category average.

The key difference from **GROUP BY** is that individual product rows remain present.

## SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    category,  
    price,  
  
    ROUND(  
      AVG(price) OVER (  
        PARTITION BY category  
      ),  
      2  
    ) AS category_average  
  
  FROM products  
  
  ORDER BY  
    category,  
    price DESC;  
  ""  
)
```

## SQL

```
AVG(price) OVER (  
  PARTITION BY category  
)
```

## Comparing a Row with Its Partition

Window outputs can participate in further analytical expressions.

For example, each product can be compared with its category average.

Because many SQL dialects do not allow a window alias to be immediately reused in the same SELECT expression, a CTE can create the window output first and an outer query can perform the comparison.



## SQL

```

query(
  """
  WITH product_context AS (
    SELECT
      product_id,
      product_name,
      category,
      price,

      AVG(price) OVER (
        PARTITION BY category
      ) AS category_average

    FROM products
  )

  SELECT
    product_name,
    category,
    price,

    ROUND(
      category_average,
      2
    ) AS category_average,

    ROUND(
      price - category_average,
      # ...
      category,
      difference_from_category_average DESC;
  """
)

```

**ORDER BY Inside OVER**

The **ORDER BY** inside a window specification defines analytical sequence.

Example:

means:

This window ordering is distinct from the final query's outer **ORDER BY**.

The outer ordering controls result presentation.

The inner window ordering controls the window calculation.

## PYTHON

```
ROW_NUMBER() OVER (
    ORDER BY price DESC
)
```

## PYTHON

```
assign sequence numbers
according to descending price
```

**ROW\_NUMBER**

`ROW_NUMBER()` assigns a unique sequential number to rows according to the window ordering.

For example:

When ordering values tie, `ROW_NUMBER` still produces distinct row numbers.

If deterministic assignment among ties matters, include an explicit tie-breaker in the window ordering.

## SQL

```
query(
    """
    SELECT
        product_id,
        product_name,
        price,

        ROW_NUMBER() OVER (
            ORDER BY
                price DESC,
                product_id ASC
        ) AS price_row_number

    FROM products

    ORDER BY price_row_number;
    """
)
```

## SQL

```
most expensive → 1
next           → 2
next           → 3
```

**ROW\_NUMBER within Partitions**

Combining `PARTITION BY` and `ORDER BY` resets numbering for each partition.

For example:

assigns product positions separately within every category.

#### SQL

```
query(  
    """  
    SELECT  
        product_id,  
        product_name,  
        category,  
        price,  
  
        ROW_NUMBER() OVER (  
            PARTITION BY category  
  
            ORDER BY  
                price DESC,  
                product_id ASC  
        ) AS position_in_category  
  
    FROM products  
  
    ORDER BY  
        category,  
        position_in_category;  
    """  
)
```

## Ranking and Ties

To understand ranking functions, consider values:

Several ranking functions handle the tied `90` values differently.

The three most important are:

#### PYTHON

```
100  
90  
90  
80
```

#### PYTHON

```
ROW_NUMBER  
RANK  
DENSE_RANK
```

## RANK

`RANK()` gives tied values the same rank and leaves gaps after ties.

For:

the ranks are:

Rank 3 is skipped because two rows occupy rank 2.

PYTHON

```
100
90
90
80
```

PYTHON

```
1
2
2
4
```

## DENSE\_RANK

`DENSE_RANK()` also gives tied values the same rank, but it does not leave gaps.

For:

dense ranks are:

## SQL

```

query(
  """
  WITH scores(
    person,
    score
  ) AS (
    VALUES
      ('A', 100),
      ('B', 90),
      ('C', 90),
      ('D', 80)
    )

  SELECT
    person,
    score,

    ROW_NUMBER() OVER (
      ORDER BY score DESC
    ) AS row_number_value,

    RANK() OVER (
      ORDER BY score DESC
    ) AS rank_value,

    DENSE_RANK() OVER (
      ORDER BY score DESC
    ) AS dense_rank_value
  # ...
  score DESC,
  person;
  """
)

```

## Choosing a Ranking Function

Use `ROW_NUMBER` when every row must receive a unique sequential position.

Use `RANK` when ties should share rank and subsequent rank numbers should reflect how many rows came before.

Use `DENSE_RANK` when ties share rank but rank values should remain consecutive.

The correct function depends on business meaning, not visual preference.

## NTILE

`NTILE(n)` divides ordered rows into approximately `n` numbered groups.

Example:

creates four buckets, often discussed as quartile-like buckets.

This is useful for:

- segmentation;
- coarse ranking groups;
- analytical bucketing.

However, `NTILE(4)` is not identical to mathematically calculating exact quartile boundary values. It distributes rows into ordered buckets.

SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
  
    NTILE(4) OVER (  
      ORDER BY  
        price ASC,  
        product_id ASC  
    ) AS price_bucket  
  
  FROM products  
  
  ORDER BY  
    price_bucket,  
    price;  
  ""  
)
```

SQL

```
NTILE(4)
```

## PERCENTRANK and CUMEDIST Awareness

Many modern SQL engines provide ranking-distribution functions such as:

`PERCENT_RANK` expresses relative rank position.

`CUME_DIST` expresses the proportion of rows whose ordering value is less than or equal to the current row under the window ordering.

Exact details matter around ties and small partitions.

These functions are useful in percentile-style analytics but should not be confused automatically with statistical quantile estimators.

## SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
  
    ROUND(  
      PERCENT_RANK() OVER (  
        ORDER BY price  
      ),  
      3  
    ) AS percent_rank_value,  
  
    ROUND(  
      CUME_DIST() OVER (  
        ORDER BY price  
      ),  
      3  
    ) AS cumulative_distribution  
  
  FROM products  
  
  ORDER BY price;  
  ""  
)
```

## SQL

```
PERCENT_RANK  
CUME_DIST
```

## Aggregate Functions as Window Functions

Functions such as:

can often operate as window functions when given `OVER`.

For example:

can attach the overall row count to every row.

Or:

can attach category size to each product.

## SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    category,  
    price,  
  
    COUNT(*) OVER ()  
      AS total_product_count,  
  
    COUNT(*) OVER (  
      PARTITION BY category  
    ) AS category_product_count  
  
  FROM products  
  
  ORDER BY  
    category,  
    product_id;  
  ""  
)
```

## Percent of Total with Window Aggregation

Window aggregates make percentage-of-total calculations elegant.

For each product:

can be calculated while preserving one row per product.



## SQL

```
query(  
  ""  
  SELECT  
    product_name,  
    price,  
  
    ROUND(  
      100.0  
      * price  
      / NULLIF(  
        SUM(price) OVER (),  
        0  
      ),  
      2  
    ) AS percent_of_catalog_price_sum  
  
  FROM products  
  
  ORDER BY  
    percent_of_catalog_price_sum DESC;  
  ""  
)
```

## SQL

```
price  
-----  
sum of all product prices
```

## GROUP BY and Window Functions Together

Grouped aggregation and window functions can appear in the same analytical pipeline.

For example:

1. group orders by status;
2. count each status;
3. use a window over those grouped rows to calculate each group's share of all orders.

The important point is that the window operates on the row set available at that stage of logical processing.

## SQL

```

query(
    """
    WITH status_counts AS (
        SELECT
            status,
            COUNT(*) AS frequency

        FROM orders

        GROUP BY status
    )

    SELECT
        status,
        frequency,

        SUM(frequency) OVER ()
        AS total_orders,

        ROUND(
            100.0
            * frequency
            / NULLIF(
                SUM(frequency) OVER (),
                0
            ),
            2
        ) AS percentage
    # ...
        frequency DESC,
        status;
    """
)

```

## Running Totals

An aggregate window with ordering can produce cumulative values.

Suppose order values are arranged chronologically.

A running total asks:

An explicit frame makes this intention clear:

## SQL

```

query(
  """
  WITH order_totals AS (
    SELECT
      o.order_id,
      o.order_date,

      SUM(
        oi.quantity * oi.unit_price
      ) AS order_value

    FROM orders AS o

    JOIN order_items AS oi
      ON o.order_id = oi.order_id

    GROUP BY
      o.order_id,
      o.order_date
  )

  SELECT
    order_id,
    order_date,
    order_value,

    SUM(order_value) OVER (
      ORDER BY
        # ...
        order_date,
        order_id;
    )
  """
)

```

## Window Frames

A window specification has several distinct concepts.

**PARTITION BY** defines the broad group.

Window **ORDER BY** defines sequence inside that group.

A frame defines which ordered rows around the current row participate in a frame-sensitive function.

Conceptually:

Common frame boundaries include:

Window frames are one of the most subtle parts of analytical SQL.

## PYTHON

```
partition  
↓  
ordered partition  
↓  
current row  
↓  
frame around current row
```

## PYTHON

```
UNBOUNDED PRECEDING  
n PRECEDING  
CURRENT ROW  
n FOLLOWING  
UNBOUNDED FOLLOWING
```

## ROWS Frames

**ROWS** describes frame boundaries in terms of physical row positions in the window ordering.

For example:

means:

This is a three-row moving window when at least two preceding rows exist.

## SQL

```

query(
    """
    WITH daily_values(
        day,
        value
    ) AS (
        VALUES
        ( '2026-03-01', 10.0),
        ( '2026-03-02', 20.0),
        ( '2026-03-03', 30.0),
        ( '2026-03-04', 50.0),
        ( '2026-03-05', 40.0)
    )

    SELECT
        day,
        value,

        ROUND(
            AVG(value) OVER (
                ORDER BY day

                ROWS BETWEEN
                    2 PRECEDING
                    AND CURRENT ROW
            ),
            2
        ) AS moving_average_3_rows

    FROM daily_values

    ORDER BY day;
    """
)

```

## SQL

```

ROWS BETWEEN
    2 PRECEDING
    AND CURRENT ROW

```

## Moving Sums

The same frame can support moving sums.

For example:

can form a rolling three-row total.

## SQL

```

query(
  """
  WITH daily_values(
    day,
    value
  ) AS (
    VALUES
    ('2026-03-01', 10),
    ('2026-03-02', 20),
    ('2026-03-03', 30),
    ('2026-03-04', 40),
    ('2026-03-05', 50)
  )

  SELECT
    day,
    value,

    SUM(value) OVER (
      ORDER BY day

      ROWS BETWEEN
        2 PRECEDING
        AND CURRENT ROW
    ) AS rolling_three_row_sum

  FROM daily_values

  ORDER BY day;
  """
)

```

## ROWS Versus RANGE

**ROWS** and **RANGE** are not interchangeable.

**ROWS** reasons about row positions.

**RANGE** reasons about ordering values and peers according to database/window semantics.

This difference becomes especially important when several rows share the same ordering value.

Window-function defaults can also involve **RANGE**-like peer behavior in some database systems.

When exact cumulative semantics matter, explicitly declaring the intended frame is safer than relying on a default you have not verified.

## Peer Rows

Rows tied on the window's ordering expressions are called peers in many window-function discussions.

Peer behavior matters for:

- `RANK` ;
- `DENSE_RANK` ;
- `RANGE` frames;
- default window frames;
- cumulative calculations.

A result that appears correct on unique ordering keys can behave differently once ties occur.

### Default Window Frames

When a window aggregate contains `ORDER BY` but no explicit frame, the database applies a default frame according to SQL and dialect rules.

That default is not always equivalent to:

especially when ordering values tie.

For educational and production calculations where exact cumulative row semantics matter, explicit frames make intention much clearer.

The tied ordering value `2` demonstrates why default-frame behavior deserves attention.

For exact cumulative calculations, specify:

when that is truly the intended meaning.

## SQL

```

query(
    """
    WITH events(
        event_order,
        amount
    ) AS (
        VALUES
            (1, 10),
            (2, 20),
            (2, 30),
            (3, 40)
    )

    SELECT
        event_order,
        amount,

        SUM(amount) OVER (
            ORDER BY event_order
        ) AS default_frame_sum,

        SUM(amount) OVER (
            ORDER BY event_order

            ROWS BETWEEN
                UNBOUNDED PRECEDING
                AND CURRENT ROW
        ) AS explicit_rows_sum
    # ...
        event_order,
        amount;
    """
)

```

## SQL

```

ROWS BETWEEN
    UNBOUNDED PRECEDING
    AND CURRENT ROW

```

## Running Totals within Partitions

Running calculations can reset independently for each partition.

For example, cumulative order value can be calculated separately for each customer.

The grain remains one row per order, but each row gains customer-history context.



## SQL

```

query(
  """
  WITH order_totals AS (
    SELECT
      o.order_id,
      o.customer_id,
      o.order_date,

      SUM(
        oi.quantity * oi.unit_price
      ) AS order_value

    FROM orders AS o

    JOIN order_items AS oi
      ON o.order_id = oi.order_id

    GROUP BY
      o.order_id,
      o.customer_id,
      o.order_date
  )

  SELECT
    customer_id,
    order_id,
    order_date,
    order_value,
    # ...
    order_date,
    order_id;
  """
)

```

**LAG**

`LAG()` accesses a value from an earlier row in the window ordering.

Conceptually:

This is extremely useful for:

- previous transaction values;
- previous timestamps;
- period-over-period change;
- event sequences;
- time-series features.

## SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    order_date,  
    status,  
  
    LAG(order_date) OVER (  
      ORDER BY  
        order_date,  
        order_id  
    ) AS previous_order_date  
  
  FROM orders  
  
  ORDER BY  
    order_date,  
    order_id;  
  ""  
)
```

### Change from the Previous Row

Once the previous value is available, the current row can be compared with it.

SQLite's `JULIANDAY()` can convert date text into a numerical day representation.

This lets us calculate the gap between consecutive order dates.

## SQL

```

query(
  """
  WITH ordered_orders AS (
    SELECT
      order_id,
      order_date,

      LAG(order_date) OVER (
        ORDER BY
          order_date,
          order_id
      ) AS previous_order_date

    FROM orders
  )

  SELECT
    order_id,
    order_date,
    previous_order_date,

    ROUND(
      JULIANDAY(order_date)
      - JULIANDAY(previous_order_date),
      2
    ) AS days_since_previous_order

  FROM ordered_orders

  ORDER BY
    order_date,
    order_id;
  """
)

```

**LAG within Each Entity**

For customer behavior, the useful "previous order" is usually the previous order for the same customer, not the previous order globally.

`PARTITION BY customer_id` creates independent customer histories.

## SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    order_id,  
    order_date,  
  
    LAG(order_date) OVER (  
      PARTITION BY customer_id  
  
      ORDER BY  
        order_date,  
        order_id  
    ) AS previous_customer_order_date  
  
  FROM orders  
  
  ORDER BY  
    customer_id,  
    order_date,  
    order_id;  
  ""  
)
```

**LEAD**

`LEAD()` accesses a later row in the window ordering.

If `LAG` means:

then `LEAD` means:

This is useful for sequence analysis and labeling future events.

In ML, however, future-looking values must be handled carefully. A future event can be a label definition, but it must not accidentally become an input feature available at prediction time.

## SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    order_id,  
    order_date,  
  
    LEAD(order_date) OVER (  
      PARTITION BY customer_id  
  
      ORDER BY  
        order_date,  
        order_id  
    ) AS next_customer_order_date  
  
  FROM orders  
  
  ORDER BY  
    customer_id,  
    order_date,  
    order_id;  
  ""  
)
```

## LAG and LEAD Offsets

`LAG` and `LEAD` can usually receive an offset.

Conceptually:

means:

A default value can also often be supplied for cases where the requested preceding/following row does not exist.

Syntax details should be checked for the target database.

SQL

```
query(  
  ""  
  WITH sequence(value) AS (  
    VALUES  
      (10),  
      (20),  
      (30),  
      (40)  
  )  
  
  SELECT  
    value,  
  
    LAG(value, 1) OVER (  
      ORDER BY value  
    ) AS previous_1,  
  
    LAG(value, 2) OVER (  
      ORDER BY value  
    ) AS previous_2  
  
  FROM sequence  
  
  ORDER BY value;  
  ""  
)
```

SQL

```
LAG(value, 2)
```

## FIRST\_VALUE

`FIRST_VALUE()` returns the first value according to the window ordering and frame semantics.

For example, every customer order row can receive the customer's first order date.

This can be useful for:

- customer tenure;
- cohort assignment;
- first-event analysis.

## SQL

```

query(
    """
    SELECT
        customer_id,
        order_id,
        order_date,

        FIRST_VALUE(order_date) OVER (
            PARTITION BY customer_id

            ORDER BY
                order_date,
                order_id

            ROWS BETWEEN
                UNBOUNDED PRECEDING
                AND UNBOUNDED FOLLOWING
        ) AS first_customer_order_date

    FROM orders

    ORDER BY
        customer_id,
        order_date,
        order_id;
    """
)

```

**LAST\_VALUE**

`LAST_VALUE()` is more subtle than its name initially suggests.

Its result depends on the current window frame.

With some default ordered frames, the frame ends at the current row, so `LAST_VALUE()` may simply return the current row's value rather than the final value in the entire partition.

To obtain the final value across the full partition, an explicit frame such as:

is often required.

## SQL

```
query(  
    ""  
    SELECT  
        customer_id,  
        order_id,  
        order_date,  
  
        LAST_VALUE(order_date) OVER (  
            PARTITION BY customer_id  
  
            ORDER BY  
                order_date,  
                order_id  
  
            ROWS BETWEEN  
                UNBOUNDED PRECEDING  
                AND UNBOUNDED FOLLOWING  
        ) AS last_customer_order_date  
  
    FROM orders  
  
    ORDER BY  
        customer_id,  
        order_date,  
        order_id;  
    ""  
)
```

## First and Last Event Context

Combining first and last event values can provide entity-level context while preserving event-level rows.

For customer behavior, this can expose:

on every order record.



SQL

```

query(
  """
  SELECT
    customer_id,
    order_id,
    order_date,

    FIRST_VALUE(order_date) OVER (
      PARTITION BY customer_id

      ORDER BY
        order_date,
        order_id

      ROWS BETWEEN
        UNBOUNDED PRECEDING
        AND UNBOUNDED FOLLOWING
    ) AS first_order_date,

    LAST_VALUE(order_date) OVER (
      PARTITION BY customer_id

      ORDER BY
        order_date,
        order_id

      ROWS BETWEEN
        UNBOUNDED PRECEDING
        # ...
        order_date,
        order_id;
  """
)

```

SQL

```

first order date
current order date
last order date

```

## NTH\_VALUE Awareness

Some database systems provide `NTH_VALUE()`.

Conceptually, it asks:

Its usefulness and support vary by database.

As with `FIRST_VALUE` and `LAST_VALUE`, frame semantics matter.

## PYTHON

What **is** the nth value **in** this ordered window/frame?

## Top-N per Group

A classic SQL problem is:

Examples:

A normal global `LIMIT` cannot solve this because it limits the complete result rather than each partition.

A common solution is:

## SQL

```
query(
    """
    WITH ranked_products AS (
        SELECT
            product_id,
            product_name,
            category,
            price,

            ROW_NUMBER() OVER (
                PARTITION BY category

                ORDER BY
                    price DESC,
                    product_id ASC
            ) AS category_position

        FROM products
    )

    SELECT
        product_id,
        product_name,
        category,
        price,
        category_position

    FROM ranked_products
    # ...
        category,
        category_position;
    """
)
```

## Why Filter Window Results in an Outer Query?

Window functions are conceptually evaluated after `WHERE`.

Therefore a window alias such as:

is not generally available to the same query's `WHERE` clause.

A CTE or subquery first computes the window result.

The outer query then filters it.

Some modern analytical databases support `QUALIFY`, which provides direct filtering of window-function results. SQLite does not use `QUALIFY` in this workflow.

```
PYTHON
```

```
category_position
```

### Top-N with Ties

`ROW_NUMBER()` guarantees exactly numbered rows.

If the requirement is:

`RANK()` or `DENSE_RANK()` may better express the intended semantics.

This distinction matters whenever ties are possible.

## SQL

```

query(
  """
  WITH scores(
    group_name,
    person,
    score
  ) AS (
    VALUES
      ('A', 'P1', 100),
      ('A', 'P2', 90),
      ('A', 'P3', 90),
      ('A', 'P4', 80),
      ('B', 'P5', 95),
      ('B', 'P6', 85),
      ('B', 'P7', 85)
  ),

  ranked AS (
    SELECT
      group_name,
      person,
      score,

      RANK() OVER (
        PARTITION BY group_name
        ORDER BY score DESC
      ) AS rank_value

    # ...
    rank_value,
    person;
  """
)

```

## SQL

**all** records tied within the top N ranks

## Latest Record per Entity

Another classic pattern is:

For example:

A reliable approach is:

1. partition by entity;
2. order newest first;
3. assign `ROW_NUMBER`;
4. retain row number 1.

A deterministic tie-breaker is important when timestamps can tie.

SQL

```
query(
  """
  WITH ranked_orders AS (
    SELECT
      order_id,
      customer_id,
      order_date,
      status,

      ROW_NUMBER() OVER (
        PARTITION BY customer_id

        ORDER BY
          order_date DESC,
          order_id DESC
      ) AS recency_position

    FROM orders
  )

  SELECT
    order_id,
    customer_id,
    order_date,
    status

  FROM ranked_orders

  WHERE recency_position = 1

  ORDER BY customer_id;
  """
)
```

## Window Functions for Deduplication

Suppose a dirty staging table contains several versions of the same business entity.

A common requirement is:

`ROW_NUMBER()` can rank candidate versions.

However, this requires a deterministic policy:

Deleting duplicates without a clear retention rule can destroy legitimate information.

## SQL

```

query(
  """
  WITH staging(
    customer_id,
    updated_at,
    value
  ) AS (
    VALUES
    (1, '2026-03-01 10:00:00', 'old'),
    (1, '2026-03-03 09:00:00', 'new'),
    (2, '2026-03-01 12:00:00', 'only'),
    (3, '2026-03-02 08:00:00', 'v1'),
    (3, '2026-03-04 08:00:00', 'v2')
  ),

  ranked AS (
    SELECT
      customer_id,
      updated_at,
      value,

      ROW_NUMBER() OVER (
        PARTITION BY customer_id

        ORDER BY
          updated_at DESC
      ) AS version_position

    # ...

    ORDER BY customer_id;
  """
)

```

## SQL

keep the latest **row for each** business **key**

## Deduplication Requires Deterministic Ties

If two candidate rows have the same:

then ordering only by `updated_at` does not determine which row becomes `ROW_NUMBER() = 1`.

Production deduplication often needs a secondary rule such as:

- ingestion sequence;
- source priority;
- version number;
- event ID.

Without deterministic ordering, deduplication may not be reproducible.

**PYTHON**

```
customer_id  
updated_at
```

## Period-over-Period Differences

`LAG()` makes change calculations straightforward.

Conceptually:

This can calculate:

- day-over-day change;
- previous transaction difference;
- previous model metric change;
- sequential feature deltas.

## SQL

```

query(
    """
    WITH measurements(
        period,
        value
    ) AS (
        VALUES
            (1, 100.0),
            (2, 120.0),
            (3, 90.0),
            (4, 135.0)
    ),

    with_previous AS (
        SELECT
            period,
            value,

            LAG(value) OVER (
                ORDER BY period
            ) AS previous_value

        FROM measurements
    )

    SELECT
        period,
        value,
        # ...

    ORDER BY period;
    """
)

```

## Percentage Change and Zero

Percentage change:

is undefined when the previous value is zero.

Using:

makes that undefined denominator explicit rather than inventing an arbitrary numerical result.

## PYTHON

```

current - previous
-----
previous

```



## PYTHON

```
NULLIF(previous_value, 0)
```

## Detecting Gaps in Sequences

`LAG` can help detect missing or discontinuous sequences.

Suppose expected integer IDs increase by one.

The difference:

can reveal gaps larger than one.

This technique also generalizes to time gaps.

## SQL

```
query(  
    """  
    WITH ids(id) AS (  
        VALUES  
            (1),  
            (2),  
            (5),  
            (6),  
            (10)  
    ),  
  
    sequenced AS (  
        SELECT  
            id,  
  
            LAG(id) OVER (  
                ORDER BY id  
            ) AS previous_id  
  
        FROM ids  
    )  
  
    SELECT  
        id,  
        previous_id,  
        id - previous_id AS gap_size  
  
    FROM sequenced  
    # ...  
  
    ORDER BY id;  
    """  
)
```

SQL

```
current_id - previous_id
```

## Cumulative Counts

Window aggregates can count observations cumulatively.

This can answer:

Explicit row framing makes the cumulative intention clear.

SQL

```
query(  
  ""  
  SELECT  
    order_id,  
    order_date,  
  
    COUNT(*) OVER (  
      ORDER BY  
        order_date,  
        order_id  
  
      ROWS BETWEEN  
        UNBOUNDED PRECEDING  
        AND CURRENT ROW  
    ) AS cumulative_order_count  
  
  FROM orders  
  
  ORDER BY  
    order_date,  
    order_id;  
  ""  
)
```

## Expanding Average

A cumulative frame can also calculate an expanding mean.

For each row:

This differs from a fixed-size moving average.

## SQL

```

query(
    """
    WITH values_table(
        step,
        value
    ) AS (
        VALUES
        (1, 10.0),
        (2, 20.0),
        (3, 30.0),
        (4, 50.0)
    )

    SELECT
        step,
        value,

        ROUND(
            AVG(value) OVER (
                ORDER BY step

                ROWS BETWEEN
                    UNBOUNDED PRECEDING
                    AND CURRENT ROW
            ),
            2
        ) AS expanding_average

    FROM values_table

    ORDER BY step;
    """
)

```

## SQL

average **from** the beginning  
through the **current row**

## Following Rows in a Frame

Frames can include future rows.

For example:

uses the current row plus the next two rows.

This is valid analytical SQL.

But in machine learning, a future-looking window cannot normally be used as a historical input feature unless those future values are actually available at prediction time.

The same SQL capability can be valid for retrospective analytics and invalid for predictive feature engineering.

SQL

```
query(  
  ""  
  WITH values_table(  
    step,  
    value  
  ) AS (  
    VALUES  
      (1, 10),  
      (2, 20),  
      (3, 30),  
      (4, 40),  
      (5, 50)  
  )  
  
  SELECT  
    step,  
    value,  
  
    SUM(value) OVER (  
      ORDER BY step  
  
      ROWS BETWEEN  
        CURRENT ROW  
        AND 2 FOLLOWING  
    ) AS current_plus_next_two  
  
  FROM values_table  
  
  ORDER BY step;  
  ""  
)
```

SQL

```
ROWS BETWEEN  
  CURRENT ROW  
  AND 2 FOLLOWING
```

## Centered Moving Windows

A frame may include preceding and following rows:

This creates a centered three-row window.

Centered windows are useful for smoothing and retrospective analysis.

They generally use future information relative to the current row and therefore require caution in forecasting or online prediction.

## SQL

```

query(
  """
  WITH values_table(
    step,
    value
  ) AS (
    VALUES
    (1, 10.0),
    (2, 20.0),
    (3, 40.0),
    (4, 30.0),
    (5, 50.0)
  )

  SELECT
    step,
    value,

    ROUND(
      AVG(value) OVER (
        ORDER BY step

        ROWS BETWEEN
          1 PRECEDING
          AND 1 FOLLOWING
      ),
      2
    ) AS centered_average

  FROM values_table

  ORDER BY step;
  """
)

```

## Causal Windows for Machine Learning

For historical prediction, feature windows are usually causal.

A causal feature for row  $t$  can use information available at or before  $t$ , depending on the exact prediction timing.

Conceptually safe historical frame:

Potentially leaky frame:

For example:

is backward-looking.

But:

includes future information.

Window-frame design therefore becomes part of ML leakage prevention.

PYTHON

```
past
+
possibly current information
```

PYTHON

```
future rows
```

## Temporal Ordering Requires Tie-Breakers

Suppose two events share the same timestamp.

Ordering only by timestamp does not define which event is first among the tie.

If sequence matters, include a stable tie-breaker such as:

For example:

Deterministic event ordering is important for reproducible `LAG`, `LEAD`, running totals, and sequence features.

PYTHON

```
event_id
ingestion_sequence
transaction_id
```

PYTHON

```
ORDER BY
    event_timestamp,
    event_id
```

## Multiple Window Calculations

A single query can use several window functions with different specifications.

For example, a product can receive:

- global rank;
- category rank;
- category average;
- total product count.

SQL evaluates each according to its own window definition.

## SQL

```

query(
    """
    SELECT
        product_id,
        product_name,
        category,
        price,

        RANK() OVER (
            ORDER BY price DESC
        ) AS global_price_rank,

        RANK() OVER (
            PARTITION BY category
            ORDER BY price DESC
        ) AS category_price_rank,

        ROUND(
            AVG(price) OVER (
                PARTITION BY category
            ),
            2
        ) AS category_average,

        COUNT(*) OVER ()
        AS total_products

    FROM products

    ORDER BY
        global_price_rank,
        product_id;
    """
)

```

## Named Window Definitions

SQL supports named window definitions in many databases.

A named window can reduce repeated specifications when several functions share the same partition/order configuration.

General concept:

Then:

can reuse that definition.

Support and syntax details should be checked for the target database.

SQL

```

query(
  """
  SELECT
    product_name,
    category,
    price,

    ROW_NUMBER() OVER category_window
      AS row_number_value,

    RANK() OVER category_window
      AS rank_value

  FROM products

  WINDOW category_window AS (
    PARTITION BY category

    ORDER BY
      price DESC,
      product_id ASC
  )

  ORDER BY
    category,
    row_number_value;
  """
)

```

SQL

```

WINDOW w AS (
  PARTITION BY ...
  ORDER BY ...
)

```

## Named Windows Improve Reuse, Not Meaning Automatically

A named window should represent a coherent analytical ordering.

Do not reuse the same named window merely to reduce typing if different functions actually require different frames or ordering semantics.

Shared syntax should follow shared analytical meaning.

## Where Window Functions Fit Logically

A simplified logical model is:

This explains why ordinary `WHERE` cannot generally filter directly on a window result calculated in the same query block.



A CTE/subquery calculates the window first; the outer query can then filter it.

Some analytical SQL dialects provide `QUALIFY` specifically for this purpose.

SQL

FROM / JOIN

↓

WHERE

↓

GROUP BY

↓

HAVING

↓

window functions

↓

SELECT result

↓

ORDER BY

↓

LIMIT

## WHERE Changes the Window Population

Because `WHERE` filters rows before window calculations, filtering affects which rows the window can see.

Compare:

with:

The second window does not know about excluded pending or cancelled rows.

This can materially change ranks, lags, totals, and moving calculations.

## SQL

```
query(  
    ""  
    SELECT  
        order_id,  
        order_date,  
        status,  
  
        ROW_NUMBER() OVER (  
            ORDER BY  
                order_date,  
                order_id  
        ) AS completed_sequence  
  
    FROM orders  
  
    WHERE status = 'completed'  
  
    ORDER BY completed_sequence;  
    ""  
)
```

## NULL Values in Window Calculations

Window aggregate functions generally follow their ordinary aggregate NULL semantics.

For example:

normally ignores NULL inputs.

Ranking and ordering involving NULL depend on the database's ordering semantics.

`LAG` or `LEAD` can return NULL because:

- there is no preceding/following row;
- the preceding/following row's actual value is NULL.

Those are logically distinct causes that may require domain-aware interpretation.

## PYTHON

```
AVG(value) OVER (...)
```

## Default Values for Missing LAG/LEAD Rows

Some window functions allow a default argument.

For example, in SQLite:

can return a supplied fallback if the requested prior row does not exist.

Be careful: replacing "no previous event exists" with zero or another value changes representation and should follow domain meaning.

SQL

```
query(  
  ""  
  WITH sequence(  
    step,  
    value  
  ) AS (  
    VALUES  
      (1, 10),  
      (2, 20),  
      (3, 30)  
  )  
  
  SELECT  
    step,  
    value,  
  
    LAG(  
      value,  
      1,  
      -1  
    ) OVER (  
      ORDER BY step  
    ) AS previous_with_default  
  
  FROM sequence  
  
  ORDER BY step;  
  ""  
)
```

SQL

```
LAG(value, 1, default_value)
```

## Entity Event Count without Collapsing Events

Suppose we want every order row to know how many orders that customer has.

`GROUP BY customer_id` would collapse orders.

A window count preserves individual orders.

## SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    order_id,  
    order_date,  
    status,  
  
    COUNT(*) OVER (  
      PARTITION BY customer_id  
    ) AS customer_order_count  
  
  FROM orders  
  
  ORDER BY  
    customer_id,  
    order_date,  
    order_id;  
  ""  
)
```

## Historical Event Counts as Features

A subtle distinction now appears.

This:

gives every order the customer's total order count across the complete partition, including future rows relative to early orders.

That is useful for retrospective analysis.

It can be leaky as a historical ML feature.

For a causal historical count, add ordering and a frame ending at the current row.

SQL

```

query(
    """
    SELECT
        customer_id,
        order_id,
        order_date,

        COUNT(*) OVER (
            PARTITION BY customer_id

            ORDER BY
                order_date,
                order_id

            ROWS BETWEEN
                UNBOUNDED PRECEDING
                AND CURRENT ROW
        ) AS orders_observed_so_far

    FROM orders

    ORDER BY
        customer_id,
        order_date,
        order_id;
    """
)

```

SQL

```

COUNT(*) OVER (
    PARTITION BY customer_id
)

```

## Excluding the Current Row from Historical Features

Sometimes a feature for the current event should use only strictly previous events.

Then the frame can end at:

rather than `CURRENT ROW`.

Conceptually:

This distinction is critical when the current event itself contains information that should not enter its prediction features.

For the first order in a customer's partition, the prior-only frame is empty.

Depending on the database/function, the aggregate may therefore produce a representation that needs explicit handling.

For feature engineering, decide whether "no prior events" should become:  
according to domain semantics.

SQL

```
query(  
  ""  
  SELECT  
    customer_id,  
    order_id,  
    order_date,  
  
    COUNT(*) OVER (  
      PARTITION BY customer_id  
  
      ORDER BY  
        order_date,  
        order_id  
  
      ROWS BETWEEN  
        UNBOUNDED PRECEDING  
        AND 1 PRECEDING  
    ) AS prior_order_count  
  
  FROM orders  
  
  ORDER BY  
    customer_id,  
    order_date,  
    order_id;  
  ""  
)
```

SQL

```
1 PRECEDING
```

## Sequence Features

Window functions can create sequence-aware categorical features.

For example:

may capture behavior transition patterns.

## SQL

```

query(
  """
  SELECT
    customer_id,
    order_id,
    order_date,
    status,

    LAG(status) OVER (
      PARTITION BY customer_id

      ORDER BY
        order_date,
        order_id
    ) AS previous_status

  FROM orders

  ORDER BY
    customer_id,
    order_date,
    order_id;
  """
)

```

## Sessionization Preview

Event-stream SQL often groups nearby events into sessions.

A typical strategy is:

1. use `LAG(timestamp)` to obtain the prior event;
2. calculate the time gap;
3. flag a new session when the gap exceeds a threshold;
4. cumulatively sum the new-session flags.

This combines multiple window-function ideas.

Full sessionization becomes particularly useful in clickstream, app, and user-activity datasets.

In the sessionization example, the threshold is:

A gap larger than 30 minutes begins a new session.

The query demonstrates a powerful analytical pattern:

Window functions often become most powerful when composed in stages.

## SQL

```

query(
  """
  WITH events(
    user_id,
    event_time
  ) AS (
    VALUES
      (1, '2026-03-01 10:00:00'),
      (1, '2026-03-01 10:05:00'),
      (1, '2026-03-01 11:00:00'),
      (1, '2026-03-01 11:10:00'),
      (2, '2026-03-01 09:00:00'),
      (2, '2026-03-01 10:00:00')
  ),

  with_previous AS (
    SELECT
      user_id,
      event_time,

      LAG(event_time) OVER (
        PARTITION BY user_id
        ORDER BY event_time
      ) AS previous_event_time

    FROM events
  ),

  # ...
  user_id,
  event_time;
  """
)

```

## SQL

```
1800 seconds = 30 minutes
```

## Window Functions Usually Need Staging for Reuse

SQL generally does not allow arbitrary nesting such as:

inside one query level.

Instead, use relational stages:

This makes both semantics and debugging clearer.



## PYTHON

```
SUM(
  LAG(value) OVER (...)
) OVER (...)
```

## PYTHON

```
CTE 1
→ compute LAG

CTE 2 / outer query
→ aggregate or window the result
```

## Median and Percentile Awareness

Some database systems provide ordered-set aggregate or percentile functions such as:

Support differs by database.

Window ranking functions can help with percentile-style analysis, but they should not automatically be substituted for mathematically defined median/quantile estimators.

For advanced statistical analysis, understand the exact estimator used by the database or Python statistical library.

## PYTHON

```
PERCENTILE_CONT
PERCENTILE_DISC
```

## Window Performance

Window functions can require substantial work.

Common physical costs include:

- sorting;
- partitioning;
- buffering;
- temporary storage;
- memory;
- disk spill.

A query containing several windows with compatible partition/order definitions may be optimized efficiently, while incompatible windows may require additional sorting.

Performance depends on:

- data volume;

- partition size;
- indexes;
- ordering;
- database engine;
- memory configuration.

Use execution plans rather than assuming that elegant analytical SQL is automatically cheap.

## Large Partitions

A partition containing millions of rows can be expensive even when the total query logic looks simple.

For example:

may produce one enormous partition for a dominant country.

Data distribution matters.

When performance is important, inspect:

- partition cardinality;
- skew;
- ordering cost;
- memory use;
- spill behavior.

```
PYTHON
```

```
PARTITION BY country
```

## Indexes and Window Functions

Indexes can sometimes help provide rows in a useful partition/order arrangement, but window optimization is database-specific.

An index matching:

may be relevant.

However, you should not create indexes solely because a window clause visually contains those columns.

Index design must consider complete workload and write cost.

```
PYTHON
```

```
partition keys  
+  
ordering keys
```

## Window Functions Versus Self Joins

Before modern window-function support, some analytical problems were solved with complex self joins or correlated subqueries.

Functions such as:

often express those problems much more directly.

The best SQL is not always the shortest SQL, but window functions frequently align very naturally with sequential analytical questions.

### PYTHON

```
LAG
LEAD
ROW_NUMBER
RANK
```

## Window SQL and Pandas

Many window operations have Pandas analogues.

Conceptual parallels include:

The implementations and semantics are not identical, but the conceptual connection is useful.

When data already lives in a database, performing appropriate window calculations in SQL can reduce transfer and leverage database execution.

### PYTHON

| SQL                  | Pandas              |
|----------------------|---------------------|
| -----                |                     |
| PARTITION BY         | groupby             |
| LAG                  | shift               |
| LEAD                 | shift(-1)           |
| ROW_NUMBER           | cumcount-like logic |
| running SUM          | cumsum              |
| moving AVG           | rolling().mean()    |
| group contextual AVG | transform("mean")   |

## Window Functions for AI Feature Engineering

Window functions are especially valuable for sequential and event-based features.

Examples include:

The crucial ML requirement is temporal correctness.

A feature for time `t` should normally use only information genuinely available by that prediction point.

A SQL window can make future information extremely easy to access; that power must be used carefully.

## PYTHON

```
previous transaction amount
time since previous event
number of prior events
running spend
rolling average
previous model score
recent failure count
customer event position
```

## LEAD for Label Construction

Future-looking functions can be appropriate for constructing labels.

Suppose the task is:

A future event may legitimately define the target.

The important separation is:

Do not accidentally feed the future-looking label construction back into the feature set.

## PYTHON

```
predict whether another event occurs next
```

## Window Reproducibility

Window results can become nondeterministic when ordering does not uniquely determine row sequence.

For example:

is not fully deterministic when several events have the same timestamp.

A stable tie-breaker improves reproducibility:

Determinism matters for:

- feature generation;
- deduplication;
- model training;
- testing;
- debugging.

## PYTHON

```
ROW_NUMBER() OVER (  
    ORDER BY event_time  
)
```

## PYTHON

```
ORDER BY
    event_time,
    event_id
```

## Window Functions and Sensitive History

Window functions make it easy to expose historical context around each row.

For example:

These derived values can be more sensitive than the original isolated records.

Privacy and authorization reviews should consider derived analytical outputs, not only source columns.

## PYTHON

```
previous payment
running medical total
previous location
recent behavior
```

## Common Window-Function Mistakes

Important mistakes include:

- confusing window functions with `GROUP BY`;
- assuming `PARTITION BY` collapses rows;
- confusing window `ORDER BY` with final result ordering;
- omitting deterministic tie-breakers;
- using `ROW_NUMBER` when tie-aware ranking is required;
- using `RANK` when exactly N rows are required;
- misunderstanding `RANK` versus `DENSE_RANK`;
- treating `NTILE` as identical to exact statistical quantiles;
- relying on unspecified default window frames;
- assuming `ROWS` and `RANGE` are identical;
- ignoring peer rows;
- misunderstanding `LAST_VALUE` because of frame boundaries;
- attempting to filter a window alias directly in ordinary `WHERE`;
- calculating total lifetime history for every row and using it as a historical feature;
- accidentally including the current event when only prior history is allowed;
- using future rows in predictive features;
- applying centered moving averages to forecasting features without recognizing leakage;
- deduplicating with `ROW_NUMBER` without a deterministic tie-breaker;

- assuming large partitions are cheap;
- assuming window SQL is automatically faster than alternative formulations.

Most mistakes come from not distinguishing partition, ordering, frame, and prediction-time semantics.

## Window-Function Checklist

Before using a window function, ask:

Answering these questions usually makes the correct window specification much clearer.

### PYTHON

What does one result row represent?

Should detail rows be preserved?

What defines the partition?

What defines sequence inside each partition?

Can ordering values tie?

Do I need a deterministic tie-breaker?

What frame should participate?

Should the current row be included?

Should future rows be excluded?

Am I calculating:

row position,  
rank,  
running value,  
moving value,  
previous/next value,  
or group context?

For ML:

would every value in this window have been available  
at prediction time?

## SQL Window Functions Summary

This part developed analytical window functions from first principles.

The major concepts included:

- window-function motivation;
- `OVER` ;
- `OVER ()` ;

- `PARTITION BY` ;
- window `ORDER BY` ;
- preserving row grain;
- group contextual aggregates;
- comparing rows with partition statistics;
- `ROW_NUMBER` ;
- deterministic row numbering;
- partitioned numbering;
- `RANK` ;
- `DENSE_RANK` ;
- tie semantics;
- `NTILE` ;
- `PERCENT_RANK` ;
- `CUME_DIST` ;
- aggregate window functions;
- partition counts;
- percent-of-total calculations;
- grouped aggregation followed by windows;
- running totals;
- explicit cumulative frames;
- window frames;
- `ROWS` ;
- `RANGE` awareness;
- peer rows;
- default-frame behavior;
- moving averages;
- moving sums;
- partitioned running totals;
- `LAG` ;
- `LEAD` ;
- offsets;
- previous-value differences;
- per-entity sequence analysis;
- `FIRST_VALUE` ;
- `LAST_VALUE` ;

- full-partition frames;
- `NTH_VALUE` awareness;
- top-N per group;
- tie-aware top-N;
- latest record per entity;
- window-based deduplication;
- deterministic deduplication;
- period-over-period changes;
- gap detection;
- cumulative counts;
- expanding statistics;
- future-looking frames;
- centered windows;
- causal ML windows;
- temporal tie-breakers;
- multiple windows;
- named windows;
- logical processing order;
- WHERE effects on window populations;
- NULL behavior;
- event counts without collapse;
- prior-event-only features;
- sequence features;
- sessionization;
- staged window calculations;
- percentile awareness;
- performance considerations;
- partition skew;
- index awareness;
- Pandas analogues;
- AI sequence features;
- LEAD-based label construction;
- leakage prevention;
- reproducibility;
- privacy implications.



The next SQL part will move into data modification and transaction engineering:

## SQL

**INSERT** in depth  
**UPDATE**  
**DELETE**  
 UPSERT awareness  
 transactions  
**COMMIT**  
**ROLLBACK**  
 SAVEPOINT  
 ACID  
 concurrency  
**isolation**  
 locking  
 safe modification workflows  
 Python **transaction** management  
 AI metadata consistency

## Data Modification and Transaction Engineering

Reading data is only one side of database engineering. Real applications must also create, modify, and remove persistent state safely.

An e-commerce application creates orders. A payment service changes payment status. A feature pipeline records completed runs. A model registry stores model versions. An annotation system updates labels. A monitoring service writes inference events. These operations change the state of the database, which means mistakes can have consequences far beyond returning an incorrect query result.

This section develops the engineering principles behind SQL data modification and transaction management.

We will study:

- **INSERT** in greater depth
- inserting several rows
- **INSERT ... SELECT**
- constraint failures
- generated identifiers
- **UPDATE**
- conditional updates
- arithmetic and expression-based updates
- **DELETE**
- safe deletion
- UPSERT concepts
- SQLite **ON CONFLICT**
- transactions

- autocommit
- `BEGIN`, `COMMIT`, and `ROLLBACK`
- transaction boundaries
- `SAVEPOINT`
- ACID properties
- concurrency
- isolation
- common concurrency anomalies
- database locking
- SQLite transaction behavior
- safe production modification workflows
- Python transaction management
- idempotency
- auditability
- AI/ML metadata consistency

The central principle is simple:

Data modification should be treated as a controlled state transition, not merely as a SQL statement.

A statement can be syntactically valid and still be operationally dangerous. Good database engineering therefore considers correctness, transaction boundaries, constraints, concurrency, recoverability, and observability together.

## Preparing an Isolated Transaction Laboratory

The earlier sections built reusable SQL examples around our main learning database. Transaction experiments are different because we intentionally perform updates, deletes, rollbacks, constraint failures, and locking demonstrations.

Using a separate in-memory SQLite database makes these examples repeatable and prevents transaction experiments from damaging earlier examples.

We will create a small banking-style schema. Banking examples are useful because transactional consistency is easy to reason about: transferring money should not create or destroy money accidentally.

The concepts are not limited to finance. The same principles apply to inventories, orders, experiment trackers, annotation systems, model registries, job queues, and AI application metadata.

## SQL

```

import sqlite3
from IPython.display import display
import pandas as pd

tx_conn = sqlite3.connect(":memory:")

tx_conn.execute("PRAGMA foreign_keys = ON")

tx_conn.executescript("""
CREATE TABLE accounts (
    account_id INTEGER PRIMARY KEY,
    owner TEXT NOT NULL,
    balance REAL NOT NULL CHECK (balance >= 0),
    status TEXT NOT NULL DEFAULT 'active'
    CHECK (status IN ('active', 'frozen', 'closed'))
);

CREATE TABLE transfers (
    transfer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    from_account INTEGER NOT NULL,
    to_account INTEGER NOT NULL,
    amount REAL NOT NULL CHECK (amount > 0),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (from_account) REFERENCES accounts(account_id),
    FOREIGN KEY (to_account) REFERENCES accounts(account_id),
    CHECK (from_account <> to_account)
);

# ...
display(pd.read_sql_query(
    "SELECT * FROM accounts ORDER BY account_id",
    tx_conn
))

```

## INSERT in Depth

**INSERT** creates rows.

The basic conceptual operation is:

Although the syntax is simple, inserts interact with almost every important relational concept:

- data types
- **NOT NULL**
- primary keys
- unique constraints
- foreign keys
- **CHECK** constraints
- default values

- generated identifiers
- transaction boundaries

A robust insert normally names its target columns explicitly.

For example:

Explicit column lists make the statement easier to understand and less dependent on the physical order of columns in the table definition.

Compare that with:

The second form may work, but it couples the statement to the complete table layout. Schema evolution can therefore make such code fragile.

Production code should generally prefer explicit column lists.

SQL

```
tx_conn.execute(
    """
    INSERT INTO accounts (account_id, owner, balance)
    VALUES (?, ?, ?)
    """,
    (5, "Riya", 500.0),
)

tx_conn.commit()

display(pd.read_sql_query(
    "SELECT * FROM accounts WHERE account_id = 5",
    tx_conn
))
```

## Default Values During INSERT

Columns do not always need an explicitly supplied value.

Our `accounts.status` column has:

When the column is omitted from the insert, SQLite uses its default.

Defaults are useful for values whose normal initial state is known by the database schema itself. They reduce duplication across clients and help make valid state creation consistent.

A default is different from `NULL`. A default is an automatically selected value when appropriate; `NULL` represents the absence of a known value.

SQL

```
row = tx_conn.execute(  
    """  
    SELECT account_id, owner, balance, status  
    FROM accounts  
    WHERE account_id = 5  
    """  
).fetchone()  
  
print(row)
```

SQL

```
DEFAULT 'active'
```

## Inserting Multiple Rows

Applications frequently need to insert batches rather than individual rows.

SQL dialects often support a multi-row `VALUES` form:

Python DB-API drivers also commonly provide `executemany()`. This is preferable to constructing SQL manually by concatenating strings.

Batch insertion matters for performance because issuing one network/database round trip for every row can become expensive at scale.

For very large ingestion workloads, production database systems may also provide dedicated bulk-loading tools. The principle remains the same: preserve validation and transactional guarantees while reducing unnecessary per-row overhead.

## SQL

```

new_accounts = [
    (6, "Ishaan", 450.0),
    (7, "Anaya", 850.0),
]

tx_conn.executemany(
    """
    INSERT INTO accounts (account_id, owner, balance)
    VALUES (?, ?, ?)
    """,
    new_accounts,
)

tx_conn.commit()

display(pd.read_sql_query(
    """
    SELECT *
    FROM accounts
    WHERE account_id IN (6, 7)
    ORDER BY account_id
    """,
    tx_conn
))

```

## SQL

```

INSERT INTO table_name (a, b)
VALUES
    (1, 'x'),
    (2, 'y'),
    (3, 'z');

```

**INSERT ... SELECT**

New rows do not have to come from literal values. They can come from another query.

Conceptually:

This pattern is extremely important in data engineering.

Typical use cases include:

- moving validated staging rows into production tables
- creating snapshots
- archiving records
- copying selected data between tables
- materializing transformed datasets
- constructing reporting tables

- storing feature computations

The destination columns and query expressions must be structurally compatible.

An important engineering benefit is that transformation and insertion can happen inside the database rather than transferring all rows through application code.

SQL

```
tx_conn.execute("""
CREATE TABLE IF NOT EXISTS high_value_accounts (
    account_id INTEGER PRIMARY KEY,
    owner TEXT NOT NULL,
    captured_balance REAL NOT NULL
)
""")

tx_conn.execute("DELETE FROM high_value_accounts")

tx_conn.execute("""
INSERT INTO high_value_accounts (
    account_id,
    owner,
    captured_balance
)
SELECT
    account_id,
    owner,
    balance
FROM accounts
WHERE balance >= 800
""")

tx_conn.commit()

display(pd.read_sql_query(
    "SELECT * FROM high_value_accounts ORDER BY account_id",
    tx_conn
))
```

## Generated Identifiers

Databases frequently generate identifiers for newly inserted rows.

Our `transfers` table uses:

The application therefore does not need to invent the next ID itself.

This distinction is important under concurrency.

A dangerous strategy would be:

1. query `MAX(transfer_id)`

2. add one

3. insert that value

Two concurrent clients could observe the same maximum and attempt to create the same next identifier.

Database-managed identity generation avoids this race.

Different database systems provide different mechanisms, including identity columns, sequences, auto-incrementing fields, and `RETURNING` clauses.

SQL

```
cursor = tx_conn.execute(
    """
    INSERT INTO transfers (
        from_account,
        to_account,
        amount
    )
    VALUES (?, ?, ?)
    """,
    (1, 2, 25.0),
)

print("Generated transfer_id:", cursor.lastrowid)

tx_conn.commit()

display(pd.read_sql_query(
    """
    SELECT *
    FROM transfers
    ORDER BY transfer_id DESC
    LIMIT 3
    """,
    tx_conn
))
```

SQL

```
transfer_id INTEGER PRIMARY KEY AUTOINCREMENT
```

## Constraint Failures Are Part of the Write Model

Constraints are not documentation. They are executable rules enforced by the database.

Suppose we attempt to insert an account with a negative balance. The schema contains:

SQLite should reject the write.

Applications need to expect constraint failures. They may occur because of invalid user input, duplicated identifiers, inconsistent foreign keys, concurrency races, or bugs.



Do not disable constraints simply because they reveal errors. In many systems, constraints are the final defensive layer protecting persistent data.

SQL

```
try:
    tx_conn.execute(
        """
        INSERT INTO accounts (account_id, owner, balance)
        VALUES (?, ?, ?)
        """,
        (100, "Invalid Example", -50.0),
    )
    tx_conn.commit()
except sqlite3.IntegrityError as exc:
    tx_conn.rollback()
    print("Insert rejected:", exc)

count = tx_conn.execute(
    "SELECT COUNT(*) FROM accounts WHERE account_id = 100"
).fetchone()[0]

print("Rows actually inserted:", count)
```

SQL

```
CHECK (balance >= 0)
```

## UPDATE

**UPDATE** changes existing rows.

Basic form:

The **WHERE** clause determines which rows are targeted.

For example:

The most dangerous beginner mistake is forgetting the **WHERE** clause:

That statement is valid SQL. It means every row in the table should be changed.

This is one reason write permissions should be treated more carefully than read permissions. SQL will faithfully execute a logically incorrect request if it is syntactically valid and does not violate database constraints.

## SQL

```
tx_conn.execute(  
    """  
    UPDATE accounts  
    SET status = ?  
    WHERE account_id = ?  
    """,  
    ("frozen", 3),  
)  
  
tx_conn.commit()  
  
display(pd.read_sql_query(  
    "SELECT * FROM accounts WHERE account_id = 3",  
    tx_conn  
))
```

## UPDATE Can Use Existing Column Values

The right side of `SET` can be an expression.

For example:

This is conceptually different from reading the balance into Python, calculating a new value, and issuing a second statement.

Performing the arithmetic directly in SQL can reduce round trips and may avoid certain read-modify-write races.

However, the complete business operation may involve several rows or tables. In those cases, the statements usually need to be grouped inside a transaction.

## SQL

```

before = tx_conn.execute(
    "SELECT balance FROM accounts WHERE account_id = 1"
).fetchone()[0]

tx_conn.execute(
    """
    UPDATE accounts
    SET balance = balance + ?
    WHERE account_id = ?
    """,
    (40.0, 1),
)

tx_conn.commit()

after = tx_conn.execute(
    "SELECT balance FROM accounts WHERE account_id = 1"
).fetchone()[0]

print("Before:", before)
print("After :", after)

```

## SQL

```

UPDATE accounts
SET balance = balance + 100
WHERE account_id = 1;

```

## Conditional UPDATE

Updates can encode conditional logic using `CASE`.

For example:

This can be useful for controlled migrations and bulk state transitions.

Still, application logic and database logic should have clear ownership. Complex business rules scattered across ad-hoc SQL statements become difficult to test and maintain.

Use database-side expressions when they naturally describe set-based data transformation, not merely because SQL can express them.

SQL

```

tx_conn.execute("""
UPDATE accounts
SET status =
    CASE
        WHEN balance = 0 THEN 'closed'
        WHEN status = 'closed' THEN 'closed'
        ELSE status
    END
""")

tx_conn.commit()

display(pd.read_sql_query(
    """
    SELECT account_id, owner, balance, status
    FROM accounts
    ORDER BY account_id
    """,
    tx_conn
))

```

SQL

```

UPDATE ...
SET category =
    CASE
        WHEN score >= 90 THEN 'excellent'
        WHEN score >= 70 THEN 'good'
        ELSE 'review'
    END;

```

## Verify the Target Before Modifying It

One of the safest habits in interactive SQL work is to first express the target as a `SELECT`.

Before:

run:

Inspect:

- which rows match
- how many rows match
- whether `NULL` behavior is correct
- whether joins accidentally multiply rows
- whether the environment is actually the intended database

The same approach applies to `DELETE`.

For high-risk production modifications, teams may additionally use explicit transactions, backups/snapshots, peer review, maintenance scripts, affected-row thresholds, audit logs, and change-management procedures.

## SQL

```
preview = pd.read_sql_query(
    """
    SELECT account_id, owner, balance, status
    FROM accounts
    WHERE balance < 400
    """,
    tx_conn
)

print("Rows that would be affected:", len(preview))
display(preview)
```

## DELETE

**DELETE** removes rows.

As with **UPDATE**, omitting **WHERE** changes the meaning dramatically:

means delete every row from the table.

**DELETE** is also constrained by relationships. If another table references a row through a foreign key, the database may reject the deletion unless the relationship specifies an appropriate referential action.

Possible foreign-key strategies in relational databases include:

- reject deletion
- cascade deletion to dependent records
- set the referencing key to **NULL**
- apply another schema-defined action

These policies should reflect data semantics rather than convenience. Cascading deletion is powerful but can remove far more data than the initial statement visibly targets.

## SQL

```

tx_conn.execute(
    """
    INSERT INTO accounts (account_id, owner, balance)
    VALUES (?, ?, ?)
    """,
    (50, "Temporary Account", 10.0),
)
tx_conn.commit()

print("Before deletion:")
display(pd.read_sql_query(
    "SELECT * FROM accounts WHERE account_id = 50",
    tx_conn
))

tx_conn.execute(
    "DELETE FROM accounts WHERE account_id = ?",
    (50,),
)
tx_conn.commit()

print("After deletion:")
display(pd.read_sql_query(
    "SELECT * FROM accounts WHERE account_id = 50",
    tx_conn
))

```

## SQL

```

DELETE FROM table_name
WHERE condition;

```

## Soft Delete Versus Hard Delete

A physical `DELETE` removes the row from the active table.

Some systems instead perform a soft delete by changing a column such as:

For example:

Soft deletion can be useful when:

- recovery is important
- historical records are required
- downstream references must remain interpretable
- audit requirements exist

But soft deletion has costs. Every relevant query must understand whether deleted rows should be included. Unique constraints can become more complicated, storage continues to grow, and accidentally forgetting the deletion predicate can expose logically removed records.

Neither approach is universally correct. The data lifecycle and retention requirements should determine the design.

SQL

```
deleted_at  
is_deleted  
status
```

SQL

```
UPDATE documents  
SET deleted_at = CURRENT_TIMESTAMP  
WHERE document_id = ?;
```

## UPSERT

Many workflows mean:

Create this row if it does not exist; otherwise update the existing row.

This operation is commonly called an UPSERT.

Without native support, developers sometimes attempt:

1. `SELECT` to check whether the row exists
2. `INSERT` if absent
3. otherwise `UPDATE`

That sequence can race under concurrency. Another transaction may insert the same key after the check but before the insert.

Modern database systems therefore provide atomic conflict-handling mechanisms. Syntax varies by database.

SQLite and PostgreSQL support `ON CONFLICT` forms. MySQL has `ON DUPLICATE KEY UPDATE`. SQL Server and other systems provide their own mechanisms.

Always learn the exact semantics of the database you are using; UPSERT syntax is not fully portable across SQL engines.

## SQL

```

tx_conn.execute("""
CREATE TABLE IF NOT EXISTS model_metrics (
    model_name TEXT NOT NULL,
    model_version TEXT NOT NULL,
    metric_name TEXT NOT NULL,
    metric_value REAL NOT NULL,
    PRIMARY KEY (model_name, model_version, metric_name)
)
""")

tx_conn.execute("""
INSERT INTO model_metrics (
    model_name,
    model_version,
    metric_name,
    metric_value
)
VALUES (?, ?, ?, ?)
ON CONFLICT(model_name, model_version, metric_name)
DO UPDATE SET
    metric_value = excluded.metric_value
""", ("fraud_model", "v1", "f1", 0.81))

tx_conn.commit()

display(pd.read_sql_query(
    "SELECT * FROM model_metrics",
    tx_conn
))

```

## Understanding excluded in SQLite UPSERT

In SQLite's `ON CONFLICT ... DO UPDATE`, `excluded` refers to the row that was proposed for insertion but encountered the conflict.

Suppose the existing database contains:

and a new insert proposes:

The composite primary key conflicts. The update clause can access the proposed `0.84` as:

This gives an atomic way to replace or merge selected values.

Be deliberate about what gets updated. An UPSERT should not silently overwrite immutable provenance fields merely because a conflict occurred.



SQL

```

tx_conn.execute("""
INSERT INTO model_metrics (
    model_name,
    model_version,
    metric_name,
    metric_value
)
VALUES (?, ?, ?, ?)
ON CONFLICT(model_name, model_version, metric_name)
DO UPDATE SET
    metric_value = excluded.metric_value
""", ("fraud_model", "v1", "f1", 0.84))

tx_conn.commit()

display(pd.read_sql_query(
    "SELECT * FROM model_metrics",
    tx_conn
))

```

SQL

|             |    |    |      |
|-------------|----|----|------|
| fraud_model | v1 | f1 | 0.81 |
|-------------|----|----|------|

## Transactions

A transaction groups database operations into one logical unit of work.

Consider transferring ₹100 from account A to account B.

At least two modifications are required:

If the first update succeeds but the second fails, the database would contain a broken partial transfer unless the two updates are coordinated.

A transaction lets us express the intended rule:

Either the complete transfer succeeds, or none of it becomes permanent.

Conceptually:

If something goes wrong:

Transactions are therefore not simply a database optimization. They are a fundamental correctness mechanism.

## SQL

```
A = A - 100  
B = B + 100
```

## SQL

```
BEGIN;  
  
UPDATE ...;  
UPDATE ...;  
INSERT ...;  
  
COMMIT;
```

## ROLLBACK

**ROLLBACK** abandons uncommitted changes in the current transaction.

Rollback is essential when any step in a logically atomic workflow fails.

For a money transfer:

1. verify relevant conditions
2. debit source
3. credit destination
4. record transfer
5. commit

If step 3 or 4 fails, the earlier debit should not remain.

Rollback allows the database to restore the state associated with the beginning of the transaction rather than leaving a half-completed operation.

The first **UPDATE** was executed, but it was never committed. When the second statement violated the balance constraint, Python caught the database error and called **rollback()**.

The database therefore did not retain the partial debit.

This example illustrates a critical distinction:

Statement success does not necessarily mean transaction success.

A multi-statement workflow is complete only when the transaction reaches its intended commit.

## SQL

```
def show_two_balances():
    return pd.read_sql_query(
        """
        SELECT account_id, owner, balance
        FROM accounts
        WHERE account_id IN (1, 2)
        ORDER BY account_id
        """,
        tx_conn,
    )

print("Before transaction:")
display(show_two_balances())

try:
    tx_conn.execute("BEGIN")

    tx_conn.execute(
        """
        UPDATE accounts
        SET balance = balance - ?
        WHERE account_id = ?
        """,
        (100.0, 1),
    )

    # Intentionally trigger a constraint failure:
    tx_conn.execute(
        # ...
    )
    tx_conn.rollback()

print("After rollback:")
display(show_two_balances())
```

## A Successful Atomic Transfer

Now we can implement the same logical operation correctly.

A robust transfer should verify more than arithmetic. For example:

- the source exists
- the destination exists
- both accounts are usable
- the source has sufficient funds
- the amount is positive
- the source and destination differ

Some rules are enforced by schema constraints, others by transaction logic, and some may belong to application-level authorization.

The exact division depends on system architecture, but persistent invariants should be protected as close to the data as practical.

## SQL

```
def transfer_money(conn, from_id, to_id, amount):
    if amount <= 0:
        raise ValueError("Transfer amount must be positive.")

    try:
        conn.execute("BEGIN")

        source = conn.execute(
            """
            SELECT balance, status
            FROM accounts
            WHERE account_id = ?
            """,
            (from_id,),
        ).fetchone()

        destination = conn.execute(
            """
            SELECT balance, status
            FROM accounts
            WHERE account_id = ?
            """,
            (to_id,),
        ).fetchone()

        if source is None or destination is None:
            raise ValueError("Source or destination account does not exist.")

        # ...
    transfer_money(tx_conn, 1, 2, 75.0)

    print("After:")
    display(show_two_balances())
```

## Transaction Invariants

An invariant is a property that should remain true before and after valid operations.

For our simplified transfer system, one useful invariant is:

A transfer should redistribute money, not create or destroy it.

Testing invariants is often more powerful than checking only individual output values.

In AI engineering, analogous invariants might include:

- one active deployment pointer per environment
- every prediction event references an existing model version

- every feature batch points to a valid source snapshot
- an experiment cannot be marked complete without required metrics
- a production model version must have an associated artifact record

Transactions help maintain such multi-table invariants.

SQL

```
total = tx_conn.execute(  
    """  
    SELECT SUM(balance)  
    FROM accounts  
    WHERE account_id IN (1, 2)  
    """  
).fetchone()[0]  
  
print("Combined balance:", total)
```

SQL

```
total money across the two accounts
```

## ACID Properties

Transactions are commonly described through four properties: ACID.

The acronym stands for:

- Atomicity
- Consistency
- Isolation
- Durability

These terms are sometimes memorized without understanding. They are more useful when connected to concrete failure modes.

ACID does not mean every database behaves identically, nor does it remove the need for correct schema and application design. Instead, it describes core transactional guarantees that database systems work to provide.

### Atomicity

Atomicity means a transaction is treated as one unit with respect to completion.

Either its intended changes commit together, or its uncommitted changes are rolled back.

For our transfer:

should not become three unrelated durable events.

Atomicity protects against partial execution becoming permanent.

It does not imply that a transaction contains only one SQL statement. In fact, atomicity is most valuable when several statements must succeed or fail together.

PYTHON

```
debit source
credit destination
record transfer
```

## Consistency

Consistency means a successful transaction takes the database from one valid state to another valid state, according to the integrity rules being enforced.

Those rules can include:

- primary keys
- foreign keys
- uniqueness
- `CHECK` constraints
- `NOT NULL`
- application-level invariants
- transaction logic

A subtle point is that the database cannot enforce business rules it does not know about.

If the schema allows logically nonsensical states, ACID does not magically infer the intended business meaning.

Consistency therefore depends on good schema design and correct transaction logic in addition to database machinery.

## Isolation

Isolation concerns interactions between concurrent transactions.

Imagine two users modifying the database at approximately the same time. Without coordination, one transaction could observe another transaction at an inconvenient intermediate state or overwrite changes unexpectedly.

Isolation mechanisms attempt to give transactions controlled views of concurrent activity.

The exact guarantee depends on the database system and isolation level. Stronger isolation can simplify reasoning but may reduce concurrency or increase conflict/retry behavior.

Understanding isolation requires thinking about interleavings, not just individual SQL statements.

## Durability

Durability means that once a transaction is successfully committed, the database is responsible for preserving that committed state according to its durability guarantees, even if a later process or system failure occurs.

Database engines use techniques such as transaction logs, write-ahead logging, checkpoints, replication, and storage synchronization to provide durability.

The exact guarantees depend on configuration and infrastructure.

A commit acknowledges a database-level persistence guarantee; it does not eliminate the need for backups, disaster recovery, replication strategy, or infrastructure reliability.

## Autocommit and Explicit Transactions

Database APIs differ in how transaction boundaries are exposed.

Some environments automatically commit individual statements unless an explicit transaction is opened. Others implicitly start transactions and require the application to commit or roll back.

Python's built-in `sqlite3` module also has configurable transaction behavior, and details can differ across Python versions and connection settings.

The engineering rule is more important than memorizing one driver's default:

Know exactly where your transaction begins, where it commits, and what happens on exceptions.

Do not assume that because a statement executed successfully it has necessarily been durably committed.

Likewise, do not assume closing a connection is an acceptable substitute for deliberate transaction management.

PYTHON

```
print("sqlite3 module version:", sqlite3.version)
print("SQLite library version:", sqlite3.sqlite_version)
print("Connection isolation_level:", repr(tx_conn.isolation_level))
```

## Python Context Managers for Transactions

Python's `sqlite3.Connection` can be used as a context manager.

A pattern such as:

provides convenient transaction handling: successful completion commits appropriately, while an exception causes rollback of pending transactional work.

The connection itself remains usable after the `with` block; the context manager manages transaction completion rather than serving as a general connection-closing mechanism.

For production systems, always consult the exact driver documentation because context-manager semantics vary between libraries.

SQL

```

before = tx_conn.execute(
    "SELECT balance FROM accounts WHERE account_id = 4"
).fetchone()[0]

try:
    with tx_conn:
        tx_conn.execute(
            """
            UPDATE accounts
            SET balance = balance + 10
            WHERE account_id = 4
            """
        )

        tx_conn.execute(
            """
            UPDATE accounts
            SET balance = -1
            WHERE account_id = 4
            """
        )

except sqlite3.IntegrityError as exc:
    print("Rolled back automatically:", exc)

after = tx_conn.execute(
    "SELECT balance FROM accounts WHERE account_id = 4"
).fetchone()[0]

print("Before:", before)
print("After :", after)

```

SQL

```

with conn:
    conn.execute(...)
    conn.execute(...)

```

## SAVEPOINT

Sometimes a transaction contains a subsection that we may want to undo without discarding the entire outer transaction.

A savepoint creates an intermediate recovery point.

Conceptually:



Savepoints are useful for complex workflows, import pipelines, migration utilities, and systems that need finer-grained recovery inside a transaction.

A savepoint is not the same as committing. Changes before the savepoint are still part of the surrounding transaction until the outer transaction commits.

SQL

```
initial_status = tx_conn.execute(
    "SELECT status FROM accounts WHERE account_id = 4"
).fetchone()[0]

try:
    tx_conn.execute("BEGIN")

    tx_conn.execute(
        """
        UPDATE accounts
        SET balance = balance + 20
        WHERE account_id = 4
        """
    )

    tx_conn.execute("SAVEPOINT status_change")

    tx_conn.execute(
        """
        UPDATE accounts
        SET status = 'frozen'
        WHERE account_id = 4
        """
    )

    # Decide that the status change should be undone,
    # but retain the earlier balance modification.
    tx_conn.execute("ROLLBACK TO status_change")
    # ...
).fetchone()

print("Original status:", initial_status)
print("Final balance/status:", result)
```

## SAVEPOINT Lifecycle

Three operations are worth distinguishing:

**SAVEPOINT** establishes the point.

**ROLLBACK TO** undoes work performed after that savepoint while keeping the broader transaction alive.

**RELEASE** removes the named savepoint from the active savepoint structure.

Details can vary across database engines, particularly with nested transactions and error handling. Libraries and frameworks may emulate nested transaction behavior using savepoints even when the underlying database does not support independent nested top-level transactions.

**PYTHON**

```
SAVEPOINT name;  
ROLLBACK TO name;  
RELEASE name;
```

## Concurrency

Concurrency means multiple transactions may execute during overlapping periods of time.

This is normal in real systems.

Examples:

- two customers purchase the final inventory item
- two workers claim the same job
- two analysts update experiment metadata
- two services modify an account balance
- multiple inference workers write telemetry
- a training pipeline registers a model while deployment automation reads the registry

The challenge is that each transaction may individually look correct while their combined interleaving creates an invalid result.

Concurrency therefore introduces correctness questions that cannot be understood by reading one transaction in isolation.

## The Lost Update Problem

Consider a counter with value 10.

Transaction A:

Transaction B:

Then A writes 11, and B also writes 11.

Two increments occurred logically, but the final value is only 11 rather than 12.

One update has effectively been lost.

This is one reason set-based atomic SQL such as:

can be preferable to unnecessary application-side read-modify-write logic.

More complex cases may require row locks, optimistic concurrency checks, stronger isolation, or retry logic depending on the database.

PYTHON

```
reads 10  
calculates 11
```

PYTHON

```
reads 10  
calculates 11
```

## Dirty Reads

A dirty read occurs when one transaction can read another transaction's changes before those changes are committed.

Suppose transaction A changes a balance from 1000 to 100 but later rolls back.

If transaction B was allowed to observe the temporary value `100`, B made a decision based on state that never became committed reality.

Many database configurations prevent dirty reads by default, but exact behavior depends on the database and isolation settings.

The broader lesson is that "what a query sees" is partly a transaction-isolation question, not merely a `SELECT` question.

## Non-Repeatable Reads

Suppose transaction A reads a row and sees:

Transaction B then updates that row to:

and commits.

If transaction A reads the same row again and now sees `approved`, it has experienced a non-repeatable read.

Whether this can occur depends on the isolation semantics of the database and the transaction.

Some workloads tolerate this. Others require a transaction to reason over a stable view of the data.

PYTHON

```
status = pending
```

## Phantom Reads

A phantom concerns a set of rows rather than only a previously read row.

Transaction A runs:

and sees 20 rows.

Transaction B inserts a new pending order and commits.

If transaction A repeats the predicate and now sees 21 rows, a new matching row has appeared—a phantom.

Phantom behavior is particularly important when application logic assumes that a predicate describes a stable set throughout a transaction.

SQL

```
SELECT COUNT(*)  
FROM orders  
WHERE status = 'pending';
```

## Write Skew and Multi-Row Invariants

Some concurrency anomalies are subtler than dirty reads or lost updates.

Suppose two rows jointly encode an invariant such as:

At least one doctor must remain on call.

Two transactions independently read that both doctors are currently on call. Transaction A takes doctor 1 off call. Transaction B takes doctor 2 off call. Each transaction may update a different row and individually believe the invariant remains satisfied.

After both commit, nobody is on call.

This type of problem demonstrates why correct concurrent systems require reasoning about business invariants, not only individual row conflicts.

Depending on the database and workload, solutions may involve stronger isolation, explicit locking, redesigning the invariant, serialization, or retryable transactions.

## Isolation Levels

SQL database systems commonly expose isolation levels with names such as:

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

Some systems also expose snapshot-oriented modes.

The names are useful, but their exact implementation and observable guarantees can differ across database products.

Very roughly:

`Read Uncommitted` permits relatively weak isolation.

**Read Committed** generally prevents reading uncommitted data, while successive statements may observe newly committed changes.

**Repeatable Read** generally provides a more stable view for rows already read, though exact phantom behavior varies.

**Serializable** aims to make concurrent transactions behave as though some valid serial ordering had occurred.

Do not build correctness assumptions from the level name alone. Consult the documentation of the specific database engine.

### Isolation Is a Trade-Off

Stronger isolation can make application reasoning easier, but concurrency control has costs.

Possible costs include:

- lock contention
- waiting
- deadlocks
- transaction aborts
- serialization failures
- additional retries
- reduced throughput for conflicting workloads

Weaker isolation may improve concurrency but pushes more correctness responsibility onto application design.

The correct isolation policy therefore depends on the workload and invariant.

For example, an approximate analytics dashboard may tolerate more concurrency variation than a financial ledger or deployment-state transition.

### Locking

Locks are one mechanism databases use to coordinate concurrent access.

Conceptually, locks can restrict incompatible operations while a transaction is working with data.

Different databases can lock at different granularities:

- database
- table
- page
- row
- key range
- predicate-like structures

They may also use mechanisms such as multi-version concurrency control rather than relying only on conventional read/write locking.

Therefore, statements such as "SQL locks the row" should not be treated as universal rules. Concurrency behavior is database-specific.

### SQLite Concurrency Awareness

SQLite is an embedded database stored primarily in a file rather than a separate database-server process.

It provides full transactional behavior, but its concurrency model differs from server databases such as PostgreSQL or MySQL.

SQLite supports multiple concurrent readers, while write concurrency is more constrained. Its locking and journaling modes determine how readers and writers interact.

Write-Ahead Logging, commonly enabled through:

can improve some reader/writer concurrency patterns.

SQLite is excellent for local applications, prototypes, tests, embedded systems, and many production workloads. But choosing a database should account for expected concurrency, write throughput, operational architecture, and scaling requirements.

PYTHON

```
PRAGMA journal_mode = WAL;
```

### BEGIN Modes in SQLite

SQLite supports forms such as:

Their differences concern when locks or write intentions are established.

**DEFERRED** delays some locking decisions until an operation actually requires them.

**IMMEDIATE** attempts to begin the write transaction in a way that reserves writing capability earlier.

**EXCLUSIVE** requests stronger exclusivity, with behavior also affected by journaling mode.

These are SQLite-specific transaction controls, not portable SQL assumptions. They become relevant when applications need predictable behavior under competing writers.

PYTHON

```
BEGIN DEFERRED;  
BEGIN IMMEDIATE;  
BEGIN EXCLUSIVE;
```

### Deadlocks

In database systems with fine-grained locking, two transactions can sometimes wait on each other.

For example:

Neither can proceed indefinitely.

Database systems generally detect or otherwise resolve such situations by aborting one participant, allowing the other to continue.

Applications therefore need to distinguish permanent errors from transient transactional conflicts. A safe operation may need to be retried.

Common techniques for reducing deadlock risk include:

- accessing shared resources in a consistent order
- keeping transactions short
- avoiding unnecessary locks
- indexing queries appropriately
- implementing bounded retry logic for retryable failures

#### PYTHON

```
Transaction A holds resource 1 and waits for resource 2.  
Transaction B holds resource 2 and waits for resource 1.
```

## Transactions Should Usually Be Short

A transaction should be long enough to cover one logical database operation but not remain open while unrelated work happens.

Potentially problematic operations inside an open transaction include:

- waiting for user input
- making slow external API calls
- performing large unrelated computations
- sleeping
- downloading files
- calling an LLM endpoint unnecessarily
- waiting for human approval

Long-running transactions can retain locks or old row versions, increase contention, complicate cleanup, and reduce system throughput.

A good architecture often performs expensive external computation first, then opens a short transaction to validate current state and persist the final database transition.

## Never Put Uncontrolled External Side Effects Inside a Database Transaction

A database rollback can undo database changes. It cannot automatically undo an email, payment request, HTTP API call, uploaded file, or message already delivered to another system.

Consider:

1. begin database transaction
2. charge an external payment service
3. update database
4. database update fails
5. rollback

The external charge may already have happened.

Distributed side effects require additional patterns such as idempotency keys, transactional outboxes, workflow orchestration, sagas, compensating actions, or message-system guarantees.

A local SQL transaction is atomic only with respect to the resources controlled by that database transaction.

## **Idempotency**

An operation is idempotent when repeating it does not incorrectly apply the intended effect multiple times.

This matters because distributed systems often retry operations after timeouts or temporary failures.

Imagine an API request:

If the client does not receive the response, it may retry. Without an idempotency mechanism, two database records or two external effects might be created.

A common strategy is to assign each logical request a unique idempotency key and enforce uniqueness at the database level.

Then a retry can recognize that the logical operation has already been processed.

Database uniqueness constraints are particularly valuable here because application-level "check then insert" logic alone may race.

`INSERT OR IGNORE` is SQLite-specific conflict behavior and should not be applied blindly. Ignoring all applicable conflicts can hide situations an application should instead treat as errors.

The important idea in this example is the unique request identifier.

For more sophisticated systems, an idempotency record may also store:

- request fingerprint
- final status
- response payload/reference
- creation time
- expiration policy

That lets retries distinguish "same request repeated" from "same key reused for a different operation."



## SQL

```

tx_conn.execute("""
CREATE TABLE IF NOT EXISTS ingestion_jobs (
    request_id TEXT PRIMARY KEY,
    dataset_name TEXT NOT NULL,
    status TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
)
""")

request = ("req-2026-001", "customer_features", "completed")

tx_conn.execute(
    """
    INSERT OR IGNORE INTO ingestion_jobs (
        request_id,
        dataset_name,
        status
    )
    VALUES (?, ?, ?)
    """,
    request,
)

# Simulated retry of the same logical request.
tx_conn.execute(
    """
    INSERT OR IGNORE INTO ingestion_jobs (
        request_id,
        # ...
    )
    VALUES (?, ?, ?)
    """,
    request,
)

display(pd.read_sql_query(
    "SELECT * FROM ingestion_jobs",
    tx_conn
))

```

## SQL

```
register payment of ₹500
```

## Optimistic Concurrency Control

Sometimes an application reads a row, lets a user or service work with it, and later attempts to update it. Meanwhile, another transaction may have changed the row.

One strategy is optimistic concurrency control.

A table can include a version number:

The application reads version 7, then updates with:

If the affected-row count is zero, the row no longer has version 7. Another writer probably changed it, so the application can reject, reload, merge, or retry rather than silently overwriting newer data.

This pattern is especially useful when conflicts are relatively uncommon.

SQL

```
tx_conn.execute("""
CREATE TABLE IF NOT EXISTS model_descriptions (
    model_id INTEGER PRIMARY KEY,
    description TEXT NOT NULL,
    version INTEGER NOT NULL DEFAULT 1
)
""")

tx_conn.execute("""
INSERT OR IGNORE INTO model_descriptions (
    model_id,
    description,
    version
)
VALUES (1, 'Initial fraud model', 1)
""")

tx_conn.commit()

current = tx_conn.execute(
    """
    SELECT description, version
    FROM model_descriptions
    WHERE model_id = 1
    """
).fetchone()

print("Read:", current)
# ...
display(pd.read_sql_query(
    "SELECT * FROM model_descriptions",
    tx_conn
))
```

## Row Count Is an Important Write Signal

For `UPDATE` and `DELETE`, affected-row count can be part of correctness.

Suppose an API expects exactly one model record to be updated.

If the database reports zero matching rows:

- the identifier may be wrong
- the record may have been deleted
- an optimistic version check may have failed
- the state may have changed concurrently

If unexpectedly many rows are affected, the predicate may be dangerously broad.

Production code can validate expected row counts rather than assuming the write did what the developer intended.

SQL

```
cursor = tx_conn.execute(
    """
    UPDATE model_descriptions
    SET description = ?
    WHERE model_id = ?
    """,
    ("Updated description", 999999),
)

print("Affected rows:", cursor.rowcount)

tx_conn.rollback()
```

### Safe UPDATE Workflow

For an important production update, a disciplined workflow might look like this:

1. define the intended business state transition
2. identify the precise target rows with `SELECT`
3. count and inspect the target
4. verify the environment/database
5. ensure indexes support the predicate when needed
6. understand constraints, triggers, and foreign keys
7. take appropriate backup/snapshot precautions
8. use an explicit transaction when appropriate
9. perform the modification
10. verify affected-row counts
11. query postconditions/invariants
12. commit only after validation
13. retain an audit trail where required

This does not mean every application update requires manual inspection. Automated systems encode these safeguards through code, tests, schema constraints, observability, permissions, and transaction logic.

### Safe DELETE Workflow

Deletion deserves even greater care because restoration may require backup recovery.

Before a high-risk delete:

Then:

Understand foreign-key behavior and cascades.

When possible, test the statement in a representative non-production environment.

For manual production maintenance, an explicit transaction may allow verification before commit:

If the result is incorrect before commit:

This safety depends on the engine and operation. Some database systems have DDL operations or administrative commands with transaction semantics different from ordinary DML.

SQL

```
SELECT ...  
FROM ...  
WHERE ...;
```

SQL

```
SELECT COUNT(*)  
FROM ...  
WHERE ...;
```

## Parameterization Applies to Writes Too

Parameterized SQL is not only for `SELECT`.

Use placeholders for values in inserts, updates, and deletes:

Do not interpolate untrusted values into SQL text:

Parameterization helps prevent SQL injection and correctly handles quoting and value conversion.

Parameters represent data values, not arbitrary SQL structure. Table names, column names, sort directions, and SQL keywords usually cannot be supplied through ordinary value placeholders. Dynamic SQL structure should therefore use strict allowlists or carefully designed query-building mechanisms.

PYTHON

```
conn.execute(  
    "UPDATE accounts SET status = ? WHERE account_id = ?",  
    (status, account_id)  
)
```

## Transactions and DDL

DDL means schema-changing operations such as:

- `CREATE`
- `ALTER`
- `DROP`

Transaction behavior for DDL varies by database system.

Some databases support strongly transactional schema changes. Others implicitly commit around certain operations or have special restrictions.

Do not assume that:

behaves identically across database products.

For production schema evolution, use migration tooling and understand the exact database's DDL transaction semantics, locking behavior, and deployment implications.

#### PYTHON

```
BEGIN
DROP TABLE ...
ROLLBACK
```

## Schema Migrations

Applications evolve, so schemas evolve too.

A migration is a controlled transition from one schema version to another.

Examples:

- adding a column
- introducing an index
- creating a new table
- backfilling data
- changing constraints
- splitting a table
- migrating enum-like state

Production migrations should be reproducible and version controlled.

Large migrations may need special designs because a single huge transaction can create long locks, massive logs, replication lag, or operational risk.

Safe database evolution is therefore both a SQL problem and a deployment-engineering problem.

## Permissions and Least Privilege

A client that only needs to read data should not automatically receive permission to delete tables.

Production databases typically support authorization mechanisms that separate capabilities such as:

- reading
- inserting
- updating
- deleting

- creating schema objects
- administrative operations

Least privilege reduces the blast radius of application bugs and credential compromise.

For example, an inference service may need to read model configuration and insert telemetry but have no reason to alter model registry history.

Database security should align permissions with service responsibilities.

## **Auditability**

For important state transitions, knowing the current value may not be enough.

Teams may need to know:

- what changed
- when it changed
- who or which service changed it
- what the previous value was
- what request initiated the change
- why the change occurred

Auditability can be implemented using dedicated history tables, event records, database features, immutable logs, or application-level audit systems.

Be careful when storing sensitive data in audit logs. Logging every previous value can create privacy and security risks.

Audit design should therefore consider both traceability and data governance.

## **AI Engineering: Metadata Is State**

AI systems often treat metadata as secondary, but operational metadata frequently determines what actually happens.

Examples include:

If these records disagree, the AI system can become operationally incorrect even when the model file itself is valid.

A transaction can protect coordinated metadata changes so downstream systems do not observe a partial state.

## PYTHON

```
model_version = fraud_v17
deployment_status = production
feature_snapshot = 2026-09-30
training_run = run_2841
artifact_uri = ...
schema_version = 4
approval_status = approved
```

## Atomic Model Registration

Imagine registering a trained model requires:

1. creating the model-version record
2. storing evaluation metrics
3. recording the artifact URI
4. associating the training dataset snapshot
5. marking registration complete

If these writes occur independently and one fails, the registry might contain an apparently valid model version missing essential provenance.

A transaction can make registration atomic:

The exact schema depends on the platform, but the engineering principle is general: related metadata that must agree should cross a transactional boundary together when the database architecture permits it.

## SQL

```

tx_conn.executescript("""
CREATE TABLE IF NOT EXISTS ai_models (
    model_id INTEGER PRIMARY KEY AUTOINCREMENT,
    model_name TEXT NOT NULL,
    model_version TEXT NOT NULL,
    artifact_uri TEXT NOT NULL,
    dataset_snapshot TEXT NOT NULL,
    status TEXT NOT NULL,
    UNIQUE(model_name, model_version)
);

CREATE TABLE IF NOT EXISTS ai_model_metrics (
    model_id INTEGER NOT NULL,
    metric_name TEXT NOT NULL,
    metric_value REAL NOT NULL,
    PRIMARY KEY (model_id, metric_name),
    FOREIGN KEY (model_id) REFERENCES ai_models(model_id)
);
""")

tx_conn.commit()

def register_model(
    conn,
    model_name,
    model_version,
    artifact_uri,
    # ...
    ),

print("Registered model_id:", registered_id)

```

## Database Atomicity Does Not Automatically Include Model Files

Suppose the database transaction records:

while the model file is being written to object storage or a filesystem.

A SQL transaction cannot normally roll back the external file write.

This creates a distributed consistency problem.

Possible robust architectures include:

- write and verify the artifact first, then register it
- use immutable artifact paths
- store checksums in metadata
- register only after upload verification



- use explicit lifecycle states such as `uploading`, `ready`, and `failed`
- reconcile orphaned artifacts
- make registration idempotent
- use workflow orchestration

The database transaction protects database state. It does not magically make external storage part of the same atomic unit.

PYTHON

```
artifact_uri = models/fraud/v2/model.bin
```

## Deployment Pointer Updates

A production AI system may have many registered model versions but only one active version for a particular environment.

For example:

Changing this pointer is a state transition.

If deployment metadata is spread across multiple rows, an atomic transaction can prevent intermediate states such as:

- both versions marked active
- no version marked active
- active pointer referencing an unapproved version

Better schemas can also encode invariants directly. Sometimes a single deployment row referencing the active model is simpler and safer than maintaining multiple independent Boolean `is_active` flags.

PYTHON

```
environment = production
active_model = fraud_detector:v17
```

## Feature Pipeline Transaction Boundaries

Feature pipelines commonly produce data in stages.

A dangerous workflow is:

1. delete yesterday's feature rows
2. insert part of today's features
3. job crashes
4. downstream training reads incomplete data

A safer architecture may use:

- staging tables
- batch identifiers
- transactional publication
- immutable snapshots
- status metadata
- atomic pointer changes

The broad pattern is:

This same pattern appears repeatedly in reliable AI engineering, from dataset versions to model artifacts and deployment configuration.

#### PYTHON

```
build privately  
validate  
publish atomically
```

### Exactly-Once Is Harder Than a Transaction

Suppose a prediction service:

1. receives a request
2. generates a prediction
3. writes a database event
4. returns a response

If the response is lost after the database commit, the client may retry. The database transaction succeeded, but the client does not know that.

This is why networked systems often operate with at-least-once retries and need idempotency or deduplication.

"Exactly once" behavior across clients, databases, queues, external APIs, and object stores is a distributed-systems problem.

SQL transactions are essential, but they solve a defined local consistency problem rather than every end-to-end reliability problem.

### Retry Logic Must Be Selective

Not every transaction failure should be retried.

Potentially retryable situations include certain:

- deadlocks
- serialization conflicts
- temporary lock-contention errors

- transient connection failures

Non-retryable failures often include:

- invalid data
- violated business rules
- malformed SQL
- missing required permissions
- deterministic constraint failures

Blind retries can make incidents worse.

A robust retry strategy usually includes:

- classification of transient errors
- bounded retry counts
- exponential backoff
- jitter
- idempotent operation design
- logging and monitoring

## Transaction Error Handling Pattern

A general application pattern is:

One subtle issue is that after some database errors, a transaction may remain in a failed state until explicitly rolled back. Behavior varies by driver/database.

Therefore, exception handling should follow the exact transaction semantics of the library being used rather than assuming every driver automatically resets itself.

PYTHON

```
begin transaction

try:
    perform validated writes
    verify required postconditions
    commit
except:
    rollback
    classify error
    retry only if appropriate
```

## Keep Business Decisions and Persistence Consistent

Suppose an application performs:

1. query current state

2. close transaction
3. decide what to do
4. later open another transaction
5. update based on the old state

Another client may change the database between steps 2 and 4.

For decisions that depend on current persistent state, validation and modification often need to occur within a concurrency-safe strategy.

Possible strategies include:

- conditional `UPDATE`
- optimistic version checks
- appropriate row locking
- serializable transactions
- uniqueness constraints
- state-transition predicates

Rather than saying "I checked earlier," robust code often says "perform the change only if the required state is still true now."

The predicate:

does more than locate the row. It encodes the precondition for the state transition.

If another worker has already changed the status, the affected-row count becomes zero.

This pattern is useful for job queues and workflow coordination because it combines validation and modification more tightly than separate "check then update" statements.

## SQL

```

tx_conn.execute("""
CREATE TABLE IF NOT EXISTS training_jobs (
    job_id INTEGER PRIMARY KEY,
    status TEXT NOT NULL
    CHECK (status IN ('queued', 'running', 'completed', 'failed'))
)
""")

tx_conn.execute("""
INSERT OR IGNORE INTO training_jobs (job_id, status)
VALUES (1, 'queued')
""")
tx_conn.commit()

# State transition succeeds only if the job is still queued.
cursor = tx_conn.execute(
    """
    UPDATE training_jobs
    SET status = 'running'
    WHERE job_id = ?
    AND status = 'queued'
    """,
    (1,),
)

tx_conn.commit()

print("Transitioned rows:", cursor.rowcount)

display(pd.read_sql_query(
    "SELECT * FROM training_jobs",
    tx_conn
))

```

## SQL

```

WHERE job_id = ?
AND status = 'queued'

```

## Transactions Do Not Replace Constraints

It is tempting to think application transactions make schema constraints less important.

The opposite is generally true.

Transactions and constraints solve complementary problems.

Transactions coordinate multiple changes.

Constraints reject invalid states.

For example, a transaction may transfer funds correctly, while:

provides an additional invariant at the schema boundary.

Similarly:

prevents duplicate model identities even if two clients race.

The strongest designs often combine:

- correct schema
- declarative constraints
- correct transactions
- appropriate isolation
- application validation
- operational monitoring

PYTHON

```
CHECK (balance >= 0)
```

## Common Data Modification Mistakes

Several mistakes recur in real SQL systems:

- forgetting the `WHERE` clause in `UPDATE`
- forgetting the `WHERE` clause in `DELETE`
- building SQL with untrusted string interpolation
- assuming a prior `SELECT` remains true under concurrency
- committing halfway through one logical operation
- holding transactions open during slow external work
- ignoring affected-row counts
- implementing identifiers with `MAX(id) + 1`
- disabling constraints to make writes "work"
- treating UPSERT syntax as portable across all databases
- retrying non-idempotent operations blindly
- assuming database rollback also undoes external API calls
- confusing application validation with database integrity
- using transactions without understanding isolation
- using overly broad service credentials

These are not merely syntax errors. They are failures of state-management design.

## A Practical Write Checklist

Before designing a database write, ask:

- What exact state transition is intended?
- Which rows are allowed to change?
- What invariants must remain true?
- Can constraints encode those invariants?
- Does the operation require multiple statements?
- Which statements belong to the same transaction?
- What should trigger rollback?
- What happens if two clients execute concurrently?
- Can the operation safely be retried?
- Is an idempotency key needed?
- Are external side effects involved?
- What affected-row count is expected?
- What should be audited?
- What is the recovery strategy if the operation fails?
- Does the service have only the permissions it needs?

These questions scale from small applications to production AI platforms.

### Data Modification and Transaction Engineering Summary

SQL writes are state transitions, not isolated syntax exercises.

`INSERT` creates rows, `UPDATE` modifies existing rows, and `DELETE` removes rows. All three interact with constraints, keys, relationships, permissions, and transaction boundaries.

UPSERT operations provide atomic conflict-aware insertion/update behavior, but syntax varies across database engines.

Transactions group related operations into logical units. `COMMIT` finalizes successful work, while `ROLLBACK` discards uncommitted changes. `SAVEPOINT` enables partial recovery inside larger transactions.

ACID provides a framework for understanding transactional guarantees:

- Atomicity protects against partial completion.
- Consistency preserves declared and correctly implemented invariants.
- Isolation controls interactions between concurrent transactions.
- Durability protects committed state.

Concurrency introduces anomalies such as lost updates, dirty reads, non-repeatable reads, phantoms, and more subtle multi-row consistency failures. Correct solutions depend on the database, workload, constraints, isolation level, and application architecture.

For AI engineering, transaction design matters anywhere metadata controls operational state: datasets, training jobs, model versions, metrics, artifact references, feature batches, and deployments.

The key engineering principle is:

Persist related state changes in a way that remains correct under failures, retries, and concurrency.

## What Comes Next

The next SQL section will move from transactional correctness into database performance and physical query execution.

Topics will include:

- indexes
- B-tree intuition
- single-column and composite indexes
- index selectivity
- covering-index awareness
- how predicates interact with indexes
- query plans
- `EXPLAIN QUERY PLAN` in SQLite
- sargability
- function-wrapped columns
- index trade-offs for writes
- statistics and optimizer awareness
- avoiding unnecessary scans
- query-performance methodology
- AI feature and retrieval workload implications

This connects logical SQL knowledge with how databases actually execute queries efficiently.

The next SQL section will move into database schema design and normalization in greater depth.

Topics will include:

- functional dependencies
- insertion anomalies
- update anomalies
- deletion anomalies
- first normal form
- second normal form
- third normal form
- BCNF awareness



- denormalization
- junction tables
- one-to-one relationships
- one-to-many relationships
- many-to-many relationships
- surrogate versus natural keys
- schema evolution
- modeling AI/ML metadata
- dimensional-modeling awareness
- facts and dimensions
- star-schema foundations

This will connect relational theory with practical database architecture.

The next SQL section will focus on production database architecture and operational engineering.

Topics will include:

- views
- temporary tables
- materialized-view awareness
- stored procedures and functions awareness
- triggers
- generated columns
- database migrations
- connection management
- connection pooling
- database backups
- restore strategy
- replication awareness
- read replicas
- sharding awareness
- partitioning in greater context
- OLTP versus OLAP architecture
- PostgreSQL/MySQL/SQLite differences
- database security
- roles and privileges
- secrets and credentials

- observability
- reliability
- choosing a database for AI applications

This will connect SQL fundamentals to production database systems.

The next SQL section will complete the major foundational SQL journey with advanced practical patterns and end-to-end database engineering.

It will cover topics such as:

- reusable analytical query design
- cohort-style SQL
- retention analysis
- funnels
- deduplication patterns
- gaps and islands
- relational division awareness
- pivoting and conditional aggregation
- dynamic SQL awareness
- JSON SQL awareness
- full-text-search awareness
- temporal SQL awareness
- audit/history patterns
- batch ingestion patterns
- incremental processing
- watermarking
- CDC awareness
- ETL versus ELT
- data quality checks
- testing SQL
- SQL style and maintainability
- production review checklist
- AI feature-pipeline SQL patterns
- end-to-end SQL engineering architecture

This final integration will connect querying, modeling, transactions, performance, and production operations into complete workflows.

The next SQL part should serve as the final consolidation and professional SQL engineering section.

It can cover:

- SQL dialect portability
- SQLite versus PostgreSQL versus MySQL syntax differences
- database drivers and Python DB-API patterns
- SQLAlchemy awareness
- ORM trade-offs
- repository/service architecture
- secure configuration
- realistic end-to-end relational application design
- database code review
- debugging incorrect SQL results
- debugging slow SQL
- production incident reasoning
- SQL interview foundations
- complete SQL revision map
- final AI-engineering relational case study

After that, the comprehensive SQL notebook can receive a dedicated end-of-unit practice notebook and separate solutions if desired, while keeping practice outside this theory notebook.

## Indexes, Query Plans, and SQL Performance Engineering

Writing logically correct SQL is only part of database engineering.

Two queries can return exactly the same result while requiring dramatically different amounts of work.

Consider:

If a database has millions of customers, should it inspect every row to find one identifier?

Often, no.

A database can maintain an additional data structure called an **index** that helps it locate relevant rows much more efficiently.

Performance engineering, however, is not simply:

Add indexes everywhere.

Indexes consume storage. They must be maintained during writes. Their usefulness depends on query patterns, predicates, ordering, selectivity, table size, data distribution, and database-engine behavior.

This section develops the foundations needed to reason about SQL performance rather than guessing.

We will study:

- why database queries become expensive
- table scans
- indexes
- B-tree intuition
- indexed lookup
- primary-key indexing awareness
- single-column indexes
- unique indexes
- composite indexes
- column order
- leftmost-prefix intuition
- covering-index awareness
- selectivity and cardinality
- indexes and `NULL`
- indexes and sorting
- indexes and grouping
- indexes and joins
- query plans
- SQLite `EXPLAIN QUERY PLAN`
- `SCAN` versus `SEARCH`
- sargability
- function-wrapped columns
- expression indexes
- partial indexes
- index write overhead
- over-indexing
- optimizer statistics
- `ANALYZE`
- performance measurement
- query rewriting
- pagination performance
- aggregation performance
- AI/data-engineering workload implications

The key principle is:

Optimize measured workloads using evidence from query plans and realistic data, not intuition alone.

SQL

```
SELECT *  
FROM customers  
WHERE customer_id = 918273;
```

## Performance Begins with Work

A SQL query describes the result we want.

It usually does not specify every physical operation required to obtain that result.

For example:

The database might:

1. inspect every row,
2. use an index,
3. use an index that already contains all requested columns,
4. choose another strategy based on statistics.

The database component responsible for selecting an execution strategy is commonly called the **query optimizer**.

The optimizer considers candidate access paths and attempts to choose a sufficiently efficient plan.

Therefore SQL performance has at least two layers:

- logical query design
- physical execution strategy

Understanding both is necessary for systematic optimization.

SQL

```
SELECT customer_id, email  
FROM customers  
WHERE email = 'user@example.com';
```

## Creating a Performance Laboratory

Performance concepts become easier to understand with enough rows for different access strategies to be meaningful.

We will create a separate in-memory SQLite connection and populate a synthetic event table.

The data is generated entirely inside this notebook. No external dataset is required.

The examples focus on query-plan structure rather than claiming universal benchmark timings, because actual runtimes depend on hardware, SQLite version, caching, row counts, and system load.

SQL

```
import sqlite3
import random
import time
import statistics
import pandas as pd
from IPython.display import display

perf_conn = sqlite3.connect(":memory:")

perf_conn.execute("PRAGMA foreign_keys = ON")

perf_conn.executescript("""
CREATE TABLE users (
    user_id INTEGER PRIMARY KEY,
    region TEXT NOT NULL,
    plan TEXT NOT NULL,
    signup_date TEXT NOT NULL
);

CREATE TABLE events (
    event_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    event_type TEXT NOT NULL,
    event_date TEXT NOT NULL,
    score REAL,
    processed INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
# ...
print(
    "Events:",
    perf_conn.execute("SELECT COUNT(*) FROM events").fetchone()[0]
)
```

## A Table Scan

Suppose we search for:

and no useful index exists on `event_type`.

One possible strategy is to examine the table row by row and test the predicate.

Conceptually:

This is commonly called a **table scan** or full scan.

A scan is not automatically bad.

Scanning may be perfectly reasonable when:

- the table is small
- most rows are required
- no useful index exists
- the query performs a large analytical aggregation
- using an index would require many expensive row lookups
- sequential access is cheaper for the workload

Performance engineering therefore should not treat the word `SCAN` as an automatic failure.

SQL

```
def explain(conn, sql, params=()):
    rows = conn.execute(
        "EXPLAIN QUERY PLAN " + sql,
        params
    ).fetchall()

    return pd.DataFrame(
        rows,
        columns=["id", "parent", "notused", "detail"]
    )

query = """
SELECT event_id, user_id, event_date
FROM events
WHERE event_type = ?
"""

display(explain(perf_conn, query, ("purchase",)))
```

SQL

```
WHERE event_type = 'purchase'
```

## EXPLAIN QUERY PLAN

SQLite provides:

It describes the high-level strategy SQLite intends to use for a query.

For example:

The output may contain descriptions such as:

or:

The exact wording can vary between SQLite versions.

A query plan is evidence about execution strategy. It is generally much more useful than staring at SQL and guessing whether an index is being used.

Other database systems provide their own explain facilities, often with substantially more detail.

SQL

```
EXPLAIN QUERY PLAN ...
```

SQL

```
EXPLAIN QUERY PLAN
SELECT ...
FROM events
WHERE event_type = 'purchase';
```

## What Is an Index?

A database index is an auxiliary data structure designed to make particular data-access patterns faster.

A useful analogy is the index at the back of a textbook.

Without a textbook index, finding every reference to "transactions" might require reading every page.

With an index, you locate the term and obtain pointers to the relevant pages.

A database index works at a much more sophisticated level, but the broad idea is similar:

Because the index is maintained separately from the table, it introduces trade-offs:

An index is therefore not a free acceleration switch.

PYTHON

```
indexed value -> row location / row identity
```

## B-Tree Intuition

Many traditional relational database indexes are implemented using B-tree-family structures.

A B-tree keeps keys ordered in a balanced tree so the engine can eliminate large portions of the search space rather than scanning every entry.

Conceptually, instead of:

the structure behaves more like:

At each level, the search chooses a smaller relevant range.

B-trees are also useful for range queries because indexed values are ordered.

Examples include:

The actual storage representation is more complex, involving pages, node capacities, row references, balancing, and database-specific implementation details. The important intuition is that ordered tree structure enables efficient point and range navigation.



## PYTHON

```
1 -> 2 -> 3 -> 4 -> ... -> 1,000,000
```

## PYTHON

```
root
  /   |   \
  /   |   \
 lower middle higher
  /   |   \
  / ...   ...   ...
```

## Creating a Single-Column Index

SQLite uses:

For our event query:

After creating the index, we can inspect the same query plan again.

The important experiment is to compare strategy before and after the schema change rather than assuming the optimizer will necessarily use every index we create.

## SQL

```
perf_conn.execute("""
CREATE INDEX idx_events_event_type
ON events(event_type)
""")

perf_conn.commit()

display(explain(
    perf_conn,
    """
    SELECT event_id, user_id, event_date
    FROM events
    WHERE event_type = ?
    """,
    ("purchase",)
))
```

## SQL

```
CREATE INDEX index_name
ON table_name(column_name);
```

## SEARCH Versus SCAN

In SQLite query-plan output, a plan containing:

usually indicates that SQLite is using an index access strategy to identify qualifying rows.

A plan containing:

usually indicates broader scanning behavior.

However, interpretation needs context.

For example:

naturally requires all rows. A scan is appropriate.

Similarly, if a predicate matches most of a table, an index may provide little benefit because the database would still need to access a large fraction of the rows.

The target is not:

Eliminate every scan.

The target is:

Avoid unnecessary work for the actual workload.

PYTHON

```
SEARCH events USING INDEX ...
```

## Primary Keys and Automatic Indexing

Primary keys and unique constraints often cause the database to maintain index-like structures automatically.

For example:

allows efficient row identity lookup in SQLite, with SQLite-specific implementation details for integer primary keys.

Other constraints such as:

also normally require an efficient mechanism for checking uniqueness.

This means you should inspect the existing schema before creating redundant indexes.

Creating another index on a column already efficiently indexed by a primary-key or uniqueness mechanism may waste storage and write effort.

SQL

```
display(explain(  
  perf_conn,  
  ""  
  SELECT *  
  FROM users  
  WHERE user_id = ?  
  "" ,  
  (2500,)  
))
```

SQL

```
user_id INTEGER PRIMARY KEY
```

## Index Selectivity

**Selectivity** describes how strongly a condition narrows the data.

Suppose a table has one million rows.

A lookup returning one row:

is highly selective.

A condition matching 900,000 rows:

is not very selective.

Highly selective predicates are often strong index candidates because the index can reduce the amount of data accessed substantially.

Low-selectivity columns are more nuanced.

A Boolean column with only:

may not be useful as a standalone general-purpose index if one value dominates the table. But workload-specific designs, composite indexes, or partial indexes can still make such data useful.

Selectivity is therefore a property of both values and workload.

SQL

```
display(pd.read_sql_query(
    """
    SELECT
        event_type,
        COUNT(*) AS rows_in_group,
        ROUND(
            100.0 * COUNT(*) /
            (SELECT COUNT(*) FROM events),
            2
        ) AS percent_of_table
    FROM events
    GROUP BY event_type
    ORDER BY rows_in_group DESC
    """,
    perf_conn
))
```

SQL

```
WHERE customer_id = 417821
```

## Cardinality and Selectivity Are Related but Different Ideas

Cardinality often refers to the number of distinct values in a column or relation.

For example:

Columns with high cardinality often support selective equality lookups, but cardinality alone does not determine index usefulness.

A date column might have moderate cardinality yet be extremely useful for range queries.

A low-cardinality status column might become useful when combined with another column in a composite index.

Always connect data distribution to actual predicates.

SQL

```
display(pd.read_sql_query(
    """
    SELECT
        COUNT(*) AS total_rows,
        COUNT(DISTINCT user_id) AS user_cardinality,
        COUNT(DISTINCT event_type) AS event_type_cardinality,
        COUNT(DISTINCT event_date) AS event_date_cardinality,
        COUNT(DISTINCT processed) AS processed_cardinality
    FROM events
    """,
    perf_conn
))
```

## Composite Indexes

An index can contain more than one column.

For example:

This is called a composite or multi-column index.

Composite indexes are essential because real queries frequently filter or order using combinations of columns.

For example:

The index ordering:

organizes entries first by `user_id` and then, within a user's entries, by `event_date`.

That structure aligns naturally with the query above.

SQL

```
perf_conn.execute("""
CREATE INDEX idx_events_user_date
ON events(user_id, event_date)
""")

perf_conn.commit()

display(explain(
    perf_conn,
    """
    SELECT event_id, event_date, event_type
    FROM events
    WHERE user_id = ?
    AND event_date >= ?
    """,
    (1234, "2026-06-01")
))
```

SQL

```
CREATE INDEX idx_events_user_date
ON events(user_id, event_date);
```

## Composite Index Column Order Matters

These indexes are not conceptually identical:

and:

The tree is ordered according to the index column sequence.

For:

rows are conceptually organized like:

This is useful when the query first narrows by `user_id`.

Index design therefore requires understanding common predicate combinations rather than simply collecting all frequently used columns.

PYTHON

```
(user_id, event_date)
```

PYTHON

```
(event_date, user_id)
```

### Leftmost-Prefix Intuition

A useful B-tree composite-index rule is the **leftmost-prefix** idea.

Given:

the ordered structure naturally begins with `user_id`.

Queries that constrain leading columns often align well with the index.

Conceptually useful patterns include:

A predicate only on:

does not directly use the leading organization of this index in the same way.

Modern optimizers can have additional strategies, and exact behavior is engine-specific, so this should be treated as strong indexing intuition rather than a universal simplistic law.

## SQL

```

print("Predicate on leading column:")
display(explain(
    perf_conn,
    """
    SELECT event_id
    FROM events
    WHERE user_id = ?
    """,
    (1234,)
))

print("\nPredicate only on second composite-index column:")
display(explain(
    perf_conn,
    """
    SELECT event_id
    FROM events
    WHERE event_date = ?
    """,
    ("2026-06-15",)
))

```

## Equality Before Range — Useful Design Intuition

Consider:

The predicate on `user_id` is equality, while the date predicate defines a range.

An index:

is naturally useful because the engine can locate one user's portion of the index and then navigate the relevant date range.

A practical index-design heuristic is often:

Place important equality predicates before range predicates.

But heuristics are not substitutes for plans and workload analysis.

Ordering may also need to support `ORDER BY`, grouping, uniqueness, or other queries. The best index for one query may not be the best overall index set for the application.

## PYTHON

```

WHERE user_id = ?
AND event_date >= ?

```

## PYTHON

```
(user_id, event_date)
```

## Covering Indexes

Normally an index helps identify rows, after which the database may need to fetch additional columns from the table.

But suppose an index already contains every value required by the query.

Then the engine may be able to answer the query directly from the index.

This is commonly called a **covering index** situation.

For example, if we have:

and query only:

the required values may all be available within the index structure.

SQLite query plans may explicitly mention:

when such an optimization is used.

## SQL

```
perf_conn.execute("""
CREATE INDEX idx_events_user_date_type
ON events(user_id, event_date, event_type)
""")

perf_conn.commit()

display(explain(
    perf_conn,
    """
    SELECT event_date, event_type
    FROM events
    WHERE user_id = ?
    """,
    (1234,)
))
```

## SQL

```
INDEX (user_id, event_date, event_type)
```

## Why Covering Indexes Can Help

Suppose an index contains only the search key.

Execution can conceptually require:



A covering index may reduce this to:

Avoiding extra table accesses can improve performance, particularly for frequent queries.

However, adding more columns makes indexes larger and increases write-maintenance cost.

Do not create giant covering indexes for every query. This is another read/write/storage trade-off.

PYTHON

```
search index
->
obtain row reference
->
visit table row
->
read requested columns
```

## Indexes and ORDER BY

Because B-tree indexes maintain ordered keys, they can sometimes help satisfy `ORDER BY`.

Consider:

An index on:

already organizes one user's events by date.

The optimizer may therefore avoid building a separate temporary sorting structure.

Without a compatible ordering, SQLite may report the use of a temporary B-tree for sorting.

This is one reason indexes affect more than `WHERE`.

SQL

```
display(explain(
  perf_conn,
  """
  SELECT event_id, event_date
  FROM events
  WHERE user_id = ?
  ORDER BY event_date
  """,
  (1234,)
))
```

SQL

```
SELECT ...
FROM events
WHERE user_id = ?
ORDER BY event_date;
```

## Descending Order

Database indexes may support ascending and descending traversal.

For many B-tree implementations, an index can be scanned in reverse, so an ascending index may still support:

in useful cases.

Multi-column mixed-direction ordering can be more nuanced, and database engines have different capabilities.

Again, verify with the execution plan instead of relying entirely on assumptions.

SQL

```
display(explain(  
  perf_conn,  
  ""  
  SELECT event_id, event_date  
  FROM events  
  WHERE user_id = ?  
  ORDER BY event_date DESC  
  LIMIT 10  
  "" ,  
  (1234,)  
))
```

SQL

```
ORDER BY event_date DESC
```

## Indexes and LIMIT

Indexes become especially useful when the database needs only the first few rows in an ordered subset.

For example:

With a compatible index, the engine may locate the relevant region and stop after obtaining ten qualifying rows.

Without a useful index, it might need to:

1. inspect many rows,
2. collect matches,
3. sort them,
4. return only ten.

This illustrates an important performance idea:

The number of returned rows and the amount of work performed are not necessarily the same.

SQL

```
SELECT ...  
FROM events  
WHERE user_id = ?  
ORDER BY event_date DESC  
LIMIT 10;
```

## Indexes and JOIN

Join performance often depends strongly on efficient lookup of join keys.

Consider:

`users.user_id` is a primary key, so looking up a user by ID is efficient.

Foreign-key columns are particularly important to consider for indexing because they frequently participate in joins and referential operations.

However, foreign-key declarations do not universally mean that every database engine automatically creates every index needed for optimal join performance.

Schema designers should inspect plans and workload patterns.

SQL

```
display(explain(  
  perf_conn,  
  ""  
  SELECT  
    e.event_id,  
    e.event_type,  
    u.region  
  FROM events AS e  
  JOIN users AS u  
    ON u.user_id = e.user_id  
  WHERE e.user_id = ?  
  "" ,  
  (1234,) ) )
```

SQL

```
SELECT ...  
FROM events AS e  
JOIN users AS u  
  ON u.user_id = e.user_id;
```

## Join Order

For multi-table queries, the optimizer must decide more than which indexes to use.

It may also choose:

- which table to access first
- which table becomes the outer input
- which lookup strategy applies to each subsequent table

Suppose one predicate reduces a million-row table to ten rows. Starting with those ten rows and performing indexed lookups may be much cheaper than generating a massive intermediate join first.

Modern optimizers attempt to reason about these alternatives using statistics and cost estimates.

This is why data distribution matters to execution planning.

## Indexes and GROUP BY

Indexes can sometimes help grouping when their ordering aligns with the grouping keys.

For example:

may benefit from an index ordered by `user_id`.

But analytical aggregation can still require reading a large portion of the data. An index cannot make unavoidable work disappear.

For example:

must account for all relevant event scores.

An index may influence ordering/access, but the aggregation still processes the contributing rows.

For large analytics workloads, database architecture, partitioning, columnar storage, materialized summaries, and distributed execution may matter more than ordinary OLTP-style indexing alone.

SQL

```
GROUP BY user_id
```

SQL

```
SELECT user_id, SUM(score)
FROM events
GROUP BY user_id;
```

## Indexes and NULL

Indexes can interact with `NULL` differently depending on the database and operation.

Queries such as:

may use an index in many modern relational engines.

However, `NULL` also affects:

- uniqueness semantics
- comparisons
- ordering
- filtered/partial indexes
- selectivity

Do not rely on folklore such as "indexes do not store NULL" as a universal SQL rule. Behavior varies by database system and index type.

PYTHON

```
WHERE column_name IS NULL
```

## Sargability

A predicate is often called **sargable** when it can effectively use a search argument against an index.

Compare:

with:

The first predicate compares the indexed column directly against boundaries.

The second applies a function to the column before comparison.

Traditional B-tree indexes on the raw column often work more naturally with the first form.

Sargability is therefore about expressing conditions in a way compatible with available access structures.

SQL

```

perf_conn.execute("""
CREATE INDEX IF NOT EXISTS idx_events_event_date
ON events(event_date)
""")

perf_conn.commit()

print("Range predicate:")
display(explain(
    perf_conn,
    """
SELECT COUNT(*)
FROM events
WHERE event_date >= ?
      AND event_date < ?
""",
    ("2026-06-01", "2026-07-01")
))

print("\nFunction-wrapped predicate:")
display(explain(
    perf_conn,
    """
SELECT COUNT(*)
FROM events
WHERE substr(event_date, 1, 7) = ?
""",
    ("2026-06",)
))

```

SQL

```

WHERE event_date >= '2026-06-01'
AND event_date < '2026-07-01'

```

## Why Function-Wrapped Columns Can Hurt Index Use

Imagine an index storing:

The query:

directly describes a contiguous range in that ordering.

But:

asks the database to evaluate a transformed expression.

A normal index on the original column is not necessarily organized by that expression.

Some databases support expression/function indexes specifically to solve this problem.

## PYTHON

```
2026-01-01
2026-01-02
2026-01-03
...
```

## PYTHON

```
event_date >= '2026-06-01'
AND event_date < '2026-07-01'
```

## Expression Indexes

SQLite supports indexes on expressions in suitable cases.

For example:

Now a query using the same expression may be able to use that index.

Expression indexes can be valuable when applications frequently search by a deterministic transformation.

However, before adding one, consider whether the underlying schema should represent the information differently or whether a range predicate is simpler.

Every new index still adds storage and maintenance cost.

## SQL

```
perf_conn.execute("""
CREATE INDEX idx_events_month
ON events(substr(event_date, 1, 7))
""")

perf_conn.commit()

display(explain(
    perf_conn,
    """
    SELECT COUNT(*)
    FROM events
    WHERE substr(event_date, 1, 7) = ?
    """,
    ("2026-06",)
))
```

## Sargable Date Filtering

Date filtering is one of the most common places where sargability matters.

Instead of conceptually writing:

a B-tree-friendly approach is often:

The half-open interval:

is especially useful for timestamps because it avoids trying to represent the final instant of the year.

This same pattern works for days, months, weeks, and feature-extraction windows.

PYTHON

```
WHERE YEAR(created_at) = 2026
```

PYTHON

```
WHERE created_at >= '2026-01-01'
AND created_at < '2027-01-01'
```

## LIKE and Index Use

Text-search performance depends on the pattern and database rules.

A prefix search such as:

can sometimes use an appropriate ordered index.

A leading-wildcard search:

usually cannot navigate a normal B-tree directly to the beginning of the matching range because the beginning of the string is unknown.

For substring search at scale, specialized full-text indexing or search-engine technology may be more appropriate.

Collation and case-sensitivity rules also influence index usability.

PYTHON

```
WHERE name LIKE 'data%'
```

PYTHON

```
WHERE name LIKE '%data%'
```

## Implicit Type Conversion

Predicates that compare incompatible types may cause conversion behavior that harms correctness or index usability.

For example, storing numeric identifiers as inconsistent text values can force conversions and make query behavior harder to reason about.

Good schema design starts with appropriate types.

Performance tuning should not be used to compensate for avoidable type inconsistency.



Different database engines have different coercion rules, so never assume a query comparing a numeric column to arbitrary textual data behaves identically everywhere.

## Partial Indexes

A partial index stores only rows satisfying a condition.

SQLite example:

This can be useful if:

- unprocessed rows are a small subset,
- applications frequently query them,
- processed historical rows dominate the table.

Instead of indexing the entire table for that workload, the index focuses on the relevant subset.

Partial indexes can reduce index size and maintenance compared with a broader index.

SQL

```
perf_conn.execute("""
CREATE INDEX idx_unprocessed_events
ON events(event_date)
WHERE processed = 0
""")

perf_conn.commit()

display(explain(
    perf_conn,
    """
    SELECT event_id, event_date
    FROM events
    WHERE processed = 0
    ORDER BY event_date
    LIMIT 20
    """)
))
```

## Partial Indexes and AI Job Queues

Partial indexes map naturally to workflow tables.

Suppose an inference-processing queue contains millions of completed tasks but only a few thousand currently waiting:

A partial index over pending records can support worker queries without maintaining a large general-purpose index containing every historical completed job.

For example, in a database supporting the required syntax:

This is especially useful when active operational state is small relative to historical state.

The exact support and syntax vary by database engine.

PYTHON

```
status = pending
```

PYTHON

```
CREATE INDEX ...  
ON jobs(created_at)  
WHERE status = 'pending';
```

## Unique Indexes

A unique index combines lookup support with an integrity rule.

For example:

The database can efficiently locate the key while also preventing duplicates.

However, if the same rule can be expressed naturally as a schema-level **UNIQUE** constraint, that form may communicate intent more clearly.

Use indexing not only as a performance concern but in coordination with relational integrity design.

PYTHON

```
CREATE UNIQUE INDEX ...  
ON model_versions(model_name, version);
```

## Too Many Indexes

If indexes accelerate reads, why not index every column and every possible combination?

Because every index has costs.

For an insert, the database may need to:

Updates to indexed columns similarly require index maintenance.

Additional indexes also consume:

- disk/storage
- cache memory
- backup size
- maintenance effort
- optimizer planning possibilities

Redundant indexes can therefore make write-heavy systems slower.

Index design is a workload trade-off, not a contest to maximize index count.

## SQL

```
write table row
update index A
update index B
update index C
update index D
...
```

## Read-Heavy Versus Write-Heavy Workloads

Indexing strategy depends on workload.

A read-heavy reporting or serving system may benefit from additional indexes that accelerate frequent queries.

A write-heavy ingestion system may need a more conservative index set because every event insertion must maintain each relevant index.

For AI engineering:

may favor different physical design from:

and both may differ from:

Architecture should reflect workload characteristics rather than applying one indexing template everywhere.

## PYTHON

```
offline analytics
```

## PYTHON

```
online inference telemetry ingestion
```

## Measuring Write Overhead Conceptually

It is possible to benchmark inserts with different index configurations, but microbenchmarks must be interpreted carefully.

Results depend on:

- transaction batching
- storage device
- journaling mode
- cache state
- database configuration
- data size
- page layout
- index count

- index width

A notebook benchmark can demonstrate that indexes require maintenance, but it should not be used to claim universal percentages such as "an index makes inserts 40% slower."

Measure your actual production-like workload.

## Query Optimizer

The query optimizer attempts to choose an efficient physical execution plan for declarative SQL.

It may consider:

- available indexes
- table sizes
- estimated selectivity
- join order
- sort requirements
- grouping requirements
- covering indexes
- temporary structures
- available statistics

The optimizer is one reason SQL is powerful: developers specify what data they need while the database chooses many physical execution details.

However, the optimizer depends on information and estimates. Poor statistics, unusual data distributions, or difficult predicates can lead to suboptimal decisions.

## Statistics

The optimizer needs information about data distribution to estimate costs.

Statistics may describe properties such as:

- row counts
- distinct-value counts
- index distributions
- value frequencies
- histograms

The exact statistics system varies by database.

If the optimizer incorrectly estimates that a predicate returns 10 rows when it actually returns 10 million, it may choose a poor join order or access strategy.

Database maintenance therefore includes keeping optimizer statistics sufficiently representative for the engine being used.

## ANALYZE in SQLite

SQLite provides:

to gather statistics that can help the query planner make better choices.

The collected information is stored in SQLite's internal statistics tables, depending on SQLite build/version/features.

Production databases have their own statistics collection and auto-maintenance systems.

Do not casually manipulate optimizer statistics manually. Use documented database mechanisms.

SQL

```
perf_conn.execute("ANALYZE")

stats_tables = perf_conn.execute("""
SELECT name
FROM sqlite_master
WHERE type = 'table'
      AND name LIKE 'sqlite_stat%'
ORDER BY name
""").fetchall()

print("SQLite statistics tables:")
for row in stats_tables:
    print(row[0])
```

SQL

```
ANALYZE;
```

## EXPLAIN Versus EXPLAIN QUERY PLAN

SQLite provides both concepts.

`EXPLAIN QUERY PLAN` gives a relatively high-level description of the intended strategy.

`EXPLAIN` can expose lower-level SQLite virtual-machine instructions.

For learning index behavior, `EXPLAIN QUERY PLAN` is usually easier to interpret.

For deep SQLite internals, the lower-level form becomes useful.

Other databases may provide options such as:

The exact tools differ substantially by platform.

## PYTHON

```
EXPLAIN
EXPLAIN ANALYZE
actual row counts
buffers
timing
I/O metrics
```

## Estimated Plans Versus Actual Execution

Some database systems distinguish between:

- estimated plan
- actual execution information

An estimated plan predicts what will happen using optimizer estimates.

Actual execution tooling may run the query and report observed row counts, timing, loops, I/O, or memory behavior.

This distinction matters because a plan can reveal:

Such discrepancies often indicate a statistics or data-distribution problem.

Be cautious with tools that actually execute queries. Running an expensive `UPDATE`, `DELETE`, or massive analytical query merely to inspect runtime behavior can have real consequences.

## PYTHON

```
estimated rows = 10
actual rows = 1,000,000
```

## Query Timing

Wall-clock timing can be useful, but a single timing measurement is weak evidence.

Database performance is affected by caching.

The first query might load pages from storage. Later queries may find those pages already cached.

Other influences include:

- concurrent workload
- CPU scheduling
- filesystem cache
- network latency
- connection setup
- transaction state
- background maintenance

Therefore:

- measure multiple times
- separate warm and cold-cache questions
- use realistic parameters
- inspect plans
- benchmark representative data volumes
- report distributions rather than one lucky timing

The timing above is specific to this notebook environment.

Its purpose is to demonstrate a measurement method, not establish a universal performance number.

For reliable engineering decisions, compare candidate designs under representative:

- data size
- data distribution
- concurrency
- hardware
- database configuration
- query mix

## SQL

```
def benchmark_query(conn, sql, params=(), repeats=7):
    durations = []

    # Warm-up
    conn.execute(sql, params).fetchall()

    for _ in range(repeats):
        start = time.perf_counter()
        conn.execute(sql, params).fetchall()
        durations.append(time.perf_counter() - start)

    return {
        "runs": repeats,
        "median_ms": statistics.median(durations) * 1000,
        "min_ms": min(durations) * 1000,
        "max_ms": max(durations) * 1000,
    }

result = benchmark_query(
    perf_conn,
    """
    SELECT event_id, event_date
    FROM events
    WHERE user_id = ?
    ORDER BY event_date
    """,
    (1234,),
)

result
```

## Do Not Use SELECT \* Without Thinking

`SELECT *` is not inherently forbidden, but production queries should consider whether every column is required.

Fetching unnecessary columns can increase:

- storage reads
- memory usage
- network transfer
- serialization cost
- Python object creation
- downstream processing

It can also prevent a query from being satisfied using a narrower covering index.

For APIs and feature pipelines, select the columns actually required by the operation.



## SQL

```
print("Narrow query plan:")

display(explain(
    perf_conn,
    """
    SELECT event_date, event_type
    FROM events
    WHERE user_id = ?
    """,
    (1234,)
))

print("\nWide query plan:")

display(explain(
    perf_conn,
    """
    SELECT *
    FROM events
    WHERE user_id = ?
    """,
    (1234,)
))
```

## Filtering Early: A Nuanced Rule

People often say:

Filter as early as possible.

This is useful as logical intuition, but SQL is declarative. The optimizer can often rearrange relational operations while preserving semantics.

You therefore do not need to force every filter into deeply nested subqueries merely to make it "run first."

Instead:

- express correct predicates clearly
- make them sargable where appropriate
- provide useful indexes
- inspect the plan

There are situations where query shape affects optimization, especially with CTE materialization, complex views, correlated subqueries, or engine-specific optimizer limitations. But generic "SQL executes exactly top-to-bottom" reasoning is incorrect.

## OR Predicates

Predicates using `OR` can be harder to optimize than simple single-index searches.

For example:

Depending on available indexes and optimizer capabilities, the engine may combine multiple index searches or choose another strategy.

In some situations, logically equivalent `UNION ALL` or `UNION` formulations can perform differently.

Do not rewrite automatically. Compare correctness first and plans second.

Remember that `UNION` removes duplicates while `UNION ALL` does not, so they are not interchangeable.

SQL

```
display(explain(  
    perf_conn,  
    """  
    SELECT event_id  
    FROM events  
    WHERE user_id = ?  
       OR event_type = ?  
    """,  
    (1234, "purchase")  
))
```

SQL

```
WHERE user_id = ?  
    OR event_type = ?
```

## IN Predicates

Queries such as:

can often make effective use of an index.

The database can conceptually perform several indexed key lookups rather than scanning the entire table.

Very large `IN` lists, however, may become awkward. Depending on the workload, it may be better to:

- load IDs into a temporary/staging table
- join against a relation
- use database-specific bulk techniques

Query design should remain relational and manageable as input size grows.

PYTHON

```
WHERE user_id IN (10, 20, 30)
```

## NOT EQUAL and Broad Predicates

Conditions such as:

may match most of a table.

Even with an index on `status`, the optimizer may determine that scanning is more economical.

This illustrates an important point:

An index existing does not imply that using it is cheaper.

The optimizer is allowed to ignore indexes.

PYTHON

```
status <> 'deleted'
```

## OFFSET Pagination Performance

Earlier we used:

for pagination.

Large offsets can become inefficient because the database may still need to navigate or process many preceding rows before returning the desired page.

The application requests only 20 rows, but the engine may perform much more work.

This is one reason large production feeds frequently use **keyset pagination**.

PYTHON

```
LIMIT 20 OFFSET 100000
```

## Stable Pagination Requires Deterministic Ordering

Ordering only by:

may not uniquely order rows because many events can share the same date.

A robust pagination key therefore often includes a unique tiebreaker:

The cursor then contains both values.

This is both a correctness issue and a performance-design issue.

Stable ordering prevents rows from being unpredictably duplicated or skipped between pages merely because equal sort keys have no deterministic order.

PYTHON

```
ORDER BY event_date
```

PYTHON

```
ORDER BY event_date, event_id
```

## COUNT Performance

A query such as:

looks simple but may require significant work depending on the database architecture.

Do not assume every relational database stores an instantly available exact row count.

Likewise:

can require substantial additional work because uniqueness must be determined.

For large analytics systems, approximate counts, metadata estimates, pre-aggregated tables, or specialized engines may be appropriate depending on accuracy requirements.

SQL

```
SELECT COUNT(*)  
FROM huge_table;
```

## DISTINCT Can Be Expensive

`DISTINCT` is often used casually to "fix duplicates":

But duplicate elimination may require sorting, hashing, or other work.

More importantly, unexpected duplicates frequently indicate a logical problem:

- incorrect join cardinality
- missing join key
- many-to-many relationship
- wrong grain
- accidental data duplication

Performance tuning should not hide correctness problems behind `DISTINCT`.

First understand why duplicates exist. Then decide whether duplicate elimination is semantically necessary.

SQL

```
SELECT DISTINCT ...
```

## Sorting Is Not Free

`ORDER BY` can require the database to materialize and sort a large result.

Sorting cost depends on:

- number of rows
- width of rows
- available memory
- sort keys
- index compatibility
- whether sorting spills to storage

A query should not add `ORDER BY` unless ordering is actually required.

SQL relations do not have an inherent presentation order. If deterministic order matters, request it explicitly—but understand its cost.

## Temporary B-Trees in SQLite Plans

SQLite query plans may report messages such as:

or for grouping/distinct operations.

This indicates that SQLite expects to construct a temporary structure to satisfy part of the query.

That is not automatically a problem.

But for a high-frequency query over large data, seeing such a step can motivate investigation:

- can an existing index provide the ordering?
- is the sort actually necessary?
- are too many rows being sorted?
- can the result be narrowed first?

SQL

```
# A query whose ordering is not aligned with the main filtering index.
display(explain(
  perf_conn,
  """
  SELECT event_id, user_id, score
  FROM events
  WHERE event_type = ?
  ORDER BY score DESC
  LIMIT 20
  """,
  ("purchase",)
))
```

SQL

```
USE TEMP B-TREE FOR ORDER BY
```

## Index Width Matters

An index on:

is generally smaller than an index containing several large text columns.

Wider indexes can:

- consume more storage
- fit fewer entries per page
- require more cache memory
- increase write traffic
- deepen or expand structures

This is another reason covering indexes should be designed selectively.

An index should contain columns because they support an important access pattern, not because "more indexed columns must be faster."

PYTHON

```
(short_integer_key)
```

## Redundant Indexes

Suppose a workload already has:

Do we also need:

Possibly not.

The composite index begins with `user_id`, so it may already support many `user_id` access patterns.

But index redundancy analysis is database- and workload-specific. A narrower index can sometimes still have advantages due to size, cache efficiency, or optimizer behavior.

The right approach is to inspect:

- index definitions
- query plans
- workload usage
- storage/write costs

before removing an index.

SQL

```
display(pd.read_sql_query(
    """
    SELECT
        name,
        tbl_name,
        sql
    FROM sqlite_master
    WHERE type = 'index'
        AND tbl_name IN ('events', 'users')
    ORDER BY tbl_name, name
    """,
    perf_conn
))
```

SQL

```
INDEX (user_id, event_date)
```

## Inspecting Index Metadata in SQLite

SQLite provides pragmas such as:

and:

These tools help answer:

- which indexes exist?
- which columns participate?
- in what order?
- is the index unique?

Before adding an index, always inspect what already exists.

PYTHON

```
index_list = perf_conn.execute(
    "PRAGMA index_list('events')"
).fetchall()

display(pd.DataFrame(
    index_list,
    columns=[
        "seq",
        "name",
        "unique",
        "origin",
        "partial"
    ]
))
```

## PYTHON

```
index_info = perf_conn.execute(
    "PRAGMA index_info('idx_events_user_date_type')"
).fetchall()

display(pd.DataFrame(
    index_info,
    columns=["seqno", "cid", "column_name"]
))
```

## Indexes and Updates

Updating a non-indexed column may primarily modify table storage.

Updating an indexed column can additionally require changes to the corresponding index entries.

For example:

If `region` is indexed, the database may need to remove the old index key and insert the new one.

This makes frequently changing columns important to consider during index design.

## SQL

```
UPDATE users
SET region = 'north'
WHERE user_id = ?;
```

## Indexes and DELETE

Deleting a row requires removing its relevant index entries as well as the row itself.

Thus, a table with many indexes may incur more maintenance during:

- bulk deletion
- archival
- retention cleanup
- high-volume event expiration

Large maintenance operations should be tested carefully because transaction logs, locks, storage reclamation, and index maintenance can all become operational concerns.

## Bulk Loading Strategy

For very large data loads, teams sometimes reduce index-maintenance overhead by changing the loading workflow.

Depending on the database and reliability requirements, strategies may include:

- bulk-loading APIs
- loading into staging tables



- creating indexes after loading
- temporarily changing maintenance settings
- batching commits
- partition exchange/swap mechanisms

These operations are highly database-specific.

Never disable integrity or indexes in production merely because a tutorial says bulk loads become faster. Understand recovery, constraints, concurrent readers, and operational consequences first.

### Query Rewriting: EXISTS Versus JOIN

Suppose we only need to know whether a related row exists.

A query using:

can communicate that intention directly.

A `JOIN` may multiply rows if multiple matches exist, requiring `DISTINCT` afterward.

For example:

The correct choice starts with semantics, but expressing the true relational requirement can also give the optimizer useful opportunities.

SQL

```
EXISTS (...)
```

SQL

```
SELECT u.user_id
FROM users AS u
WHERE EXISTS (
  SELECT 1
  FROM events AS e
  WHERE e.user_id = u.user_id
  AND e.event_type = 'purchase'
);
```

### Correlated Subquery Performance

A correlated subquery conceptually depends on values from an outer row.

A naïve mental model might imagine:

Database optimizers can often transform or optimize such patterns, but expensive correlated execution is still possible.

When a correlated query performs poorly:

- inspect the plan

- verify indexes on correlation keys
- consider pre-aggregation
- consider joins or CTEs
- verify that the rewrite preserves semantics

Never rewrite a query solely because "subqueries are slow." That statement is too broad to be useful.

PYTHON

```
for every outer row:  
    run entire inner query from scratch
```

### CTE Performance Is Engine-Specific

Common table expressions improve readability, but their physical execution differs between database engines and versions.

A CTE may be:

- inlined
- materialized
- optimized as part of the larger query
- affected by explicit materialization controls

Therefore:

A CTE is not automatically a temporary table.

and:

A CTE is not automatically faster or slower than a subquery.

Use CTEs for clear logical structure, then inspect execution behavior when performance matters.

### Window Function Performance

Window functions often require ordered partitions.

For:

the engine needs rows organized according to the partition/order specification.

An index such as:

may align with that processing order in useful ways.

But window queries can still process very large datasets and may require sorting or temporary structures depending on the full query.

The same rule applies: inspect the plan and actual workload.

SQL

```
display(explain(  
  perf_conn,  
  ""  
  SELECT  
    user_id,  
    event_date,  
    ROW_NUMBER() OVER (  
      PARTITION BY user_id  
      ORDER BY event_date  
    ) AS event_number  
  FROM events  
  WHERE user_id BETWEEN ? AND ?  
  "" ,  
  (100, 110)  
))
```

## Avoid N+1 Query Patterns

An N+1 query problem often occurs in application code:

1. query 100 users
2. loop over users
3. execute one additional SQL query per user

The application performs:

rather than one appropriately designed relational query.

This can be extremely expensive when a database is accessed over a network because round-trip latency accumulates.

Solutions may involve:

- joins
- batch queries
- `IN`
- eager loading
- prefetching
- aggregation
- application ORM configuration

SQL performance is therefore partly an application architecture concern.

## PYTHON

```
1 + 100 queries
```

## Network Cost Matters

A database server may execute a query quickly while the application still performs poorly because it transfers too much data.

For example:

is not made practical merely because database execution is efficient.

End-to-end latency includes:

Performance engineering should measure the user-visible or pipeline-visible operation, not only isolated database CPU time.

## SQL

```
SELECT *  
FROM billion_row_table;
```

## SQL

```
database work  
+  
serialization  
+  
network transfer  
+  
client deserialization  
+  
application processing
```

## Connection Management

Opening a database connection can have a nontrivial cost for server databases.

Production applications often use connection pools to reuse established connections.

Poor pooling can lead to:

- connection storms
- exhausted server limits
- unnecessary authentication overhead
- high latency
- resource pressure

SQLite uses a different embedded connection architecture, but applications still need correct connection lifecycle and transaction management.

Connection engineering belongs to database performance just as query design does.

## Prepared Statements and Repeated Queries

Applications frequently execute the same SQL shape using different values:

Parameterization provides security and correctness benefits, and database drivers/engines may also be able to reuse aspects of statement preparation.

Exact plan caching and prepared-statement behavior is database- and driver-specific.

Do not manually concatenate values into SQL in an attempt to improve performance.

SQL

```
SELECT ...  
FROM events  
WHERE user_id = ?;
```

## Partitioning Awareness

Very large database systems may partition tables.

For example, events might be physically divided by:

A query filtering by the partition key may allow **partition pruning**, avoiding irrelevant partitions.

Partitioning is not the same thing as indexing.

Indexes organize access within a table or partitioned structure, while partitioning divides data into larger physical/logical subsets.

Partitioning introduces complexity and should solve an actual scale or lifecycle problem rather than being added prematurely.

PYTHON

```
date  
region  
tenant
```

## Materialized Aggregation Awareness

If a dashboard repeatedly computes:

over billions of raw inference events, reprocessing all raw events for every page load may be inefficient.

Systems may instead maintain:

- summary tables
- materialized views
- incremental aggregates

- OLAP stores

This trades computation at query time for precomputation and refresh complexity.

The key question becomes:

How fresh must the result be?

Performance architecture often balances latency, freshness, storage, and computational cost.

PYTHON

```
GROUP BY day, region, model_version
```

## OLTP and OLAP Performance Priorities

Online Transaction Processing (OLTP) workloads often involve:

- short transactions
- point lookups
- small range queries
- inserts/updates
- many concurrent users

Online Analytical Processing (OLAP) workloads often involve:

- large scans
- aggregations
- wide transformations
- historical data
- fewer but computationally heavy queries

The physical designs that work best can differ substantially.

A scan over a billion-columnar-row analytics table may be normal, while scanning a transactional table to find one customer may be unacceptable.

Always evaluate performance in workload context.

## AI Engineering: Feature Lookup Performance

Online ML systems may need features with strict latency requirements.

A request might need:

before model inference can begin.

Slow SQL feature lookup directly increases model-serving latency.

Possible design considerations include:

- indexes on entity IDs
- composite indexes matching temporal lookup patterns
- precomputed features
- dedicated online feature stores
- caching
- point-in-time correctness
- bounded result sizes

The fastest query is still wrong if it leaks future information or retrieves the wrong entity state. Correctness comes first.

PYTHON

```
customer aggregate  
recent transaction count  
risk flag  
account attributes
```

### AI Engineering: Offline Feature Queries

Training pipelines frequently execute large analytical SQL queries.

For example:

Here the workload differs from online serving.

Performance may depend on:

- partition pruning
- pre-aggregation
- join strategy
- column selection
- data layout
- distributed execution
- materialization
- avoiding repeated computation

Indexes alone are not a universal solution for large-scale offline feature engineering.

## PYTHON

```
join customers
join transactions
aggregate historical behavior
construct labels
produce training snapshot
```

## AI Engineering: Point-in-Time Queries

Point-in-time feature retrieval often uses predicates such as:

combined with:

A conceptual index pattern could be:

because the query typically:

1. locates one entity,
2. searches historical values up to a cutoff,
3. chooses the latest valid record.

This is an example where ML correctness requirements and database index design align naturally.

Performance tuning must never remove the temporal condition merely because a broader join is faster; doing so could introduce target leakage.

## PYTHON

```
feature_time <= prediction_time
```

## PYTHON

```
entity_id
```

## AI Engineering: Vector Search Is Different

Traditional B-tree indexes are excellent for ordered scalar values such as:

- IDs
- dates
- names
- numeric ranges

Vector similarity search is a different access problem.

Searching high-dimensional embeddings by nearest-neighbor similarity generally requires specialized vector index structures and algorithms.

Examples in the broader ecosystem include families of methods such as:



- HNSW
- IVF
- product quantization approaches

Those belong to advanced retrieval/vector-database study rather than this foundational SQL section.

The important awareness is:

Different query geometry requires different index structures.

A B-tree should not be expected to solve arbitrary embedding similarity search efficiently.

### AI Engineering: Telemetry Tables

Inference systems can generate large volumes of telemetry:

Indexing every telemetry field can severely increase ingestion cost.

A better design starts with actual operational queries:

Then indexes are designed around those access patterns.

High-volume event analytics may eventually be better served by specialized analytical storage rather than an ever-growing collection of indexes on an OLTP database.

PYTHON

```
request_id
model_version
timestamp
latency
prediction
confidence
status
```

### Performance and Correctness

Never make a query faster by changing its meaning accidentally.

Common dangerous "optimizations" include:

- dropping necessary join predicates
- removing point-in-time conditions
- replacing `UNION` with `UNION ALL` without checking duplicate semantics
- replacing `LEFT JOIN` with `INNER JOIN`
- removing deterministic ordering from pagination
- removing constraints
- reading stale replicas without understanding consistency

- using approximate results where exact results are required

Performance is a requirement that exists alongside correctness, not above it.

## A Systematic Query Optimization Workflow

A practical workflow is:

1. establish query correctness
2. identify the actual slow operation
3. obtain realistic query parameters
4. inspect table sizes and data distribution
5. inspect existing indexes
6. inspect the execution plan
7. find large scans, sorts, repeated loops, or poor joins
8. compare estimated and actual behavior when tooling allows
9. improve predicates or query shape if appropriate
10. add or modify an index only when justified
11. benchmark again
12. check write/storage side effects
13. test under realistic concurrency
14. monitor after deployment

Optimization is an iterative measurement process.

## Common Performance Mistakes

Common mistakes include:

- adding indexes without inspecting plans
- indexing every column
- ignoring composite-index column order
- wrapping indexed columns in unnecessary functions
- requesting columns that are never used
- using enormous `OFFSET` values for deep pagination
- solving duplicate-producing joins with `DISTINCT`
- performing N+1 queries from Python
- assuming a CTE is automatically materialized
- assuming every table scan is bad
- assuming every index lookup is good
- benchmarking tiny toy tables

- measuring only one execution
- ignoring cache effects
- ignoring network transfer
- tuning before establishing correctness
- copying another system's indexes without matching its workload

Performance engineering should be evidence-driven.

## Index Design Checklist

Before creating an index, ask:

- Which real query needs it?
- Which predicates does that query use?
- Are they equality or range predicates?
- Which columns participate in joins?
- Is ordering required?
- Can the index support `ORDER BY`?
- Is the predicate selective?
- Would a composite index be more appropriate?
- What should the column order be?
- Is an existing index already sufficient?
- Could a partial index represent the workload better?
- Could an expression index be justified?
- Could the index cover a critical query?
- How often is the table written?
- How wide will the index become?
- What does the query plan show before and after?
- Does measured latency actually improve?

If these questions cannot be answered, the index may be premature.

## Reading a Query Plan Checklist

When reading a plan, investigate:

- which tables are scanned
- which tables are searched
- which indexes are selected
- whether expected indexes are ignored
- whether temporary sorting structures appear

- join access order
- repeated operations
- whether a covering index is used
- whether predicates align with index keys
- whether broad predicates justify scanning

For advanced database systems, also inspect:

- estimated rows
- actual rows
- execution time per operator
- buffer/page reads
- loops
- memory use
- disk spills
- parallel execution

Do not focus only on the total query duration.

## SQL Performance Engineering Summary

Indexes are auxiliary structures that exchange storage and write cost for faster access patterns.

B-tree-family indexes are particularly effective for:

- equality lookup
- range lookup
- ordered access
- many join-key lookups

Single-column indexes solve simple access patterns, while composite indexes support multi-column filtering and ordering. Composite column order matters because the index has a defined key ordering.

Covering indexes can answer some queries without additional table access. Partial indexes can focus on important subsets. Expression indexes can support frequently queried transformations.

Sargable predicates align naturally with index structures. Applying functions or transformations to indexed columns can sometimes prevent ordinary indexes from being useful.

`EXPLAIN QUERY PLAN` provides evidence about SQLite's selected strategy. The optimizer may choose scans, indexed searches, join orders, and temporary structures based on estimated cost and available statistics.

More indexes are not always better. Inserts, updates, deletes, storage, caching, and maintenance all pay for unnecessary indexes.

The central performance principle is:

Measure the real workload, inspect the execution plan, optimize the dominant work, and verify the improvement.

## Schema Design, Normalization, and Data Modeling

SQL syntax tells us how to interact with relational databases. Data modeling determines what those databases mean.

Poor schema design can make even perfectly written SQL difficult, error-prone, and inconsistent.

Consider a table such as:

At first glance, storing everything together seems convenient.

But what happens when:

- the same customer places 500 orders?
- a customer changes their email address?
- a product appears in thousands of orders?
- a product name changes?
- one order contains several products?
- we want to store a customer before their first order?

Repeated facts create opportunities for inconsistency.

Relational schema design asks questions such as:

- What real-world entities exist?
- Which facts belong to each entity?
- What uniquely identifies each row?
- Which attributes depend on which keys?
- Which relationships exist between entities?
- What integrity rules must always hold?
- Which historical facts should remain immutable?
- Which values represent current state versus event history?

This section develops schema design from relational principles through practical AI/data-engineering examples.

We will study:

- entities and attributes
- grain
- functional dependencies

- determinants
- candidate keys
- superkeys
- primary keys
- alternate keys
- natural keys
- surrogate keys
- composite keys
- update anomalies
- insertion anomalies
- deletion anomalies
- normalization
- first normal form
- second normal form
- third normal form
- transitive dependencies
- BCNF awareness
- relationship modeling
- one-to-one
- one-to-many
- many-to-many
- junction tables
- recursive relationships
- optional relationships
- referential actions
- denormalization
- schema evolution
- lookup/reference tables
- temporal/history modeling awareness
- dimensional modeling
- facts
- dimensions
- star schemas
- slowly changing dimension awareness
- AI/ML metadata modeling

- model registries
- experiment tracking
- dataset provenance
- feature metadata

The central principle is:

Store each fact according to its true meaning, dependency, and grain.

PYTHON

```
order_id
order_date
customer_id
customer_name
customer_email
product_id
product_name
product_price
quantity
```

## Schema Design Starts Before CREATE TABLE

A common beginner workflow is:

Reliable schema design starts earlier.

We first identify the domain.

For an online store, possible entities include:

Then we ask what each row represents.

For example:

This row-level meaning is the table's **grain**.

Grain is one of the most important ideas in both transactional databases and analytical modeling.

If grain is unclear, keys, constraints, joins, and aggregation become difficult to reason about.

PYTHON

```
I have some columns
-> create a table
-> start inserting data
```

## PYTHON

Customer  
Product  
Order  
Order Item  
Payment  
Shipment

## Entities

An entity is a distinguishable thing or concept about which we want to store information.

Examples include:

- customer
- employee
- product
- order
- invoice
- training run
- model version
- dataset snapshot
- deployment
- inference request

Entities do not need to be physical objects.

A `training_run` is an event/process entity. A `model_version` is a versioned artifact concept. A `deployment` represents operational state.

A relational table often corresponds to an entity type, but the mapping is not always one table per everyday noun. Good modeling reflects semantics, lifecycle, relationships, and integrity requirements.

## Attributes

Attributes describe entities.

For a customer:

For a model version:

The important design question is:

Does this attribute describe the entity represented by this row?

For example, `customer_name` naturally belongs to a customer record.



If it is copied into every order row, it becomes repeated customer data rather than an intrinsic property of the order itself.

Some repetition is deliberate—particularly historical snapshots—but it should result from an explicit design decision rather than accidental schema construction.

```
PYTHON
```

```
customer_id
name
email
created_at
```

## Grain Revisited

The grain specifies what exactly one row represents.

Suppose we have:

If one order can contain multiple products, then:

cannot be the grain of this table.

A more accurate grain is:

That immediately suggests a possible key:

if a product can occur only once per order.

Or we might use:

as a surrogate identifier while preserving an appropriate uniqueness constraint.

Always state the grain before choosing keys.

```
PYTHON
```

```
order_id
product_id
quantity
```

```
PYTHON
```

```
one row = one order
```

## Functional Dependencies

A functional dependency captures a relationship between attributes.

We write:

to mean:

If two rows have the same value of X, they must have the same value of Y.

For example, if `customer_id` uniquely identifies a customer:

Or compactly:

This means customer attributes are functionally determined by `customer_id`.

Functional dependencies are based on the semantics and rules of the data—not merely the values currently present in a sample dataset.

PYTHON

```
X -> Y
```

PYTHON

```
customer_id -> customer_name  
customer_id -> customer_email
```

## Functional Dependency Is Not Correlation

Suppose every row in a small sample happens to have a unique customer name.

That does not prove:

Two different customers can legitimately share the same name.

Similarly, a dataset may currently show one product per category, but that does not establish:

Functional dependency is a semantic rule that must hold for every valid database state.

This distinction is critical when normalizing schemas.

Do not infer database constraints simply because a limited sample happens to contain unique values.

PYTHON

```
customer_name -> customer_id
```

## Determinants

In:

`X` is called the determinant.

For:

`employee_id` determines the other values.

A determinant can contain several attributes.

For an enrollment system:

The combination of student and course may determine facts about that enrollment.

Functional dependencies involving composite determinants become especially important when studying second normal form.

PYTHON

`X -> Y`

PYTHON

`employee_id -> employee_name, department_id`

## Primary Keys and Alternate Keys

A table chooses one candidate key as its primary key.

Other candidate keys are often called alternate keys.

Example:

Here:

is the chosen primary identifier, while `email` remains protected against duplication.

A primary key serves as the table's canonical row identity and becomes a common target for foreign-key references.

PYTHON

`customer_id` PRIMARY KEY  
`email` UNIQUE

PYTHON

`customer_id`

## Natural Versus Surrogate Keys

Neither strategy is universally superior.

A useful design asks:

- Is the natural identifier truly stable?
- Is it guaranteed unique?
- Is it compact?
- Is it sensitive?
- Can external organizations redefine it?
- Will several systems need to integrate?

- Does the identifier have lifecycle issues?

For many application tables, a surrogate primary key plus explicit uniqueness constraints on true business keys provides a flexible design.

But a stable, compact natural identifier can also be an excellent key.

Choose based on semantics rather than ideology.

### The Problem of Repeated Facts

Consider this unnormalized order-like structure:

| ORDER_ID | CUSTOMER_ID | CUSTOMER_NAME | CUSTOMER_EMAIL | PRODUCT_ID | PRODUCT_NAME | P |
|----------|-------------|---------------|----------------|------------|--------------|---|
| 1        | 10          | Asha          | a@example.com  | 101        | Keyboard     | 5 |
| 1        | 10          | Asha          | a@example.com  | 102        | Mouse        | 2 |
| 2        | 10          | Asha          | a@example.com  | 103        | Monitor      | 2 |

Customer information is repeated.

If Asha changes email address, every row containing her customer data may need to change.

Product information can also repeat across many orders.

Repeated facts increase the number of locations that must agree with one another.

Normalization provides systematic tools for reducing inappropriate redundancy.

### Update Anomaly

An update anomaly occurs when one logical fact is stored in multiple places and must be updated consistently.

Suppose a customer's email appears in 300 order rows.

Changing the email means updating all 300 copies.

If 299 are changed and one is missed, the database now contains contradictory values for the same customer.

Normalized design would typically store the current customer email once:

Orders reference the customer:

One logical fact has one authoritative representation.

## PYTHON

```
customers
-----
customer_id
email
```

**Insertion Anomaly**

An insertion anomaly occurs when the schema makes it difficult or impossible to record one fact without an unrelated fact.

Suppose product information exists only inside an order-line table.

How do we store a new product that has never been ordered?

If product existence requires creating a fake order row, the schema has mixed independent facts together.

Separating:

from:

allows products to exist independently of purchases.

## PYTHON

```
products
```

## PYTHON

```
order_items
```

**Deletion Anomaly**

A deletion anomaly occurs when deleting one fact unintentionally removes another important fact.

Suppose the only row containing information about product 500 is its final order record.

Deleting that historical order might also remove the only stored copy of:

The schema has coupled unrelated information lifecycles.

Normalization separates facts whose existence should not depend on one another.

## PYTHON

```
product name
product category
product metadata
```

**Normalization**

Normalization is a systematic approach to organizing relational schemas around dependencies and keys.

Its goals include:

- reducing inappropriate redundancy
- avoiding modification anomalies
- representing dependencies clearly
- improving integrity

Normalization is described using **normal forms**.

Common levels include:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Boyce-Codd Normal Form (BCNF)

Higher normal forms also exist, but 1NF through 3NF plus BCNF awareness provide an important practical foundation.

### First Normal Form

First Normal Form is often introduced using the idea that each row/column intersection should contain a single value from its domain rather than a repeating group.

Consider:

The `phone_numbers` value is being used as a list encoded inside one field.

This creates problems:

- difficult validation
- difficult searching
- difficult uniqueness enforcement
- difficult foreign-key relationships
- string parsing requirements

A normalized design might use:

where each phone number is represented as its own relational row.

PYTHON

| customer_id | phone_numbers   |
|-------------|-----------------|
| 1           | "111, 222, 333" |

### Atomic Values Need Context

"Atomic" does not mean the database must split every value into microscopic components.

A date can reasonably be one value even though it contains year, month, and day.

An address might be one textual value in one application but need structured fields in another.

Atomicity is influenced by how the application needs to query, validate, and relate the data.

The important rule is to avoid stuffing repeating relational structures into opaque values merely to avoid modeling them properly.

### **A 1NF-Style Phone Model**

Instead of:

we can model:

One customer can now have many phone rows.

The database can:

- search by phone
- enforce uniqueness where appropriate
- validate each value
- add phone metadata
- delete one phone independently

SQL

```

import sqlite3
import pandas as pd
from IPython.display import display

design_conn = sqlite3.connect(":memory:")
design_conn.execute("PRAGMA foreign_keys = ON")

design_conn.executescript("""
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE
);

CREATE TABLE customer_phones (
    customer_id INTEGER NOT NULL,
    phone_number TEXT NOT NULL,
    phone_type TEXT,
    PRIMARY KEY (customer_id, phone_number),
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE CASCADE
);

INSERT INTO customers
VALUES
    (1, 'Asha', 'asha@example.com'),
    (2, 'Rohan', 'rohan@example.com');
# ...
ORDER BY c.customer_id, p.phone_number
""",
design_conn
))

```

SQL

| customer_id | phones      |
|-------------|-------------|
| 1           | 111,222,333 |

## Second Normal Form

Second Normal Form matters primarily when a table has a composite candidate key.

A relation is conceptually in 2NF when:

- it satisfies 1NF, and
- non-key attributes depend on the whole candidate key rather than only part of a composite key.

Consider:

Suppose the table also stores:



Then:

These attributes depend on only part of the composite key.

Those are **partial dependencies**.

PYTHON

```
(student_id, course_id) -> grade
```

PYTHON

```
student_name  
course_name
```

### Partial Dependency Example

Consider:

with key:

Dependencies include:

Only `grade` depends on the complete enrollment identity.

A better decomposition is:

Each fact now resides with the key that determines it.

## SQL

```

design_conn.executescript("""
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT NOT NULL
);

CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY,
    course_name TEXT NOT NULL
);

CREATE TABLE enrollments (
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    grade TEXT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id)
        REFERENCES students(student_id),
    FOREIGN KEY (course_id)
        REFERENCES courses(course_id)
);
""")

design_conn.commit()

print("2NF-style enrollment schema created.")

```

## Why 2NF Is Often Invisible with a Single-Column Key

If a table's candidate key contains only one attribute, a non-key attribute cannot depend on only part of that key because there is no smaller non-empty part to depend on.

Therefore partial dependency issues arise most visibly with composite keys.

This does not mean single-key tables are automatically well designed. They may still violate third normal form through transitive dependencies.

## Third Normal Form

Third Normal Form addresses dependencies where a non-key attribute depends on another non-key attribute rather than directly on the key.

Suppose:

Then:

holds transitively.

If `department_name` is copied into every employee row, the same department name may be repeated hundreds of times.

A normalized design separates department facts:

## PYTHON

```
employee_id -> department_id
department_id -> department_name
```

## PYTHON

```
employee_id -> department_name
```

## Transitive Dependency

A transitive dependency has a chain-like form:

which means:

through **B**.

In a table where **A** is the key and both **B** and **C** are non-key attributes, storing **C** with **A** may create redundancy if **C** really describes **B**.

Example:

`department_location` describes the department, not the employee.

It therefore belongs naturally with the department entity.

## SQL

```
design_conn.executescript("""
CREATE TABLE departments (
    department_id INTEGER PRIMARY KEY,
    department_name TEXT NOT NULL,
    department_location TEXT
);

CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT NOT NULL,
    department_id INTEGER,
    FOREIGN KEY (department_id)
        REFERENCES departments(department_id)
);
""")

design_conn.commit()

print("3NF-style employee/department schema created.")
```

## SQL

```
A -> B
B -> C
```

## A Practical Memory Aid for 2NF and 3NF

A popular informal memory aid is:

The key, the whole key, and nothing but the key.

It is useful intuition, but formal normalization is more precise.

Roughly:

- 2NF asks whether non-key facts depend on the whole composite key.
- 3NF asks whether non-key facts are improperly determined through other non-key facts.

Do not rely solely on memorized slogans. Write down candidate keys and functional dependencies explicitly when the schema becomes difficult.

## Boyce-Codd Normal Form

BCNF strengthens the dependency condition.

Informally, for every non-trivial functional dependency:

BCNF requires  $X \twoheadrightarrow Y$  to be a superkey.

Many schemas that satisfy ordinary practical 3NF also satisfy BCNF, but there are edge cases involving overlapping candidate keys where a relation can satisfy 3NF yet violate BCNF.

At foundational level, the key idea is:

If some determinant controls other information, ask whether that determinant is actually a key.

BCNF becomes especially useful when schemas contain multiple candidate keys with complex dependencies.

PYTHON

$X \twoheadrightarrow Y$

## BCNF Awareness Example

Imagine a relation:

with rules:

Assume each instructor teaches exactly one course.

The dependency:

has a determinant (`instructor`) that may not be a superkey for the original relation because many students can have the same instructor.

That signals a BCNF issue.

A possible decomposition separates instructor-course assignment from student-instructor association.

The exact correct schema depends on the actual domain rules, which illustrates why normalization cannot be performed correctly without understanding semantics.

PYTHON

```
student
course
instructor
```

PYTHON

```
(student, course) -> instructor
instructor -> course
```

## Lossless Decomposition

When splitting a relation into multiple tables, we want to be able to reconstruct the original valid information through joins without inventing spurious combinations.

This property is called a **lossless decomposition**.

Normalization is not arbitrary table splitting.

A decomposition should preserve the relationships required to reconstruct the original information meaningfully.

Keys and shared attributes play an important role in lossless decomposition.

## Dependency Preservation

Another design consideration is whether important functional dependencies can still be enforced easily after decomposition.

A decomposition may be logically lossless yet make certain constraints difficult to verify without joining tables.

Normalization theory therefore considers both:

- lossless decomposition
- dependency preservation

In practical engineering, database support, write patterns, query complexity, and operational constraints also influence the final schema.

## Junction Tables

A junction table represents associations between two entities.

For students and courses:

`enrollments` contains:

and possibly relationship-specific attributes such as:

These values belong to the relationship itself, not independently to the student or the course.

SQL

```
design_conn.executescript("""
CREATE TABLE tags (
    tag_id INTEGER PRIMARY KEY,
    tag_name TEXT NOT NULL UNIQUE
);

CREATE TABLE products_demo (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT NOT NULL
);

CREATE TABLE product_tags (
    product_id INTEGER NOT NULL,
    tag_id INTEGER NOT NULL,
    PRIMARY KEY (product_id, tag_id),
    FOREIGN KEY (product_id)
        REFERENCES products_demo(product_id)
        ON DELETE CASCADE,
    FOREIGN KEY (tag_id)
        REFERENCES tags(tag_id)
        ON DELETE CASCADE
);
""")

design_conn.commit()

print("Many-to-many product/tag schema created.")
```

SQL

```
students
courses
enrollments
```

## Relationship Attributes Belong in the Junction

Suppose an employee can work on several projects, and each project has several employees.

The relationship may have:

These attributes do not belong solely to `employee`.

They do not belong solely to `project`.

They describe:

Therefore the junction entity is the natural location.

```
PYTHON
```

```
assigned_at  
role_on_project  
allocation_percent
```

## Optional Relationships

Relationships are not always mandatory.

An employee might not yet belong to a department.

Then:

may be nullable.

By contrast, if every order must belong to a customer, the foreign key may be:

Nullability is therefore part of relationship semantics.

Do not make every foreign key nullable merely to make inserts easier. Decide whether the relationship is actually optional.

```
PYTHON
```

```
department_id
```

```
PYTHON
```

```
customer_id INTEGER NOT NULL
```

## Recursive Relationships

An entity can relate to another row of the same entity type.

For example:

where a manager is also an employee.

This can be modeled with a self-referencing foreign key:

Top-level managers may have `NULL` as their manager.

Recursive structures occur in:

- organization charts
- category hierarchies
- threaded comments
- dependency trees

Recursive CTEs can then query hierarchical relationships.

SQL

```
design_conn.executescript("""
CREATE TABLE organization_people (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT NOT NULL,
    manager_id INTEGER,
    FOREIGN KEY (manager_id)
        REFERENCES organization_people(employee_id)
);
""")

design_conn.executemany(
    """
    INSERT INTO organization_people (
        employee_id,
        employee_name,
        manager_id
    )
    VALUES (?, ?, ?)
    """,
    [
        (1, "CEO", None),
        (2, "Engineering Lead", 1),
        (3, "ML Engineer", 2),
        (4, "Data Engineer", 2),
    ],
)

design_conn.commit()
# ...
ORDER BY e.employee_id
""",
design_conn
))
```

SQL

```
employee -> manager
```

## Referential Actions

Foreign keys raise an important lifecycle question:

What should happen to dependent rows when a referenced row changes or disappears?

Common actions include:

Exact support and semantics vary by database.



For example:

can be appropriate for data that has no independent meaning after the parent disappears.

But cascading deletion of business history may be disastrous.

Referential actions should model lifecycle semantics, not merely reduce manual SQL.

PYTHON

```
RESTRICT / NO ACTION
CASCADE
SET NULL
SET DEFAULT
```

## Historical Facts Versus Current Attributes

Normalization discussions sometimes lead beginners to believe repeated historical values are always bad.

Consider an order.

The product currently costs ₹1200, but the customer bought it for ₹1000 six months ago.

The order should normally preserve the transaction-time amount.

A historical line item might contain:

Even though current product price exists in `products`, the historical purchase price is a different fact.

This is intentional snapshotting, not necessarily a normalization mistake.

Always ask what time the fact describes.

PYTHON

```
product_id
quantity
unit_price_at_purchase
```

## Mutable and Immutable Data

Some data represents current state:

Other data represents historical events:

Historical events are often better modeled as append-oriented records rather than repeatedly overwriting one row.

Choosing between current-state and event-history modeling affects:

- auditing
- analytics
- reproducibility

- temporal queries
- storage growth

## PYTHON

```
customer.current_email  
deployment.active_model_id
```

## PYTHON

```
order created  
payment received  
prediction produced  
model deployed
```

## Lookup Tables

A lookup/reference table stores a controlled set of domain values.

For example:

A foreign key can then ensure that only known values are referenced.

However, not every tiny enumeration needs its own table. A **CHECK** constraint may be simpler for stable, small states.

Use a lookup table when the values have independent metadata, relationships, lifecycle, or administrative management.

## PYTHON

```
countries  
currencies  
departments  
model_statuses
```

## Denormalization

Denormalization deliberately introduces redundancy or combines data structures to improve a specific workload.

Examples include:

- storing precomputed totals
- materialized summary tables
- duplicating display labels
- flattened analytical tables
- cached aggregates

Denormalization is not the same as accidental poor design.

A good denormalization is intentional and answers:

- What workload does this accelerate?
- Which source remains authoritative?
- How is redundant data synchronized?
- What happens when synchronization fails?
- What measurable benefit justifies the complexity?

### Normalize First, Denormalize with Evidence

A useful default is:

Premature denormalization makes integrity difficult before there is evidence that the extra complexity is necessary.

Conversely, refusing all denormalization can also be impractical for large analytical systems.

Relational purity is not the final goal. Correct, maintainable, performant systems are.

#### PYTHON

```
understand dependencies  
-> design a correct normalized model  
-> observe workload  
-> denormalize only when justified
```

### Derived Data

Suppose:

Should the table also store:

If `line_total` is always exactly:

then it is derived data.

Storing it introduces synchronization risk unless the database can guarantee consistency.

But persisted derived values can be reasonable when:

- calculation is expensive
- historical calculation rules must be preserved
- the value has independent business meaning
- generated-column/database features enforce consistency
- performance measurements justify materialization

Derived-data decisions require explicit ownership and refresh rules.

PYTHON

```
quantity = 3
unit_price = 100
```

PYTHON

```
line_total = 300
```

## Wide Tables

A wide table has many columns.

Wide tables are not automatically wrong. Some entities genuinely have many attributes.

But excessive width can indicate that:

- several entities have been merged
- optional subtypes are mixed together
- repeating groups were flattened
- unrelated lifecycle data shares one row
- analytics output is being confused with transactional storage

Ask whether every column describes the same grain and entity lifecycle.

## Sparse Optional Attributes

Suppose a generic `assets` table supports:

and contains dozens of type-specific nullable columns.

This may lead to a schema where most columns are `NULL` for most rows.

Possible alternatives include:

- subtype tables
- separate type-specific entities
- carefully constrained JSON for genuinely flexible metadata
- inheritance-like relational patterns

There is no universal answer. Query patterns and integrity requirements matter.

PYTHON

```
image
video
audio
document
```

## JSON Inside Relational Tables

Modern relational databases often support JSON data.

JSON can be appropriate for:

- flexible external payloads
- rarely queried metadata
- schema-evolving auxiliary properties
- heterogeneous integration data

But JSON should not automatically replace relational modeling.

If an attribute frequently needs:

- joins
- foreign keys
- uniqueness
- indexing
- type constraints
- aggregation

a proper relational column/table may be more appropriate.

Use JSON intentionally, not as an escape from schema design.

## Schema Evolution

Production schemas change over time.

Examples include:

- adding columns
- introducing tables
- adding indexes
- changing constraints
- splitting entities
- backfilling data
- deprecating fields

Schema evolution should be performed through reproducible migrations rather than manual undocumented database edits.

Migration tooling depends on the technology stack, but the principle is universal:

Database schema is versioned application infrastructure.

## Backward-Compatible Schema Changes

Deployments often contain multiple application versions temporarily.

Suppose application version A expects the old schema while version B expects a new column.

A safer migration strategy may use an expand/contract pattern:

This reduces the chance that one deployment step immediately breaks another service version.

Schema migration therefore interacts with CI/CD and distributed-system deployment.

PYTHON

```
expand schema
-> deploy compatible application changes
-> migrate/backfill data
-> switch reads/writes
-> remove obsolete structure later
```

## Renaming and Removing Columns

Destructive schema changes deserve caution.

Before dropping a column, determine:

- which applications read it
- which applications write it
- which dashboards use it
- which ML pipelines depend on it
- whether historical jobs need it
- whether external integrations consume it

Database schemas often have more consumers than the core application team realizes.

Data lineage becomes increasingly valuable as systems grow.

## Dimensional Modeling

Normalized transactional schemas are excellent for managing operational data.

Analytics often has different priorities.

Dimensional modeling organizes data around measurable business events and descriptive context.

The two central concepts are:

- fact tables

- dimension tables

A common structure is the **star schema**.

Dimensional modeling does not mean normalization was wrong. It solves a different workload.

### Fact Tables

A fact table stores measurements or events at a clearly defined grain.

Examples:

Possible measures include:

Fact tables frequently contain foreign keys to dimensions.

#### PYTHON

```
one row per order line
one row per website session
one row per daily product metric
one row per prediction request
```

#### PYTHON

```
quantity
revenue
latency_ms
token_count
prediction_score
```

### Fact Table Grain Must Be Explicit

Before creating an analytical fact table, state:

One row represents \_\_\_\_.

Examples:

Never mix multiple grains casually in one fact table.

If some rows represent daily aggregates and others individual events, downstream `SUM`, `AVG`, and `COUNT` operations can become meaningless.

## PYTHON

```
one row per completed order item  
  
one row per model inference  
  
one row per customer per day  
  
one row per model version per evaluation dataset
```

## Dimension Tables

Dimensions provide descriptive context for facts.

Examples include:

A sales fact may contain:

while dimensions store descriptive values such as:

Dimensions make analytical queries easier to understand and organize.

## PYTHON

```
customer  
product  
date  
region  
model  
experiment
```

## PYTHON

```
customer_key  
product_key  
date_key
```

## Star Schema

A star schema places a central fact table around several dimensions.

Conceptually:

The structure resembles a star.

Benefits often include:

- intuitive analytical queries
- predictable joins
- clear grain
- convenient BI usage
- simplified aggregation



Fact tables can become very large, while dimensions are often smaller and descriptive.

PYTHON

```
dim_customer
|
dim_date ---- fact_sales ---- dim_product
|
dim_region
```

## SQL Star-Schema Example

Suppose inference telemetry has this grain:

We might design:

Analysts can aggregate facts using descriptive dimensions.

SQL

```
design_conn.executescript("""
CREATE TABLE dim_model (
    model_key INTEGER PRIMARY KEY,
    model_name TEXT NOT NULL,
    model_version TEXT NOT NULL,
    UNIQUE (model_name, model_version)
);

CREATE TABLE dim_date (
    date_key INTEGER PRIMARY KEY,
    calendar_date TEXT NOT NULL UNIQUE,
    year INTEGER NOT NULL,
    month INTEGER NOT NULL
);

CREATE TABLE fact_predictions (
    prediction_id INTEGER PRIMARY KEY,
    date_key INTEGER NOT NULL,
    model_key INTEGER NOT NULL,
    latency_ms REAL NOT NULL,
    prediction_score REAL,
    was_error INTEGER NOT NULL CHECK (was_error IN (0, 1)),
    FOREIGN KEY (date_key)
        REFERENCES dim_date(date_key),
    FOREIGN KEY (model_key)
        REFERENCES dim_model(model_key)
);
""")

design_conn.commit()

print("Foundational prediction star schema created.")
```

## Facts Can Be Additive or Non-Additive

Not every numeric column should be summed.

Examples:

are often additive across several dimensions.

A percentage such as:

usually should not be summed.

Likewise:

cannot generally be averaged across groups unless weighting information is preserved.

Dimensional modeling requires understanding measure semantics, not merely storing numbers in fact tables.

```
PYTHON
```

```
revenue  
quantity
```

```
PYTHON
```

```
conversion_rate
```

## Store Components Needed for Correct Aggregation

Suppose a daily table stores only:

Combining days correctly may be impossible if each day had a different request count.

A better summary might preserve:

so the overall average can be reconstructed:

This connects schema design directly with the weighted-metric principles studied earlier.

```
PYTHON
```

```
average_latency
```

```
PYTHON
```

```
request_count  
total_latency
```

## Slowly Changing Dimensions Awareness

Dimension attributes can change over time.

A customer may move regions. A product may change category. A model may change governance status.

Analytics must decide whether historical facts should use:

- the latest dimension value
- the historical value valid when the fact occurred
- both

Dimensional modeling calls these evolving attributes **slowly changing dimensions**.

Common strategies are conventionally called Type 1, Type 2, and others.

At foundation level:

- Type 1 broadly overwrites old descriptive state.
- Type 2 broadly preserves historical versions as separate dimension records.

The correct choice depends on analytical requirements.

### Operational Schema Versus Analytical Schema

An operational database might store:

in normalized form.

An analytical warehouse may transform that data into:

Neither schema universally replaces the other.

The operational schema prioritizes transactional integrity.

The analytical schema prioritizes historical analysis and convenient aggregation.

Modern data platforms frequently maintain both representations for different purposes.

#### PYTHON

```
customers
orders
products
payments
```

### AI Engineering: Model Registry Data Model

A model registry may need to represent:

A poor design might place everything into one giant table:

That quickly creates repeated facts and incompatible grains.

For example:

may have:

These are distinct relationships and should be modeled accordingly.

## PYTHON

```
models
model_versions
artifacts
metrics
datasets
deployments
approvals
```

## PYTHON

```
model_name
version
artifact_uri
accuracy
precision
recall
dataset_name
dataset_version
deployment_environment
deployment_status
...
```

## AI Engineering: Models and Model Versions

A useful distinction is:

versus:

For example:

A possible schema is:

This avoids repeating model-level information across every version and gives versions their own lifecycle.

SQL

```

design_conn.executescript("""
CREATE TABLE ai_models_demo (
    model_id INTEGER PRIMARY KEY,
    model_name TEXT NOT NULL UNIQUE,
    description TEXT
);

CREATE TABLE ai_model_versions_demo (
    model_version_id INTEGER PRIMARY KEY,
    model_id INTEGER NOT NULL,
    version_label TEXT NOT NULL,
    artifact_uri TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (model_id, version_label),
    FOREIGN KEY (model_id)
        REFERENCES ai_models_demo(model_id)
);
""")

design_conn.commit()

print("Model/model-version schema created.")

```

SQL

model

## AI Engineering: Metric Grain

A metric is not meaningful without context.

Consider:

Questions immediately arise:

- For which model version?
- Which dataset?
- Which split?
- Which threshold?
- Which evaluation run?
- Which metric implementation/version?

A robust metric table might have grain such as:

or another explicitly defined grain.

Metric modeling is therefore a schema-design problem, not merely a logging problem.

## SQL

```

design_conn.executescript("""
CREATE TABLE evaluation_runs (
    evaluation_run_id INTEGER PRIMARY KEY,
    model_version_id INTEGER NOT NULL,
    dataset_snapshot TEXT NOT NULL,
    evaluated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (model_version_id)
        REFERENCES ai_model_versions_demo(model_version_id)
);

CREATE TABLE evaluation_metrics (
    evaluation_run_id INTEGER NOT NULL,
    metric_name TEXT NOT NULL,
    metric_value REAL NOT NULL,
    PRIMARY KEY (
        evaluation_run_id,
        metric_name
    ),
    FOREIGN KEY (evaluation_run_id)
        REFERENCES evaluation_runs(evaluation_run_id)
        ON DELETE CASCADE
);
""")

design_conn.commit()

print("Evaluation-run and metric schema created.")

```

## AI Engineering: Dataset Identity

A dataset name alone is usually insufficient for reproducibility.

For example:

does not reveal whether the file changed between training runs.

Dataset identity may need:

- dataset logical name
- version
- snapshot ID
- creation time
- schema version
- storage URI
- checksum
- source lineage

A training run should ideally reference an immutable or otherwise reproducibly identifiable dataset snapshot rather than an ambiguous mutable name.

PYTHON

`customers.csv`

## AI Engineering: Provenance Relationships

A reproducible ML lineage graph may conceptually connect:

Trying to flatten this entire graph into one table creates duplication and ambiguous grain.

Relational modeling allows each concept to have its own identity and relationships.

This supports questions such as:

- Which dataset produced this model?
- Which models used this dataset snapshot?
- Which evaluation approved this deployment?
- Which artifact belongs to this version?

PYTHON

```
source datasets
  ->
dataset snapshot
  ->
feature transformation
  ->
training run
  ->
model version
  ->
evaluation run
  ->
deployment
```

## AI Engineering: Training Runs

A training run is distinct from a model version.

A training run may fail and never produce a registered model.

Several runs may attempt the same configuration.

A run may produce multiple artifacts.

A useful schema can therefore distinguish:

rather than assuming every training attempt is automatically one successful model version.

## PYTHON

```

training_runs

models

model_versions

artifacts

```

## AI Engineering: Hyperparameters

Hyperparameters pose an interesting modeling choice.

We might store them as:

for flexibility.

Or as rows:

Or use typed structured columns for a fixed set of important parameters.

There is no universal best choice.

Consider:

- query frequency
- typing requirements
- schema stability
- number of algorithms
- indexing needs
- reproducibility
- validation

Flexible metadata and strongly relational metadata can coexist.

## PYTHON

```

JSON

```

## PYTHON

```

run_hyperparameters
-----
run_id
parameter_name
parameter_value

```

## AI Engineering: Feature Definitions

A feature platform might need to distinguish:



These concepts have different grains.

For example:

Storing definition metadata and feature observations as though they were the same entity creates confusion.

Grain remains the foundation.

PYTHON

```
feature definition
feature version
feature values
feature materialization run
entity
source dataset
```

PYTHON

```
feature definition:
customer_30d_purchase_count

feature value:
customer 123,
as of 2026-09-30,
value = 8
```

## AI Engineering: Deployment Modeling

Deployment data may involve:

A simple system may have:

A more sophisticated system may support:

Schema design must follow actual deployment semantics.

Do not model `is_active BOOLEAN` on model versions if the real system supports multiple environments and weighted traffic allocations.

PYTHON

```
environment
model version
deployment event
current deployment state
approval
rollback
traffic allocation
```

## AI Engineering: Inference Event Grain

For an inference log, define one row carefully.

Possible grain:

Then attributes might include:

But if one request predicts 100 items, another table may be needed:

Otherwise request-level and item-level data become mixed in one relation.

This directly affects metrics such as latency, error rate, and prediction count.

PYTHON

```
one row per API prediction request
```

PYTHON

```
request_id  
model_version_id  
request_timestamp  
latency_ms  
response_status
```

## Multi-Tenancy

Software-as-a-Service systems often contain a tenant or organization dimension.

A business key may be unique only within a tenant.

Instead of:

the real rule may be:

Multi-tenant schema design also influences:

- authorization
- indexes
- foreign keys
- partitioning
- data deletion
- audit requirements

Tenant boundaries should be explicit in both data modeling and security design.

PYTHON

```
project_name UNIQUE
```

PYTHON

```
UNIQUE(tenant_id, project_name)
```

## Composite Business Uniqueness

Business uniqueness often involves several columns.

Examples:

A surrogate primary key does not make these rules disappear.

For example:

still needs:

if duplicate version labels within one model are forbidden.

PYTHON

```
(model_id, version_label)

(student_id, course_id)

(tenant_id, project_slug)

(order_id, line_number)
```

## Avoid Meaningful Encoded IDs Unless Truly Required

Developers sometimes create identifiers such as:

that encode:

- country
- year
- entity type
- sequence

Such identifiers can become difficult when business rules change.

If the customer moves countries or numbering policies change, should the identity change?

Stable surrogate identity plus separate attributes is often more resilient.

Human-readable reference numbers can still exist independently of the internal primary key.

PYTHON

```
US-2026-CUSTOMER-000123
```

## Boolean Explosion

A schema sometimes accumulates state columns:

Now impossible combinations can occur:

If the states are mutually exclusive, one constrained state column may model the domain more clearly:  
with allowed values.

Multiple Booleans are appropriate when the properties are genuinely independent.

PYTHON

```
is_pending
is_running
is_complete
is_failed
is_cancelled
```

PYTHON

```
is_complete = 1
is_failed = 1
```

## NULL and Schema Meaning

**NULL** should represent missing, unknown, or inapplicable information according to a deliberate model.

Do not make everything nullable for convenience.

**NOT NULL** communicates that a value is required for every valid row.

At the same time, avoid fake sentinel values such as:

when the true state is absence of a known value.

Schema nullability should reflect domain semantics.

PYTHON

```
'N/A'
-1
'unknown'
1900-01-01
```

## Constraints Are Part of the Model

A diagram may describe a schema visually, but database constraints make important rules executable.

Use appropriate:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK

- defaults

For example:

documents and enforces that an order line cannot contain zero or negative quantity.

Do not rely entirely on application code for invariants that the database can safely enforce.

PYTHON

```
CHECK (quantity > 0)
```

## Naming Conventions

Consistent naming reduces cognitive load.

A project may choose conventions such as:

rather than mixing:

No single naming style is mandatory.

Consistency matters more than arguing over one convention.

Good names should reveal meaning and grain without requiring the developer to inspect every query.

PYTHON

```
customer_id  
created_at  
model_version_id
```

PYTHON

```
CustomerID  
customerId  
cust_id  
idCustomer
```

## Schema Documentation

Important tables should be documented with:

- grain
- purpose
- primary key
- important candidate keys
- foreign keys
- major constraints
- lifecycle

- source of truth
- historical behavior
- ownership

For analytical tables also document:

- measure semantics
- dimensions
- refresh cadence
- late-arriving data policy
- timezone
- data-quality expectations

A schema without semantic documentation becomes increasingly difficult to use safely as teams grow.

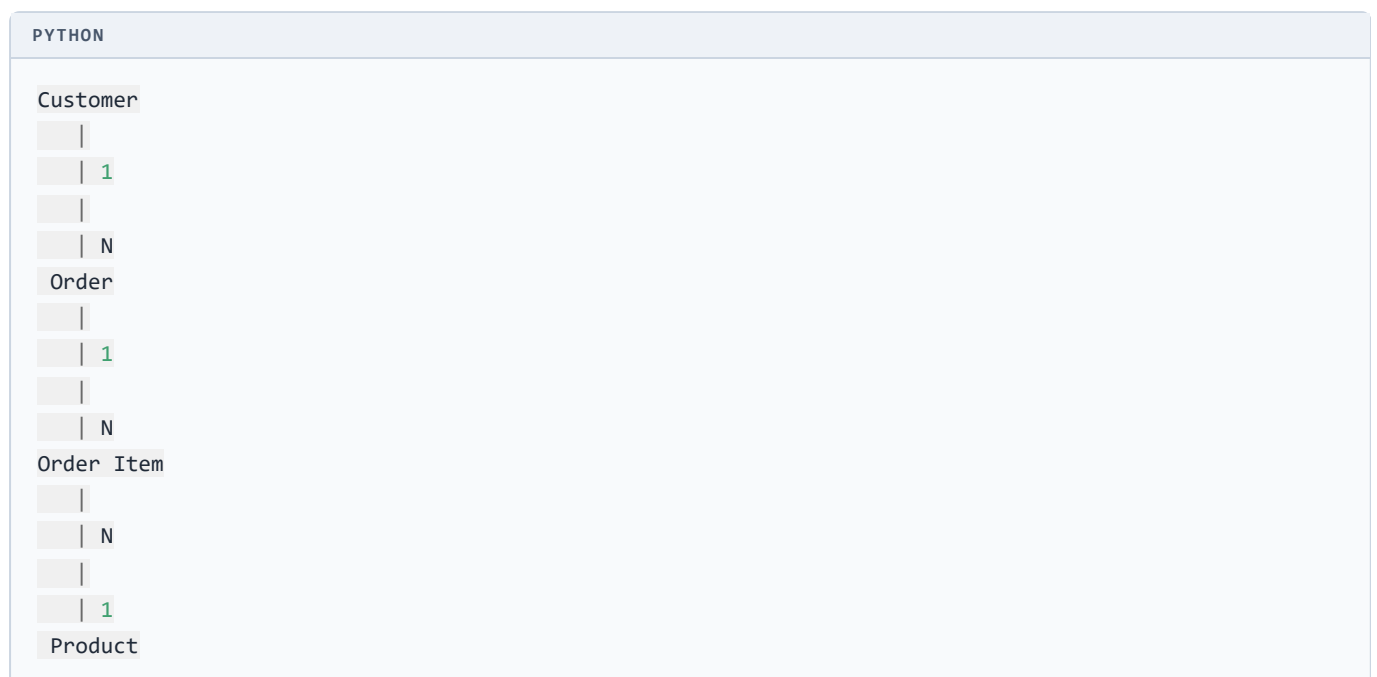
## Entity-Relationship Diagrams

Entity-relationship diagrams provide a visual representation of:

- entities
- attributes
- identifiers
- relationships
- cardinality

A simplified commerce model may look conceptually like:

The diagram helps communicate structure, but the actual database must still enforce important rules with keys and constraints.



## Cardinality

Relationship cardinality describes how many rows can participate.

Common patterns:

But optionality also matters.

Examples:

That is more informative than simply saying "one-to-many."

Cardinality should come from business semantics, not from current row counts.

PYTHON

1:1

1:N

N:M

PYTHON

customer -> orders

one customer may have zero **or** many orders

each order must have exactly one customer

## Modeling Time

Many schemas become difficult because they fail to distinguish:

from:

Examples include:

- customer address
- product price
- model deployment
- employee department
- feature value
- account status

Temporal requirements should be discovered during design.

If historical reconstruction will matter later, repeatedly overwriting current state may permanently destroy required information.

PYTHON

what **is** true now

## PYTHON

```
what was true at a particular time
```

## Effective-Dated Records

One temporal pattern stores validity intervals:

Then a time-specific query asks for the record whose interval contains the target timestamp.

For example:

followed by:

Temporal modeling introduces important issues such as:

- overlapping intervals
- open-ended current records
- boundary semantics
- point-in-time joins

It deserves deliberate constraints and query design.

A common history design stores:

For example:

A row with:

often represents the current version.

A historical query needs the version whose validity interval contains the target time.

## PYTHON

```
entity_id  
valid_from  
valid_to  
value
```

## Soft Deletes and Uniqueness

Soft deletion complicates schema design.

Suppose users have:

If a deleted user's row remains, can a new account reuse the same email?

Different products may answer differently.

Potential solutions vary by database and policy:

- never reuse identity



- reactivate the old row
- partial unique index for active rows
- archive old identity elsewhere

Soft deletion should therefore be included in key and uniqueness design rather than added later as a Boolean flag.

PYTHON

```
email UNIQUE
deleted_at
```

## Schema Design and Index Design Are Different

A key or constraint represents semantics.

An index represents a physical access structure.

They overlap because databases often implement keys using indexes, but the concepts are distinct.

For example:

is a semantic rule.

is an access-pattern requirement.

The schema should not distort business meaning solely to fit a particular index.

PYTHON

```
customer email must be unique
```

PYTHON

```
queries frequently filter by created_at
```

## Schema Design and Transactions Work Together

Normalization separates facts into appropriate tables.

That means one business operation may need to update multiple tables.

For example, model registration may create:

Transactions ensure these related normalized writes remain consistent as one logical operation.

Normalization does not eliminate transaction complexity; it gives transactions well-defined entities and invariants to protect.

## PYTHON

```
model_version  
artifact metadata  
evaluation metadata  
provenance links
```

## Over-Normalization Awareness

Not every conceptual noun requires its own table.

A schema with dozens of tiny tables for values that have no independent lifecycle or integrity requirements can become difficult to query and maintain.

Examples of questionable decomposition might include creating separate entities for trivial attributes purely to claim a higher level of normalization.

Good normalization follows dependencies.

It is not a competition to maximize the number of joins.

## Under-Normalization Awareness

The opposite problem is putting unrelated facts into one giant table.

Symptoms include:

- many repeated descriptive columns
- comma-separated lists
- dozens of nullable type-specific fields
- repeated update logic
- conflicting copies of the same fact
- difficulty enforcing uniqueness
- deletion of one record destroying unrelated information

When these symptoms appear, return to:

rather than patching every issue with application logic.

## PYTHON

```
grain  
keys  
functional dependencies  
relationships
```

## A Schema Design Workflow

A practical relational design workflow is:

1. understand the business/domain requirements

2. identify important entities
3. state the grain of each table
4. identify candidate keys
5. identify functional dependencies
6. identify relationships and cardinalities
7. normalize inappropriate repeated facts
8. choose primary keys
9. preserve alternate/business uniqueness
10. define foreign keys
11. define nullability
12. add `CHECK` and domain constraints
13. decide referential actions
14. consider historical/temporal requirements
15. test with realistic operations
16. design workload-driven indexes separately
17. document the model
18. evolve it through controlled migrations

Schema design should be iterative, but not accidental.

### Schema Review Questions

Before accepting a table, ask:

- What does one row represent?
- What is the primary key?
- What other candidate keys exist?
- Which attributes depend on that key?
- Are any facts repeated unnecessarily?
- Are there partial dependencies?
- Are there transitive dependencies?
- Can the entity exist independently?
- What relationships does it have?
- Are foreign keys mandatory or optional?
- What happens on deletion?
- Which values describe current state?
- Which values describe historical state?
- What must be unique?

- Which invariants can the database enforce?
- How will the schema evolve?

If the grain cannot be explained in one clear sentence, the table likely needs more design work.

### Common Schema Design Mistakes

Common mistakes include:

- designing tables before understanding grain
- using names as keys simply because they look unique
- adding surrogate keys but forgetting business uniqueness
- storing comma-separated foreign IDs
- mixing several grains in one table
- duplicating mutable attributes everywhere
- misunderstanding historical snapshots as accidental duplication
- storing derived values without synchronization rules
- making every column nullable
- using sentinel values instead of `NULL`
- creating many-to-many relationships without junction tables
- placing relationship attributes on the wrong entity
- using cascades without understanding deletion consequences
- blindly normalizing or denormalizing
- ignoring temporal requirements
- storing ambiguous dataset/model identifiers in AI systems
- treating schema design as only a SQL syntax problem

### Normalization Decision Summary

A useful reasoning sequence is:

This is more useful than memorizing definitions without applying them to actual tables.

## PYTHON

What does one row mean?

|

What uniquely identifies it?

|

What facts depend on that identity?

|

Do some facts depend only on part of a composite key?

|

Do non-key facts determine other non-key facts?

|

Are independent entities mixed together?

|

Can relationships be represented **with** foreign keys?

|

Is **any** redundancy intentional **and** justified?

## AI Engineering Schema Checklist

For AI/ML metadata, additionally ask:

- Is model identity separate from model-version identity?
- Is a training run separate from the resulting model?
- Can failed runs exist?
- What uniquely identifies a dataset snapshot?
- Can dataset versions be reproduced?
- What is the grain of a metric?
- Which evaluation dataset produced that metric?
- Can one model have several artifacts?
- Can one model version have several deployments?
- Are environments modeled explicitly?
- Is deployment history preserved?
- Can a prediction be traced to the exact model version?
- Can a model be traced back to its training data?
- Are feature values point-in-time identifiable?
- Which metadata is mutable?
- Which provenance data should be immutable?

These questions turn AI metadata from ad-hoc logging into an engineered relational system.

## Schema Design, Normalization, and Data Modeling Summary

Schema design defines the meaning and integrity of relational data before query optimization begins.

The grain states what one row represents.

Functional dependencies describe which attributes determine other attributes.

Candidate keys uniquely identify rows with no unnecessary attributes. A primary key is the selected candidate key, while other business identifiers may require `UNIQUE` constraints.

Normalization reduces inappropriate redundancy and modification anomalies.

First Normal Form addresses relationally structured values and repeating groups.

Second Normal Form removes improper partial dependencies on composite keys.

Third Normal Form addresses transitive dependencies among non-key facts.

BCNF strengthens the principle by requiring determinants of non-trivial dependencies to be superkeys.

Relationships are modeled explicitly through foreign keys and junction tables. One-to-one, one-to-many, many-to-many, optional, and recursive relationships each encode different semantics.

Denormalization can improve measured workloads, but redundant data requires synchronization and ownership.

Operational schemas and analytical schemas solve different problems. Dimensional models organize measurable facts around descriptive dimensions, with star schemas providing a common analytical structure.

For AI engineering, careful relational modeling supports reproducible datasets, model registries, experiments, evaluation metrics, feature metadata, deployment history, and inference observability.

The core principle remains:

Define the grain, identify the dependencies, encode the relationships, and protect the invariants.

## Production Database Features and Operational Engineering

Knowing SQL syntax is not enough to operate a database-backed production system.

A real database exists inside a larger engineering environment involving:

- applications
- APIs
- background workers
- analytics pipelines
- authentication systems
- backups
- storage
- monitoring
- deployment infrastructure

- replication
- security controls
- migrations
- failure recovery

An SQL statement may be perfectly correct while the surrounding database architecture is unreliable.

For example:

- a query may work but exhaust all database connections
- an application may write correctly but have no recoverable backup
- a schema migration may succeed locally but lock a production table
- a read replica may return stale information
- excessive privileges may turn one compromised service into a database-wide incident
- an ML service may register a model correctly but lose metadata after an unrecoverable database failure

This section connects relational SQL foundations to production database engineering.

We will study:

- views
- view limitations
- temporary tables
- temporary data scope
- materialized-view awareness
- generated columns
- triggers
- trigger risks
- stored procedure and function awareness
- schema migrations
- migration safety
- connection lifecycle
- connection pooling
- pool exhaustion
- timeouts
- backups
- restoration
- recovery objectives
- write-ahead logging awareness

- replication
- primary/replica architectures
- replication lag
- read-after-write consistency
- partitioning
- partition pruning
- sharding
- distributed database trade-offs
- roles and privileges
- least privilege
- database credentials
- TLS awareness
- SQL injection defense
- observability
- slow-query monitoring
- database health metrics
- PostgreSQL awareness
- MySQL awareness
- SQLite awareness
- OLTP and OLAP systems
- selecting databases for AI systems
- production AI metadata architecture

The central principle is:

A production database must be correct not only when everything works, but also when systems scale, fail, retry, recover, and operate concurrently.

### A Separate Production-Features Laboratory

We will use another in-memory SQLite database for executable examples.

SQLite supports several production-relevant relational features, including:

- views
- temporary tables
- triggers
- generated columns in modern SQLite versions
- transactions



- constraints

Other topics in this section—such as server-side connection pooling, read replicas, sharding, database roles, and stored procedures—are architectural concepts that SQLite does not model in the same way as PostgreSQL or MySQL.

Those concepts will therefore be explained carefully without pretending that SQLite provides identical behavior.

## SQL

```
import sqlite3
import pandas as pd
from IPython.display import display

prod_conn = sqlite3.connect(":memory:")
prod_conn.execute("PRAGMA foreign_keys = ON")

prod_conn.executescript("""
CREATE TABLE customers_prod (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL,
    region TEXT NOT NULL
);

CREATE TABLE orders_prod (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date TEXT NOT NULL,
    status TEXT NOT NULL
    CHECK (
        status IN (
            'pending',
            'paid',
            'cancelled',
            'shipped'
        )
    ),
    total_amount REAL NOT NULL
    # ...
ORDER BY order_id
""",
prod_conn
))
```

## Views

A view is a named query.

Instead of repeatedly writing:

we can define a view:

Applications can then query:

A normal view generally stores the query definition rather than a permanently materialized copy of all query results.

When the underlying data changes, querying the view normally reflects the changed data.

SQL

```
prod_conn.execute("""
CREATE VIEW customer_order_details AS
SELECT
    o.order_id,
    o.order_date,
    o.status,
    o.total_amount,
    c.customer_id,
    c.customer_name,
    c.region
FROM orders_prod AS o
JOIN customers_prod AS c
    ON c.customer_id = o.customer_id
""")

display(pd.read_sql_query(
    """
    SELECT *
    FROM customer_order_details
    ORDER BY order_id
    """,
    prod_conn
))
```

SQL

```
SELECT
    c.customer_id,
    c.customer_name,
    c.region,
    o.order_id,
    o.order_date,
    o.status,
    o.total_amount
FROM customers AS c
JOIN orders AS o
    ON o.customer_id = c.customer_id;
```

## Why Views Are Useful

Views can provide several benefits.

First, they can simplify repeated SQL.

Instead of repeating a complicated join in many applications, developers can query a meaningful relational interface.

Second, views can centralize definitions.

For example, a business may define "completed revenue" using one standardized query rather than allowing every analyst to invent a slightly different rule.

Third, views can provide abstraction.

Applications can query a stable logical interface while parts of the underlying schema evolve.

Fourth, database systems may use views as part of access-control designs by exposing only selected rows or columns.

However, views are not automatically performance optimizations. A normal view commonly causes its underlying query logic to participate in execution when the view is queried.

### Views Do Not Automatically Store Results

This distinction is essential:

is usually:

rather than:

Suppose a view computes:

Querying that view may still require the database to perform the aggregation.

If persistent precomputation is required, a materialized structure may be needed.

SQL

```
prod_conn.execute("""
CREATE VIEW customer_revenue AS
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(total_amount) AS total_order_amount
FROM orders_prod
WHERE status IN ('paid', 'shipped')
GROUP BY customer_id
""")

display(pd.read_sql_query(
    """
    SELECT *
    FROM customer_revenue
    ORDER BY customer_id
    """,
    prod_conn
))
```

SQL

ordinary **view**

## Views and Query Composition

Views can be queried like relational sources:

This lets developers build higher-level queries from reusable logical components.

However, deeply nested layers of views can make systems difficult to understand.

A query may appear simple:

while `final_report` depends on several views, each containing joins and aggregations.

Good abstraction reduces complexity. Excessive abstraction can hide it.

Document important views and inspect execution plans when performance matters.

SQL

```
display(pd.read_sql_query(  
    """  
    SELECT  
        c.customer_name,  
        r.order_count,  
        r.total_order_amount  
    FROM customer_revenue AS r  
    JOIN customers_prod AS c  
        ON c.customer_id = r.customer_id  
    WHERE r.total_order_amount >= 1000  
    ORDER BY r.total_order_amount DESC  
    """,  
    prod_conn  
))
```

## Can Views Be Updated?

Whether a view can be directly modified depends on:

- database system
- view definition
- joins
- aggregation
- grouping
- computed expressions
- database-specific rules
- triggers or special mechanisms

A simple view over one table may be writable in some database systems.

A view containing:

is generally not directly equivalent to one underlying row and therefore may not be naturally updateable.

Do not assume every view behaves like a physical table for writes.

PYTHON

```
GROUP BY
SUM
multiple joined tables
```

## Views as Security Boundaries

Suppose an employee table contains:

A reporting user might need employee names and departments but not sensitive fields.

A database platform may allow access to a restricted view such as:

and grant access to the view instead of the base table.

This can be useful, but secure design requires understanding the database's actual privilege model. A view is not automatically a security boundary merely because sensitive columns were omitted from its SQL definition.

SQL

```
employee_id
employee_name
department
salary
government_identifier
```

SQL

```
CREATE VIEW employee_directory AS
SELECT
    employee_id,
    employee_name,
    department
FROM employees;
```

## Materialized Views

A materialized view stores the result of a query physically rather than recalculating the complete result whenever it is queried.

Conceptually:

Materialized views can accelerate expensive:

- aggregations

- reporting queries
- joins
- analytical summaries

The trade-off is freshness.

If source data changes, the materialized result must be refreshed according to database-specific mechanisms.

PYTHON

```
ordinary view
=
saved query

materialized view
=
saved query result
```

### Materialized View Freshness

Suppose a dashboard uses a materialized daily revenue summary.

The source orders table changes continuously.

If the materialized result refreshes every hour, the dashboard may be up to roughly one refresh interval behind the source system.

This creates a design trade-off:

For some workloads, a one-hour delay is acceptable.

For fraud detection, account balance, or live deployment state, stale data may be unacceptable.

Materialization decisions therefore depend on data semantics, not just speed.

PYTHON

```
freshness
versus
query cost
```

### SQLite and Materialized Views

SQLite does not provide native materialized views in the same sense as database systems such as PostgreSQL.

A similar pattern can be implemented manually using:

- summary tables
- scheduled refresh jobs
- triggers in limited cases
- application-controlled rebuilds

But those are application designs rather than native SQLite materialized-view semantics.

This distinction is important because SQL features are not perfectly portable across database systems.

## Temporary Tables

A temporary table stores intermediate relational data for a limited scope.

SQLite supports:

Temporary tables are useful when a workflow needs to:

- stage intermediate results
- break a complicated transformation into steps
- repeatedly reuse an expensive intermediate relation
- compare batches
- perform data cleaning
- support migration operations

A temporary table is physically distinct from a CTE.

A CTE belongs to a SQL statement. A temporary table can generally be reused across multiple statements within its applicable connection/session scope.

SQL

```
prod_conn.execute("""
CREATE TEMP TABLE high_value_orders AS
SELECT
    order_id,
    customer_id,
    total_amount
FROM orders_prod
WHERE total_amount >= 800
""")

display(pd.read_sql_query(
    """
    SELECT *
    FROM high_value_orders
    ORDER BY total_amount DESC
    """,
    prod_conn
))
```

SQL

```
CREATE TEMP TABLE ...
```

## Temporary Tables Versus CTEs

Consider a transformation used once inside one query.

A CTE may be appropriate:

But suppose the intermediate result will be reused by five separate queries.

A temporary table may make the workflow clearer and can sometimes avoid repeated computation.

The trade-offs differ:

Performance behavior remains database-specific.

SQL

```
WITH filtered AS (...)  
SELECT ...  
FROM filtered;
```

## Temporary Tables and Connection Pools

A subtle production issue is that temporary tables often belong to a database session or connection.

In an application using a connection pool:

1. request A obtains connection 7
2. it creates a temporary table
3. connection 7 returns to the pool
4. another operation receives a different connection

That operation may not see the temporary object.

Or worse, pooled connection reuse may expose temporary session state unexpectedly if cleanup is poor.

Do not build application correctness around connection-local state unless connection ownership is explicit.

## Generated Columns

A generated column derives its value from other values in the row according to a database-defined expression.

Conceptually:

This can centralize deterministic derived logic and prevent application clients from manually storing inconsistent values.

Database systems differ in:

- generated-column syntax
- virtual versus stored behavior
- permitted expressions



- index support

Always consult the target database documentation.

SQL

```
sqlite_version = tuple(
    int(part)
    for part in sqlite3.sqlite_version.split(".")[:3]
)

print("SQLite version:", sqlite3.sqlite_version)

if sqlite_version >= (3, 31, 0):
    prod_conn.execute("""
        CREATE TABLE invoice_lines (
            line_id INTEGER PRIMARY KEY,
            quantity INTEGER NOT NULL
                CHECK (quantity > 0),
            unit_price REAL NOT NULL
                CHECK (unit_price >= 0),
            line_total REAL
                GENERATED ALWAYS AS (
                    quantity * unit_price
                ) STORED
        )
    """)

    prod_conn.executemany(
        """
        INSERT INTO invoice_lines (
            line_id,
            quantity,
            unit_price
            # ...
        )
        print(
            "Generated-column example skipped because "
            "SQLite 3.31.0+ is required."
        )
```

SQL

```
quantity INTEGER,
unit_price REAL,
line_total REAL GENERATED ALWAYS AS (
    quantity * unit_price
)
```

## Generated Columns Versus Application Calculation

Suppose an application calculates:

in Python and writes all three values independently.

A bug could write:

If `line_total` is generated by the database, the derived value cannot independently drift from its defining expression.

However, not every computation belongs in a generated column.

Complex business logic, external data, non-deterministic operations, or expensive ML inference generally belong elsewhere.

Generated columns are most useful for deterministic row-local expressions supported by the database.

PYTHON

```
line_total = quantity * unit_price
```

PYTHON

```
quantity = 2
unit_price = 100
line_total = 999
```

## Triggers

A trigger is database logic that automatically executes in response to certain database events.

Triggers can commonly respond to operations such as:

A trigger might:

- create an audit record
- enforce a specialized rule
- maintain a derived table
- synchronize related state
- reject invalid transitions

Triggers are powerful because their behavior occurs regardless of which application client performed the write.

But they also introduce hidden behavior.

SQL

```
INSERT
UPDATE
DELETE
```

## An Audit Trigger Example

Suppose we want a record whenever an order status changes.

Instead of relying on every application client to remember to insert an audit row, a trigger can respond to the update.

We will create:

and an `AFTER UPDATE OF status` trigger.

SQL

```
prod_conn.executescript("""
CREATE TABLE order_status_audit (
    audit_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    old_status TEXT NOT NULL,
    new_status TEXT NOT NULL,
    changed_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER audit_order_status_change
AFTER UPDATE OF status ON orders_prod
WHEN OLD.status <> NEW.status
BEGIN
    INSERT INTO order_status_audit (
        order_id,
        old_status,
        new_status
    )
    VALUES (
        OLD.order_id,
        OLD.status,
        NEW.status
    );
END;
""")

prod_conn.execute(
    """
    # ...
    ORDER BY audit_id
    """,
    prod_conn
))
```

SQL

```
order_status_audit
```

## OLD and NEW in Triggers

Trigger systems often expose conceptual old and new row values.

In SQLite:

represents the previous row value where appropriate.

represents the new row value where appropriate.

Our trigger used:

so the audit record is created only when the status actually changes.

This avoids recording an audit event for an update that leaves the status unchanged.

| PYTHON     |
|------------|
| OLD.column |

| PYTHON     |
|------------|
| NEW.column |

## Trigger Advantages

Triggers can be useful when a rule must apply to every database client.

If five applications write directly to the same database, implementing a critical audit rule only in one application's code is insufficient.

Database-side trigger logic can centralize behavior.

Potential uses include:

- audit history
- invariant enforcement
- change propagation
- derived data maintenance

However, triggers should be used selectively because they make database writes less explicit from the calling application's perspective.

## Trigger Risks

Triggers can surprise developers.

An application executes:

but the database may internally perform additional writes through triggers.

Potential problems include:

- hidden performance cost
- difficult debugging
- recursive trigger behavior
- unexpected lock duration
- complicated migration dependencies

- duplicated business logic
- surprising side effects

Before modifying a production table, engineers should understand whether triggers exist on it.

Triggers are not inherently bad. Hidden, undocumented trigger logic is.

SQL

```
display(pd.read_sql_query(  
    """  
    SELECT  
        name,  
        tbl_name,  
        sql  
    FROM sqlite_master  
    WHERE type = 'trigger'  
    ORDER BY name  
    """,  
    prod_conn  
))
```

## Stored Procedures

Many server-oriented relational databases support stored procedures.

A stored procedure is a named program stored and executed in the database environment.

Depending on the database, procedures can:

- accept parameters
- run multiple SQL statements
- contain branching
- perform loops
- manage transactions or transaction-related behavior
- return result sets
- encapsulate database operations

Stored-procedure languages and capabilities vary significantly across database products.

SQLite does not have a stored-procedure system equivalent to PostgreSQL PL/pgSQL or MySQL stored procedures.

## Database Functions

Database systems also support built-in and sometimes user-defined functions.

We have already used scalar functions such as:

Some databases allow developers to define custom functions that execute server-side.

Possible categories include:

- scalar functions
- aggregate functions
- table-returning functions

Functions can centralize reusable logic, but excessive database-specific procedural logic can reduce portability and complicate application architecture.

PYTHON

```
LOWER(...)  
ROUND(...)  
COALESCE(...)
```

## Stored Logic Versus Application Logic

A recurring architecture question is:

Should this logic live in the database or application?

Database-side logic can provide:

- centralized enforcement
- fewer network round trips
- close access to data
- transactional integration

Application-side logic can provide:

- familiar programming languages
- easier application testing
- broader libraries
- service-level ownership
- easier portability

There is no universal rule that all business logic belongs in one place.

The important requirement is clear ownership and avoidance of duplicated contradictory implementations.

## Schema Migrations in Production

A migration is a reproducible change to database structure or controlled data state.

Examples include:

Migrations should normally be:

- version controlled
- reviewable
- repeatable
- observable
- compatible with deployment strategy
- tested against realistic data volumes

A migration that takes 20 milliseconds on a development database may behave very differently on a billion-row production table.

PYTHON

```
create a table
add a column
add an index
introduce a constraint
backfill values
rename an object
remove obsolete structure
```

## Migration Files

A migration system often stores ordered changes conceptually like:

Each deployment can determine which schema versions have already been applied.

Popular application frameworks and database tools provide migration systems, but the underlying engineering principle is independent of framework:

Schema history should be explicit and reproducible.

PYTHON

```
001_create_users
002_create_orders
003_add_order_status
004_add_orders_index
```

## Expand and Contract Migrations

Suppose an application currently uses:

but the next schema wants:

Dropping `full_name` immediately can break old application instances still running during deployment.

A safer process can be:

This pattern is especially valuable in zero-downtime or rolling deployments.

PYTHON

`full_name`

PYTHON

`first_name``last_name`

## Large Backfills

Adding a nullable column can be cheap in some databases, while rewriting values for hundreds of millions of rows may be expensive.

Large backfills can create:

- transaction-log growth
- replication lag
- lock contention
- storage pressure
- cache disruption
- long-running transactions

Safer approaches may use:

- batches
- resumable jobs
- throttling
- checkpoints
- progress tracking
- idempotent updates

Migration engineering depends on both logical correctness and operational scale.

## Index Creation in Production

Creating an index on a large table requires substantial work.

Depending on the database and index-building mode, it can:

- scan large amounts of data
- consume CPU
- consume disk I/O
- require temporary storage
- block or interfere with writes
- increase replication load



Some databases provide concurrent or online index-building options with their own trade-offs.

Never assume:

has negligible operational impact merely because the SQL statement is short.

```
PYTHON
```

```
CREATE INDEX ...
```

## Connection Lifecycle

A database connection represents a communication/session context between an application and database.

A typical server-database connection may involve:

- network establishment
- authentication
- session initialization
- server memory/resources
- transaction state

Applications should not create unlimited connections casually.

Connections must also be cleaned up correctly so that:

- transactions do not remain open
- locks are released
- server resources are returned
- credentials are handled safely

## Python Connection Management

For SQLite, we can explicitly open and close a connection:

Context management can help with transaction handling, but exact close semantics depend on the object/library.

For server databases, connection lifecycle is commonly managed by higher-level libraries and connection pools.

Always distinguish:

- transaction scope
- connection scope
- cursor/result scope

They are related but not identical.

## SQL

```
example_conn = sqlite3.connect(":memory:")

try:
    example_conn.execute("""
        CREATE TABLE example (
            id INTEGER PRIMARY KEY,
            value TEXT NOT NULL
        )
    """)

    with example_conn:
        example_conn.execute(
            """
            INSERT INTO example (id, value)
            VALUES (?, ?)
            """,
            (1, "safe transaction example"),
        )

    print(
        example_conn.execute(
            "SELECT * FROM example"
        ).fetchall()
    )

finally:
    example_conn.close()

print("Connection closed explicitly.")
```

## Connection Pooling

Server-based applications frequently use a connection pool.

Instead of opening a new database connection for every request:

the application maintains reusable connections:

Pooling can reduce connection setup overhead and protect the database from unlimited connection creation.

## PYTHON

```
request
-> create connection
-> query
-> destroy connection
```

## PYTHON

```
connection pool

request A -> borrow connection
request B -> borrow connection
...
return connections to pool
```

## A Connection Pool Is Not Infinite

Suppose an application has a pool of 20 connections.

If 20 requests each acquire a connection and hold it while waiting for a slow external API, request 21 cannot immediately obtain one.

This is **pool exhaustion**.

Symptoms may appear as:

- request timeouts
- growing queues
- apparent database latency
- cascading service failures

The database query itself may not be the root problem.

Applications should hold database connections only for the period they actually need them.

## Pool Size

A larger connection pool does not automatically improve throughput.

If the database can efficiently process only a certain level of concurrent work, opening hundreds more active connections can increase:

- context switching
- memory pressure
- lock contention
- transaction contention
- queueing inside the database

Pool size should reflect:

- database capacity
- workload characteristics
- application instance count
- query duration

- concurrency requirements

Remember that if 50 application instances each open a 20-connection pool, the database may see up to approximately 1000 potential application connections.

## Connection Timeouts

Production applications need bounded waiting.

Useful timeout categories may include:

- connection-acquisition timeout
- connection-establishment timeout
- statement/query timeout
- transaction timeout
- lock timeout
- network timeout

Exact support varies by driver and database.

Without appropriate timeouts, failed dependencies can cause requests to wait indefinitely and consume resources needed by healthy work.

## Backups

A backup is a recoverable copy of database state.

Having a production database without a tested backup strategy means accepting the possibility of permanent data loss.

Backups may include:

- physical database backups
- logical exports
- snapshots
- continuous log/WAL archiving
- managed-cloud backup systems

Different methods have different restoration characteristics.

A backup strategy must be designed around recovery requirements rather than merely checking a box saying "backups enabled."

## A Backup Is Useful Only If It Can Be Restored

A backup process can report success while the backup is still unusable.

Potential failures include:

- corrupted backup

- missing encryption key
- incomplete files
- incompatible tooling
- undocumented restoration steps
- missing transaction logs
- incorrect permissions

Therefore:

Backups must be tested through restoration exercises.

Recovery is the actual capability. Backup creation is only one part of achieving it.

### Logical Backups

A logical backup represents database objects and data in a logical form.

Examples in various ecosystems include exports containing:

- schema definitions
- SQL statements
- rows
- metadata

Logical backups can be useful for:

- portability
- selected-object restoration
- smaller systems
- migration workflows

However, restoring massive databases through logical row-by-row operations can be slower than physical recovery techniques.

### Physical Backups

Physical backups capture database storage structures more directly.

They can often restore large databases efficiently but may be more tightly coupled to:

- database engine
- version
- storage layout
- operating environment

Production backup design often uses combinations of snapshots, physical backups, and transaction-log archiving.

## RPO and RTO

Two important disaster-recovery concepts are:

### Recovery Point Objective (RPO)

How much recent data loss is acceptable?

For example:

means the architecture should aim to avoid losing more than roughly five minutes of committed data in the specified disaster scenario.

### Recovery Time Objective (RTO)

How long can recovery take?

For example:

describes the targeted time to restore service.

Backup frequency alone does not define a complete recovery strategy.

```
PYTHON
```

```
RPO = 5 minutes
```

```
PYTHON
```

```
RTO = 30 minutes
```

## Point-in-Time Recovery

Some database systems combine a base backup with transaction logs so the database can be reconstructed to a selected point in time.

This can be useful after incidents such as:

Instead of restoring only last night's backup, point-in-time recovery may allow recovery to just before the destructive event.

Capabilities vary by database system and configuration.

```
PYTHON
```

```
10:02 valid database  
10:05 accidental DELETE  
10:06 mistake discovered
```

## SQLite Backup Awareness

SQLite databases are often contained primarily in files, but copying a database file blindly while writes are occurring is not always the correct backup strategy.

SQLite provides documented backup mechanisms and APIs designed to obtain consistent copies.

Journaling mode and active transactions also matter.

Production SQLite applications should use SQLite-supported backup procedures rather than assuming an arbitrary filesystem copy is always transactionally consistent.

### PYTHON

```
source_conn = sqlite3.connect(":memory:")
source_conn.execute(
    "CREATE TABLE important_data (id INTEGER, value TEXT)"
)
source_conn.executemany(
    "INSERT INTO important_data VALUES (?, ?)",
    [
        (1, "alpha"),
        (2, "beta"),
        (3, "gamma"),
    ],
)
source_conn.commit()

backup_conn = sqlite3.connect(":memory:")

source_conn.backup(backup_conn)

rows = backup_conn.execute(
    "SELECT * FROM important_data ORDER BY id"
).fetchall()

print("Rows in backup copy:", rows)

backup_conn.close()
source_conn.close()
```

## Write-Ahead Logging Awareness

Database systems commonly use logs to protect transactional state.

In write-ahead logging, the general principle is:

Record enough information in a durable log before treating corresponding data-page changes as safely persisted.

The log supports:

- crash recovery
- durability
- transaction reconstruction
- replication in some architectures
- point-in-time recovery in some systems

Implementation details differ substantially between SQLite, PostgreSQL, MySQL/InnoDB, and other database engines.

### SQLite WAL Mode

SQLite supports a Write-Ahead Logging journaling mode:

WAL mode can improve some concurrency patterns because readers and a writer can coexist differently than under rollback-journal modes.

However, WAL is not a universal "make SQLite faster" switch.

Operational details include:

- checkpointing
- WAL-file growth
- filesystem behavior
- backup handling
- multiple-process access

Configuration should reflect the actual application.

```
PYTHON
```

```
PRAGMA journal_mode = WAL;
```

### Replication

Replication maintains copies of data on multiple database nodes or instances.

A common architecture is:

Writes go to the primary, while replicas receive changes.

Replicas may provide:

- read scaling
- disaster-recovery options
- geographical distribution
- analytical offloading
- failover support



Replication introduces consistency and operational complexity.

PYTHON

```
primary
|
+---- replica 1
|
+---- replica 2
```

### Primary and Read Replicas

A typical primary/replica configuration routes:

while some reads can be sent to replicas.

This can reduce read load on the primary.

But replication is not necessarily instantaneous.

A replica may be behind the primary.

That delay is called **replication lag**.

SQL

```
INSERT / UPDATE / DELETE
->
primary
```

### Replication Lag

Imagine:

The application may observe the previous deployment.

This is not necessarily corruption. It may be expected asynchronous replication behavior.

Systems that require immediate read-after-write visibility may need to:

- read from the primary after a write
- use session-consistency mechanisms
- wait for replication position
- use stronger database-specific consistency guarantees

Architecture must align with application semantics.

## PYTHON

```
12:00:00
application commits new model deployment on primary

12:00:00.100
application reads deployment state from replica

replica has not received update yet
```

## AI Engineering and Stale Replicas

Replica lag can matter for AI systems.

Suppose the primary records:

but an inference service immediately reads an asynchronously replicated database and still sees:

Whether this is acceptable depends on the deployment architecture.

For critical configuration, model routing, permissions, and feature freshness, consistency requirements should be explicit rather than assumed.

## PYTHON

```
production model = fraud_v18
```

## Replication Does Not Replace Backups

If an application accidentally executes:

replication may faithfully copy that deletion to every replica.

Now all replicas contain the mistake.

Replication protects against some hardware/node failures.

Backups and historical recovery protect against other failure classes.

A reliable architecture often needs both.

## SQL

```
DELETE FROM important_table;
```

## High Availability

High availability aims to reduce service interruption when infrastructure fails.

A database architecture may include:

- primary node
- standby nodes

- health checks
- automated failover
- connection rerouting
- replication
- quorum mechanisms

Failover creates difficult questions:

- Which node is authoritative?
- Could two primaries temporarily exist?
- Was every transaction replicated?
- How are applications redirected?
- What happens to in-flight transactions?

Managed database platforms automate much of this complexity, but engineers still need to understand failure semantics.

## Partitioning

Partitioning divides one logical table into larger physical/logical pieces.

For example, an event table might be partitioned by month:

while the database presents a unified logical table interface.

Partitioning can help with:

- pruning irrelevant data
- retention
- archival
- maintenance
- very large table management

Partitioning is not the same as sharding.

PYTHON

```
events_2026_01
events_2026_02
events_2026_03
...
```

## Partition Pruning

Suppose a huge event table is partitioned by date.

A query:

may allow the engine to avoid reading unrelated partitions.

This is called partition pruning.

But if the query hides the partition key inside an incompatible transformation, pruning may become more difficult depending on the database.

Partition-aware query design is closely related to the sargability concepts studied earlier.

PYTHON

```
WHERE event_date >= '2026-09-01'  
AND event_date < '2026-10-01'
```

## Partitioning and Retention

Suppose inference telemetry is retained for 90 days.

Without partitioning, deleting old rows may require a massive operation:

With date partitions, some systems can remove or detach an entire old partition much more efficiently.

This can simplify lifecycle management for:

- logs
- telemetry
- events
- time-series data
- historical analytics

But partitioning adds schema and operational complexity, so it should solve a real scale problem.

SQL

```
DELETE ...  
WHERE event_time < ...;
```

## Sharding

Sharding distributes data across independent database nodes based on a shard key.

Conceptually:

Unlike ordinary partitioning inside one database system, sharding often creates distributed ownership across multiple database instances.

This can increase total capacity but introduces substantial complexity.

## PYTHON

```
tenant IDs 1-1000  
-> shard A
```

```
tenant IDs 1001-2000  
-> shard B
```

```
tenant IDs 2001-3000  
-> shard C
```

## Choosing a Shard Key

A shard key determines where data lives.

A poor shard key can create hotspots.

For example, if all new traffic goes to one time-based shard while older shards are idle, the workload may remain unbalanced.

A good shard key considers:

- data distribution
- request routing
- locality
- join patterns
- growth
- hot tenants
- resharding

Once sharding is introduced, operations that used to be simple local SQL may become distributed-system problems.

## Cross-Shard Operations

Before sharding:

may query one database.

After sharding, a global result may require:

Cross-shard joins and transactions can also become significantly harder.

Therefore:

Sharding should usually be introduced because measured scale requirements demand it, not because it sounds architecturally advanced.

## SQL

```
SELECT SUM(revenue)
FROM orders;
```

## Vertical and Horizontal Scaling

Database capacity can be increased in different ways.

Vertical scaling means giving one database machine more:

- CPU
- memory
- storage performance

Horizontal scaling means distributing workload across more machines, potentially through:

- read replicas
- sharding
- distributed database systems

Vertical scaling is often operationally simpler.

Horizontal scaling can provide greater aggregate capacity but introduces coordination and consistency complexity.

## Database Security

A production database usually contains valuable data and credentials.

Security therefore involves more than parameterizing SQL.

Important areas include:

- authentication
- authorization
- network controls
- transport encryption
- encryption at rest
- credential management
- auditing
- patching
- backup protection
- least privilege
- secure configuration

## Authentication Versus Authorization

Authentication answers:

Who are you?

Authorization answers:

What are you allowed to do?

A database account may successfully authenticate but still be restricted to certain:

- schemas
- tables
- views
- operations

Production systems should avoid giving every application administrative database credentials.

### Least Privilege

Least privilege means granting only the permissions needed for a role or service.

For example:

An analytics dashboard may need:

but not:

An inference telemetry writer may need:

and perhaps selected reads, but it probably should not be able to modify model registry history.

Reducing permissions reduces the blast radius of application bugs and credential compromise.

SQL

**SELECT**

SQL

**DROP TABLE**

**DELETE**

**ALTER**

### Database Roles

Server database systems commonly support roles or users representing permission groups.

Possible roles might include:

Permissions can then be assigned according to responsibility.

Exact syntax differs by database.

SQLite has a different embedded security model and does not provide a server-style role system equivalent to PostgreSQL.

PYTHON

```
app_readonly
app_writer
migration_runner
analytics_reader
backup_operator
database_admin
```

## Never Hard-Code Production Database Passwords

Credentials such as:

should not be committed into source code or notebooks.

Credentials should come from secure configuration mechanisms such as:

- environment variables
- secret managers
- workload identity
- managed service authentication
- protected deployment configuration

Even environment variables are only one layer of secret handling and must be managed carefully in production.

PYTHON

```
password = "production-password-123"
```

## Credential Rotation

Production credentials should be replaceable without redesigning the application.

Credential rotation reduces the lifetime of compromised secrets.

Architectures using:

- managed identities
- short-lived tokens
- dynamic credentials



can reduce reliance on long-lived passwords.

Database security should be designed around the full credential lifecycle:

#### PYTHON

```
creation
distribution
use
rotation
revocation
audit
```

## Encryption in Transit

For networked databases, application traffic may contain:

- personal information
- authentication data
- financial information
- model metadata
- proprietary features

TLS can protect network traffic from passive interception and certain network attacks when configured and verified correctly.

Production clients should verify certificates according to the database platform's security guidance rather than disabling TLS verification merely to resolve configuration issues.

## Encryption at Rest

Encryption at rest protects stored database files, disks, snapshots, and backups under relevant threat models.

Managed cloud databases may provide disk/storage encryption.

However, encryption at rest does not mean:

- every database administrator cannot read data
- an SQL injection cannot access rows
- an authenticated application cannot leak data

Security controls address different threat layers.

## SQL Injection Revisited

Production security begins with proper parameterization.

Unsafe:

Preferred:

Parameterization separates SQL structure from data values.

But remember that ordinary parameters cannot safely substitute arbitrary SQL identifiers or keywords.

Dynamic table names, columns, or sort directions require strict allowlisting or safe query-building mechanisms.

PYTHON

```
sql = (  
    "SELECT * FROM users "  
    "WHERE email = '" + email + "'"   
)
```

PYTHON

```
cursor.execute(  
    "SELECT * FROM users WHERE email = ?",  
    (email,)   
)
```

## Database Firewalls and Network Boundaries

A production database should generally not be exposed to the public internet merely because applications need connectivity.

Network design may restrict access using:

- private networks
- virtual private clouds
- security groups
- firewall rules
- private endpoints
- bastion/administrative access controls

Only intended services and operators should be able to establish database connections.

Network controls complement—not replace—database authentication and authorization.

## Observability

A production database should be observable.

Operators need evidence when answering:

- Is the database healthy?
- Are queries becoming slower?
- Are connections exhausted?
- Are transactions blocked?

- Is replication behind?
- Is storage nearly full?
- Are backups succeeding?
- Did error rate increase?
- Which query consumes most resources?

Without observability, database incidents become guesswork.

### Useful Database Metrics

Exact metrics vary by database, but commonly useful categories include:

- query latency
- query throughput
- active connections
- connection-pool usage
- transaction rate
- rollback rate
- lock waits
- deadlocks
- CPU utilization
- memory usage
- disk latency
- storage consumption
- cache hit behavior
- replication lag
- checkpoint behavior
- backup success
- error rate

One metric in isolation rarely explains the complete incident.

### Slow Query Monitoring

A slow-query system helps identify statements consuming excessive time or resources.

Questions to investigate include:

- Is the query suddenly slower?
- Did data volume grow?
- Did the plan change?
- Is an index missing?

- Are statistics stale?
- Is the database waiting on locks?
- Is storage slow?
- Is the application fetching too many rows?
- Is this query executed far too frequently?

A moderately slow query executed once per day may matter less than a 20-millisecond query executed hundreds of thousands of times per minute.

## Latency Percentiles

Average query latency can hide important behavior.

Suppose:

The average may not reveal how painful the slow tail is for users.

Production systems often monitor percentiles such as:

Tail latency is especially important when one API request performs several dependent database operations.

PYTHON

```
99 requests = 10 ms
1 request   = 5 seconds
```

## Lock Monitoring

A query may appear "slow" because it is not actively computing.

It may be waiting for another transaction to release a lock.

Useful production questions include:

- Which session holds the blocking lock?
- Which session is waiting?
- How old is the blocking transaction?
- Why has it remained open?
- Did an application forget to commit?
- Is a migration blocking traffic?

Database-specific monitoring tools expose this information differently.

## Long-Running Transactions

Long transactions can create several operational problems:

- locks held for too long
- old row versions retained

- cleanup delayed
- replication effects
- increased conflict probability
- large rollback cost

Monitoring should identify unexpectedly old transactions.

A database query taking a long time and an idle transaction left open are different problems, but both may harm production systems.

### **Query Logging and Sensitive Data**

Logging SQL can help diagnose production problems.

But query logs may accidentally capture:

- passwords
- tokens
- emails
- personal information
- confidential prompts
- model inputs

Observability systems themselves therefore require data-governance controls.

Prefer parameterized logging and deliberate redaction where appropriate.

Do not solve a debugging problem by creating a security incident in the logs.

### **PostgreSQL Awareness**

PostgreSQL is a powerful open-source relational database frequently used for production applications.

Capabilities include, among many others:

- strong transactional behavior
- rich SQL support
- sophisticated indexing
- JSON support
- extensibility
- views and materialized views
- advanced data types
- window functions
- CTEs
- stored functions/procedures

- roles and privileges
- replication
- partitioning

It is widely used in APIs, SaaS products, data platforms, and AI application backends.

### **MySQL Awareness**

MySQL is another widely used relational database.

Modern MySQL installations commonly use the InnoDB storage engine, which provides transactional behavior and foreign-key support.

MySQL is common in:

- web applications
- SaaS systems
- content platforms
- operational application databases

It supports:

- transactions
- indexes
- replication
- views
- stored routines
- JSON
- partition-related capabilities
- roles/privileges in modern versions

SQL syntax and behavior differ in important details from PostgreSQL and SQLite.

### **SQLite Awareness**

SQLite is fundamentally different architecturally from PostgreSQL and MySQL because it is an embedded database library rather than a separate database-server process.

Advantages include:

- simple deployment
- single-file database architecture
- no separate server administration
- full transactional SQL capabilities
- excellent local-development experience

- usefulness in desktop/mobile/embedded systems
- strong testing utility

SQLite can also support real production applications when its concurrency and operational model fits the workload.

### SQLite Is Not Just a Toy Database

SQLite is used extensively in real software.

The important question is not:

Is SQLite production capable?

The useful question is:

Does SQLite's architecture match this production workload?

SQLite may be excellent for:

- local applications
- edge software
- embedded services
- per-user data
- prototypes
- smaller server workloads
- caches and metadata stores in suitable architectures

A high-write, multi-server application needing large concurrent write throughput may be better matched to a server database.

### SQL Is a Family, Not One Perfectly Uniform Language

Core relational concepts transfer across databases, but syntax and behavior differ.

Examples include differences in:

- auto-incrementing keys
- UPSERT
- date/time functions
- string functions
- JSON
- generated columns

- stored procedures
- index types
- full-text search
- transaction isolation
- locking
- DDL transactions
- materialized views
- roles
- partitioning

Learning standard concepts is essential, but production engineers must also learn the exact database they operate.

### A Small Engine-Comparison Perspective

Very broadly:

These descriptions are intentionally broad.

Database selection should consider the workload and organization rather than internet popularity or simplistic "database X is always faster" claims.

#### PYTHON

##### SQLite

embedded, simple deployment, local file architecture

##### PostgreSQL

feature-rich server relational database, extensible, strong general-purpose choice

##### MySQL

widely deployed server relational database **with** a large operational ecosystem

### OLTP Databases

Online Transaction Processing systems focus on operational transactions.

Typical workload:

Characteristics often include:

- small queries
- indexed lookups
- concurrent users
- short transactions
- strong integrity
- frequent writes



PostgreSQL and MySQL are common general-purpose OLTP choices.

#### SQL

```
create order
update payment
read account
insert event
change deployment state
```

## OLAP Systems

Online Analytical Processing systems focus on analytical workloads.

Typical query:

Analytical engines often optimize for:

- columnar storage
- compression
- large scans
- vectorized execution
- distributed computation

A transactional relational database can perform analytics, but very large analytical workloads may be better served by dedicated analytical systems or warehouses.

#### PYTHON

```
scan large historical dataset
join large relations
aggregate by many dimensions
compute trends
```

## HTAP Awareness

Some modern systems attempt to support both transactional and analytical workloads, often described as Hybrid Transactional/Analytical Processing.

This can reduce the need to move data between completely separate systems for some workloads.

However, the architectural trade-offs remain.

A single database technology does not automatically make every transactional and analytical workload equally optimal.

## Choosing a Database

Database selection should begin with requirements.

Ask:

- Is the application local or networked?
- How many concurrent writers exist?
- How much data is expected?
- What latency is required?
- What transaction guarantees are required?
- Are complex joins important?
- Is JSON important?
- Is full-text search required?
- Is vector search required?
- Is the workload transactional or analytical?
- Is horizontal scaling actually needed?
- What backup/recovery guarantees are required?
- What operational expertise does the team have?
- Does a managed service exist?
- What compliance requirements apply?

Technology should follow workload.

## Managed Databases

Cloud providers offer managed relational databases that automate substantial operational work.

Managed services may provide:

- automated backups
- patching
- monitoring
- replicas
- failover
- storage management
- encryption
- maintenance automation

Managed does not mean responsibility disappears.

Engineers still need to understand:

- schema
- SQL performance
- capacity
- security

- recovery configuration
- consistency
- cost
- migration strategy

## Database Capacity Planning

Production databases need capacity headroom.

Important resources include:

- CPU
- memory
- storage capacity
- storage IOPS/throughput
- network
- connection slots

A database running permanently at its limits has little room for:

- traffic spikes
- maintenance
- failover
- backups
- index creation
- batch workloads

Capacity planning should account for future growth, not only today's average load.

## Data Growth

Database design should estimate growth.

Suppose inference telemetry generates:

Then approximately:

before considering indexes, metadata overhead, backups, and replicas.

Questions become:

- How long should raw data be retained?
- Should old data move to cheaper analytical storage?
- Do we need partitions?
- Which indexes remain affordable?
- What is the backup size?

- How long does restoration take?

Data lifecycle is a production architecture concern.

PYTHON

2 million rows/day

PYTHON

60 million rows/month

730 million rows/year

## Retention Policies

Not every row must remain forever.

Retention policies may specify:

Retention may be driven by:

- product requirements
- cost
- legal requirements
- privacy
- reproducibility
- security policy

Deletion itself must also be engineered safely and observably.

PYTHON

API logs: 30 days

inference telemetry: 180 days

audit history: 7 years

temporary staging data: 24 hours

## Data Privacy

Databases can contain personally identifiable or sensitive information.

Schema and operational architecture should consider:

- data minimization
- retention
- access controls
- encryption
- deletion requests
- auditability

- masking
- environment separation

Do not copy production data into development notebooks casually.

AI projects are particularly prone to creating uncontrolled dataset copies during experimentation.

## Health Checks

An application may expose a health endpoint.

But a health check should distinguish:

from:

A readiness check may verify database connectivity carefully.

However, aggressive health checks can themselves create load or trigger cascading failures.

Keep database health probes lightweight and purpose-specific.

```
PYTHON
```

```
process is alive
```

## Failure Modes

Production database systems should be designed with failure in mind.

Possible failures include:

- database process failure
- machine failure
- storage failure
- network partition
- DNS failure
- credential expiration
- pool exhaustion
- deadlock
- lock timeout
- replica lag
- full disk
- bad migration
- accidental deletion
- application retry storm

Reliable engineering asks:

## What happens next?

before the failure actually occurs.

### Retry Storms

Suppose a database becomes temporarily slow.

Applications time out and retry immediately.

Those retries create more database traffic.

The extra traffic makes the database slower.

More requests time out.

More retries occur.

This creates a positive feedback loop.

Resilient systems use controls such as:

- exponential backoff
- jitter
- bounded retries
- circuit breakers
- concurrency limits
- queueing

Retries are not free recovery.

### Idempotency Revisited

Retry-safe systems benefit from idempotency.

Suppose an API request creates a training job.

The client times out after the database commit and retries.

Without a unique logical request ID, the system might create two training jobs.

A database constraint such as:

can help deduplicate the retry.

Idempotency therefore combines:

- application protocol
- database uniqueness

- transaction design

```
PYTHON
```

```
request_id UNIQUE
```

## AI Engineering: Metadata Databases

AI systems often require an operational relational database for metadata such as:

- users
- organizations
- projects
- dataset versions
- training jobs
- model versions
- evaluation results
- deployments
- API keys
- job states
- audit events

This data is relational and transactional.

A relational database is often a strong fit even when model artifacts and large datasets live in other storage systems.

## AI Engineering: Do Not Store Everything in One Database

A production AI platform may use several storage technologies.

For example:

No single storage technology must solve every problem.

Choose storage according to data shape and access pattern.

## PYTHON

## Relational database

```
-> model metadata
-> users
-> deployments
-> jobs
```

## Object storage

```
-> model files
-> large datasets
-> checkpoints
```

## Analytical warehouse

```
-> large inference/event history
```

## Cache

```
-> frequently accessed ephemeral state
```

## Vector database/index

```
-> embedding similarity retrieval
```

## AI Engineering: Model Artifacts

Large model artifacts generally do not need to be stored directly as giant binary values inside the primary relational metadata database.

A common architecture stores:

while the relational database stores:

This separates large immutable object storage from transactional relational metadata.

## PYTHON

## artifact file

```
->
```

```
object/blob storage
```

## AI Engineering: Artifact Integrity

An artifact URI alone is not enough for strong integrity.

Metadata can also include:

A consumer can then verify that the retrieved artifact matches the registered metadata.

This connects relational database engineering with reliable model serialization.



## PYTHON

```
SHA-256 checksum
artifact size
serialization format
schema/version
creation timestamp
```

## AI Engineering: Training Job Coordination

A training system may use a relational table such as:

with states:

Multiple workers may attempt to claim work concurrently.

Correct job claiming requires transaction/concurrency design so two workers do not accidentally process the same logical job.

Production job queues often use specialized queue systems, database locking patterns, or conditional atomic updates depending on workload.

## PYTHON

```
training_jobs
```

## PYTHON

```
queued
running
completed
failed
```

## AI Engineering: Deployment Consistency

A deployment operation may involve:

1. validate model approval
2. change active model metadata
3. create deployment history
4. publish configuration
5. notify serving infrastructure

A database transaction can protect the relational steps.

But external serving systems are outside that local transaction.

Therefore production deployment architecture may need:

- idempotent events
- transactional outbox patterns

- workflow orchestration
- reconciliation
- rollback procedures

Database atomicity is necessary but may not be sufficient for distributed deployment workflows.

### The Transactional Outbox Pattern Awareness

Suppose an application needs to:

If it updates the database and crashes before publishing the message, downstream systems never learn about the change.

If it publishes first and database commit fails, consumers may receive an event describing state that never became committed.

A common distributed-systems pattern is a **transactional outbox**.

The application writes:

in the same database transaction.

A separate reliable process publishes committed outbox events and records progress.

This does not magically remove every delivery problem, but it provides a strong foundation for reliable event publication.

SQL

```
update database  
AND  
publish message
```

### AI Engineering: Online Versus Offline Storage

An ML system often has two different feature requirements.

Offline training may need:

Online inference may need:

These workloads can require different storage systems.

Using one database for both without evaluating workload characteristics can create performance and consistency problems.

PYTHON

```
years of historical records  
large scans  
point-in-time joins  
analytical aggregation
```

## PYTHON

```
one entity's latest features  
millisecond-scale lookup  
high request concurrency
```

## AI Engineering: Observability Data

Model-serving systems may generate enormous event volumes.

Examples:

An OLTP relational database might be appropriate for recent operational state, but long-term event analytics may belong in:

- warehouse
- lake/lakehouse
- columnar analytics system
- observability platform

Retention and aggregation should be planned from the beginning.

## PYTHON

```
prediction request  
latency  
model version  
confidence  
error type  
drift signal
```

## AI Engineering: Database Credentials

Model-serving code should not contain database passwords in:

- notebooks
- Dockerfiles
- Git repositories
- model artifacts
- frontend JavaScript

Production AI services should obtain credentials through secure deployment mechanisms.

Different services should ideally use distinct identities and least-privilege permissions.

A compromised inference service should not automatically gain schema-administrator privileges.

## AI Engineering: Database Migration and Model Services

A model-serving application may depend on metadata columns or feature schemas.

If the database schema changes incompatibly while older serving instances are still active, production inference can fail.

Database migrations and application/model-service deployment therefore need coordinated compatibility.

This is another reason expand/contract migrations are important in production AI infrastructure.

### **PostgreSQL for AI Application Backends**

PostgreSQL is often a strong general-purpose choice for AI application metadata because it supports:

- relational transactions
- rich indexing
- JSON when flexibility is required
- mature operational tooling
- strong ecosystem
- managed cloud offerings

Extensions can add additional capabilities, but architecture should still separate different workload types when appropriate.

Selecting PostgreSQL should come from requirements, not from the assumption that every AI system must use the same database.

### **Database Abstraction Layers**

Application code may interact with databases through:

- raw SQL
- query builders
- database drivers
- ORMs

Object-Relational Mappers can improve developer productivity by mapping application objects to relational structures.

But abstraction does not eliminate the need to understand:

- SQL
- joins
- transactions
- indexing
- N+1 problems
- constraints
- query plans

An ORM can generate inefficient SQL just as a developer can write inefficient SQL manually.

## ORM Awareness

ORMs can provide:

- model definitions
- migrations
- relationship management
- parameterization
- transaction APIs
- query composition

But common ORM mistakes include:

- accidental N+1 queries
- loading entire tables
- holding sessions too long
- unclear transaction boundaries
- relying on application validation instead of database constraints

Learn the SQL generated by your ORM.

## Repository and Service Layers

Larger applications may isolate database operations behind repository or service abstractions.

This can improve:

- testability
- ownership
- transaction coordination
- API boundaries
- maintainability

However, abstraction should not hide critical behavior such as:

- whether a function commits
- whether it opens a transaction
- whether it performs 100 queries
- whether it acquires locks

Good abstractions hide implementation complexity without hiding operational semantics.

## Production Database Checklist

Before calling a database-backed application production-ready, ask:

- Is schema integrity enforced?
- Are migrations version controlled?
- Are queries parameterized?
- Are indexes based on measured workload?
- Are transactions explicit where required?
- Is the connection pool bounded?
- Are timeouts configured?
- Are long transactions monitored?
- Are credentials stored securely?
- Is least privilege applied?
- Is traffic encrypted where required?
- Are backups automated?
- Has restoration been tested?
- Are RPO and RTO understood?
- Is replication lag monitored?
- Are storage and capacity monitored?
- Are slow queries observable?
- Is retention defined?
- Is sensitive data governed?
- Is failure/retry behavior understood?
- Is operational documentation available?

Production readiness is a system property.

## Common Production Database Mistakes

Common mistakes include:

- treating views as cached results
- assuming temporary tables survive arbitrary pooled connections
- putting critical logic into undocumented triggers
- running untested migrations directly in production
- creating huge indexes during peak traffic
- opening a new connection for every small operation unnecessarily
- configuring enormous connection pools
- leaving transactions open during external API calls
- assuming replicas are instantly consistent
- treating replication as backup

- never testing restoration
- keeping database passwords in source code
- giving application services administrator permissions
- exposing the database publicly without need
- logging sensitive query parameters
- retrying every database error blindly
- sharding before scale requires it
- storing all AI artifacts directly in the operational metadata database
- ignoring data retention
- assuming managed databases eliminate operational responsibility

### **Production Architecture Reasoning**

A useful way to reason about production database design is to separate concerns.

A system can perform well in one category while remaining dangerously weak in another.

## PYTHON

## Correctness

- schema
- constraints
- transactions
- isolation

## Performance

- indexes
- query plans
- pooling
- caching
- workload design

## Availability

- replication
- failover
- health monitoring

## Durability

- transaction logs
- backups
- restore procedures

## Security

- authentication
- authorization
- encryption
- secrets
- # ...
- logs
- migrations
- runbooks
- alerts

## Production Database Features and Operational Engineering Summary

Production database engineering extends far beyond writing SQL queries.

Views provide reusable relational interfaces, while materialized views or summary tables can persist expensive query results when freshness trade-offs allow it.

Temporary tables support reusable session-scoped intermediate relations. Generated columns can keep deterministic derived values consistent. Triggers provide automatic database-side behavior but must be documented because they create implicit side effects.

Production schemas evolve through controlled migrations. Large migrations and index creation can become operational events rather than trivial SQL statements.

Connection pools reduce connection setup overhead and bound database concurrency, but poor connection ownership can exhaust pools and destabilize applications.



Backups must be designed around actual recovery. Restoration testing, RPO, RTO, and point-in-time recovery are more meaningful than merely reporting that backups exist.

Replication improves availability and read scaling but introduces replica lag and consistency considerations. Replication does not replace backups.

Partitioning divides large logical tables inside database architectures, while sharding distributes data across independent nodes and introduces distributed-system complexity.

Database security requires authentication, authorization, least privilege, credential management, encryption, network controls, and careful observability.

SQLite, PostgreSQL, and MySQL share relational foundations but differ substantially in architecture and features. Production engineers must understand the exact system they operate.

AI systems commonly combine relational databases with object storage, analytical systems, caches, and specialized retrieval infrastructure. Relational databases are especially valuable for transactional metadata, provenance, jobs, model registries, deployments, and operational state.

The central engineering principle is:

Choose database architecture from correctness, workload, recovery, security, and operational requirements—not merely from SQL syntax or technology popularity.

## Advanced SQL Patterns and Data Pipeline Engineering

The previous sections developed SQL from relational foundations through querying, aggregation, joins, subqueries, window functions, transactions, indexing, modeling, and production database architecture.

This section integrates those ideas into patterns commonly found in analytics, data engineering, product systems, and AI pipelines.

The difficulty of advanced SQL is usually not syntax alone.

The difficult questions are often:

- What is the grain of the source data?
- What exactly is the metric?
- Which event counts as the first event?
- Which timestamp should define the cohort?
- Can the same entity appear more than once?
- Are late-arriving rows possible?
- Is the transformation idempotent?
- Is the pipeline using event time or ingestion time?
- Can historical data change after it was processed?
- Is the feature point-in-time correct?

- Can the query be tested automatically?
- What happens when the pipeline is rerun?

We will study:

- reusable analytical query structure
- cohort analysis
- retention analysis
- funnel analysis
- ordered funnels
- deduplication
- latest-row patterns
- gaps and islands
- streak analysis
- relational division awareness
- conditional pivoting
- pivot/unpivot awareness
- dynamic SQL awareness
- JSON in SQL
- full-text-search awareness
- temporal/history modeling
- effective-dated records
- audit tables
- snapshot versus event data
- ETL
- ELT
- staging layers
- incremental processing
- high-water marks
- compound watermarks
- late-arriving data
- lookback windows
- idempotency
- batch identifiers
- merge/upsert concepts
- change data capture awareness
- soft deletes in pipelines

- data-quality checks
- SQL assertions
- SQL testing
- reconciliation
- maintainable SQL style
- lineage and provenance
- feature pipeline patterns
- label leakage
- point-in-time features
- end-to-end SQL pipeline architecture

The central principle is:

Advanced SQL engineering is the design of correct, reproducible transformations over changing data—not merely writing complicated queries.

### Building an Integration Laboratory

We will create a dedicated SQLite database containing:

- users
- product events
- subscriptions
- historical customer tiers
- ingestion batches

The data is deliberately small enough to inspect manually while containing enough structure to demonstrate advanced analytical patterns.

These examples are educational. Production schemas and SQL syntax can differ substantially between SQLite, PostgreSQL, MySQL, warehouses, and distributed analytical systems.

## SQL

```

import sqlite3
import pandas as pd
from IPython.display import display

adv_conn = sqlite3.connect(":memory:")
adv_conn.execute("PRAGMA foreign_keys = ON")

adv_conn.executescript("""
CREATE TABLE app_users_adv (
    user_id INTEGER PRIMARY KEY,
    signup_date TEXT NOT NULL,
    region TEXT NOT NULL
);

CREATE TABLE product_events_adv (
    event_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    event_time TEXT NOT NULL,
    event_name TEXT NOT NULL,
    ingestion_time TEXT NOT NULL,
    FOREIGN KEY (user_id)
        REFERENCES app_users_adv(user_id)
);

CREATE TABLE subscriptions_adv (
    subscription_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    started_at TEXT NOT NULL,
    # ...
    ORDER BY user_id, event_time, event_id
""",
adv_conn
))

```

## Analytical SQL Starts with Metric Definition

Before writing a query for a metric such as retention, conversion, churn, or active users, define the metric precisely.

Consider:

What is a monthly active user?

Possible definitions include:

- any user producing any event
- any user logging in
- any user performing a meaningful product action

- any paying user
- any user with at least N actions
- any user excluding internal/test accounts

All of these can produce different SQL while each query is syntactically correct.

Therefore analytical SQL begins with semantics.

A useful workflow is:

Do not let the query silently define the metric after the fact.

#### PYTHON

```
business question
-> precise metric definition
-> grain
-> eligible population
-> time boundaries
-> SQL implementation
-> validation
```

## Cohort Analysis

A cohort groups entities according to a shared starting characteristic.

A common example is the month in which users signed up.

Users who signed up in January belong to one cohort; users who signed up in February belong to another.

Cohorts help answer questions such as:

- Do newer users retain better?
- Did a product change improve long-term engagement?
- Does one acquisition period behave differently?
- Do users from different onboarding versions convert differently?

The most important decision is defining the **cohort anchor**.

Possible anchors include:

Different anchors answer different questions.

## SQL

```
cohorts = pd.read_sql_query(
    """
    SELECT
        user_id,
        signup_date,
        strftime('%Y-%m', signup_date) AS signup_cohort
    FROM app_users_adv
    ORDER BY user_id
    """,
    adv_conn
)

display(cohorts)
```

## SQL

```
signup time
first purchase
first paid subscription
first model deployment
first successful API call
```

**Cohort Grain**

The cohort assignment should generally have one row per entity being analyzed.

For user retention:

If cohort assignment accidentally has several rows per user, joining it with activity events can multiply activity and corrupt counts.

This is a recurring advanced SQL principle:

Validate the grain of every intermediate relation before joining it.

## PYTHON

```
one row per user
```

**Activity Periods**

Retention analysis also needs an activity period.

For monthly retention we might convert event timestamps into:

However, a user may produce hundreds of events in one month.

Retention normally asks whether the user was active, not how many events they generated.

So an intermediate relation often has grain:

This prevents highly active users from being counted repeatedly.

SQL

```
user_month_activity = pd.read_sql_query(
    """
    SELECT DISTINCT
        user_id,
        strftime('%Y-%m', event_time) AS activity_month
    FROM product_events_adv
    WHERE event_name <> 'signup'
    ORDER BY user_id, activity_month
    """,
    adv_conn
)

display(user_month_activity)
```

SQL

```
2026-01
2026-02
2026-03
```

## Monthly Retention

A retention table usually compares:

with:

and computes how far activity occurs from the cohort start.

For a January cohort:

A common retention rate is:

The denominator must remain the original cohort population, not the number active in the previous period unless the metric explicitly defines something else.

SQL

```

retention = pd.read_sql_query(
    """
    WITH cohorts AS (
        SELECT
            user_id,
            date(
                signup_date,
                'start of month'
            ) AS cohort_month
        FROM app_users_adv
    ),
    activity AS (
        SELECT DISTINCT
            user_id,
            date(
                event_time,
                'start of month'
            ) AS activity_month
        FROM product_events_adv
        WHERE event_name <> 'signup'
    ),
    joined AS (
        SELECT
            c.user_id,
            c.cohort_month,
            a.activity_month,
            (
                # ...
                adv_conn
            )
    )

    display(retention)

```

SQL

cohort **period**

## Retention Interpretation

The query separates retention into explicit stages:

This is intentionally more verbose than forcing everything into one expression.

Well-named intermediate CTEs make analytical definitions easier to audit.

Notice that activity uses:

because the metric counts active users, not raw events.

Without this deduplication, a user with 100 events could incorrectly contribute 100 units to the numerator.



SQL

```
cohorts
activity
joined periods
cohort size
retained users
final rate
```

SQL

```
SELECT DISTINCT user_id, activity_month
```

## Retention Has Many Definitions

"Retention" is not one universal metric.

Possible definitions include:

- returned on exactly day 7
- returned during days 7–13
- active in calendar month 1
- active in every period through month N
- ever returned after signup
- still subscribed
- still paying
- performed a specific meaningful action

These metrics answer different questions.

Always document:

- anchor event
- activity event
- time window
- denominator
- timezone
- eligibility filters

A retention percentage without its definition is incomplete.

## Funnel Analysis

A funnel measures progression through a sequence of actions.

For an e-commerce product:

For an AI developer platform:

A basic funnel might ask:

- how many users viewed?
- how many added to cart?
- how many purchased?

But a serious funnel definition must answer:

- must events happen in order?
- can they happen on different days?
- how long is the allowed conversion window?
- does any purchase count?
- can users repeat steps?
- what is the population?

SQL

```
basic_funnel = pd.read_sql_query(
    """
    SELECT
        COUNT(DISTINCT CASE
            WHEN event_name = 'view_product'
            THEN user_id
            END) AS viewed_users,

        COUNT(DISTINCT CASE
            WHEN event_name = 'add_to_cart'
            THEN user_id
            END) AS cart_users,

        COUNT(DISTINCT CASE
            WHEN event_name = 'purchase'
            THEN user_id
            END) AS purchaser_users
    FROM product_events_adv
    """
    ,
    adv_conn
)

display(basic_funnel)
```

SQL

```
view product
-> add to cart
-> purchase
```

## Basic Funnel Versus Ordered Funnel

The previous query counts whether users ever performed each event.

It does not prove that:

A user could theoretically purchase first and view later and still appear in both counts.

If event order matters, SQL must encode order explicitly.

This is an important distinction between:

and:

PYTHON

```
view  
happened before  
cart  
happened before  
purchase
```

PYTHON

```
event membership
```

### First Event Time Pattern

One useful pattern is to calculate each user's first occurrence of each funnel step.

Conditional aggregation can produce:

Then the query can compare timestamps.

This works well when the funnel definition specifically concerns first occurrences.

More complicated funnels may require sequence logic, session boundaries, or repeated-attempt modeling.

SQL

```

ordered_funnel = pd.read_sql_query(
    """
    WITH first_steps AS (
        SELECT
            user_id,

            MIN(CASE
                WHEN event_name = 'view_product'
                THEN event_time
            END) AS first_view,

            MIN(CASE
                WHEN event_name = 'add_to_cart'
                THEN event_time
            END) AS first_cart,

            MIN(CASE
                WHEN event_name = 'purchase'
                THEN event_time
            END) AS first_purchase

        FROM product_events_adv
        GROUP BY user_id
    )
    SELECT
        user_id,
        first_view,
        first_cart,
        # ...
        adv_conn
    )

display(ordered_funnel)

```

SQL

```

first_view_time
first_cart_time
first_purchase_time

```

## Funnel Logic Can Become Subtle

The previous query is educational, but a production funnel needs a precisely defined sequence.

Questions include:

- Is `add_to_cart` mandatory before purchase?
- If a user views three products, which view belongs to the purchase?
- Can multiple purchase attempts exist?
- Does a new session reset the funnel?

- Must conversion happen within 30 minutes?
- What if timestamps tie?
- What if events arrive late?

For event analytics, the business definition should be written before implementing increasingly complex SQL.

## Conversion Rates

Suppose a funnel has:

Possible metrics include:

Always specify the denominator.

A dashboard showing:

without saying which stage is the denominator is ambiguous.

PYTHON

```
1000 viewers
500 cart users
200 purchasers
```

PYTHON

```
view -> cart conversion = 500 / 1000

cart -> purchase conversion = 200 / 500

overall conversion = 200 / 1000
```

## Deduplication

Real data often contains duplicates.

But "duplicate" must be defined.

Possible meanings include:

- identical rows
- same external event ID
- repeated business transaction
- multiple versions of one record
- same entity loaded by multiple batches
- retry-generated copies

Deleting rows because several columns happen to match can destroy legitimate data.

Deduplication starts by identifying the **business key** and the rule for choosing the surviving record.

## SQL

```

adv_conn.executescript("""
CREATE TABLE raw_customer_updates_adv (
    row_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    customer_name TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    ingestion_time TEXT NOT NULL
);

INSERT INTO raw_customer_updates_adv
VALUES
    (1, 100, 'Anika', '2026-01-01 09:00:00',
     '2026-01-01 09:01:00'),

    (2, 100, 'Anika Singh', '2026-01-05 12:00:00',
     '2026-01-05 12:01:00'),

    (3, 101, 'Dev', '2026-01-03 10:00:00',
     '2026-01-03 10:02:00'),

    (4, 101, 'Dev Kumar', '2026-01-03 10:00:00',
     '2026-01-03 10:03:00'),

    (5, 102, 'Sara', '2026-01-04 08:00:00',
     '2026-01-04 08:01:00');
""")

adv_conn.commit()
# ...
ORDER BY customer_id, updated_at, ingestion_time
""",
adv_conn
))

```

## Latest Row per Business Key

A classic deduplication pattern uses:

Then retain:

For our example:

is the business key.

`updated_at` indicates source recency.

But two rows can share the same `updated_at`, so we also need a deterministic tiebreaker.

We will use:

as additional ordering.

## SQL

```
latest_customers = pd.read_sql_query(
    """
    WITH ranked AS (
        SELECT
            *,
            ROW_NUMBER() OVER (
                PARTITION BY customer_id
                ORDER BY
                    updated_at DESC,
                    ingestion_time DESC,
                    row_id DESC
            ) AS rn
        FROM raw_customer_updates_adv
    )
    SELECT
        row_id,
        customer_id,
        customer_name,
        updated_at,
        ingestion_time
    FROM ranked
    WHERE rn = 1
    ORDER BY customer_id
    """,
    adv_conn
)

display(latest_customers)
```

## SQL

```
ROW_NUMBER()
OVER (
    PARTITION BY business_key
    ORDER BY recency_column DESC
)
```

## Deduplication Must Be Deterministic

Suppose two versions have the same:

If the window ordering contains only:

the database is not given a complete rule for choosing between tied records.

Results may be unstable.

A production deduplication rule should ideally define a deterministic total preference using trusted columns such as:

- source version

- event sequence
- ingestion timestamp
- batch ID
- unique row ID

The chosen rule should reflect source semantics rather than arbitrary convenience.

```
PYTHON
```

```
customer_id  
updated_at
```

```
PYTHON
```

```
ORDER BY updated_at DESC
```

## Exact Duplicates Versus Versioned Records

These are different problems.

Exact duplicates:

may require deduplication using an event ID or content fingerprint.

Versioned records:

require a "latest valid version" rule.

Do not use `SELECT DISTINCT` as a universal deduplication solution.

`DISTINCT` only removes rows identical across selected expressions. It does not understand business identity or recency.

```
PYTHON
```

```
same logical event accidentally delivered twice
```

```
PYTHON
```

```
same customer represented by several legitimate updates
```

## Gaps and Islands

"Gaps and islands" describes a family of SQL problems involving consecutive sequences.

Examples:

- consecutive active days
- consecutive machine-failure periods
- continuous subscription periods



- runs of missing observations
- consecutive IDs
- streaks of model-serving errors

An **island** is a consecutive run.

A **gap** separates islands.

Window functions are often central to solving these problems.

SQL

```
adv_conn.executescript("""
CREATE TABLE daily_activity_adv (
    user_id INTEGER NOT NULL,
    activity_date TEXT NOT NULL,
    PRIMARY KEY (user_id, activity_date)
);

INSERT INTO daily_activity_adv
VALUES
    (1, '2026-03-01'),
    (1, '2026-03-02'),
    (1, '2026-03-03'),
    (1, '2026-03-06'),
    (1, '2026-03-07'),
    (1, '2026-03-10'),

    (2, '2026-03-01'),
    (2, '2026-03-04'),
    (2, '2026-03-05');
""")

adv_conn.commit()

display(pd.read_sql_query(
    """
    SELECT *
    FROM daily_activity_adv
    ORDER BY user_id, activity_date
    """,
    adv_conn
))
```

## Gaps with LAG

The easiest first step is often to compare each date with the previous date.

For each user:

Then calculate the difference.

If the difference is greater than one day, a new island starts.

SQL

```

gaps = pd.read_sql_query(
    """
    SELECT
        user_id,
        activity_date,

        LAG(activity_date) OVER (
            PARTITION BY user_id
            ORDER BY activity_date
        ) AS previous_date,

        julianday(activity_date)
        -
        julianday(
            LAG(activity_date) OVER (
                PARTITION BY user_id
                ORDER BY activity_date
            )
        ) AS gap_days

    FROM daily_activity_adv
    ORDER BY user_id, activity_date
    """,
    adv_conn
)

display(gaps)

```

SQL

```

LAG(activity_date)
OVER (
    PARTITION BY user_id
    ORDER BY activity_date
)

```

## Building Islands

Once we identify where a new sequence begins, we can create an island identifier.

A common approach is:

1. flag the beginning of each new island
2. cumulatively sum those flags
3. group by the resulting island number

This demonstrates an important window-function composition pattern:

SQL

```

islands = pd.read_sql_query(
    """
    WITH previous_dates AS (
        SELECT
            user_id,
            activity_date,

            LAG(activity_date) OVER (
                PARTITION BY user_id
                ORDER BY activity_date
            ) AS previous_date

        FROM daily_activity_adv
    ),
    boundaries AS (
        SELECT
            user_id,
            activity_date,

            CASE
                WHEN previous_date IS NULL
                OR julianday(activity_date)
                    - julianday(previous_date) > 1
                THEN 1
                ELSE 0
            END AS starts_new_island

        FROM previous_dates
    )
    # ...
    adv_conn
)

display(islands)

```

SQL

```

LAG
-> boundary flag
-> cumulative SUM
-> GROUP BY

```

## Streak Analysis

Once islands are identified, streak questions become ordinary grouped analysis.

For example:

can be computed by:

The same architecture applies to:

- consecutive failures

- uptime intervals
- repeated daily purchases
- data-quality incident streaks
- model drift alert periods

SQL

```
longest_streak = pd.read_sql_query(
    """
    WITH previous_dates AS (
        SELECT
            user_id,
            activity_date,
            LAG(activity_date) OVER (
                PARTITION BY user_id
                ORDER BY activity_date
            ) AS previous_date
        FROM daily_activity_adv
    ),
    boundaries AS (
        SELECT
            user_id,
            activity_date,
            CASE
                WHEN previous_date IS NULL
                OR julianday(activity_date)
                    - julianday(previous_date) > 1
                THEN 1
                ELSE 0
            END AS start_flag
        FROM previous_dates
    ),
    numbered AS (
        SELECT
            user_id,
            # ...
            adv_conn
    )
    display(longest_streak)
```

SQL

longest consecutive active streak per **user**

## Relational Division Awareness

Some SQL questions ask:

Which entities are related to all required values?

Examples:

- students who completed every required course
- suppliers who provide every required component
- models evaluated on every mandatory benchmark
- users who possess every required permission

This family of problems is sometimes described as **relational division**.

SQL does not normally use a literal `DIVIDE` keyword. Instead, common patterns use:

- double `NOT EXISTS`
- grouped counts
- set comparisons

### Double NOT EXISTS Pattern

Suppose we want models evaluated on every required benchmark.

Conceptually:

This sounds awkward in English, but logically it means:

The double-`NOT EXISTS` pattern is therefore powerful for "all requirements satisfied" queries.

SQL

```

adv_conn.executescript("""
CREATE TABLE required_checks_adv (
    check_name TEXT PRIMARY KEY
);

CREATE TABLE model_checks_adv (
    model_name TEXT NOT NULL,
    check_name TEXT NOT NULL,
    passed INTEGER NOT NULL,
    PRIMARY KEY (model_name, check_name)
);

INSERT INTO required_checks_adv
VALUES
    ('accuracy'),
    ('latency'),
    ('security');

INSERT INTO model_checks_adv
VALUES
    ('model_a', 'accuracy', 1),
    ('model_a', 'latency', 1),
    ('model_a', 'security', 1),

    ('model_b', 'accuracy', 1),
    ('model_b', 'latency', 1),

    ('model_c', 'accuracy', 1),
    # ...
adv_conn
)

display(qualified_models)

```

SQL

```

find models for which
there does NOT exist
a required benchmark for which
there does NOT exist
a matching evaluation

```

## Pivoting with Conditional Aggregation

Relational data is usually stored in rows, but reports sometimes need categories transformed into columns.

Suppose event counts are stored as:

A report may want:

A portable SQL pattern uses conditional aggregation:

This is effectively a manual pivot.

SQL

```
event_pivot = pd.read_sql_query(
    """
    SELECT
        user_id,

        SUM(CASE
            WHEN event_name = 'view_product'
            THEN 1 ELSE 0
        END) AS views,

        SUM(CASE
            WHEN event_name = 'add_to_cart'
            THEN 1 ELSE 0
        END) AS cart_events,

        SUM(CASE
            WHEN event_name = 'purchase'
            THEN 1 ELSE 0
        END) AS purchases,

        SUM(CASE
            WHEN event_name = 'login'
            THEN 1 ELSE 0
        END) AS logins

    FROM product_events_adv
    GROUP BY user_id
    ORDER BY user_id
    """,
    adv_conn
)

display(event_pivot)
```

SQL

```
user_id | event_name | count
```

## Native PIVOT Features

Some database systems provide dedicated pivot syntax.

Others rely primarily on conditional aggregation or database-specific extensions.

Native pivot syntax is not fully portable across SQL engines.

Conditional aggregation remains important because it is conceptually simple and works across many relational systems.

However, manually writing hundreds of category columns is usually a sign that the reporting shape should be reconsidered or generated carefully.

## Unpivot Awareness

Unpivoting transforms columns into rows.

Suppose a wide table contains:

A long representation could be:

Long-form metric data often becomes easier to:

- filter
- aggregate
- visualize
- extend with new metric names

Some databases provide `UNPIVOT`-style syntax; otherwise `UNION ALL` or engine-specific techniques can produce long form.

PYTHON

```
model_id
accuracy
precision
recall
f1
```

PYTHON

```
model_id
metric_name
metric_value
```

## Static Versus Dynamic Pivoting

Conditional aggregation assumes known categories:

But suppose categories are discovered dynamically.

Then the SQL column list itself must change.

Value parameters cannot normally replace SQL identifiers, so dynamic pivoting may require dynamic SQL generation.

Dynamic SQL should be treated carefully because it introduces:

- injection risks
- quoting complexity
- changing output schema



- difficult testing
- database-specific syntax

Often it is simpler to return long-form data and pivot in a reporting/analysis layer.

PYTHON

```
views
purchases
logins
```

## Dynamic SQL

Dynamic SQL means constructing SQL structure programmatically.

Sometimes it is genuinely required—for example, generating a report over an allowed set of columns.

But this is fundamentally different from parameterizing values.

Safe:

This cannot generally parameterize:

When SQL structure must be dynamic, use strict allowlists and trusted query-building techniques.

SQL

```

allowed_sort_columns = {
    "event_time",
    "event_name",
    "user_id",
}

requested_sort = "event_time"

if requested_sort not in allowed_sort_columns:
    raise ValueError("Invalid sort column.")

# Safe here because the structural identifier came
# from a strict application-controlled allowlist.
sql = f"""
SELECT
    event_id,
    user_id,
    event_name,
    event_time
FROM product_events_adv
ORDER BY {requested_sort}
LIMIT ?
"""

rows = adv_conn.execute(sql, (5,)).fetchall()

for row in rows:
    print(row)

```

SQL

```

cursor.execute(
    "SELECT * FROM users WHERE user_id = ?",
    (user_id,)
)

```

## JSON in Relational Databases

Modern relational databases frequently support JSON.

JSON can be valuable when input contains flexible or semi-structured attributes.

Examples include:

- external API payloads
- model inference metadata
- experiment configuration
- optional event properties
- evolving integration fields

But JSON should not automatically replace relational schema.

If an attribute is central to:

- joins
- uniqueness
- foreign keys
- filtering
- grouping
- integrity constraints

a typed relational column is often easier to manage.

SQL

```
json_supported = False

try:
    value = adv_conn.execute(
        """
        SELECT json_extract(
            '{"model": "fraud_v2", "score": 0.87}',
            '$.model'
        )
        """
    ).fetchone()[0]

    json_supported = True
    print("JSON extraction result:", value)

except sqlite3.OperationalError as exc:
    print(
        "JSON function unavailable in this SQLite build:",
        exc
    )
```

## Querying JSON

Where supported, SQL JSON functions can extract nested values.

Conceptually:

may access a model identifier in a JSON document.

Production databases differ significantly in:

- JSON data types
- operators
- indexing
- path syntax

- validation
- performance

For frequently queried fields, consider whether they deserve first-class typed columns rather than repeated runtime JSON extraction.

PYTHON

```
json_extract(payload, '$.model')
```

## JSON and AI Event Payloads

An inference event might have stable fields:

plus flexible metadata:

A hybrid schema can be reasonable:

This preserves relational integrity for critical fields while allowing controlled flexibility where the shape legitimately changes.

PYTHON

```
request_id  
model_version_id  
timestamp  
latency_ms  
status
```

PYTHON

```
device  
client version  
experiment flags  
optional diagnostics
```

## Full-Text Search

A normal B-tree index is not designed to solve general document search such as:

Full-text search systems tokenize textual content and maintain specialized inverted indexes.

Capabilities may include:

- token search
- phrase search
- ranking
- stemming
- language-aware processing
- prefix matching

SQLite provides FTS extensions such as FTS5 when available. PostgreSQL, MySQL, and specialized search systems provide their own mechanisms.

PYTHON

```
find documents containing concepts related to these words
```

### LIKE Is Not a Full-Text Search Engine

A query such as:

can be useful for small data but is not a substitute for a full-text index at scale.

Leading-wildcard substring searches often cannot efficiently navigate a normal B-tree.

Full-text search uses a different data structure designed around terms and documents.

This reinforces a general database principle:

Index structures are designed for particular query geometries.

## SQL

```
fts_available = False

try:
    adv_conn.execute("""
    CREATE VIRTUAL TABLE docs_fts_adv
    USING fts5(title, body)
    """)

    adv_conn.executemany(
        """
        INSERT INTO docs_fts_adv (title, body)
        VALUES (?, ?)
        """,
        [
            (
                "SQL Transactions",
                "Transactions provide atomic database updates."
            ),
            (
                "Model Serving",
                "Model serving exposes machine learning inference APIs."
            ),
            (
                "SQL Indexes",
                "Indexes improve selected relational lookup patterns."
            ),
        ],
    )
    # ...
    print(
        "FTS5 is unavailable in this SQLite build:",
        exc
    )
```

## SQL

```
WHERE document_text LIKE '%transformer%'
```

## Full-Text Search Versus Vector Search

Full-text search and vector similarity search solve different problems.

Full-text search commonly uses token/term matching.

Vector search compares embedding representations to retrieve semantically similar items.

Modern retrieval systems may combine:

This foundation course only introduces the distinction. Advanced vector databases, RAG, and advanced retrieval engineering belong to later LLM-focused study.

## PYTHON

```
keyword search  
+  
vector search  
+  
metadata filtering
```

## Temporal Data

Many databases need to answer not only:

What is true now?

but:

What was true at time T?

Examples include:

- customer's region at purchase time
- employee's department on a historical date
- model deployed when a prediction occurred
- feature value available at prediction time
- account status before a transaction

Temporal requirements affect schema design, query logic, indexes, and correctness.

### Half-Open Validity Intervals

A particularly useful temporal convention is:

meaning:

The start is included and the end is excluded.

For open-ended current rows:

This convention prevents two adjacent versions from both claiming the exact boundary timestamp.

SQL

```
tier_at_event = pd.read_sql_query(
    """
    SELECT
        e.event_id,
        e.user_id,
        e.event_time,
        e.event_name,
        h.tier
    FROM product_events_adv AS e
    JOIN customer_tier_history_adv AS h
        ON h.user_id = e.user_id
        AND date(e.event_time) >= h.valid_from
        AND (
            h.valid_to IS NULL
            OR date(e.event_time) < h.valid_to
        )
    WHERE e.user_id IN (1, 5)
    ORDER BY e.user_id, e.event_time
    """,
    adv_conn
)

display(tier_at_event)
```

SQL

```
[valid_from, valid_to)
```

## Temporal Join Correctness

A naïve join:

may attach today's customer state to historical events.

That can rewrite history accidentally.

For some analyses, current state is exactly what we want.

For historical attribution, we need state valid at the event time.

This distinction is critical in ML feature engineering because joining future state to past labels can introduce temporal leakage.

PYTHON

```
events
JOIN current_customer
```

## Overlapping Validity Intervals

Effective-dated tables become dangerous if one entity has overlapping versions.



For example:

What tier applies on February 15?

Both rows match.

A robust temporal model needs rules preventing or resolving overlaps.

Some databases provide advanced range types or exclusion constraints; otherwise application/database validation logic may be required.

#### PYTHON

```
standard: Jan 1 -> Mar 1  
gold:     Feb 1 -> Apr 1
```

## Audit History

An audit history records changes for accountability or reconstruction.

A useful audit record may include:

- entity ID
- changed timestamp
- actor/service
- operation
- old value
- new value
- request/correlation ID

Audit history and temporal business history are related but not necessarily identical.

An audit log answers:

Who changed what?

A temporal business table may answer:

Which state was valid at time T?

One structure does not automatically satisfy every use case.

## Snapshot Data Versus Event Data

A snapshot captures state at a point in time:

Event data records changes or occurrences:

Events preserve the sequence of activity.

Snapshots can make state reconstruction faster.

Many systems use both:

The distinction matters when designing analytical pipelines.

#### PYTHON

```
customer balance at midnight
inventory at end of day
daily model metrics
```

#### PYTHON

```
payment received
order created
model deployed
prediction generated
```

## ETL

ETL stands for:

Conceptually:

1. extract data from source
2. transform it outside the destination system
3. load the transformed result

Traditional ETL pipelines often transformed data before loading it into a warehouse.

The exact architecture depends on platform and era; ETL is a data-flow pattern, not one specific product.

#### PYTHON

```
Extract
Transform
Load
```

## ELT

ELT stands for:

Data is first loaded into the destination analytical platform, then transformed using the platform's compute engine—often with SQL.

Modern cloud warehouses made ELT especially common because they can process large datasets directly.

Neither ETL nor ELT is universally superior.

The choice depends on:

- source systems
- compute architecture
- data volume
- governance
- transformation tooling
- latency
- security

#### PYTHON

```
Extract
Load
Transform
```

## Staging Layers

A common data pipeline architecture separates raw ingestion from cleaned and modeled data.

Conceptually:

A staging area helps preserve source fidelity while allowing downstream transformations to be rerun.

The exact number and naming of layers varies across organizations.

#### PYTHON

```
source
|
v
raw / staging
|
v
cleaned / standardized
|
v
modeled
|
v
analytics / features
```

## Why Preserve Raw Input?

Suppose a transformation bug is discovered two months later.

If only transformed output was kept and the raw source was discarded, rebuilding correctly may be impossible.

Preserving appropriate raw or immutable source snapshots can support:

- reprocessing

- debugging
- lineage
- audits
- schema evolution
- reproducibility

Retention, privacy, and storage cost still apply. "Keep everything forever" is not automatically a sound policy.

## Full Refresh

The simplest pipeline can rebuild its entire destination.

Conceptually:

Advantages:

- easy reasoning
- naturally captures source corrections
- often idempotent

Disadvantages:

- expensive for large data
- long runtime
- large compute usage
- potentially disruptive publication

Full refresh is excellent when datasets are small enough. Incremental complexity should be introduced only when necessary.

SQL

```
DELETE/TRUNCATE destination  
-> recompute everything  
-> reload
```

## Incremental Processing

An incremental pipeline processes only data believed to be new or changed since the previous successful run.

For example:

This can drastically reduce work.

But incremental pipelines are harder to make correct.

Questions include:

- What if two rows share the same timestamp?

- What if data arrives late?
- What if a source row is corrected?
- What if a row is deleted?
- What if a batch partially fails?
- What if the pipeline reruns?
- What if clocks differ?

**PYTHON**

```
WHERE updated_at > last_processed_time
```

## High-Water Marks

A high-water mark records how far a pipeline has safely processed.

Example:

The next run processes:

After successful publication, the watermark advances.

The watermark itself is operational state and should usually advance only after the corresponding batch has completed successfully.

## SQL

```

adv_conn.executescript("""
CREATE TABLE source_events_incremental_adv (
    event_id INTEGER PRIMARY KEY,
    payload TEXT NOT NULL
);

CREATE TABLE processed_events_adv (
    event_id INTEGER PRIMARY KEY,
    payload TEXT NOT NULL
);

CREATE TABLE pipeline_state_adv (
    pipeline_name TEXT PRIMARY KEY,
    last_event_id INTEGER NOT NULL
);

INSERT INTO source_events_incremental_adv
VALUES
    (1, 'alpha'),
    (2, 'beta'),
    (3, 'gamma'),
    (4, 'delta'),
    (5, 'epsilon');

INSERT INTO pipeline_state_adv
VALUES
    ('event_copy', 0);
""")
# ...
display(pd.read_sql_query(
    "SELECT * FROM pipeline_state_adv",
    adv_conn
))

```

## SQL

```
last_processed_event_id = 987654
```

## Watermark Advancement Must Be Atomic with Processing State

Imagine:

1. watermark advances to 1000
2. destination write fails after row 850
3. pipeline restarts
4. it begins after 1000

Rows 851–1000 may never be processed.

The state change and the associated database writes should therefore be coordinated safely when they share the same transactional system.

If the destination is external—such as object storage or another warehouse—coordination becomes a distributed-systems problem.

## Timestamp Watermarks

A common incremental predicate is:

But timestamps can be dangerous if multiple rows share the same timestamp.

Suppose the final processed row has:

and another unprocessed row also has exactly `12:00:00`.

Using:

may skip that second row.

A robust design may need a compound cursor.

```
PYTHON
```

```
WHERE updated_at > ?
```

```
PYTHON
```

```
updated_at = 12:00:00
```

## Compound Watermarks

A compound high-water mark might use:

with deterministic ordering:

The next batch can use a keyset-style predicate:

This is conceptually the same principle used in stable keyset pagination.

A deterministic total order makes incremental progress safer.

```
PYTHON
```

```
(updated_at, row_id)
```

```
PYTHON
```

```
ORDER BY updated_at, row_id
```

## Event Time Versus Ingestion Time

Two timestamps commonly exist in data pipelines:

## Event time

When the event actually occurred.

## Ingestion time

When the pipeline received or stored it.

Example:

Using only event time as an incremental cursor can be dangerous when late events arrive with timestamps older than the current watermark.

PYTHON

```
event occurred:
```

```
10:00
```

```
network delay
```

```
event ingested:
```

```
10:15
```

## Late-Arriving Data

Suppose a pipeline has already processed event time through:

At 12:10, an event arrives whose actual event time is:

If the next query is:

the late event may never be processed.

Possible strategies include:

- ingestion-time cursors
- overlapping lookback windows
- CDC streams
- source sequence IDs
- periodic reconciliation
- idempotent reprocessing

PYTHON

```
12:00
```

PYTHON

```
11:45
```



## Lookback Windows

A common incremental strategy deliberately reprocesses recent history.

Instead of:

use:

This captures some late-arriving changes.

Because rows are intentionally reprocessed, the destination must support idempotent merge/upsert behavior.

The lookback size is a trade-off:

PYTHON

```
process strictly after last timestamp
```

PYTHON

```
reprocess previous N hours/days
```

## Idempotent Pipelines

A pipeline is idempotent when rerunning the same logical batch does not incorrectly duplicate its effects.

Suppose batch 42 is retried.

Bad result:

Desired result:

Techniques include:

- natural/business unique keys
- UPSERT
- batch IDs
- replace-partition strategies
- atomic publication
- deterministic transformations

PYTHON

```
rows inserted twice  
metrics doubled
```

PYTHON

```
same final database state
```

## Batch Identifiers

Assigning each ingestion run a batch identifier can improve traceability.

Example:

Rows or metadata can then record:

- source
- started time
- completed time
- row count
- success/failure
- checksum
- watermark
- pipeline version

Batch metadata helps answer:

Which pipeline execution produced this row or dataset?

## SQL

```

adv_conn.executescript("""
CREATE TABLE pipeline_batches_adv (
    batch_id TEXT PRIMARY KEY,
    pipeline_name TEXT NOT NULL,
    status TEXT NOT NULL
    CHECK (
        status IN (
            'running',
            'completed',
            'failed'
        )
    ),
    started_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    completed_at TEXT,
    rows_processed INTEGER
);
""")

adv_conn.execute(
    """
    INSERT INTO pipeline_batches_adv (
        batch_id,
        pipeline_name,
        status
    )
    VALUES (?, ?, ?)
    """,
    (
        # ...
display(pd.read_sql_query(
    "SELECT * FROM pipeline_batches_adv",
    adv_conn
))

```

## SQL

```
batch_id = ingest_2026_10_04_001
```

## UPSERT in Data Pipelines

Incremental loads often need:

UPSERT can support this pattern.

However, the key question is:

Which columns define record identity?

If the wrong conflict key is chosen, unrelated records can overwrite each other.

A robust pipeline defines:

- business key
- source version
- update precedence
- immutable fields
- mutable fields

before writing the UPSERT.

SQL

```
insert unseen record
update changed record
```

## MERGE Awareness

Many relational and analytical database systems support some form of **MERGE** operation.

Conceptually, it compares source and target and can perform actions such as:

Some systems can also process deletion-like semantics.

**MERGE** syntax and historical correctness guarantees differ by database product and version.

Do not copy a **MERGE** statement between platforms without reading platform-specific documentation.

PYTHON

```
WHEN MATCHED
    -> UPDATE

WHEN NOT MATCHED
    -> INSERT
```

## Change Data Capture

Change Data Capture, or CDC, captures changes occurring in a source database.

Instead of repeatedly asking:

CDC may expose a change stream derived from mechanisms such as:

- database transaction logs
- replication logs
- triggers
- change tables
- managed database streams

Events may describe:

CDC is an important bridge between operational databases and downstream analytical systems.

SQL

```
which rows changed since yesterday?
```

SQL

```
insert
update
delete
```

### CDC Is Not Just `updated_at` Filtering

An `updated_at` column can support incremental extraction, but it has limitations:

- deletions may disappear entirely
- clients may forget to update the timestamp
- multiple changes can collapse into one final state
- timestamp collisions can occur
- late transactions complicate ordering

Log-based CDC can capture a richer ordered history of database changes.

However, it introduces operational infrastructure and delivery semantics that must be understood.

### CDC Delivery Semantics

Change streams may provide delivery models such as:

- at-most-once
- at-least-once
- stronger system-specific guarantees

At-least-once delivery means duplicates are possible.

Consumers therefore often need:

- stable change identifiers
- ordering metadata
- idempotent processing
- checkpointing

Never assume receiving a change event once means the distributed pipeline cannot replay it.

### Deletes in Incremental Pipelines

Suppose source row 10 existed yesterday and is deleted today.

A pipeline that only queries:

may never learn that row 10 disappeared.

Possible designs include:

- CDC delete events
- soft-delete markers
- snapshot comparison
- tombstone rows
- periodic full reconciliation

Deletion propagation must be designed explicitly.

PYTHON

```
WHERE updated_at > last_run
```

## Tombstones

A tombstone is a record indicating that an entity or key was deleted.

Instead of silently disappearing, a change stream might produce:

Downstream systems can then remove or deactivate their corresponding representation.

Tombstones are particularly important in replicated and eventually consistent systems.

PYTHON

```
key = customer_100  
operation = delete
```

## Data Quality

A successful SQL query does not imply valid data.

A pipeline can run without exceptions and still produce:

- duplicate business keys
- missing IDs
- impossible dates
- negative prices
- orphan foreign keys
- unexpected categories
- row-count collapses
- future leakage

Data quality should therefore be tested explicitly.

## Data Quality Dimensions

Common quality dimensions include:

### Completeness

Are required values present?

### Uniqueness

Are identifiers unique where required?

### Validity

Do values satisfy domain rules?

### Referential integrity

Do references point to known entities?

### Consistency

Do related values agree?

### Timeliness

Is the data sufficiently fresh?

### Volume

Did expected data arrive?

Different datasets require different quality contracts.

## SQL Quality Check: Missing Values

A simple completeness check:

The desired count may be zero.

But quality expectations should be field-specific.

`NULL` may be valid for an optional field while unacceptable for a primary business attribute.

SQL

```
quality_missing = adv_conn.execute(
    """
    SELECT COUNT(*)
    FROM app_users_adv
    WHERE signup_date IS NULL
       OR region IS NULL
    """
).fetchone()[0]

print("Rows missing required values:", quality_missing)
```

SQL

```
SELECT COUNT(*)
FROM table
WHERE required_column IS NULL;
```

## SQL Quality Check: Duplicate Business Keys

Suppose `event_id` should be unique.

A general duplicate check is:

Even when a database constraint already enforces uniqueness, quality checks are valuable in staging/raw systems where source data may not yet be constrained.

SQL

```
duplicate_users = pd.read_sql_query(
    """
    SELECT
        user_id,
        COUNT(*) AS row_count
    FROM app_users_adv
    GROUP BY user_id
    HAVING COUNT(*) > 1
    """,
    adv_conn
)

display(duplicate_users)
```

SQL

```
SELECT business_key, COUNT(*)
FROM table
GROUP BY business_key
HAVING COUNT(*) > 1;
```

## SQL Quality Check: Orphans

An orphan is a child reference with no matching parent.



A common check is:

Production foreign keys can prevent many orphans, but raw ingestion areas may intentionally load unvalidated data before validation.

SQL

```
orphans = pd.read_sql_query(
    """
    SELECT e.*
    FROM product_events_adv AS e
    LEFT JOIN app_users_adv AS u
        ON u.user_id = e.user_id
    WHERE u.user_id IS NULL
    """,
    adv_conn
)

display(orphans)
```

SQL

```
SELECT child.*
FROM child
LEFT JOIN parent
    ON ...
WHERE parent.key IS NULL;
```

## SQL Quality Check: Accepted Values

Suppose event names are expected to belong to a controlled set.

A quality query can identify unknown categories:

In a normalized operational system, a foreign key or **CHECK** constraint may be stronger.

In an analytical pipeline, discovering a new category can be important because it may indicate:

- legitimate schema evolution
- source bug
- upstream typo
- undocumented product event

SQL

```
unexpected_events = pd.read_sql_query(
    """
    SELECT
        event_name,
        COUNT(*) AS row_count
    FROM product_events_adv
    WHERE event_name NOT IN (
        'signup',
        'view_product',
        'add_to_cart',
        'purchase',
        'login'
    )
    GROUP BY event_name
    """,
    adv_conn
)

display(unexpected_events)
```

SQL

```
WHERE event_name NOT IN (...)
```

## Volume Checks

A pipeline can technically succeed while receiving almost no data.

Suppose yesterday produced:

and today produces:

That may be possible, but it deserves investigation.

Volume checks can compare:

- row count
- distinct entities
- partition size
- event count by source
- historical baseline

Static thresholds are useful for some datasets; dynamic anomaly detection may be appropriate for others.

PYTHON

```
10,000,000 events
```

## PYTHON

```
13 events
```

## Freshness Checks

A dataset can be structurally valid but stale.

A simple freshness check might inspect:

Then compare it with expected recency.

For example:

This indicates pipeline or source failure even if all existing rows satisfy their constraints.

## SQL

```
latest_event = adv_conn.execute(  
    """  
    SELECT MAX(event_time)  
    FROM product_events_adv  
    """  
).fetchone()[0]  
  
print("Latest event in example data:", latest_event)
```

## SQL

```
MAX(event_time)
```

## Reconciliation

A migration or pipeline may be validated by comparing source and destination.

Possible reconciliation checks include:

- row counts
- distinct key counts
- sums
- minimum/maximum dates
- checksum-like aggregates
- unmatched keys
- category distributions

For example:

A matching total does not prove every row is correct, but reconciliation provides strong additional evidence when combined with other tests.

## PYTHON

```
source total payment amount
versus
destination total payment amount
```

## SQL Assertions

A quality check becomes more operationally useful when failure is explicit.

In Python:

In dedicated data-quality frameworks, assertions can be declared as tests.

The important idea is:

Bad data should fail loudly before it silently contaminates downstream models and reports.

## SQL

```
def assert_zero(conn, sql, message):
    count = conn.execute(sql).fetchone()[0]

    if count != 0:
        raise AssertionError(
            f"{message}. Failing row count: {count}"
        )

    return count

assert_zero(
    adv_conn,
    """
    SELECT COUNT(*)
    FROM product_events_adv
    WHERE user_id IS NULL
       OR event_time IS NULL
       OR event_name IS NULL
    """,
    "Required event fields contain NULL values"
)

print("Required-field assertion passed.")
```

## SQL

```
count = ...
if count != 0:
    raise RuntimeError(...)
```

## Testing SQL

SQL should be tested like application code.

Useful test categories include:

- schema tests
- constraint tests
- transformation tests
- edge-case tests
- regression tests
- integration tests
- data-quality tests
- performance tests

A complex analytical query deserves known input data and expected outputs.

Do not rely only on visually inspecting a few rows in production.

### Unit Testing a SQL Transformation

A useful SQL transformation test can:

1. create a small isolated database
2. insert carefully chosen rows
3. run the transformation
4. compare results with expected values

Test data should include edge cases such as:

- `NULL`
- duplicate timestamps
- missing related rows
- zero values
- boundary dates
- tied rankings
- users with no activity

## SQL

```

test_conn = sqlite3.connect(":memory:")

test_conn.executescript("""
CREATE TABLE values_test (
    group_id INTEGER NOT NULL,
    amount REAL
);

INSERT INTO values_test
VALUES
    (1, 10),
    (1, 20),
    (1, NULL),
    (2, NULL);
""")

actual = test_conn.execute("""
SELECT
    group_id,
    COUNT(*) AS rows_total,
    COUNT(amount) AS non_null_amounts,
    SUM(amount) AS amount_sum
FROM values_test
GROUP BY group_id
ORDER BY group_id
""").fetchall()

expected = [
    # ...

print("Transformation test passed.")

test_conn.close()

```

## Test Grain Invariants

Suppose a feature table should contain:

A direct test is:

An empty result confirms the uniqueness invariant for the current data.

Better still, where appropriate, enforce it with a database uniqueness constraint.

## SQL

```

one row per customer per snapshot_date

```

SQL

```
SELECT
  customer_id,
  snapshot_date,
  COUNT(*)
FROM feature_table
GROUP BY
  customer_id,
  snapshot_date
HAVING COUNT(*) > 1;
```

## SQL Style Matters

SQL can become difficult to maintain when every query uses different formatting and naming conventions.

Readable SQL generally benefits from:

- consistent keyword casing
- meaningful aliases
- one logical expression per line
- clear indentation
- explicit join conditions
- named CTEs
- limited nesting
- comments explaining why
- deterministic ordering when needed

Style is not cosmetic when queries implement important business logic.

## Meaningful CTE Names

Compare:

with:

The second version communicates transformation stages.

CTE names should describe the relation produced by that stage.

For complicated data pipelines, the SQL itself becomes a form of executable documentation.

SQL

```
WITH a AS (...),
     b AS (...),
     c AS (...)
```

## SQL

```
WITH customer_cohorts AS (...),  
     monthly_activity AS (...),  
     retained_users AS (...)
```

## Comments Should Explain Why

A weak comment:

The SQL already says that.

A useful comment:

Comments should explain business reasoning, non-obvious constraints, and compatibility decisions.

## PYTHON

```
-- filter date  
WHERE event_date >= ...
```

## PYTHON

```
-- Use only data available before the prediction cutoff  
-- to prevent target leakage.  
WHERE event_time < prediction_time
```

## Avoid Magic Values

A query containing:

may be incomprehensible.

Prefer meaningful values, reference tables, parameters, or documented constants.

Likewise, unexplained dates such as:

should have a reason.

Magic values make analytical logic difficult to audit and change.

## PYTHON

```
WHERE status <> 7
```

## PYTHON

```
2024-07-17
```

## Reusable SQL Models

Data transformation projects often structure SQL into reusable models.

Conceptually:



Each stage should have a clear grain and responsibility.

This modular structure improves:

- testing
- lineage
- readability
- reuse
- incremental processing

Avoid creating a single 2000-line query when meaningful relational stages can be named and validated.

PYTHON

```
raw_orders
->
clean_orders
->
customer_daily_orders
->
customer_features
```

## Lineage

Data lineage answers:

Where did this data come from?

For a derived feature:

lineage might be:

Lineage becomes important when:

- source schema changes
- data quality fails
- a model behaves unexpectedly
- metrics disagree
- compliance asks where data originated

PYTHON

```
customer_30d_spend
```

## PYTHON

```
payments table  
-> validated payments  
-> customer daily spend  
-> rolling 30-day feature
```

## Provenance

Provenance is related to lineage but often emphasizes exact origin and processing context.

Useful provenance metadata may include:

- source dataset
- source version
- transformation version
- execution ID
- code commit
- parameters
- timestamp
- destination version

For AI systems, provenance is essential for reproducibility.

Knowing a model used "customer data" is insufficient. We need to know which customer-data snapshot and which transformations.

## Feature Engineering with SQL

SQL is extremely useful for tabular feature engineering.

Examples include:

These features often combine:

- joins
- aggregation
- date filtering
- window functions
- point-in-time constraints

The most important ML-specific concern is avoiding leakage.

## PYTHON

```
30-day transaction count  
average purchase amount  
days since last login  
number of failed payments  
customer lifetime spend  
latest account state
```

## Point-in-Time Correct Features

Suppose a training row represents a prediction at:

A valid historical feature must use only information available at or before the allowed cutoff.

For example:

or another precisely defined boundary.

If the feature query accidentally uses transactions from March 11, the model sees information from the future.

This is temporal data leakage.

## PYTHON

```
2026-03-10 12:00
```

## PYTHON

```
WHERE transaction_time < prediction_time
```

## Feature Window Boundaries

Consider a 30-day purchase feature.

A conceptual definition might be:

Using a half-open interval prevents the event occurring exactly at prediction time from being ambiguously included when it should not be visible.

The correct inclusivity depends on the application, but it must be explicit.

Notice that user 1's purchase after the prediction timestamp is not included, even though its amount is very large.

This is the correct behavior for a historical feature.

A query that simply aggregated all purchases by user and joined the result to historical prediction rows would leak future information.

## SQL

```

adv_conn.executescript("""
CREATE TABLE prediction_points_adv (
    prediction_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    prediction_time TEXT NOT NULL
);

CREATE TABLE purchases_adv (
    purchase_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    purchase_time TEXT NOT NULL,
    amount REAL NOT NULL
);

INSERT INTO prediction_points_adv
VALUES
    (1, 1, '2026-03-10 12:00:00'),
    (2, 2, '2026-03-10 12:00:00');

INSERT INTO purchases_adv
VALUES
    (1, 1, '2026-02-15 10:00:00', 100.0),
    (2, 1, '2026-03-01 09:00:00', 200.0),
    (3, 1, '2026-03-11 09:00:00', 9999.0),
    (4, 2, '2026-02-01 08:00:00', 500.0),
    (5, 2, '2026-03-05 08:00:00', 75.0);
""")

# ...
adv_conn
)

display(point_in_time_features)

```

## SQL

```

prediction_time - 30 days
<= transaction_time
<
prediction_time

```

## Label Construction

ML labels also require precise SQL definitions.

Suppose a fraud model predicts whether a transaction will be followed by a chargeback within 30 days.

The label query needs:

- prediction/reference event
- outcome event

- future observation window
- boundary rules
- sufficient follow-up time

A transaction from yesterday cannot always be labeled negative merely because no chargeback has appeared yet. The observation window may not have completed.

This is called label maturity or outcome completeness.

### Feature Time and Label Time Are Different

For supervised learning:

The feature query must not cross forward into the label window.

The label query often intentionally looks forward.

Confusing these two windows is a major source of leakage.

A well-designed training dataset documents:

PYTHON

```
past / present  
-> features  
  
future  
-> label
```

PYTHON

```
prediction time  
feature cutoff  
label window  
observation end
```

### Current-State Tables Can Leak Future Information

Suppose today's customer table says:

We build a historical training example from six months ago and join the current customer table.

If the customer became delinquent only last month, the historical training row now contains future information.

Current-state tables are therefore dangerous in retrospective ML training unless the field is truly time-invariant.

Historical feature reconstruction often requires versioned or event-based source data.

## PYTHON

```
customer_id = 5  
status = delinquent
```

## Snapshot Feature Tables

One architecture computes features at periodic snapshots.

Example grain:

Columns might include:

Advantages:

- reusable training inputs
- easier auditing
- faster downstream training
- explicit point-in-time dates

Costs include:

- storage
- refresh logic
- backfills
- feature-definition versioning

## PYTHON

```
one row per customer per feature_date
```

## PYTHON

```
purchase_count_30d  
average_order_value_90d  
days_since_last_login  
failed_payment_count_7d
```

## Online and Offline Feature Consistency

A model may train using SQL-computed offline features but serve using features computed by application code.

If the definitions differ, training-serving skew occurs.

For example:

or:

Feature definitions should be shared, tested, versioned, or otherwise governed to reduce skew.

## PYTHON

```
offline:  
30 days inclusive  
  
online:  
30 * 24 hours exclusive
```

## PYTHON

```
offline:  
ignore cancelled orders  
  
online:  
include cancelled orders
```

## Incremental Feature Computation

Suppose daily customer features depend on years of history.

Recomputing everything every day may become expensive.

Incremental strategies can maintain intermediate aggregates such as:

Then longer feature windows can aggregate smaller daily tables.

This is a classic trade-off between:

- storage
- computation
- freshness
- complexity

Correctness under late-arriving events must still be addressed.

## PYTHON

```
customer daily spend  
customer daily event counts
```

## Feature Backfills

When a feature definition changes, historical values may need to be recomputed.

For example:

A feature pipeline should know:

- which version generated existing data
- what date range needs backfill
- whether online consumers can tolerate mixed versions

- how publication occurs atomically

Versioning is therefore not only for models. Data transformations also have versions.

PYTHON

```
v1:  
count all transactions  
  
v2:  
exclude reversed transactions
```

## Data Contracts

A data contract defines expectations between data producers and consumers.

It may describe:

- schema
- types
- allowed values
- keys
- nullability
- semantics
- freshness
- ownership
- compatibility expectations

A source adding a column is usually easier to tolerate than changing:

from:

to:

without notice.

Semantic changes can be more dangerous than structural changes.

PYTHON

```
amount
```

PYTHON

```
currency units
```

## Schema Drift

Schema drift occurs when incoming structure changes unexpectedly.



Examples:

- new column
- removed column
- renamed field
- type change
- nested JSON shape change

Pipelines should decide whether unexpected drift should:

- fail
- warn
- quarantine records
- accept compatible additions
- trigger schema evolution

Silently coercing everything into strings may keep a pipeline "green" while destroying data meaning.

### **Quarantine Patterns**

Bad rows do not always need to crash an entire large ingestion batch.

Some systems route invalid records into a quarantine/dead-letter structure containing:

- original record
- validation error
- source
- ingestion time
- batch ID

Valid rows continue while invalid rows remain observable for investigation.

This is appropriate only when partial acceptance matches business requirements. Financial or strongly transactional imports may instead require atomic rejection.

### **Reprocessing**

Reliable data platforms assume transformations may need to run again.

Reasons include:

- fixed bug
- new business logic
- late source data
- schema change
- corruption recovery

- feature backfill

Reprocessing becomes easier when:

- raw inputs are reproducibly available
- transformations are deterministic
- outputs are versioned
- jobs are idempotent
- lineage is recorded

### Exactly-Once Processing Awareness

"Exactly once" is often used loosely.

In a distributed pipeline:

- messages can be retried
- workers can crash
- acknowledgements can be lost
- databases and queues have separate transactions

Many robust systems instead achieve the desired final state through:

Do not assume that a framework's "exactly once" phrase eliminates the need to understand its precise failure semantics.

#### PYTHON

```
at-least-once delivery
+
idempotent processing
+
deduplication
+
transactional boundaries
```

### SQL and Orchestration

SQL transformations are often executed by an orchestration system.

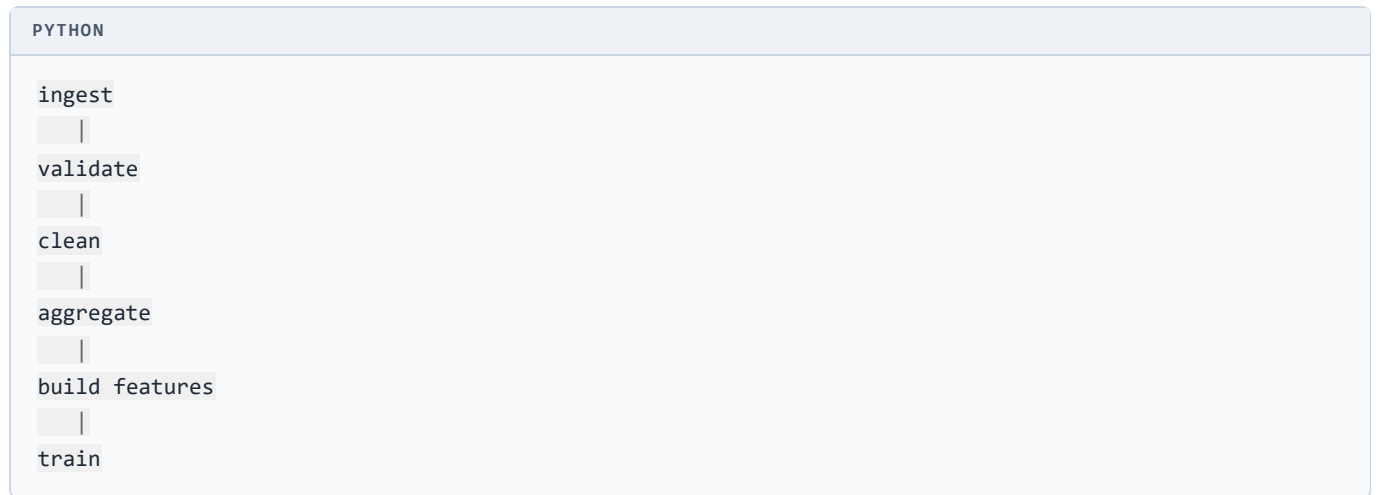
An orchestrator coordinates:

- dependencies
- schedules
- retries
- parameters
- backfills
- logging

- alerts

Conceptually:

SQL is one layer of the pipeline. Workflow state and dependencies belong to broader data engineering.



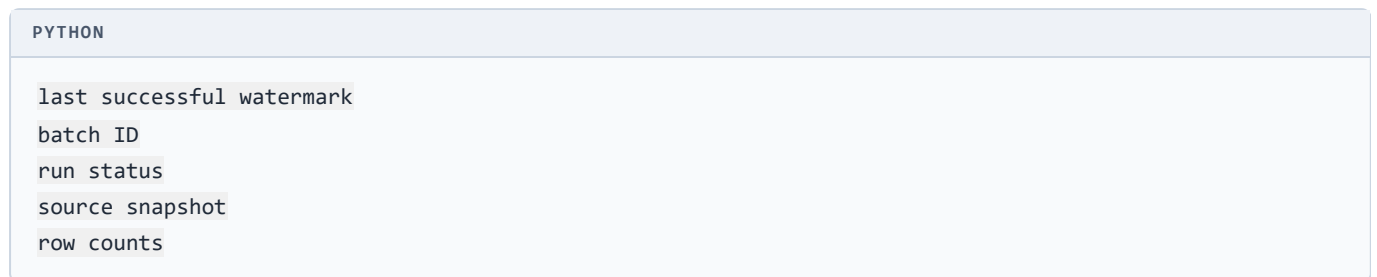
### Do Not Hide Pipeline State in Random Files

A production pipeline may need durable state such as:

Scattering this state across ad-hoc local text files can make distributed execution unreliable.

Operational metadata should have a deliberate persistence strategy.

Relational databases are often well suited for this type of transactional pipeline metadata.



### Reusable Analytical Query Design

Complex SQL becomes easier to reason about when built as stages.

For example:

For every stage, document:

This creates a transformation DAG in relational form.

## PYTHON

```

eligible_population
|
filtered_events
|
entity_level_features
|
labels
|
final_training_dataset

```

## PYTHON

```

grain
keys
filters
time semantics

```

**Avoid Accidental Many-to-Many Joins**

A common pipeline bug occurs when two intermediate relations both contain multiple rows per entity.

Suppose:

Joining only on `customer_id` can create:

per customer.

Before joining, verify that join keys represent the intended grain on both sides.

Many "mysterious duplicate" problems are actually grain mismatches.

## PYTHON

```

customer_features:
3 rows per customer

customer_labels:
2 rows per customer

```

## PYTHON

```

3 x 2 = 6 rows

```

**Reconciliation After Joins**

Useful join checks include:

If a supposedly one-to-one enrichment doubles row count, investigate before continuing.

Data pipeline correctness should be checked at transformation boundaries, not only at the final output.

## PYTHON

```
row count before
row count after
distinct entity count before
distinct entity count after
unmatched entities
duplicate key count
```

## Null Semantics in Pipelines

Pipelines often incorrectly convert `NULL` to zero.

But:

may mean:

while:

means a known numeric zero.

For ML features, this distinction can matter significantly.

Imputation policy should be deliberate and often belongs to the feature/model pipeline rather than being silently applied by every SQL transformation.

## PYTHON

```
NULL
```

## PYTHON

```
unknown / unavailable / not applicable
```

## Time Zones

Production event pipelines frequently contain multiple time zones.

Questions include:

- Are timestamps stored in UTC?
- Is the source timestamp timezone-aware?
- Which timezone defines a business day?
- What happens at daylight-saving transitions?
- Does a cohort use user-local date or UTC date?

SQLite's date/time facilities are limited compared with richer server databases, so production temporal logic should use the capabilities of the chosen platform carefully.

A date boundary is not meaningful until its timezone semantics are known.

## Deterministic Transformations

A reproducible transformation should avoid unspecified order and unstable choices.

Examples of nondeterminism include:

- `LIMIT` without `ORDER BY`
- selecting a row from tied values without a tiebreaker
- using changing current timestamps in historical rebuilds without deliberate semantics
- depending on mutable external state

Determinism makes:

- testing
- backfills
- debugging
- reproducibility

substantially easier.

## SQL Review Checklist

For an important transformation, review:

- What does one output row represent?
- What is the output key?
- Are joins one-to-one, one-to-many, or many-to-many?
- Can joins multiply rows?
- Are `NULL` semantics correct?
- Are time boundaries explicit?
- Is ordering deterministic?
- Are future values excluded where required?
- Are duplicates handled using business semantics?
- Is the query idempotent when materialized?
- Can late data change the result?
- What tests verify the output?
- Is performance acceptable?
- Is lineage documented?

## Data Pipeline Review Checklist

For an incremental pipeline, additionally ask:

- What is the source of truth?

- What defines a new row?
- What defines an updated row?
- How are deletes represented?
- What is the watermark?
- Can several rows share the watermark value?
- Can data arrive late?
- Is there a lookback strategy?
- Can the batch be safely retried?
- When does the watermark advance?
- Are writes atomic?
- Is batch metadata recorded?
- Can the pipeline be backfilled?
- How is a failed batch recovered?
- How are quality failures handled?

### **AI Training Dataset Review Checklist**

Before trusting a SQL-built training dataset, ask:

- What does one training row represent?
- What is prediction time?
- What is the feature cutoff?
- What is the label window?
- Are current-state tables leaking future information?
- Are features computed only from allowed history?
- Are labels mature?
- Are multiple source rows multiplying examples?
- Are duplicate entities intentional?
- Are missing values meaningful?
- Can the dataset be reproduced later?
- Is the source snapshot/version recorded?
- Is the feature transformation version recorded?
- Is class distribution plausible?
- Are train/validation/test time boundaries correct?

### **Common Advanced SQL Mistakes**

Common mistakes include:

- counting events instead of distinct entities in retention

- using the wrong cohort anchor
- computing funnel stages without enforcing required order
- using `DISTINCT` as generic deduplication
- choosing latest rows without deterministic tie-breaking
- joining current state into historical examples
- using event-time watermarks without handling late data
- advancing watermarks before successful publication
- ignoring source deletions
- making incremental pipelines non-idempotent
- using dynamic SQL without structural allowlists
- storing important relational fields only inside JSON
- treating `LIKE '%term%'` as scalable full-text search
- ignoring overlapping temporal intervals
- checking only whether a pipeline ran, not whether data is valid
- averaging averages without weights
- joining several detail tables before establishing grain
- building ML features with future information

### End-to-End SQL Data Pipeline Architecture

A robust SQL-centered pipeline can conceptually look like:

Each boundary should have an explicit grain and data contract.



## PYTHON

## Operational Sources

```
|
v
```

## Ingestion

```
|
v
```

## Raw / Staging Data

```
|
+---- Data Quality Checks
|
v
```

## Cleaned / Standardized Data

```
|
+---- Deduplication
+---- Schema Validation
+---- Referential Validation
|
v
```

## Modeled Relational Data

```
|
+---- Analytical Facts/Dimensions
+---- Historical State
|
v
```

## Feature / Reporting Models

```
|
+---- Point-in-Time Validation
+---- Metric Tests
# ...
+---- Version
+---- Batch Metadata
+---- Lineage
+---- Quality Results
```

**End-to-End AI Feature Pipeline Example**

Imagine building a churn model.

Operational data contains:

A robust SQL workflow might:

1. define prediction dates
2. identify eligible customers
3. construct historical payment features before prediction time
4. construct historical login features
5. construct support-interaction features
6. join each feature table at one-row-per-prediction grain
7. define churn outcome in a future label window

8. exclude examples whose label window has not matured
9. run uniqueness and leakage checks
10. publish a versioned training dataset
11. record source snapshots and transformation version

The difficult part is not `SELECT`.

The difficult part is keeping time, grain, identity, and provenance correct across the entire workflow.

PYTHON

```
customers
subscriptions
payments
logins
support tickets
```

## Advanced SQL Patterns and Data Pipeline Engineering Summary

Advanced SQL combines relational reasoning with temporal, analytical, and operational correctness.

Cohort analysis groups entities around a defined anchor. Retention then measures subsequent activity against a clearly defined denominator.

Funnels measure progression through actions, but event membership and ordered event progression are different problems.

Deduplication requires a business key and deterministic survivor rule. Window functions make latest-version and ranking patterns expressive.

Gaps-and-islands techniques identify consecutive sequences by combining `LAG`, boundary detection, cumulative windows, and grouping.

Relational division solves "all required items" questions. Conditional aggregation provides portable pivot-like transformations.

JSON adds useful flexibility but should not replace important relational structure. Full-text search requires specialized indexing and differs from vector similarity search.

Temporal databases distinguish current state from historical state. Effective-dated records and point-in-time joins prevent historical data from being incorrectly enriched with future/current information.

ETL and ELT describe different transformation locations. Staging layers, raw preservation, batch metadata, and lineage improve reproducibility.

Incremental pipelines reduce processing cost but introduce difficult problems involving watermarks, ties, late-arriving data, deletes, retries, and idempotency.

CDC provides richer change streams than ordinary timestamp filtering but introduces distributed delivery semantics.

Data quality must be tested explicitly for completeness, uniqueness, validity, integrity, freshness, and volume.

SQL transformations should be tested like code.

For AI engineering, SQL is especially powerful for feature computation, historical reconstruction, labeling, and dataset generation—but point-in-time correctness is mandatory to prevent leakage.

The central principle is:

Every transformation should have explicit grain, identity, time semantics, quality expectations, and reproducible lineage.

## Edge Cases and Boundaries

### TEST THE BOUNDARY CASES

Most SQL errors occur at boundaries:

- midnight
- month-end
- year-end
- inclusive/exclusive date endpoints
- empty tables
- single-row partitions
- tied values
- `NULL`
- duplicate keys
- no matching join row
- several matching join rows

Good test data deliberately includes these cases rather than only clean happy-path examples.

## Real-World and AI Engineering Relevance

### Production Data in Development

Using raw production data for development can expose sensitive information.

Safer approaches may include:

- synthetic datasets

- properly anonymized data
- masked snapshots
- restricted staging environments
- sampled data under governance controls

The correct approach depends on organizational policy and legal requirements.

Convenience is not sufficient justification for uncontrolled production-data copying.

### SQLite for Local AI Applications

SQLite can be extremely useful for:

- local experiment tools
- desktop AI applications
- offline metadata
- cached results
- single-user prototypes
- evaluation utilities
- embedded applications
- unit/integration tests

Its zero-server architecture dramatically reduces operational complexity.

This makes SQLite an important engineering tool even when the eventual production backend uses a server database.

### Where this shows up in AI engineering

- relational modelling, keys and normalisation
- DDL, constraints and data types
- DML, DQL and filtering
- joins and set operations
- aggregation, grouping and subqueries
- window functions and CTEs
- transactions, indexes and query performance

#### EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

# Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

SQL

LEARNING PHILOSOPHY

```
SELECT *
FROM customers;
```

PYTHON

DATA AND PERSISTENT STORAGE

```
customer_name = "Aisha"
balance = 2500
```

PYTHON

WHY NOT STORE EVERYTHING IN CSV FILES?

```
customers.csv
orders.csv
```

PYTHON

RELATIONAL DATABASES

```
customers

customer_id | name | city
-----|-----|-----
1 | Aisha | Delhi
2 | Rohan | Pune
3 | Meera | Jaipur
```

PYTHON

TABLES, ROWS, COLUMNS, AND VALUES

```
employee_id | name | department | salary
-----|-----|-----|-----
101 | Aisha | AI | 90000
102 | Rohan | Data | 85000
```

## PYTHON

## RELATION VERSUS SQL TABLE

relational theory



conceptual foundation

SQL database



practical implementation

## PYTHON

## SCHEMA

customers

|             |         |             |
|-------------|---------|-------------|
| customer_id | INTEGER | PRIMARY KEY |
| name        | TEXT    | NOT NULL    |
| email       | TEXT    | UNIQUE      |
| city        | TEXT    |             |

## PYTHON

## SCHEMA VERSUS DATA

|            |           |
|------------|-----------|
| student_id | : integer |
| name       | : text    |
| score      | : numeric |

## Glossary

### superkey

any set of attributes that uniquely identifies a row.

### UNION

Aur ML ke liye important point: CTE query ko clean bana sakta hai, but clean query automatically leakage-free nahi hoti. Time boundary phir bhi manually correct define karni padegi.

### UPDATE

conditional updates

### SAVEPOINT

ACID properties

**ANALYZE**

performance measurement

**island**

a consecutive run.

**NULL**

duplicate timestamps

## Revision Checklist

---

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

- ☐ Learning Philosophy
- ☐ Data and Persistent Storage
- ☐ Database Management Systems
- ☐ Why Not Store Everything in CSV Files?
- ☐ Relational Databases
- ☐ Tables, Rows, Columns, and Values
- ☐ Relation Versus SQL Table
- ☐ Schema
- ☐ Schema Versus Data
- ☐ Metadata
- ☐ Domains
- ☐ Tuples and Attributes
- ☐ Degree and Cardinality

☐ Keys☐ Superkeys☐ Candidate Keys☐ Primary Keys☐ Composite Keys☐ Natural Keys☐ Surrogate Keys☐ Alternate Keys☐ Foreign Keys